Introduction

> **Download PDF**: Biostatistics in Practice (PDF)

Why This Book Exists

Every bioinformatics pipeline produces numbers. P-values, fold changes, confidence intervals, FDR-adjusted q-values, effect sizes, cluster assignments. But numbers without statistical understanding are dangerous. A p-value of 0.049 does not mean your result is true. A fold change of 10 in a single sample means nothing. A clustering algorithm will always find clusters — even in random data.

This book teaches you to think statistically about biological data. Not just which test to run (though we cover that), but *why* that test is appropriate, *what* its assumptions are, *when* it fails, and *how* to interpret results honestly.

Every chapter uses real public datasets — from TCGA, GEO, gnomAD, GTEx, 1000 Genomes, ENCODE, and ClinVar. Every analysis uses real BioLang builtins. No fabricated data, no invented functions, no toy examples that work perfectly.

Who This Book Is For

- **Bioinformaticians** who run DESeq2 but aren't sure why it uses a negative binomial model instead of a t-test
- **Bench scientists** who get back a spreadsheet of genes with p-values and need to know which ones to follow up on
- **Clinical researchers** designing genomics studies who need proper sample size calculations
- **Students** learning biostatistics who want to work with real data, not textbook exercises

What You Need

BioLang installed. See lang.bio for installation.

```
curl -fsSL https://lang.bio/install.sh | sh
```

```
bl --version
```

Internet access. Most chapters download data from public repositories.

No prior statistics background required. Each chapter builds concepts from the ground up, with biological context explaining why each method exists.

How Each Chapter Works

Every chapter follows the same structure:

1. **The Problem** — A real biological or clinical question that requires statistical thinking
2. **Why It Matters** — The clinical or research significance
3. **The Data** — Real public dataset with download instructions
4. **Exploration** — Looking at the data before testing anything
5. **The Analysis** — Step-by-step statistical analysis in BioLang
6. **Interpretation** — What the results actually mean (and don't mean)
7. **Decision** — What action to take, what to report, what to investigate next
8. **Pitfalls** — Common mistakes and how to avoid them
9. **Where Next?** — Related problems this methodology applies to

The Four Parts

Part I: Foundations covers the statistical mindset — how to think about variability in biological data, how to explore data before testing, what distributions look like in genomics, and how to design experiments properly.

Part II: Hypothesis Testing covers comparing groups — two-sample tests, multi-group comparisons, when to go non-parametric, and the critical problem of multiple testing that dominates genomics.

Part III: Modeling moves beyond simple comparisons to regression, GLMs, survival analysis, and handling the batch effects and confounders that plague every real dataset.

Part IV: Multivariate & Advanced covers the high-dimensional methods that genomics requires — PCA, clustering, and meta-analysis across studies.

BioLang Statistics Builtins Used

This book uses these real BioLang builtins extensively:

Descriptive: `mean`, `median`, `stdev`, `variance`, `quantile`, `summary`, `cor`

Tests: `ttest`, `ttest_paired`, `ttest_one`, `anova`, `chi_square`, `fisher_exact`, `wilcoxon`, `ks_test`, `spearman`, `kendall`

Correction: `p_adjust` (Benjamini-Hochberg, Bonferroni, Holm, BY)

Models: `lm`, `glm`, `cox_ph`, `kaplan_meier`

Distributions: `dnorm`, `pnorm`, `qnorm`, `dbinom`, `pbinom`, `dpois`, `ppois`, `rnorm`, `rbinom`, `rpois`

Multivariate: `pca`, `kmeans`, `leiden`, `umap`, `tsne`, `cor_matrix`, `cov_matrix`, `dist_matrix`

Normalization: `normalize`, `tpm`, `fpkm`, `cpm`

Tables: `group_by`, `summarize`, `filter_by`, `arrange`, `mutate`, `pivot_longer`, `pivot_wider`, `inner_join`, `describe`

Plots: `histogram`, `boxplot`, `scatter`, `volcano`, `qq_plot`, `heatmap`, `pca_plot`, `forest_plot`, `kaplan_meier` (plot)

A Note on Honesty

Statistics is the discipline of being honest about uncertainty. This book reflects that philosophy. When a method has limitations, we say so. When a p-value doesn't answer your question, we explain what does. When a common practice is statistically dubious, we call it out.

Real biological data is messy, has batch effects, violates assumptions, and requires judgment. This book prepares you for that reality.

Thinking Statistically About Biology

The Problem

Dr. Elena Vasquez stares at her screen. Her collaborator has sent a spreadsheet with RNA-seq results from a breast cancer study. The email reads: “Great news! We found 2,847 differentially expressed genes between tumor and normal tissue ($p < 0.05$). The results are very significant.”

Elena opens the file. Three tumor samples. Three normal samples. $n = 3$ per group. She winces. Not because the biology is wrong — breast cancer certainly involves massive transcriptional changes — but because with $n = 3$, the statistics are almost guaranteed to be misleading. Some of those 2,847 genes are real. Many are not. And there is no way to tell which is which from this experiment alone.

This chapter is about developing the instinct to ask the right questions before trusting a number. What does a p-value actually mean? Why is effect size more important than significance? Why does sample size matter so profoundly in genomics? And what should you do when — as happens constantly in biology — the ideal experiment is too expensive to run?

We will work through these questions using real breast cancer RNA-seq data from GEO dataset GSE183947, a study that examined gene expression in breast cancer tissue using RNA sequencing with FPKM quantification.

Why It Matters

The reproducibility crisis in biomedical research is not abstract. A 2012 study by Amgen scientists attempted to reproduce 53 “landmark” cancer biology papers. They could reproduce only 6 — an 11% success rate. A similar effort by Bayer found that only 25% of published preclinical studies could be confirmed.

Underpowered studies are a primary driver of irreproducibility. When you have too few samples, several pathological things happen simultaneously:

1. **Inflated effect sizes.** Only the largest (often spurious) effects reach significance with small samples. This is called the “winner’s curse.” The effects that survive your $p < 0.05$ filter are systematically overestimated.

2. **High false discovery rates.** With 20,000 genes and n = 3, you are running 20,000 tests. Even with multiple testing correction, the statistical power is so low that a large fraction of “significant” results are false positives.

3. **Missed real biology.** Conversely, many genes with genuine, biologically meaningful expression changes will not reach significance. You are blind to moderate effects.

4. **Unstable variance estimates.** With only 3 observations per group, the sample variance is extremely noisy. A single outlier can dominate. The t-statistic denominator becomes unreliable.

These are not theoretical concerns. They affect every experiment, every paper, every clinical decision that rests on statistical evidence from small samples.

The Data

We will use GEO dataset GSE183947, which contains RNA-seq gene expression data from breast cancer patients. The dataset provides FPKM (Fragments Per Kilobase of transcript per Million mapped reads) values across tumor and normal tissue samples.

```
# Download the breast cancer RNA-seq expression data
# GEO accession: GSE183947 — breast cancer FPKM values
let url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let raw_file = download(url, "data/GSE183947_fpkm.csv.gz")

# Read the expression matrix
# Rows are genes, columns are samples
let expr = read_csv(raw_file)

# Examine the structure
println("Dimensions: " + to_string(len(expr)) + " genes x " +
to_string(ncols(expr)) + " samples")
println("Column names:")
println(columns(expr))

# Look at the first few rows
take(expr, 10)
```

The dataset contains FPKM expression values for thousands of genes across multiple samples. For our analysis, we will focus on the contrast between tumor and normal tissue.

```
# Identify sample groups from column names
# In this dataset, column names encode sample type information
let all_cols = columns(expr)
println("All columns in dataset:")
each(all_cols, |c| println("  " + c))

# Separate the gene identifier column from expression columns
let gene_col = "gene_id"
let sample_cols = filter(all_cols, |c| c != gene_col)
println("Number of samples: " + to_string(len(sample_cols)))
```

Exploration

Before any statistical testing, we must understand what the data looks like. This is not optional — it is the most important step. Skipping exploration is how you end up with “2,847 significant genes” that mean nothing.

What does a typical gene’s expression look like?

```
# Pick a well-known housekeeping gene to examine
# GAPDH is commonly used as a reference
let gapdh = filter_by(expr, col("gene_id") |> contains("GAPDH"))
println("GAPDH expression across samples:")
println(gapdh)

# Pick a known breast cancer gene
let brca1 = filter_by(expr, col("gene_id") |> contains("BRCA1"))
println("BRCA1 expression across samples:")
println(brca1)
```

Distribution of expression values

```
# Get expression values for the first sample column
let first_sample = sample_cols |> take(1) |> col_values(expr, _)

# Raw FPKM distribution
println("Raw FPKM summary for first sample:")
println(summary(first_sample))
println("Mean: " + to_string(mean(first_sample)))
println("Median: " + to_string(median(first_sample)))
println("Stdev: " + to_string(stdev(first_sample)))
println("Min: " + to_string(min(first_sample)))
println("Max: " + to_string(max(first_sample)))

# The mean is much larger than the median — classic right skew
# Most genes have low expression, a few have very high expression

# Visualize the raw distribution
histogram(first_sample, "plots/raw_fpkm_distribution.svg")
  |> save_plot("plots/raw_fpkm_distribution.svg")
```

Why log-transformation is essential

```
# Log2-transform the expression values
# Add a small pseudocount to handle zeros
let pseudocount = 1.0
let log_values = map(first_sample, |x| log2(x + pseudocount))

println("Log2(FPKM+1) summary:")
println("Mean: " + to_string(mean(log_values)))
println("Median: " + to_string(median(log_values)))
println("Stdev: " + to_string(stdev(log_values)))

# Now the distribution is much more symmetric
histogram(log_values, "plots/log2_fpkm_distribution.svg")
  |> save_plot("plots/log2_fpkm_distribution.svg")
```

The raw FPKM distribution is massively right-skewed. A handful of highly expressed genes (ribosomal proteins, actins, histones) dominate the scale. Log-transformation compresses this range and makes the distribution approximately normal, which is a requirement for many statistical tests.

This is not a mathematical trick. It reflects the biology: gene expression regulation is multiplicative (fold-changes), not additive. A 2-fold change from 100 to 200 FPKM is biologically comparable to a 2-fold change from 1000 to 2000. The log scale captures this: log2(200) - log2(100) = log2(2000) - log2(1000) = 1.

Checking for outlier samples

```
# Calculate summary statistics for each sample
let sample_summaries = map(sample_cols, |col_name| {
  let vals = col_values(expr, col_name)
  let log_vals = map(vals, |x| log2(x + 1.0))
  record(
    sample: col_name,
    raw_mean: mean(vals),
    raw_median: median(vals),
    log_mean: mean(log_vals),
    log_stdev: stdev(log_vals),
    zeros: len(filter(vals, |x| x == 0.0)),
    max_val: max(vals)
  )
})

let sample_table = table(sample_summaries)
println("Per-sample statistics:")
println(sample_table)

# Look for samples with unusual statistics
# An outlier sample might have very different mean, stdev, or zero count
let log_means = map(sample_summaries, |s| s.log_mean)
let overall_mean = mean(log_means)
let overall_sd = stdev(log_means)
println("Overall mean of log-means: " + to_string(overall_mean))
println("Overall SD of log-means: " + to_string(overall_sd))
```

Correlation between samples

```
# Samples within the same group should correlate highly
# This is a basic quality check
# Extract expression profiles for the first few samples
let s1_vals = col_values(expr, sample_cols[0])
let s2_vals = col_values(expr, sample_cols[1])

let log_s1 = map(s1_vals, |x| log2(x + 1.0))
let log_s2 = map(s2_vals, |x| log2(x + 1.0))

let r = cor(log_s1, log_s2)
println("Pearson correlation between first two samples: " + to_string(r))

# In a well-behaved RNA-seq experiment, within-group correlations
# should be > 0.95. Between-group correlations are typically lower.
# If within-group correlation is < 0.90, something may be wrong.

scatter(log_s1, log_s2, "plots/sample_correlation.svg")
  |> save_plot("plots/sample_correlation.svg")
```

The Analysis

Now we arrive at the core question: which genes are differentially expressed between tumor and normal tissue? But before we rush to p-values, we need to understand what we are actually computing.

What is a p-value, really?

A p-value answers exactly one question: “If there were truly no difference between the groups, how often would I see a test statistic this extreme or more extreme?”

It does NOT answer:

- “What is the probability that this gene is differentially expressed?” (That would require a Bayesian posterior.)
- “How large is the effect?” (That is the effect size.)
- “Is this result biologically important?” (That requires domain knowledge.)
- “Will this replicate?” (That depends on power and effect size.)

Let us make this concrete with a single gene.

```
# Simulate the thought experiment behind a p-value
# Suppose gene X has identical expression in tumor and normal
# What distribution of t-statistics would we see by chance?

# First, let us see a real gene comparison
# Assume we have 3 tumor and 3 normal samples
let tumor_cols = slice(sample_cols, 0, 3)
let normal_cols = slice(sample_cols, 3, 6)

# Pick an arbitrary gene (row 100)
let gene_row = slice(expr, 99, 100)
let gene_name = col_values(gene_row, gene_col) |> take(1)

let tumor_vals = map(tumor_cols, |c| {
  col_values(gene_row, c) |> take(1) |> to_float()
})
let normal_vals = map(normal_cols, |c| {
  col_values(gene_row, c) |> take(1) |> to_float()
})

# Log-transform
let tumor_log = map(tumor_vals, |x| log2(x + 1.0))
let normal_log = map(normal_vals, |x| log2(x + 1.0))

println("Gene: " + to_string(gene_name))
println("Tumor log2(FPKM+1): " + to_string(tumor_log))
println("Normal log2(FPKM+1): " + to_string(normal_log))
println("Tumor mean: " + to_string(mean(tumor_log)))
println("Normal mean: " + to_string(mean(normal_log)))
println("Log2 fold change: " + to_string(mean(tumor_log) - mean(normal_log)))
```

The t-test with n = 3

```
# Perform a t-test on this gene
let result = ttest(tumor_log, normal_log)
println("T-test result:")
println("  t-statistic: " + to_string(result.statistic))
println("  p-value: " + to_string(result.p_value))
println("  degrees of freedom: " + to_string(result.df))

# Now let us think about what this p-value means
# With n=3 per group, df = 4 (approximately)
# The t-distribution with 4 degrees of freedom has very heavy tails
# This means large t-statistics are common by chance alone
```

Why n = 3 is dangerous: the variance problem

```
# The core issue: with n=3, variance estimation is terrible
# Let us demonstrate by looking at the standard deviation
# of our tumor samples

let tumor_sd = stdev(tumor_log)
let normal_sd = stdev(normal_log)
println("Tumor SD (n=3): " + to_string(tumor_sd))
println("Normal SD (n=3): " + to_string(normal_sd))

# With only 3 values, the SD estimate has huge uncertainty
# The 95% confidence interval for sigma given n=3 is approximately
# [0.52 * s, 6.28 * s] — a 12-fold range!
# This means the denominator of your t-statistic is unreliable

let sd_lower = tumor_sd * 0.52
let sd_upper = tumor_sd * 6.28
println("95% CI for true SD: [" + to_string(round(sd_lower, 3)) + ", " +
to_string(round(sd_upper, 3)) + "]")
println("That is a " + to_string(round(sd_upper / sd_lower, 1)) + "-fold
range of uncertainty")
```

Genome-wide testing: the multiple testing catastrophe

```
# Now imagine running this test for every gene
# Let us do it for the first 1000 genes as a demonstration

let results = []
let n_genes = min(len(expr), 1000)

# Process genes in bulk
let gene_subset = take(expr, n_genes)

let test_results = map(range(0, n_genes), |i| {
  let row = slice(gene_subset, i, i + 1)
  let t_vals = map(tumor_cols, |c| {
    col_values(row, c) |> take(1) |> to_float()
  })
  let n_vals = map(normal_cols, |c| {
    col_values(row, c) |> take(1) |> to_float()
  })

  let t_log = map(t_vals, |x| log2(x + 1.0))
  let n_log = map(n_vals, |x| log2(x + 1.0))

  let t_mean = mean(t_log)
  let n_mean = mean(n_log)
  let lfc = t_mean - n_mean
  let res = ttest(t_log, n_log)

  record(
    gene_index: i,
    log2fc: lfc,
    t_stat: res.statistic,
    p_value: res.p_value
  )
})

let results_table = table(test_results)
println("Tested " + to_string(n_genes) + " genes")
```

How many “significant” results do we get?

```
# Count genes with p < 0.05 (unadjusted)
let raw_pvals = map(test_results, |r| r.p_value)
let sig_raw = filter(raw_pvals, |p| p < 0.05)
println("Genes with p < 0.05 (unadjusted): " + to_string(len(sig_raw)))
println("Percentage: " + to_string(round(100.0 * len(sig_raw) / n_genes, 1))
+ "%")

# Under the null hypothesis (no real differences), we expect
# 5% of genes to be "significant" by chance alone
# With 1000 tests, that is 50 false positives expected
println("Expected false positives at alpha=0.05: " + to_string(round(0.05 *
n_genes, 0)))

# Now apply multiple testing correction
let adjusted = p_adjust(raw_pvals, "BH")  # Benjamini-Hochberg
let sig_adj = filter(adjusted, |p| p < 0.05)
println("Genes with adjusted p < 0.05 (BH): " + to_string(len(sig_adj)))

# The Benjamini-Hochberg procedure controls the False Discovery Rate (FDR)
# FDR = expected proportion of false positives among rejected hypotheses
```

Understanding False Discovery Rate

```
# FDR is subtle. Let us be precise about what it means.
#
# Suppose we reject 100 genes at FDR < 0.05.
# This means: among those 100 genes, we expect about 5 to be
# false positives. We do not know WHICH 5, but the overall
# proportion of mistakes is controlled.
#
# This is different from FWER (Family-Wise Error Rate), which
# controls the probability of making even ONE false positive.
# Bonferroni correction controls FWER but is very conservative
# for genomics (thousands of tests).

let adjusted_bonf = p_adjust(raw_pvals, "bonferroni")
let sig_bonf = filter(adjusted_bonf, |p| p < 0.05)
println("Genes with Bonferroni p < 0.05: " + to_string(len(sig_bonf)))

# Bonferroni is typically too conservative for genomics
# BH (FDR) is the standard in the field
```

Effect size: what p-values cannot tell you

```
# A p-value tells you whether an effect exists (under certain assumptions)
# It does NOT tell you how large the effect is
# With enough samples, even trivially small differences become "significant"

# Effect size for a two-group comparison is typically measured as:
# Cohen's d = (mean1 - mean2) / pooled_SD
# Or simply log2 fold change in genomics

# Let us examine the relationship between effect size and significance
let effect_sizes = map(test_results, |r| abs(r.log2fc))
let neg_log_p = map(test_results, |r| -log10(r.p_value + 1e-300))

# Create a volcano plot: effect size (x) vs significance (y)
volcano(effect_sizes, neg_log_p, "plots/volcano_n3.svg")
  |> save_plot("plots/volcano_n3.svg")

println("Effect size statistics:")
println("  Mean |log2FC|: " + to_string(round(mean(effect_sizes), 3)))
println("  Median |log2FC|: " + to_string(round(median(effect_sizes), 3)))

# In practice, we want BOTH:
# 1. Statistical significance (adjusted p < 0.05)
# 2. Meaningful effect size (|log2FC| > 1, i.e., 2-fold change)
let sig_and_large = filter(test_results, |r| {
  let adj_p = p_adjust([r.p_value], "BH") |> take(1)
  adj_p < 0.05 && abs(r.log2fc) > 1.0
})
println("Genes with adjusted p<0.05 AND |log2FC|>1: " +
to_string(len(sig_and_large)))
```

The danger zone: statistical significance without biological significance

```
# Consider a gene with a log2 fold change of 0.1
# That is a 7% change in expression. Even if the p-value is 0.001,
# a 7% change is rarely biologically meaningful.

# Conversely, consider a gene with log2FC = 3 (8-fold change)
# but p = 0.08. With n=3, this might not reach significance
# due to high variance, but an 8-fold change in a cancer gene
# is biologically very important.

# This is why genomics has adopted the "volcano plot" standard:
# plot -log10(p) vs log2FC, and highlight genes that pass
# BOTH thresholds.

# Let us classify our results
let categories = map(test_results, |r| {
  let adj_p = p_adjust([r.p_value], "BH") |> take(1)
  let sig = adj_p < 0.05
  let large = abs(r.log2fc) > 1.0
  if sig && large then "significant_and_large"
  else if sig && !large then "significant_but_small"
  else if !sig && large then "large_but_not_significant"
  else "neither"
})

let category_counts = value_counts(categories)
println("Gene categories:")
println(category_counts)
```

Type I and Type II errors

```
# Type I error (false positive): concluding a gene is DE when it is not
# Type II error (false negative): missing a gene that truly is DE
#
# With n=3:
# - Type I rate is controlled by our alpha threshold (0.05)
# - Type II rate (1 - power) is VERY HIGH for moderate effects
#
# Power = probability of detecting a real effect
# For a t-test with n=3 per group and Cohen's d = 1.0:
# Power is approximately 0.22 (22%)
#
# This means we MISS 78% of genes with a real, large effect!

# Let us calculate approximate power for different effect sizes
# Power for a two-sample t-test depends on:
# - n (sample size per group)
# - d (effect size, Cohen's d)
# - alpha (significance level)

# Using the relationship between power and the non-central t-distribution
# For n=3, alpha=0.05 (two-sided), approximate power:
let effect_d_values = [0.5, 0.8, 1.0, 1.5, 2.0, 3.0]
let n_per_group = 3
let alpha = 0.05

# The critical t-value for df=4, alpha=0.05 two-sided
# t_crit approximately 2.776
let t_crit = 2.776

each(effect_d_values, |d| {
  # Non-centrality parameter: delta = d * sqrt(n/2)
  let delta = d * sqrt(n_per_group / 2.0)
  # Approximate power using normal approximation
  # (This is a rough approximation for illustration)
  let z_alpha = 1.96
  let z_power = delta - z_alpha
  let approx_power = pnorm(z_power, 0.0, 1.0)
  println("  d=" + to_string(d) + ": approximate power = " +
to_string(round(approx_power * 100, 1)) + "%")
})
```

What happens when we simulate the truth

```
# To really understand the n=3 problem, let us simulate data
# where we KNOW the truth

# Generate 1000 "genes":
# - 900 are truly NOT differentially expressed (null genes)
# - 100 ARE truly differentially expressed with log2FC = 1.5

let n_null = 900
let n_de = 100
let true_mean = 8.0  # average log2 expression
let true_sd = 1.5    # within-group standard deviation

# Null genes: same distribution in both groups
let null_pvals = map(range(0, n_null), |i| {
  let group1 = rnorm(3, true_mean, true_sd)
  let group2 = rnorm(3, true_mean, true_sd)
  let res = ttest(group1, group2)
  res.p_value
})

# DE genes: group2 shifted by 1.5 log2 units
let de_pvals = map(range(0, n_de), |i| {
  let group1 = rnorm(3, true_mean, true_sd)
  let group2 = rnorm(3, true_mean + 1.5, true_sd)
  let res = ttest(group1, group2)
  res.p_value
})

# How many null genes are falsely called significant?
let false_positives = len(filter(null_pvals, |p| p < 0.05))
println("False positives (null genes with p<0.05): " +
to_string(false_positives) + " / " + to_string(n_null))
println("False positive rate: " + to_string(round(100.0 * false_positives /
n_null, 1)) + "%")

# How many DE genes are correctly detected?
let true_positives = len(filter(de_pvals, |p| p < 0.05))
println("True positives (DE genes with p<0.05): " + to_string(true_positives)
+ " / " + to_string(n_de))
println("Power (sensitivity): " + to_string(round(100.0 * true_positives /
n_de, 1)) + "%")

# Among all "significant" results, what fraction are false?
let total_sig = false_positives + true_positives
let fdr_observed = false_positives / total_sig
println("Observed FDR: " + to_string(round(100.0 * fdr_observed, 1)) + "%")
println("Out of " + to_string(total_sig) + " significant calls, " +
to_string(false_positives) + " are false")
```

The same simulation with n = 10

```
# Now repeat with n=10 per group
let null_pvals_10 = map(range(0, n_null), |i| {
  let group1 = rnorm(10, true_mean, true_sd)
  let group2 = rnorm(10, true_mean, true_sd)
  let res = ttest(group1, group2)
  res.p_value
})

let de_pvals_10 = map(range(0, n_de), |i| {
  let group1 = rnorm(10, true_mean, true_sd)
  let group2 = rnorm(10, true_mean + 1.5, true_sd)
  let res = ttest(group1, group2)
  res.p_value
})

let fp_10 = len(filter(null_pvals_10, |p| p < 0.05))
let tp_10 = len(filter(de_pvals_10, |p| p < 0.05))
let total_sig_10 = fp_10 + tp_10

println("With n=10:")
println("  False positives: " + to_string(fp_10) + " / " + to_string(n_null))
println("  True positives: " + to_string(tp_10) + " / " + to_string(n_de))
println("  Power: " + to_string(round(100.0 * tp_10 / n_de, 1)) + "%")
if total_sig_10 > 0 then {
  println("  Observed FDR: " + to_string(round(100.0 * fp_10 / total_sig_10,
1)) + "%")
}
```

Exploring additional sample sizes systematically

```
# To see the full picture, run the simulation across many sample sizes
# and track power, FDR, and total significant counts

let sample_sizes_to_test = [3, 4, 5, 6, 8, 10, 12, 15, 20, 25, 30]

let sim_results = map(sample_sizes_to_test, |n| {
  let null_ps = map(range(0, n_null), |i| {
    let g1 = rnorm(n, true_mean, true_sd)
    let g2 = rnorm(n, true_mean, true_sd)
    let res = ttest(g1, g2)
    res.p_value
  })

  let de_ps = map(range(0, n_de), |i| {
    let g1 = rnorm(n, true_mean, true_sd)
    let g2 = rnorm(n, true_mean + 1.5, true_sd)
    let res = ttest(g1, g2)
    res.p_value
  })

  let fp = len(filter(null_ps, |p| p < 0.05))
  let tp = len(filter(de_ps, |p| p < 0.05))
  let total = fp + tp
  let fdr = if total > 0 then round(100.0 * fp / total, 1) else 0.0
  let power = round(100.0 * tp / n_de, 1)

  record(
    n: n,
    true_positives: tp,
    false_positives: fp,
    total_significant: total,
    power_pct: power,
    fdr_pct: fdr
  )
})

println("Comprehensive simulation results:")
println("n/group | Power   FDR     Total Sig  TP   FP")
println("--------|-------------------------------------")
each(sim_results, |r| {
  println("  " + to_string(r.n) + "     | " + to_string(r.power_pct) + "%   "
+ to_string(r.fdr_pct) + "%   " + to_string(r.total_significant) + "       "
+ to_string(r.true_positives) + "   " + to_string(r.false_positives))
})
```

```
# Visualize the power curve from the simulation
let sim_ns = map(sim_results, |r| r.n)
let sim_powers = map(sim_results, |r| r.power_pct / 100.0)

scatter(sim_ns, sim_powers, "plots/simulated_power_curve.svg")
  |> save_plot("plots/simulated_power_curve.svg")

# Also plot the FDR curve
let sim_fdrs = map(sim_results, |r| r.fdr_pct / 100.0)
scatter(sim_ns, sim_fdrs, "plots/simulated_fdr_curve.svg")
  |> save_plot("plots/simulated_fdr_curve.svg")

# Notice: as power increases, FDR decreases
# At n=3: high FDR, low power — the worst of both worlds
# At n=20+: low FDR, high power — where you want to be
```

The winner’s curse: inflated effect sizes in underpowered studies

```
# The winner's curse is subtle but devastating
# In underpowered studies, only the largest observed effects
# pass the significance threshold — but these are overestimates

# Simulate: true log2FC = 1.0, but with n=3, what LFC do we observe
# among the "significant" results?

let observed_lfcs_n3 = []
let observed_lfcs_n20 = []

let n_sims = 1000

# With n=3
let sig_lfcs_3 = map(range(0, n_sims), |i| {
  let g1 = rnorm(3, true_mean, true_sd)
  let g2 = rnorm(3, true_mean + 1.0, true_sd)  # true LFC = 1.0
  let res = ttest(g1, g2)
  let observed_lfc = mean(g2) - mean(g1)
  record(lfc: observed_lfc, significant: res.p_value < 0.05)
})

let sig_only_3 = filter(sig_lfcs_3, |r| r.significant)
let sig_lfc_vals_3 = map(sig_only_3, |r| r.lfc)

# With n=20
let sig_lfcs_20 = map(range(0, n_sims), |i| {
  let g1 = rnorm(20, true_mean, true_sd)
  let g2 = rnorm(20, true_mean + 1.0, true_sd)
  let res = ttest(g1, g2)
  let observed_lfc = mean(g2) - mean(g1)
  record(lfc: observed_lfc, significant: res.p_value < 0.05)
})

let sig_only_20 = filter(sig_lfcs_20, |r| r.significant)
let sig_lfc_vals_20 = map(sig_only_20, |r| r.lfc)

println("Winner's curse demonstration (true log2FC = 1.0):")
println("")
println("With n=3:")
println("  Significant results: " + to_string(len(sig_only_3)) + " / " +
to_string(n_sims))
if len(sig_lfc_vals_3) > 0 then {
  println("  Mean observed LFC among significant: " +
to_string(round(mean(sig_lfc_vals_3), 3)))
  println("  Median observed LFC among significant: " +
to_string(round(median(sig_lfc_vals_3), 3)))
}
println("")
println("With n=20:")
println("  Significant results: " + to_string(len(sig_only_20)) + " / " +
to_string(n_sims))
if len(sig_lfc_vals_20) > 0 then {
  println("  Mean observed LFC among significant: " +
to_string(round(mean(sig_lfc_vals_20), 3)))
  println("  Median observed LFC among significant: " +
to_string(round(median(sig_lfc_vals_20), 3)))
}

# With n=3, the observed LFC among significant results is inflated
# (well above the true value of 1.0)
# With n=20, the observed LFC is close to the truth
println("")
```

```
  println("The true effect is 1.0. With n=3, significant results")
  println("overestimate it. This is the winner's curse.")
```

Nonparametric alternatives

```
  # When you cannot assume normality (or want to be safe with small n),
  # nonparametric tests provide valid p-values without distributional
  assumptions

  # The Wilcoxon rank-sum test (Mann-Whitney U) is the nonparametric
  # equivalent of the two-sample t-test

  let group_a = [8.2, 7.9, 8.5, 9.1, 7.6]
  let group_b = [10.1, 9.8, 10.4, 11.2, 9.5]

  let t_result = ttest(group_a, group_b)
  let w_result = wilcoxon(group_a, group_b)

  println("Parametric vs nonparametric comparison:")
  println("  t-test p-value: " + to_string(round(t_result.p_value, 4)))
  println("  Wilcoxon p-value: " + to_string(round(w_result.p_value, 4)))

  # For normally distributed data, the t-test is more powerful
  # For non-normal data, the Wilcoxon test can be more powerful
  # With large n, both give similar results (by the CLT)

  # The Wilcoxon test is based on RANKS, not values
  # It tests whether one group tends to have larger values than the other
  # It is robust to outliers because ranks are bounded
```

This simulation reveals the brutal arithmetic of underpowered studies. With n = 3, you detect maybe 20-30% of truly differentially expressed genes while accumulating a false discovery rate that can exceed 50%. With n = 10, power jumps to 70-90% and the FDR drops dramatically.

The p-value distribution under null

```
# A crucial diagnostic: under the null hypothesis, p-values
# should be uniformly distributed between 0 and 1
# This is one of the most important properties in statistics

# Our null gene p-values should look uniform
histogram(null_pvals, "plots/null_pvalue_histogram.svg")
  |> save_plot("plots/null_pvalue_histogram.svg")

# Check with quantiles
println("Null p-value quantiles (should be ~0.25, 0.5, 0.75):")
println("  25th percentile: " + to_string(round(quantile(null_pvals, 0.25),
3)))
println("  50th percentile: " + to_string(round(quantile(null_pvals, 0.50),
3)))
println("  75th percentile: " + to_string(round(quantile(null_pvals, 0.75),
3)))

# A QQ-plot against uniform distribution confirms this
qq_plot(null_pvals, "plots/null_pvalue_qq.svg")
  |> save_plot("plots/null_pvalue_qq.svg")
```

When the p-value distribution is NOT uniform

```
# Mix null and DE p-values — this is what real data looks like
let all_pvals = concat(null_pvals, de_pvals)

histogram(all_pvals, "plots/mixed_pvalue_histogram.svg")
  |> save_plot("plots/mixed_pvalue_histogram.svg")

# The histogram should show:
# - A spike near 0 (from truly DE genes)
# - A flat plateau from 0.1 to 1.0 (from null genes)
#
# The height of the plateau estimates the proportion of null genes (pi0)
# The area of the spike above the plateau estimates the number of true
positives
#
# If the histogram shows strange shapes (dips, bumps, U-shapes),
# something is wrong with the data or the test assumptions
```

Interpretation

What have we learned from this analysis? Several things that every biologist working with high-throughput data should internalize:

1. p < 0.05 means less than you think.

A p-value of 0.03 for a single gene means there is a 3% chance of seeing this extreme a result if the gene is truly not differentially expressed. But when you test 20,000 genes, you expect 600 false positives at $p < 0.05$. The p-value does not account for multiple testing — you must apply FDR correction.

2. Effect size is often more important than significance.

A gene with log2FC = 0.1 and p = 0.001 has a statistically detectable but biologically trivial change. A gene with log2FC = 3.0 and p = 0.08 has a massive, potentially important change that your small study lacked power to confirm. Always report effect sizes alongside p-values.

3. n = 3 is not enough for discovery.

With 3 samples per group, you have approximately 20-30% power to detect even large effects (Cohen's d = 1.5). This means you miss 70-80% of real biology. The genes you do detect have inflated effect size estimates (winner's curse).

4. Variance estimation is the bottleneck.

The t-test requires estimating variance within each group. With n = 3, you have only 2 degrees of freedom for variance estimation. This is why methods like DESeq2 and edgeR share information across genes to stabilize variance estimates — a technique called empirical Bayes shrinkage.

5. The p-value distribution is diagnostic.

If your p-value histogram under the null is not flat, your assumptions are violated. Common causes: batch effects, sample contamination, wrong distributional assumption, or confounding variables.

What the results do NOT mean

It is equally important to be clear about limitations:

- Finding 200 significant genes does not mean there are only 200 DE genes. Many more may exist but fall below the detection threshold.
- A non-significant result (p = 0.3) does not mean the gene is not differentially expressed. It means you do not have enough evidence to conclude that it is, given your sample size.
- Genes with the smallest p-values are not necessarily the most biologically important. They might simply have the lowest variance or the most consistent expression.

Decision

Given an RNA-seq experiment with n = 3 per group, here is a principled workflow:

```
# Step 1: Quality control and exploration (always)
# - Check distributions, correlations, outliers
# - PCA to visualize sample grouping

# Step 2: Use methods designed for small samples
# DESeq2/edgeR (external tools) share variance information
# across genes. In BioLang, we can approximate this concept:

# Compute a global variance estimate across all genes
let all_gene_vars = map(range(0, min(len(expr), 500)), |i| {
  let row = slice(expr, i, i + 1)
  let all_vals = map(sample_cols, |c| {
    col_values(row, c) |> take(1) |> to_float()
  })
  let log_vals = map(all_vals, |x| log2(x + 1.0))
  variance(log_vals)
})

let global_var = median(all_gene_vars)
println("Global median variance estimate: " + to_string(round(global_var,
4)))
println("This can be used to stabilize per-gene variance estimates")

# Step 3: Apply strict thresholds
# - Use FDR < 0.05 (not raw p < 0.05)
# - Require |log2FC| > 1 (2-fold change minimum)
# - Consider these results as HYPOTHESIS-GENERATING, not confirmatory

# Step 4: Validate key findings
# - qPCR validation of top candidates
# - Check against known biology (pathway enrichment)
# - Plan a larger follow-up experiment for confirmation
```

Rules of thumb for sample size in genomics

Study Type	Minimum n per group	Ideal n per group	Notes
RNA-seq (discovery)	3	6-12	Use empirical Bayes methods
RNA-seq (confirmation)	6	12-20	Pre-registered analysis
ATAC-seq	2	3-5	Lower variance than RNA-seq
ChIP-seq	2	3-5	Need input controls

Study Type	Minimum n per group	Ideal n per group	Notes
GWAS (common variants)	1000	10,000+	Effect sizes are tiny
Single-cell RNA-seq	3 donors	6+ donors	Cells are not replicates!

Pitfalls

Pitfall 1: Treating technical replicates as biological replicates

```
# WRONG: sequencing the same RNA extraction twice
# and calling it n=2
#
# Technical replicates measure machine variability
# Biological replicates measure biological variability
# Only biological replicates count for sample size
#
# If your "3 replicates" are 3 sequencing runs from the same
# mouse, you have n=1, not n=3

# The variance you care about is BIOLOGICAL variance
# Technical variance is typically much smaller
# Using technical replicates inflates your effective sample size
# and leads to false discoveries
```

Pitfall 2: Cherry-picking genes

```
# It is tempting to focus on the most significant results
# and build a story around them. This is p-hacking.
#
# Example of what NOT to do:
# 1. Run 20,000 tests
# 2. Find 200 "significant" genes
# 3. Select the 10 that fit your hypothesis
# 4. Report only those 10 with impressive p-values
#
# This is scientifically fraudulent even if unintentional.
# The correction for multiple testing must account for ALL
# tests performed, not just the ones you report.
```

Pitfall 3: Confusing correlation with causation in expression data

```
# Finding that Gene A and Gene B are co-expressed
# (high correlation) does not mean:
# - A regulates B
# - B regulates A
# - They are in the same pathway
#
# It means their expression levels vary together across samples.
# This could be because:
# - They share a common regulator
# - They are in the same cell type (which varies across samples)
# - They are both affected by a batch effect
# - Pure coincidence (with enough genes, some will correlate)

# Demonstrate with random data
let random1 = rnorm(100, 0.0, 1.0)
let random2 = rnorm(100, 0.0, 1.0)
let r_random = cor(random1, random2)
println("Correlation between random vectors: " + to_string(round(r_random,
4)))
# Even this will sometimes be "significant" with enough tests
```

Pitfall 4: Ignoring batch effects

```
# Samples processed on different days, by different technicians,
# or on different sequencing lanes can have systematic differences
# that have nothing to do with biology.
#
# If all your tumor samples were processed in batch 1 and all
# normal samples in batch 2, EVERY gene will appear DE —
# but you are detecting the batch effect, not cancer biology.
#
# Solution: randomize sample processing across batches
# Or at minimum, include batch as a covariate in your model.
#
# PCA is the quickest way to check: if PC1 separates batches
# rather than biological groups, you have a problem.
```

Pitfall 5: The log-fold-change trap for lowly expressed genes

```
# Genes with very low expression can show enormous fold changes
# that are meaningless

# Example: gene has FPKM values of [0.01, 0.02, 0.01] in normal
# and [0.1, 0.2, 0.15] in tumor
let low_normal = [0.01, 0.02, 0.01]
let low_tumor = [0.1, 0.2, 0.15]

let lfc = mean(map(low_tumor, |x| log2(x + 1.0))) - mean(map(low_normal, |x|
log2(x + 1.0)))
println("Log2 fold change: " + to_string(round(lfc, 2)))
# This shows a large fold change, but the absolute expression
# is so low that it is likely noise

# Solution: filter out lowly expressed genes before testing
# A common threshold: keep genes with FPKM > 1 in at least
# half the samples in at least one group
```

Pitfall 6: Not checking test assumptions

```
# The t-test assumes:
# 1. Independent observations (violated by batch effects, family relatedness)
# 2. Normal distribution within each group (approximately OK for log-
expression)
# 3. Equal variances (use Welch's t-test if not)
#
# Check normality with a QQ-plot or Shapiro-Wilk test
# Check equal variance by comparing group standard deviations

# For a specific gene
let group1_log = [8.2, 7.9, 8.5]
let group2_log = [10.1, 9.8, 10.4]

println("Group 1 SD: " + to_string(round(stdev(group1_log), 3)))
println("Group 2 SD: " + to_string(round(stdev(group2_log), 3)))

# If SDs differ by more than 2-fold, equal variance assumption
# is questionable. Use Welch's t-test (default in most software).
# For non-normal data, consider the Wilcoxon rank-sum test.
let wilcox_result = wilcoxon(group1_log, group2_log)
println("Wilcoxon p-value: " + to_string(wilcox_result.p_value))
```

Where Next?

This chapter established the foundations: what p-values mean, why effect size matters, and why sample size is the single most important determinant of a study's reliability.

In **Chapter 2: Exploratory Data Analysis**, we will learn to look at data systematically before any hypothesis testing — understanding distributions, detecting outliers, and using visualization to build intuition about what the data can and cannot tell us.

In **Chapter 3: Distributions in Genomics**, we will study the specific probability distributions that appear repeatedly in genomic data — normal, Poisson, negative binomial, binomial — and learn how to check whether your data follow the distributions your statistical methods assume.

In **Chapter 4: Experimental Design and Power**, we will use the variance estimates from real data to calculate exactly how many samples you need for a well-powered study, and learn why getting this right before the experiment is far cheaper than discovering it afterwards.

Summary of key principles

Principle	Rule of Thumb
p-value threshold	Use FDR-adjusted $p < 0.05$, never raw $p < 0.05$ for genomics
Effect size	Require at minimum
Sample size	$n = 3$ is for pilots only; $n >= 6$ for discovery
Multiple testing	Always correct; BH/FDR is standard in genomics
Variance estimation	With small n, use methods that share information across genes
Replication	Biological replicates only; technical replicates do not count
Pre-registration	Decide your analysis plan before looking at the data
Reporting	Always report effect sizes, confidence intervals, and sample sizes

Exercises

```
# Exercise 1: Download the GSE183947 data and reproduce the analysis
# above. How many genes pass both FDR < 0.05 and |log2FC| > 1?

# Exercise 2: Repeat the simulation with n = 5, 7, 10, 15, 20.
# Plot power vs sample size for d = 0.5, 1.0, 1.5, 2.0.
# At what sample size does power reach 80% for d = 1.0?

# Exercise 3: Take the p-values from the real data and plot
# the p-value histogram. Does it look like a mixture of
# uniform + spike near zero? Estimate pi0 (the proportion
# of truly null genes) from the histogram.

# Exercise 4: Compute the Spearman correlation between
# -log10(p-value) and |log2FC|. Is it positive?
# What does this tell you about the relationship between
# significance and effect size?

# Exercise 5: Why is the median a better summary statistic
# than the mean for FPKM values? Compute both for 100 genes
# and compare.
```

Further reading

- Ioannidis (2005), “Why Most Published Research Findings Are False” — the paper that launched the reproducibility crisis discussion
- Leek & Storey (2007), “Capturing Heterogeneity in Gene Expression Studies by Surrogate Variable Analysis” — on batch effects
- Love, Huber & Anders (2014), “Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2” — the standard DE method
- Conesa et al. (2016), “A survey of best practices for RNA-seq data analysis” — comprehensive review of the RNA-seq analysis workflow

Exploratory Data Analysis

The Problem

A new postdoc walks into the lab meeting with a fresh dataset. The PI loaded RNA-seq expression data from the GTEx consortium — 56,200 genes across 54 human tissues — and asks the simplest possible question: “What does it look like?”

Not “which genes are significant.” Not “confirm my hypothesis.” Just: what does it look like?

This is exploratory data analysis (EDA), and it is the most undervalued step in all of bioinformatics. In a field obsessed with p-values and publication, EDA is the thing you do before the thing you want to do. It is looking at the shape of the data, finding patterns you did not expect, and discovering problems that would invalidate your downstream analysis.

John Tukey, who coined the term, put it this way: “Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise.”

This chapter teaches you how to look at biological data systematically. We will use the GTEx gene expression dataset — one of the largest and most valuable public expression resources — and build an EDA toolkit that you will use on every dataset for the rest of your career.

Why It Matters

Here are things that EDA catches that no statistical test will:

1. **Mislabeled samples.** A liver sample that clusters with brain samples on PCA is probably mislabeled. No differential expression analysis will flag this — it will simply produce wrong results.

2. **Batch effects.** If your samples separate by processing date rather than biological condition on PCA, your entire experiment is confounded. Testing will give you thousands of “significant” results that reflect the batch, not the biology.

3. **Outliers.** A single outlier sample with 10x higher expression in all genes (perhaps due to a library preparation failure) will distort every mean, every variance estimate, and every t-test.

4. **Wrong distributional assumptions.** If your data are bimodal, zero- inflated, or heteroscedastic, many standard tests will be invalid. You need to see this before choosing your method.

5. **Unexpected structure.** Your data might contain subgroups you did not know about. EDA reveals these before they become confounders.

Skipping EDA is like performing surgery without imaging. You might get lucky. You might not.

The Data

We will use the GTEx (Genotype-Tissue Expression) project's median gene expression data. This file contains the median TPM (Transcripts Per Million) for each gene across 54 human tissues, calculated from thousands of donors.

```
# Download GTEx v8 median gene-level TPM
# This is a GCT-format file (Gene Cluster Text)
let url = "https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-
seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
let raw_file = download(url, "data/gtex_median_tpm.gct.gz")

# GCT format has 2 header lines, then a tab-delimited matrix
# Line 1: #1.2
# Line 2: number_of_genes  number_of_samples
# Line 3: header row (Name, Description, tissue1, tissue2, ...)
# Line 4+: data rows

# Read as TSV, skipping the first 2 lines
let gtex = read_tsv(raw_file)

println("GTEx dataset dimensions:")
println("  Genes: " + to_string(len(gtex)))
println("  Columns: " + to_string(ncols(gtex)))

# Examine column names — these are the tissue types
let all_cols = columns(gtex)
println("First 10 columns:")
each(slice(all_cols, 0, 10), |c| println("  " + c))
```

```
# The first two columns are gene identifiers
# Name = Ensembl gene ID (e.g., ENSG00000000003.15)
# Description = gene symbol (e.g., TSPAN6)
# Remaining columns are tissue names

let id_cols = ["Name", "Description"]
let tissue_cols = filter(all_cols, |c| !contains(id_cols, c))
println("Number of tissues: " + to_string(len(tissue_cols)))
println("Tissues:")
each(tissue_cols, |t| println("  " + t))
```

Exploration

Step 1: How are expression values distributed?

This is always the first question. The shape of the distribution determines which statistics are meaningful and which are misleading.

```
# Get TPM values for a single tissue
let brain_cortex = col_values(gtex, "Brain - Cortex")

# Basic summary statistics
println("Brain Cortex TPM - Raw values:")
println("  n genes: " + to_string(len(brain_cortex)))
println("  Mean: " + to_string(round(mean(brain_cortex), 2)))
println("  Median: " + to_string(round(median(brain_cortex), 2)))
println("  SD: " + to_string(round(stdev(brain_cortex), 2)))
println("  Min: " + to_string(round(min(brain_cortex), 4)))
println("  Max: " + to_string(round(max(brain_cortex), 2)))
println("  Skewness indicator (mean/median): " +
to_string(round(mean(brain_cortex) / median(brain_cortex), 2)))

# Quantiles
println("  Q25: " + to_string(round(quantile(brain_cortex, 0.25), 3)))
println("  Q75: " + to_string(round(quantile(brain_cortex, 0.75), 3)))
println("  Q95: " + to_string(round(quantile(brain_cortex, 0.95), 2)))
println("  Q99: " + to_string(round(quantile(brain_cortex, 0.99), 2)))
```

The mean is much larger than the median. This is the hallmark of a right- skewed distribution, and it is universal in gene expression data. A handful of very highly expressed genes (think ribosomal proteins with TPM > 10,000) pull the mean upward while most genes have modest or zero expression.

```
# How many genes have zero or near-zero expression?
let zero_count = len(filter(brain_cortex, |x| x == 0.0))
let near_zero = len(filter(brain_cortex, |x| x < 0.1))
let low_count = len(filter(brain_cortex, |x| x < 1.0))
let total = len(brain_cortex)

println("Expression level breakdown:")
println("  Zero (TPM = 0): " + to_string(zero_count) + " (" +
to_string(round(100.0 * zero_count / total, 1)) + "%)")
println("  Near-zero (TPM < 0.1): " + to_string(near_zero) + " (" +
to_string(round(100.0 * near_zero / total, 1)) + "%)")
println("  Low (TPM < 1): " + to_string(low_count) + " (" +
to_string(round(100.0 * low_count / total, 1)) + "%)")
println("  Expressed (TPM >= 1): " + to_string(total - low_count) + " (" +
to_string(round(100.0 * (total - low_count) / total, 1)) + "%)")

# Visualize the raw distribution
histogram(brain_cortex, "plots/brain_cortex_raw_tpm.svg")
  |> save_plot("plots/brain_cortex_raw_tpm.svg")
```

Step 2: The log transformation

```
# Log-transform to handle the skew
# Add pseudocount of 1 to handle zeros
let brain_log = map(brain_cortex, |x| log2(x + 1.0))

println("Brain Cortex log2(TPM+1):")
println("  Mean: " + to_string(round(mean(brain_log), 3)))
println("  Median: " + to_string(round(median(brain_log), 3)))
println("  SD: " + to_string(round(stdev(brain_log), 3)))
println("  Min: " + to_string(round(min(brain_log), 3)))
println("  Max: " + to_string(round(max(brain_log), 3)))

# Now the distribution is much more symmetric
histogram(brain_log, "plots/brain_cortex_log2_tpm.svg")
  |> save_plot("plots/brain_cortex_log2_tpm.svg")

# But notice: there is still a spike at zero (from genes with TPM=0)
# This creates a zero-inflated distribution
# For many analyses, you want to filter these out first

let brain_log_expressed = filter(brain_log, |x| x > 0.0)
histogram(brain_log_expressed, "plots/brain_cortex_log2_expressed.svg")
  |> save_plot("plots/brain_cortex_log2_expressed.svg")

println("After removing zeros:")
println("  n expressed genes: " + to_string(len(brain_log_expressed)))
println("  Mean: " + to_string(round(mean(brain_log_expressed), 3)))
println("  Median: " + to_string(round(median(brain_log_expressed), 3)))
```

Step 3: Comparing distributions across tissues

```
# How does the expression distribution differ across tissues?
# Compare brain, liver, heart, and blood

let tissues_to_compare = ["Brain - Cortex", "Liver", "Heart - Left
Ventricle", "Whole Blood"]

let tissue_summaries = map(tissues_to_compare, |tissue| {
  let vals = col_values(gtex, tissue)
  let log_vals = map(vals, |x| log2(x + 1.0))
  let expressed = filter(vals, |x| x > 0.0)

  record(
    tissue: tissue,
    n_expressed: len(expressed),
    pct_expressed: round(100.0 * len(expressed) / len(vals), 1),
    mean_tpm: round(mean(vals), 2),
    median_tpm: round(median(vals), 4),
    mean_log: round(mean(log_vals), 3),
    median_log: round(median(log_vals), 3),
    sd_log: round(stdev(log_vals), 3),
    max_tpm: round(max(vals), 1)
  )
})

let summary_table = table(tissue_summaries)
println("Cross-tissue comparison:")
println(summary_table)
```

```
# Boxplots are ideal for comparing distributions
# Create a combined dataset for visualization
let boxplot_data = map(tissues_to_compare, |tissue| {
  let vals = col_values(gtex, tissue)
  let log_vals = map(vals, |x| log2(x + 1.0))
  # Filter to expressed genes for cleaner visualization
  filter(log_vals, |x| x > 0.0)
})

boxplot(boxplot_data, "plots/tissue_expression_boxplots.svg")
  |> save_plot("plots/tissue_expression_boxplots.svg")

# Density plots show the shape more clearly than boxplots
each(enumerate(tissues_to_compare), |pair| {
  let i = pair[0]
  let tissue = pair[1]
  let vals = col_values(gtex, tissue)
  let log_vals = map(vals, |x| log2(x + 1.0))
  let expressed = filter(log_vals, |x| x > 0.0)
  histogram(expressed, "plots/density_" + to_string(i) + ".svg")
    |> save_plot("plots/density_" + to_string(i) + ".svg")
})
```

Step 4: Tissue-specific genes

```
# Which genes are highly expressed in one tissue but not others?
# This is a fundamental question in biology

# Define "tissue-specific" as: expressed > 10 TPM in the tissue
# AND < 1 TPM in at least 90% of other tissues

let n_tissues = len(tissue_cols)

# For each gene, count how many tissues it is expressed in
let expression_breadth = map(range(0, min(len(gtex), 5000)), |i| {
  let row = slice(gtex, i, i + 1)
  let gene_name = col_values(row, "Description") |> take(1)

  let tissue_vals = map(tissue_cols, |t| {
    col_values(row, t) |> take(1) |> to_float()
  })

  let n_expressed = len(filter(tissue_vals, |x| x >= 1.0))
  let max_expr = max(tissue_vals)
  let mean_expr = mean(tissue_vals)

  record(
    gene: gene_name,
    n_tissues_expressed: n_expressed,
    max_tpm: round(max_expr, 2),
    mean_tpm: round(mean_expr, 2),
    specificity: if mean_expr > 0.0 then round(max_expr / mean_expr, 2) else
0.0
  )
})

let breadth_table = table(expression_breadth)

# Distribution of expression breadth
let breadth_vals = map(expression_breadth, |r| r.n_tissues_expressed)
println("Expression breadth distribution:")
println("  Genes in 0 tissues: " + to_string(len(filter(breadth_vals, |x| x
== 0))))
println("  Genes in 1-5 tissues: " + to_string(len(filter(breadth_vals, |x| x
>= 1 && x <= 5))))
println("  Genes in 6-20 tissues: " + to_string(len(filter(breadth_vals, |x|
x >= 6 && x <= 20))))
println("  Genes in 21-40 tissues: " + to_string(len(filter(breadth_vals, |x|
x >= 21 && x <= 40))))
println("  Genes in 41+ tissues: " + to_string(len(filter(breadth_vals, |x| x
> 40))))

histogram(breadth_vals, "plots/expression_breadth.svg")
  |> save_plot("plots/expression_breadth.svg")
```

```
# Find the most tissue-specific genes
let specific_genes = filter(expression_breadth, |r| {
  r.n_tissues_expressed <= 3 && r.max_tpm > 10.0
})
let specific_sorted = sort(specific_genes, |a, b| b.specificity -
a.specificity)

println("Top tissue-specific genes (expressed in <= 3 tissues, max TPM >
10):")
each(take(specific_sorted, 20), |g| {
  println("  " + to_string(g.gene) + ": " + to_string(g.n_tissues_expressed)
+ " tissues, max=" + to_string(g.max_tpm) + " TPM, specificity=" +
to_string(g.specificity))
})
```

Step 5: Correlation structure

Understanding which tissues have similar expression profiles is one of the most informative things you can learn from expression data.

```
# Compute pairwise correlations between tissues
# Use a subset of well-expressed genes for speed
let expressed_genes = filter_by(gtex, |row| {
  let vals = map(tissue_cols, |t| col_values(row, t) |> take(1) |>
to_float())
  mean(vals) > 1.0
})

println("Genes with mean TPM > 1 across tissues: " +
to_string(len(expressed_genes)))

# Extract expression matrix for correlation analysis
# Compare a few key tissues
let compare_tissues = [
  "Brain - Cortex",
  "Brain - Cerebellum",
  "Brain - Hippocampus",
  "Liver",
  "Heart - Left Ventricle",
  "Heart - Atrial Appendage",
  "Lung",
  "Kidney - Cortex",
  "Whole Blood",
  "Muscle - Skeletal"
]

# Build tissue expression vectors (log-transformed)
let tissue_vectors = map(compare_tissues, |tissue| {
  let raw = col_values(expressed_genes, tissue)
  map(raw, |x| log2(x + 1.0))
})

# Pairwise Pearson correlations
println("Tissue-tissue Pearson correlations:")
println("")
each(enumerate(compare_tissues), |pair1| {
  let i = pair1[0]
  let t1 = pair1[1]
  each(enumerate(compare_tissues), |pair2| {
    let j = pair2[0]
    let t2 = pair2[1]
    if j > i then {
      let r = cor(tissue_vectors[i], tissue_vectors[j])
      println("  " + t1 + " vs " + t2 + ": r=" + to_string(round(r, 4)))
    }
  })
})
```

```
# Build a correlation matrix for visualization
let cor_mat = cor_matrix(tissue_vectors)

println("Correlation matrix:")
println(cor_mat)

# Visualize as heatmap
heatmap(cor_mat, "plots/tissue_correlation_heatmap.svg")
  |> save_plot("plots/tissue_correlation_heatmap.svg")

# Key observation: brain tissues cluster together
# Heart tissues cluster together
# This makes biological sense — tissues with similar cell types
# have similar transcriptomes
```

Step 6: PCA — the most important EDA technique in genomics

Principal Component Analysis reduces the dimensionality of the data while preserving as much variance as possible. In tissue expression data, it reveals which tissues are most similar and which are most different.

```
# Build a matrix of tissue expression profiles
# Each row is a tissue, each column is a gene
# (Transpose of the usual gene x sample matrix)
let expr_matrix = matrix(tissue_vectors)

# PCA on the tissue profiles
let pca_result = pca(expr_matrix)

println("PCA on tissue expression profiles:")
println("Variance explained by each PC:")
let total_var = sum(pca_result.eigenvalues)
each(enumerate(pca_result.eigenvalues), |pair| {
  let i = pair[0]
  let ev = pair[1]
  let pct = round(100.0 * ev / total_var, 1)
  if i < 10 then {
    println("  PC" + to_string(i + 1) + ": " + to_string(pct) + "%")
  }
})
```

```
# Visualize the first two principal components
let pc1 = col_values(pca_result.scores, 0)
let pc2 = col_values(pca_result.scores, 1)

# Create PCA plot
pca_plot(pca_result, "plots/tissue_pca.svg")
  |> save_plot("plots/tissue_pca.svg")

# Also create a manual scatter plot for more control
scatter(pc1, pc2, "plots/tissue_pca_scatter.svg")
  |> save_plot("plots/tissue_pca_scatter.svg")

# Label each point
println("PCA coordinates:")
each(enumerate(compare_tissues), |pair| {
  let i = pair[0]
  let tissue = pair[1]
  println("  " + tissue + ": PC1=" + to_string(round(pc1[i], 2)) + ", PC2=" +
to_string(round(pc2[i], 2)))
})
```

Interpreting the PCA plot:

- **PC1** typically separates the most transcriptionally distinct tissues. Brain tissues often separate from everything else because the brain transcriptome is uniquely enriched in neuron-specific genes.
- **PC2** captures the next largest source of variation. Often this separates other tissue groups (e.g., blood from solid tissues).
- Tissues that cluster together share similar expression programs. The three brain regions should cluster tightly. The two heart tissues should be near each other.
- An outlier tissue that does not cluster with its expected group warrants investigation.

Step 7: Outlier detection

```
# Outlier detection in high-dimensional data
# Method 1: Mahalanobis distance (simplified version)
# A sample far from the center of the PCA cloud is suspicious

let pc1_mean = mean(pc1)
let pc2_mean = mean(pc2)
let pc1_sd = stdev(pc1)
let pc2_sd = stdev(pc2)

let distances = map(enumerate(compare_tissues), |pair| {
  let i = pair[0]
  let tissue = pair[1]
  let d1 = (pc1[i] - pc1_mean) / pc1_sd
  let d2 = (pc2[i] - pc2_mean) / pc2_sd
  let d = sqrt(d1 * d1 + d2 * d2)
  record(tissue: tissue, pca_distance: round(d, 3))
})

let dist_sorted = sort(distances, |a, b| b.pca_distance - a.pca_distance)
println("Tissues ranked by PCA distance from center:")
each(dist_sorted, |d| println("  " + d.tissue + ": " +
to_string(d.pca_distance)))
```

```
# Method 2: Expression profile distance
# How different is each tissue from the "average tissue"?

let mean_profile = map(range(0, len(tissue_vectors[0])), |gene_i| {
  let gene_vals = map(tissue_vectors, |tv| tv[gene_i])
  mean(gene_vals)
})

let profile_distances = map(enumerate(compare_tissues), |pair| {
  let i = pair[0]
  let tissue = pair[1]
  let tv = tissue_vectors[i]
  let diffs = map(range(0, len(tv)), |j| {
    pow(tv[j] - mean_profile[j], 2.0)
  })
  let rmsd = sqrt(mean(diffs))
  record(tissue: tissue, rmsd: round(rmsd, 4))
})

let profile_sorted = sort(profile_distances, |a, b| b.rmsd - a.rmsd)
println("Tissues ranked by expression profile distance from mean:")
each(profile_sorted, |d| println("  " + d.tissue + ": RMSD=" +
to_string(d.rmsd)))

# A tissue with RMSD > 2x the median RMSD warrants closer inspection
let median_rmsd = median(map(profile_distances, |d| d.rmsd))
println("Median RMSD: " + to_string(round(median_rmsd, 4)))
println("Outlier threshold (2x median): " + to_string(round(2.0 *
median_rmsd, 4)))
```

Step 8: Housekeeping genes vs tissue-specific genes

```
# Housekeeping genes: expressed in all tissues at moderate-high levels
# These have low coefficient of variation across tissues

let gene_variability = map(range(0, min(len(gtex), 5000)), |i| {
  let row = slice(gtex, i, i + 1)
  let gene_name = col_values(row, "Description") |> take(1)

  let tissue_vals = map(tissue_cols, |t| {
    col_values(row, t) |> take(1) |> to_float()
  })

  let m = mean(tissue_vals)
  let s = stdev(tissue_vals)
  let cv = if m > 0.0 then s / m else 0.0

  record(
    gene: gene_name,
    mean_tpm: round(m, 2),
    sd_tpm: round(s, 2),
    cv: round(cv, 3),
    min_tpm: round(min(tissue_vals), 2),
    max_tpm: round(max(tissue_vals), 2)
  )
})

# Filter for expressed genes (mean > 5 TPM)
let expressed_variable = filter(gene_variability, |g| g.mean_tpm > 5.0)

# Most stable (housekeeping) genes
let housekeeping = sort(expressed_variable, |a, b| a.cv - b.cv)
println("Top 20 most stable (housekeeping) genes:")
each(take(housekeeping, 20), |g| {
  println("  " + to_string(g.gene) + ": mean=" + to_string(g.mean_tpm) + ",
CV=" + to_string(g.cv))
})

# Most variable genes
let most_variable = sort(expressed_variable, |a, b| b.cv - a.cv)
println("")
println("Top 20 most variable genes:")
each(take(most_variable, 20), |g| {
  println("  " + to_string(g.gene) + ": mean=" + to_string(g.mean_tpm) + ",
CV=" + to_string(g.cv))
})
```

Step 9: Expression scale awareness

```
# Understanding what TPM values actually mean
# TPM = transcripts per million
# It represents the proportion of the transcriptome occupied by each gene

# The key property: TPM sums to 1 million per sample (by definition)
let total_tpm = col_values(gtex, "Brain - Cortex") |> sum()
println("Total TPM in Brain Cortex: " + to_string(round(total_tpm, 0)))
# This should be approximately 1,000,000

# What fraction of the transcriptome do the top genes consume?
let brain_sorted = sort(brain_cortex, |a, b| b - a)
let top10_sum = sum(slice(brain_sorted, 0, 10))
let top100_sum = sum(slice(brain_sorted, 0, 100))
let top1000_sum = sum(slice(brain_sorted, 0, 1000))

println("Transcriptome concentration:")
println("  Top 10 genes: " + to_string(round(100.0 * top10_sum / total_tpm,
1)) + "% of TPM")
println("  Top 100 genes: " + to_string(round(100.0 * top100_sum / total_tpm,
1)) + "% of TPM")
println("  Top 1000 genes: " + to_string(round(100.0 * top1000_sum /
total_tpm, 1)) + "% of TPM")

# A handful of genes dominate the transcriptome
# This has implications for differential expression analysis:
# changes in highly expressed genes can cause apparent changes
# in lowly expressed genes (compositional bias)
```

Step 10: QQ-plots for distribution checking

```
# QQ-plots compare your data distribution against a theoretical distribution
# If the data follow the theoretical distribution, points fall on the
diagonal

# Check if log-transformed expression is approximately normal
let brain_log_expr = filter(brain_log, |x| x > 0.0)
qq_plot(brain_log_expr, "plots/brain_log_qqplot.svg")
  |> save_plot("plots/brain_log_qqplot.svg")

# A straight line = data are approximately normal
# Deviations at the tails are common and usually acceptable
# S-shaped curvature = different distribution family

# Check normality more formally with summary statistics
let brain_skewness_indicator = mean(brain_log_expr) - median(brain_log_expr)
println("Mean - Median (skew indicator): " +
to_string(round(brain_skewness_indicator, 4)))
# Small value = approximately symmetric = consistent with normal
```

Step 11: When to use different plot types

```
# HISTOGRAMS: show the full shape of a single distribution
# Best for: understanding one variable in detail
# Trap: bin width matters — too few bins hide structure, too many create
noise
histogram(brain_log_expr, "plots/eda_histogram.svg")
  |> save_plot("plots/eda_histogram.svg")

# DENSITY PLOTS: smoothed version of histograms
# Best for: overlaying multiple distributions
# Trap: smoothing can hide modes or create artificial ones
histogram(brain_log_expr, "plots/eda_density.svg")
  |> save_plot("plots/eda_density.svg")

# BOXPLOTS: compare distributions side by side
# Best for: comparing groups (tissues, conditions)
# Trap: hides multimodality. A bimodal distribution looks like
# a distribution with outliers in a boxplot.
let multi_data = map(tissues_to_compare, |t| {
  let vals = col_values(gtex, t)
  map(vals, |x| log2(x + 1.0))
})
boxplot(multi_data, "plots/eda_boxplot.svg")
  |> save_plot("plots/eda_boxplot.svg")

# VIOLIN PLOTS: boxplot + density combined
# Best for: same as boxplots but reveals shape
# Use instead of boxplots when distribution shape matters
boxplot(multi_data, "plots/eda_violin.svg")
  |> save_plot("plots/eda_violin.svg")

# SCATTER PLOTS: relationship between two variables
# Best for: correlation, sample comparison
# Trap: overplotting with thousands of points — use density scatter
let liver_log = map(col_values(gtex, "Liver"), |x| log2(x + 1.0))
scatter(brain_log, liver_log, "plots/brain_vs_liver.svg")
  |> save_plot("plots/brain_vs_liver.svg")

# HEATMAPS: matrix visualization (correlations, expression patterns)
# Best for: showing patterns in matrices
# Trap: clustering can create false patterns if not done carefully
```

The Analysis

Now that we have explored the data visually, let us formalize our observations with systematic analyses.

Systematic tissue clustering

```
# Hierarchical clustering using correlation distance
# 1 - correlation gives a distance metric: similar tissues have small
distance

let all_tissue_vectors = map(tissue_cols, |tissue| {
  let raw = col_values(expressed_genes, tissue)
  map(raw, |x| log2(x + 1.0))
})

# Compute the full correlation matrix
let full_cor = cor_matrix(all_tissue_vectors)

# Convert to distance matrix (1 - r)
let n = len(tissue_cols)
let dist_vals = map(range(0, n), |i| {
  map(range(0, n), |j| {
    round(1.0 - full_cor[i][j], 4)
  })
})

let full_dist = dist_matrix(dist_vals)

println("Distance matrix (1 - correlation) for selected tissues:")
println(full_dist)

heatmap(full_cor, "plots/full_tissue_correlation.svg")
  |> save_plot("plots/full_tissue_correlation.svg")
```

Gene filtering for downstream analysis

```
# Before any statistical testing, filter genes appropriately
# This is a critical EDA step that affects all downstream results

# Common filter: require TPM >= 1 in at least 20% of tissues
let min_tissues = ceil(len(tissue_cols) * 0.2)
println("Requiring expression in at least " + to_string(min_tissues) + "
tissues")

let filtered_genes = filter_by(gtex, |row| {
  let vals = map(tissue_cols, |t| col_values(row, t) |> take(1) |>
to_float())
  let n_expressed = len(filter(vals, |x| x >= 1.0))
  n_expressed >= min_tissues
})

println("Genes before filtering: " + to_string(len(gtex)))
println("Genes after filtering: " + to_string(len(filtered_genes)))
println("Removed: " + to_string(len(gtex) - len(filtered_genes)) + " lowly
expressed genes")
```

Normalization check

```
# TPM is already normalized within samples (sums to 1M)
# But let us verify and understand the implications

let tpm_sums = map(tissue_cols, |t| {
  let vals = col_values(gtex, t)
  sum(vals)
})

println("TPM sum statistics across tissues:")
println("  Mean: " + to_string(round(mean(tpm_sums), 0)))
println("  Min: " + to_string(round(min(tpm_sums), 0)))
println("  Max: " + to_string(round(max(tpm_sums), 0)))
println("  SD: " + to_string(round(stdev(tpm_sums), 0)))

# These should all be approximately 1,000,000
# Small deviations are due to rounding and gene filtering
```

Detecting bimodal distributions

```
# Some genes have bimodal expression across tissues
# (highly expressed in some tissues, off in others)
# This indicates tissue-specific regulation

# A simple bimodality detector: ratio of variance to mean
# (on log scale) combined with the distribution shape

let bimodal_candidates = map(range(0, min(len(filtered_genes), 3000)), |i| {
  let row = slice(filtered_genes, i, i + 1)
  let gene = col_values(row, "Description") |> take(1)

  let vals = map(tissue_cols, |t| {
    col_values(row, t) |> take(1) |> to_float()
  })
  let log_vals = map(vals, |x| log2(x + 1.0))

  let m = mean(log_vals)
  let s = stdev(log_vals)
  let med = median(log_vals)

  # Bimodal indicator: high variance and mean != median
  let bimodal_score = s * abs(m - med)

  record(
    gene: gene,
    bimodal_score: round(bimodal_score, 3),
    mean_log: round(m, 2),
    sd_log: round(s, 2),
    median_log: round(med, 2)
  )
})

let bimodal_sorted = sort(bimodal_candidates, |a, b| b.bimodal_score -
a.bimodal_score)
println("Top candidate bimodal genes:")
each(take(bimodal_sorted, 15), |g| {
  println("  " + to_string(g.gene) + ": score=" + to_string(g.bimodal_score)
+ ", mean=" + to_string(g.mean_log) + ", sd=" + to_string(g.sd_log))
})
```

Summary statistics that matter

```
# The right summary statistic depends on the distribution shape:
#
# Symmetric (normal-ish):
#   Center: mean or median (both work)
#   Spread: standard deviation
#
# Right-skewed (raw expression):
#   Center: MEDIAN (mean is inflated by outliers)
#   Spread: IQR (interquartile range)
#
# Zero-inflated:
#   Report zero proportion separately
#   Summarize non-zero values with median
#
# Bimodal:
#   Neither mean nor median is meaningful
#   Report the two modes separately

# Demonstrate with brain cortex data
let iqr = quantile(brain_cortex, 0.75) - quantile(brain_cortex, 0.25)
println("Brain Cortex TPM - Appropriate summaries:")
println("  For raw TPM (skewed):")
println("    Median: " + to_string(round(median(brain_cortex), 4)))
println("    IQR: " + to_string(round(iqr, 4)))
println("    Mean: " + to_string(round(mean(brain_cortex), 2)) + "
(misleading!)")
println("")
println("  For log2(TPM+1) (symmetric):")
println("    Mean: " + to_string(round(mean(brain_log), 3)))
println("    SD: " + to_string(round(stdev(brain_log), 3)))
```

Sparklines for rapid overview

```
# Sparklines provide a compact visual summary
# Great for scanning many variables at once

println("Expression distribution sparklines by tissue:")
each(tissues_to_compare, |tissue| {
  let vals = col_values(gtex, tissue)
  let log_vals = map(vals, |x| log2(x + 1.0))
  let expressed = filter(log_vals, |x| x > 0.0)
  let spark = sparkline(expressed)
  println("  " + tissue + ": " + spark)
})
```

The EDA checklist

```
# Every new dataset should go through this checklist:
#
# 1. DIMENSIONS
#    - How many genes? How many samples?
#    - Any unexpected missing data?
#
# 2. DISTRIBUTIONS
#    - Raw data shape (usually right-skewed for expression)
#    - Log-transformed shape (should be ~normal for expression)
#    - Zero inflation (what fraction of genes are zero?)
#    - Outlier values (max value, 99th percentile)
#
# 3. SAMPLE QUALITY
#    - Per-sample summary statistics (mean, SD, zero fraction)
#    - Sample-sample correlation (within-group > 0.95?)
#    - PCA (samples cluster by biology, not by batch?)
#
# 4. GENE FILTERING
#    - Remove lowly expressed genes
#    - Remove genes with zero variance
#    - Document how many genes removed
#
# 5. NORMALIZATION
#    - What normalization was applied?
#    - Do library sizes differ dramatically?
#    - Are distributions comparable across samples?
#
# 6. STRUCTURE
#    - Expected groupings visible in PCA?
#    - Any unexpected subclusters?
#    - Correlation structure makes biological sense?
#
# 7. ANOMALIES
#    - Outlier samples?
#    - Batch effects visible?
#    - Any variables confounded with the factor of interest?
```

Interpretation

Exploratory data analysis of the GTEx dataset reveals several important features that are general to gene expression data:

Most genes are not expressed in any given tissue. Typically 40-60% of annotated genes have zero or near-zero expression. This reflects the fact that most genes are tissue-specific or lineage-specific. Keeping these in your analysis inflates the multiple testing burden and can distort normalization.

Expression distributions are universally right-skewed. A small number of genes consume a large fraction of the transcriptome. The top 100 genes can account for 30-50% of total TPM. This motivates log-transformation as a standard preprocessing step.

Tissues cluster by lineage. Brain regions cluster together, heart chambers cluster together, and so on. This is expected and reassuring — it means the data capture real biology. If your experimental samples do not cluster by expected groups, something is wrong.

Housekeeping genes have low variance. Genes like GAPDH, ACTB, and ribosomal proteins are expressed at similar levels across all tissues. These serve as internal controls and reference genes for normalization.

Tissue-specific genes drive the principal components. The genes with the highest PC1 loadings are typically the most tissue-specific. In a disease study, the genes driving separation between conditions are your candidates for follow-up.

Decision

After thorough EDA, you should be able to answer:

1. **Is the data quality sufficient for your question?** If samples do not cluster by biology, or if outliers dominate, no amount of statistical testing will save the experiment.
2. **Which genes to include?** Your filtering strategy affects everything downstream. Document it clearly.
3. **Which normalization to use?** TPM, FPKM, CPM, or quantile normalization each have trade-offs. EDA helps you choose.
4. **What statistical method is appropriate?** If distributions are non-normal even after transformation, use nonparametric tests. If there are confounders, use regression.
5. **What are the expected effect sizes?** Looking at the variance in your data tells you what effects you can realistically detect.

```
# Decision framework summary
# After EDA, decide on:

# 1. Gene filtering threshold
let min_tpm = 1.0
let min_samples_pct = 0.2  # expressed in at least 20% of samples

# 2. Transformation
# log2(TPM + 1) for most analyses
# Rank-based for robustness to outliers

# 3. Normalization method
# TPM: within-sample normalization (gene proportions)
# Quantile normalization: makes all distributions identical

# 4. Distance/similarity metric
# Pearson correlation: captures linear relationships
# Spearman correlation: robust to outliers and non-linearity
let r_pearson = cor(brain_log, liver_log)
let r_spearman = spearman(brain_log, liver_log)
println("Brain vs Liver:")
println("  Pearson r: " + to_string(round(r_pearson, 4)))
println("  Spearman rho: " + to_string(round(r_spearman, 4)))
# If these differ substantially, there are outliers or nonlinear effects

# 5. Dimensionality reduction method
# PCA: linear, fast, interpretable
# UMAP: nonlinear, captures local structure, good for clusters
# t-SNE: nonlinear, good for visualization, but distances are not preserved
```

Pitfalls

Pitfall 1: Analyzing without looking

```
# The most common mistake is jumping straight to testing
# without exploring the data first.
#
# Example: You run DESeq2 on samples that include a mislabeled
# sample. You get hundreds of "significant" genes that are
# actually driven by the mislabeled sample. A PCA plot would
# have caught this in 30 seconds.
#
# Rule: ALWAYS plot PCA before any statistical test.
# It takes 5 minutes and can save weeks of wasted analysis.
```

Pitfall 2: Over-filtering genes

```
# If you filter too aggressively, you remove real biology.
# If you filter too loosely, you test too many genes and
# inflate the multiple testing burden.
#
# A gene with TPM = 0.5 in all normal samples and TPM = 5.0
# in all tumor samples would be removed by a "TPM > 1 in all
# samples" filter, even though it shows a 10-fold change.
#
# Better: require expression above threshold in at least one GROUP
# Not in all samples.

# WRONG: Filter genes with mean TPM > 1 across ALL samples
# RIGHT: Filter genes with mean TPM > 1 in at least one group
```

Pitfall 3: Log-transforming zeros

```
# log2(0) is negative infinity. You cannot log-transform zeros.
# Common solutions:
# 1. Add a pseudocount: log2(x + 1) or log2(x + 0.5)
# 2. Use a variance-stabilizing transformation (VST)
# 3. Filter zeros before transforming
#
# The pseudocount choice matters for lowly expressed genes:
# log2(0 + 1) = 0
# log2(0 + 0.1) = -3.32
# log2(0 + 0.01) = -6.64
#
# Different pseudocounts give different results for low-expression genes
# This is why purpose-built transformations (VST, rlog) are preferred

let test_vals = [0.0, 0.01, 0.1, 1.0, 10.0, 100.0]
println("Effect of pseudocount on log-transformation:")
each(test_vals, |x| {
  let l1 = log2(x + 1.0)
  let l05 = log2(x + 0.5)
  let l01 = log2(x + 0.01)
  println("  TPM=" + to_string(x) + ": log2(x+1)=" + to_string(round(l1, 2))
+ ", log2(x+0.5)=" + to_string(round(l05, 2)) + ", log2(x+0.01)=" +
to_string(round(l01, 2)))
})
```

Pitfall 4: Ignoring the compositional nature of expression data

```
# TPM values sum to 1 million per sample.
# This means they are compositional: increasing one gene's TPM
# necessarily decreases others'.
#
# If one highly expressed gene changes 10-fold, it can shift
# the TPM of ALL other genes even if they are unchanged.
#
# This is called "compositional bias" and it is a real problem
# in RNA-seq analysis.
#
# Consider:
# Before: Gene A = 100,000 TPM (10% of transcriptome)
# After:  Gene A = 500,000 TPM (50% of transcriptome)
#
# Gene B was 1000 TPM in both conditions. But:
# Before: Gene B = 1000 / 1,000,000 = 0.1%
# After:  Gene B = 1000 / 1,000,000 = 0.1%... wait, TPM re-normalizes
# After normalization, Gene B's TPM drops even though it did not change!
#
# Methods that use size factors (DESeq2) or TMM normalization (edgeR)
# account for this. Raw TPM comparisons do not.
```

Pitfall 5: Overinterpreting PCA

```
# PCA shows the LARGEST sources of variation
# These may not be biologically interesting
#
# PC1 might separate by:
# - Sex (if you have mixed-sex samples)
# - Batch (if processing dates differ)
# - Library size (if some samples have more reads)
# - Cell type composition (if tissue heterogeneity varies)
#
# None of these are your biological variable of interest
# but they can dominate the PCA
#
# Always check what is driving the principal components
# before concluding that samples cluster by condition

# Example: correlate PC1 scores with known covariates
# If PC1 correlates with batch more than condition,
# you need batch correction before proceeding
```

Pitfall 6: Mean as representative value for skewed data

```
# The mean of gene expression values (especially raw counts or TPM)
# is NOT a good representative value

let example_tpms = [0.0, 0.0, 0.0, 0.1, 0.3, 1.2, 3.5, 15.0, 120.0, 8500.0]
println("Example TPMs: " + to_string(example_tpms))
println("Mean: " + to_string(round(mean(example_tpms), 2)))
println("Median: " + to_string(round(median(example_tpms), 2)))

# The mean (864) is dominated by the single high value (8500)
# The median (0.75) better represents the "typical" gene
# Always use median for skewed distributions
```

Where Next?

This chapter has equipped you with a systematic approach to exploring any gene expression dataset. The key lessons are:

1. **Always look before you test.** Five minutes of EDA can prevent weeks of wasted analysis.
2. **Understand your distributions.** Gene expression is right-skewed, zero-inflated, and compositional. These properties determine which methods work.
3. **PCA is your single most valuable tool.** It reveals sample quality, batch effects, outliers, and biological structure simultaneously.
4. **The right summary statistic depends on the distribution.** Median for skewed data, mean for symmetric. Never report a mean without also reporting the shape.
5. **Document your preprocessing.** Every filtering, transformation, and normalization choice affects downstream results. Write it down.

In **Chapter 3: Distributions in Genomics**, we will formalize the distribution knowledge we developed here — learning exactly which probability distributions describe read counts, variant quality scores, and allele frequencies, and how to check whether your data match the distributions your methods assume.

In **Chapter 4: Experimental Design and Power**, we will use the variance estimates from our EDA to calculate how many samples are needed for a well-powered study. The variability you observe in EDA directly determines the sample size you need.

Exercises

```
# Exercise 1: Download the GTEx data and identify the 5 most
# tissue-specific genes for each of: liver, brain cortex,
# skeletal muscle, and whole blood. Are they known markers?

# Exercise 2: Build a correlation matrix of all 54 tissues.
# Which two non-brain tissues are most correlated?
# Which tissue is most different from all others?

# Exercise 3: Compare Pearson and Spearman correlations for
# Brain vs Liver. Which is higher? Why?
# Hint: think about what each correlation measures.

# Exercise 4: Create a PCA plot using all 54 tissues.
# How much variance does PC1 explain? PC2?
# Color-code points by organ system. Do related tissues cluster?

# Exercise 5: Find genes with bimodal expression across tissues
# (high in some, zero in others). These are tissue-specific
# markers. Pick one and verify it makes biological sense.

# Exercise 6: What happens to the distribution of TPM values
# if you use different pseudocounts (0.01, 0.1, 0.5, 1.0)?
# Plot the log-transformed distributions side by side.
# At what pseudocount do the distributions stabilize?

# Exercise 7: Calculate the coefficient of variation (SD/mean)
# for each gene across tissues. What is the distribution of CV?
# Is it bimodal? What does each mode represent?
```

Distributions in Genomics

The Problem

A senior bioinformatician reviews a variant calling pipeline's quality metrics. The distribution of variant quality scores does not look right. Instead of the expected bell curve for true variants and a clear separation from artifacts, the histogram shows an unexpected shoulder at intermediate quality scores. Is this a real biological signal — perhaps a heterogeneous tumor sample with subclonal variants — or a technical artifact from a failing sequencing run?

To answer this question, you need to know what the quality score distribution should look like when everything is working. You need to know which probability distributions describe different types of genomic data, how to check whether your data match those distributions, and what it means when they do not.

This chapter is about the mathematical distributions that appear everywhere in genomics. Not as abstract theory, but as practical tools for understanding your data and detecting problems.

Why It Matters

Every statistical test makes assumptions about the underlying distribution of the data. Use the wrong distribution, and your p-values, confidence intervals, and effect estimates are all wrong.

Here are the distributions that matter most in genomics and why:

1. **Normal (Gaussian):** Log-transformed expression values, measurement error, test statistics. Most parametric tests assume normality.
2. **Poisson:** Read counts at a genomic position under simple models, rare variant counts, random sampling events. The starting point for count models.
3. **Negative binomial:** Overdispersed counts — the real distribution of RNA-seq read counts. DESeq2 and edgeR use this instead of Poisson because biological variance exceeds what Poisson allows.

4. **Binomial:** Allele counts in genotyping, variant allele frequencies, the basis of Hardy-Weinberg equilibrium tests.
5. **Uniform:** P-values under the null hypothesis. If your null p-values are not uniform, your test is miscalibrated.

Understanding these distributions is not academic. It is the difference between detecting a real variant and calling an artifact, between finding a real expression change and chasing noise.

The Data

We will use ClinVar variant data to explore real distributions of variant properties. ClinVar is NCBI's database of clinically relevant genetic variants, containing hundreds of thousands of variants with their clinical significance, genomic coordinates, and associated metadata.

```
# Download ClinVar variant summary data
# This file contains variant-level information including
# chromosomal location, clinical significance, review status, etc.
let url =
"https://ftp.ncbi.nlm.nih.gov/pub/clinvar/tab_delimited/variant_summary.txt.g
z"
let raw_file = download(url, "data/variant_summary.txt.gz")

# Read the tab-delimited file
let variants = read_tsv(raw_file)

println("ClinVar dataset:")
println("  Total variants: " + to_string(len(variants)))
println("  Columns: " + to_string(ncols(variants)))
println("  Column names:")
each(columns(variants), |c| println("    " + c))

# Examine the first few rows
take(variants, 5)
```

```
# Understanding the dataset structure
# Key columns we will use:
# - Type: variant type (single nucleotide variant, deletion, etc.)
# - ClinicalSignificance: pathogenic, benign, uncertain, etc.
# - Assembly: genome build (GRCh37, GRCh38)
# - Chromosome: chromosomal location
# - Start, Stop: genomic coordinates

# Count variant types
let type_counts = count_by(variants, "Type")
println("Variant types:")
println(type_counts)

# Count clinical significance categories
let clin_counts = count_by(variants, "ClinicalSignificance")
println("Clinical significance:")
println(take(clin_counts, 15))
```

Exploration

The Normal Distribution

The normal (Gaussian) distribution is the foundation of classical statistics. It describes continuous measurements that cluster symmetrically around a mean, with frequency falling off exponentially in the tails.

```
# The normal distribution is defined by two parameters:
# mu (mean) — the center
# sigma (standard deviation) — the spread

# In genomics, log-transformed expression values are approximately normal
# Measurement errors are approximately normal
# Many test statistics follow normal distributions (by CLT)

# Generate normal samples and examine their properties
let normal_data = rnorm(10000, 0.0, 1.0)

println("Standard normal sample (n=10000):")
println("  Mean: " + to_string(round(mean(normal_data), 4)) + " (expected:
0)")
println("  SD: " + to_string(round(stdev(normal_data), 4)) + " (expected:
1)")
println("  Median: " + to_string(round(median(normal_data), 4)) + "
(expected: 0)")

# The 68-95-99.7 rule
let within_1sd = len(filter(normal_data, |x| abs(x) < 1.0))
let within_2sd = len(filter(normal_data, |x| abs(x) < 2.0))
let within_3sd = len(filter(normal_data, |x| abs(x) < 3.0))
println("  Within 1 SD: " + to_string(round(100.0 * within_1sd / 10000.0, 1))
+ "% (expected: 68.3%)")
println("  Within 2 SD: " + to_string(round(100.0 * within_2sd / 10000.0, 1))
+ "% (expected: 95.4%)")
println("  Within 3 SD: " + to_string(round(100.0 * within_3sd / 10000.0, 1))
+ "% (expected: 99.7%)")

histogram(normal_data, "plots/normal_distribution.svg")
  |> save_plot("plots/normal_distribution.svg")
```

```
# Normal distribution probability functions

# dnorm: probability density function (height of the curve)
# pnorm: cumulative distribution function (area to the left)
# qnorm: quantile function (inverse of CDF)
# rnorm: random number generator

# Example: What fraction of a standard normal is above 1.96?
let p_above_196 = 1.0 - pnorm(1.96, 0.0, 1.0)
println("P(Z > 1.96) = " + to_string(round(p_above_196, 4)))
# This is 0.025 — the basis for 95% confidence intervals

# What Z-score corresponds to the 95th percentile?
let z_95 = qnorm(0.95, 0.0, 1.0)
println("Z for 95th percentile: " + to_string(round(z_95, 4)))

# What about more extreme values?
let p_above_3 = 1.0 - pnorm(3.0, 0.0, 1.0)
let p_above_5 = 1.0 - pnorm(5.0, 0.0, 1.0)
println("P(Z > 3.0) = " + to_string(round(p_above_3, 6)))
println("P(Z > 5.0) = " + to_string(round(p_above_5, 10)))
# Very extreme values are vanishingly rare under normality
```

The Normal Distribution in Real Genomic Data

```
# Where does the normal distribution appear in genomics?

# 1. Log-transformed expression values
# Generate realistic log2(TPM+1) values
let simulated_expr = rnorm(5000, 4.5, 2.8)
# Center around 4.5 (median expression), SD around 2.8

println("Simulated log2 expression:")
println("  Mean: " + to_string(round(mean(simulated_expr), 2)))
println("  SD: " + to_string(round(stdev(simulated_expr), 2)))

# QQ-plot against normal distribution
qq_plot(simulated_expr, "plots/log_expr_qqplot.svg")
  |> save_plot("plots/log_expr_qqplot.svg")

# 2. Measurement error
# Technical replicates of the same sample
# The differences between replicates follow a normal distribution
let replicate_diffs = rnorm(1000, 0.0, 0.15)
# Mean near 0 (no systematic bias), SD reflects technical noise
println("Replicate differences:")
println("  Mean: " + to_string(round(mean(replicate_diffs), 4)))
println("  SD: " + to_string(round(stdev(replicate_diffs), 4)))
```

The Poisson Distribution

The Poisson distribution describes the number of events occurring in a fixed interval when events happen independently at a constant average rate. In genomics, it models read counts at a locus under the simplest assumptions.

```
# The Poisson distribution has ONE parameter: lambda (the mean)
# Key property: mean = variance (for a true Poisson)

# Example: sequencing reads at a genomic position
# If the expected coverage is 30x, the number of reads
# at any position follows approximately Poisson(30)

let lambda = 30.0
let poisson_data = rpois(10000, lambda)

println("Poisson(lambda=30) sample:")
println("  Mean: " + to_string(round(mean(poisson_data), 2)) + " (expected:
30)")
println("  Variance: " + to_string(round(variance(poisson_data), 2)) + "
(expected: 30)")
println("  SD: " + to_string(round(stdev(poisson_data), 2)) + " (expected:
~5.48)")
println("  Mean/Variance ratio: " + to_string(round(mean(poisson_data) /
variance(poisson_data), 3)) + " (expected: 1.0)")

histogram(poisson_data, "plots/poisson_distribution.svg")
  |> save_plot("plots/poisson_distribution.svg")
```

```
# Poisson probability functions

# dpois: probability mass function (exact probability of k events)
# ppois: cumulative distribution function
# rpois: random number generator

# Example: If expected coverage is 30x, what is the probability
# of seeing 0 reads at a position?
let p_zero = dpois(0, 30.0)
println("P(X=0 | lambda=30) = " + to_string(round(p_zero, 10)))
# Essentially zero — this is why 30x coverage is considered adequate

# What about 10 reads? (half the expected coverage)
let p_10 = dpois(10, 30.0)
println("P(X=10 | lambda=30) = " + to_string(round(p_10, 6)))

# What fraction of positions have < 15 reads?
let p_below_15 = ppois(14, 30.0)
println("P(X < 15 | lambda=30) = " + to_string(round(p_below_15, 6)))

# For variant calling: if we need >= 10 reads for a confident call,
# what fraction of positions are callable at 30x?
let p_callable = 1.0 - ppois(9, 30.0)
println("P(X >= 10 | lambda=30) = " + to_string(round(p_callable, 4)))
# Very high — almost all positions are callable at 30x
```

Why RNA-seq Is NOT Poisson

```
# The critical insight: real RNA-seq counts are OVERDISPERSED
# The variance exceeds the mean, violating the Poisson assumption

# In Poisson: Var(X) = mean(X)
# In real RNA-seq: Var(X) = mean(X) + alpha * mean(X)^2
# where alpha is the dispersion parameter

# Let us demonstrate with simulated biological replicates
# If gene expression truly followed Poisson:
let poisson_counts = rpois(100, 500.0)  # 100 samples, mean=500
println("Poisson counts (mean=500):")
println("  Mean: " + to_string(round(mean(poisson_counts), 1)))
println("  Variance: " + to_string(round(variance(poisson_counts), 1)))
println("  Ratio Var/Mean: " + to_string(round(variance(poisson_counts) /
mean(poisson_counts), 2)))

# But real RNA-seq has biological variation ON TOP of technical variation
# This makes the variance much larger than the mean
# Simulate this with a negative binomial
let alpha = 0.1  # typical dispersion for RNA-seq
let nb_mean = 500.0
# Negative binomial: Var = mu + alpha * mu^2
# For mu=500, alpha=0.1: Var = 500 + 0.1 * 250000 = 25500
# SD = sqrt(25500) = ~160

# Approximate NB by sampling means from a gamma distribution
# then drawing Poisson counts
let nb_counts = map(range(0, 100), |i| {
  let true_rate = rnorm(1, nb_mean, nb_mean * sqrt(alpha)) |> take(1)
  let clamped_rate = max(true_rate, 1.0)
  rpois(1, clamped_rate) |> take(1)
})

println("Negative binomial-like counts (mean~500, overdispersed):")
println("  Mean: " + to_string(round(mean(nb_counts), 1)))
println("  Variance: " + to_string(round(variance(nb_counts), 1)))
println("  Ratio Var/Mean: " + to_string(round(variance(nb_counts) /
mean(nb_counts), 2)))
println("  Expected ratio: ~" + to_string(round(1.0 + alpha * nb_mean, 1)))

# The variance-to-mean ratio is much greater than 1
# This is OVERDISPERSION and it is universal in RNA-seq data
# Using Poisson when data are overdispersed gives p-values
# that are FAR too small — massive false positive rates
```

The Negative Binomial Distribution

```
# The negative binomial (NB) is the standard distribution for RNA-seq counts
# It has TWO parameters: mean (mu) and dispersion (alpha)
# Variance = mu + alpha * mu^2
#
# When alpha = 0: NB reduces to Poisson (no overdispersion)
# As alpha increases: variance grows much faster than mean
#
# Typical alpha values in RNA-seq:
# - Highly expressed genes: alpha ~ 0.01 (almost Poisson)
# - Moderately expressed genes: alpha ~ 0.1
# - Lowly expressed genes: alpha ~ 1.0 (very overdispersed)

# Demonstrate the effect of dispersion
let mu = 100.0
let dispersions = [0.0, 0.01, 0.1, 0.5, 1.0]

each(dispersions, |alpha| {
  let expected_var = mu + alpha * mu * mu
  let expected_sd = sqrt(expected_var)
  let cv = expected_sd / mu
  println("alpha=" + to_string(alpha) + ": Var=" +
to_string(round(expected_var, 1)) + ", SD=" + to_string(round(expected_sd,
1)) + ", CV=" + to_string(round(cv, 3)))
})

# At alpha=0 (Poisson): CV = 1/sqrt(mu) = 0.1 (10% variation)
# At alpha=0.1: CV = 0.33 (33% variation)
# At alpha=1.0: CV = 1.005 (100% variation!)
# Biological variability is the dominant source of noise
```

The Binomial Distribution

```
# The binomial distribution describes the number of "successes" in n
# independent trials, each with probability p of success.
#
# In genomics, it appears in:
# 1. Genotype calling: expected allele counts given ploidy
# 2. Allele frequency estimation
# 3. Hardy-Weinberg equilibrium testing
# 4. Variant allele frequency (VAF) in mixed samples

# Example: at a heterozygous site, each read has p=0.5
# probability of showing the reference allele
# With 60 reads, the number showing ref follows Binomial(60, 0.5)

let n_reads = 60
let p_ref = 0.5
let binom_data = rbinom(10000, n_reads, p_ref)

println("Binomial(n=60, p=0.5) — heterozygous site:")
println("  Mean: " + to_string(round(mean(binom_data), 2)) + " (expected:
30)")
println("  SD: " + to_string(round(stdev(binom_data), 2)) + " (expected:
~3.87)")
println("  Min observed: " + to_string(min(binom_data)))
println("  Max observed: " + to_string(max(binom_data)))

histogram(binom_data, "plots/binomial_heterozygous.svg")
  |> save_plot("plots/binomial_heterozygous.svg")
```

```
# Binomial probability functions
# dbinom: P(X = k)
# pbinom: P(X <= k)
# rbinom: random number generator

# For variant calling: at a heterozygous site with 60 reads,
# what is the probability of seeing 20 or fewer ref alleles?
let p_20_or_less = pbinom(20, 60, 0.5)
println("P(ref_count <= 20 | n=60, p=0.5) = " + to_string(round(p_20_or_less,
6)))
# This is quite unlikely (~0.003), suggesting allelic imbalance
# or that the site might not actually be heterozygous

# For a somatic mutation with VAF = 10% (early tumor clone):
# Expected variant allele count at 100x coverage
let vaf = 0.10
let coverage = 100
let expected_alt = vaf * coverage
println("Expected alt reads: " + to_string(expected_alt))

# What is the probability of seeing >= 5 alt reads?
let p_ge_5 = 1.0 - pbinom(4, coverage, vaf)
println("P(alt >= 5 | coverage=100, VAF=10%) = " + to_string(round(p_ge_5,
4)))

# What about at 30x coverage?
let p_ge_5_30x = 1.0 - pbinom(4, 30, vaf)
println("P(alt >= 5 | coverage=30, VAF=10%) = " + to_string(round(p_ge_5_30x,
4)))
# Much lower — this is why low-frequency variant detection requires
# deep sequencing
```

Analyzing Real Variant Data

```
# Analyze ClinVar variant characteristics
# Filter to single nucleotide variants on GRCh38
let snvs = filter_by(variants, col("Type") == "single nucleotide variant")
let snvs_grch38 = filter_by(snvs, col("Assembly") == "GRCh38")

println("Single nucleotide variants (GRCh38): " +
to_string(len(snvs_grch38)))

# Examine the distribution of variants across chromosomes
let chr_counts = count_by(snvs_grch38, "Chromosome")
println("Variants per chromosome:")
println(chr_counts)
```

```
# Variant distribution across chromosomes should roughly
# correlate with chromosome length
# This is approximately a Poisson process along the genome

# Known approximate chromosome lengths (Mb) for GRCh38
let chr_lengths = record(
  chr1: 248.9, chr2: 242.2, chr3: 198.3, chr4: 190.2,
  chr5: 181.5, chr6: 170.8, chr7: 159.3, chr8: 145.1,
  chr9: 138.4, chr10: 133.8, chr11: 135.1, chr12: 133.3,
  chr13: 114.4, chr14: 107.0, chr15: 101.9, chr16: 90.3,
  chr17: 83.3, chr18: 80.4, chr19: 58.6, chr20: 64.4,
  chr21: 46.7, chr22: 50.8, chrX: 156.0
)

# If variants were uniformly distributed, the count per chromosome
# would be proportional to chromosome length
# Deviations from this indicate biological enrichment or depletion
```

The Uniform Distribution and P-values Under the Null

```
# Under the null hypothesis (no effect), p-values follow a
# Uniform(0, 1) distribution. This is a FUNDAMENTAL property
# that must hold for any valid test.
#
# If your null p-values are not uniform, either:
# 1. The null hypothesis is false for some tests (expected)
# 2. Your test is invalid (a problem)
# 3. There are confounders or batch effects (a problem)

# Demonstrate: random t-tests between identical groups
let null_pvals = map(range(0, 5000), |i| {
  let group1 = rnorm(10, 0.0, 1.0)
  let group2 = rnorm(10, 0.0, 1.0)
  let result = ttest(group1, group2)
  result.p_value
})

# These should be uniform
println("Null p-value distribution (should be uniform):")
println("  Mean: " + to_string(round(mean(null_pvals), 3)) + " (expected:
0.5)")
println("  Median: " + to_string(round(median(null_pvals), 3)) + " (expected:
0.5)")
println("  < 0.05: " + to_string(round(100.0 * len(filter(null_pvals, |p| p <
0.05)) / 5000.0, 1)) + "% (expected: 5%)")
println("  < 0.01: " + to_string(round(100.0 * len(filter(null_pvals, |p| p <
0.01)) / 5000.0, 1)) + "% (expected: 1%)")

# Check with quantiles (should be 0.1, 0.2, ..., 0.9)
each([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9], |q| {
  let observed = quantile(null_pvals, q)
  println("  Q" + to_string(round(q * 100, 0)) + ": " +
to_string(round(observed, 3)) + " (expected: " + to_string(q) + ")")
})

histogram(null_pvals, "plots/null_pvalue_uniform.svg")
  |> save_plot("plots/null_pvalue_uniform.svg")
```

```
# What does the p-value distribution look like when there IS a real effect?
# Mix null tests with tests that have a real difference

let mixed_pvals = map(range(0, 5000), |i| {
  if i < 4000 then {
    # Null: no difference
    let g1 = rnorm(10, 0.0, 1.0)
    let g2 = rnorm(10, 0.0, 1.0)
    let res = ttest(g1, g2)
    res.p_value
  } else {
    # Alternative: real difference of d=1.0
    let g1 = rnorm(10, 0.0, 1.0)
    let g2 = rnorm(10, 1.0, 1.0)
    let res = ttest(g1, g2)
    res.p_value
  }
})

histogram(mixed_pvals, "plots/mixed_pvalue_distribution.svg")
  |> save_plot("plots/mixed_pvalue_distribution.svg")

# The histogram should show:
# - A flat region (from 4000 null tests — uniform distribution)
# - A spike near 0 (from 1000 tests with real effects)
# The spike height above the flat region tells you how many
# true positives you have
```

The Analysis

QQ-Plots: The Universal Distribution Check

```
# A QQ (Quantile-Quantile) plot compares the distribution of your data
# against a theoretical distribution.
#
# How it works:
# 1. Sort your data
# 2. For each data point, compute its expected value under the
#    theoretical distribution
# 3. Plot observed (y-axis) vs expected (x-axis)
# 4. If the data follow the theoretical distribution,
#    points fall on the diagonal y = x

# Check if our Poisson data looks Poisson
qq_plot(poisson_data, "plots/poisson_qqplot.svg")
  |> save_plot("plots/poisson_qqplot.svg")

# Check if overdispersed data looks Poisson (it should not)
qq_plot(nb_counts, "plots/nb_qqplot.svg")
  |> save_plot("plots/nb_qqplot.svg")

# The overdispersed data will show tails that deviate from the line
# — more extreme values than Poisson predicts
```

Testing for Distribution Fit

```
# The Kolmogorov-Smirnov (KS) test compares a sample distribution
# to a theoretical distribution, or two sample distributions to each other

# Test if our normal data is actually normal
let normal_sample = rnorm(500, 0.0, 1.0)
let reference_sample = rnorm(500, 0.0, 1.0)
let ks_result = ks_test(normal_sample, reference_sample)
println("KS test — normal vs normal:")
println("  statistic: " + to_string(round(ks_result.statistic, 4)))
println("  p-value: " + to_string(round(ks_result.p_value, 4)))
# High p-value = no evidence of different distributions (expected)

# Test if overdispersed data differs from Poisson
let true_poisson = rpois(500, 100.0)
let overdispersed = map(range(0, 500), |i| {
  let rate = rnorm(1, 100.0, 30.0) |> take(1)
  rpois(1, max(rate, 1.0)) |> take(1)
})

let ks_overdispersed = ks_test(true_poisson, overdispersed)
println("KS test — Poisson vs overdispersed:")
println("  statistic: " + to_string(round(ks_overdispersed.statistic, 4)))
println("  p-value: " + to_string(round(ks_overdispersed.p_value, 4)))
# Low p-value = distributions differ (overdispersion detected)
```

The Central Limit Theorem

```
# The CLT is the most important theorem in statistics:
# The mean of many independent random variables is approximately
# normally distributed, REGARDLESS of the underlying distribution.
#
# This is why:
# 1. The t-test works even when individual observations are not normal
#    (as long as sample size is large enough)
# 2. Confidence intervals based on the normal distribution are valid
# 3. Many biological measurements appear normal (they are sums of
#    many small effects)

# Demonstrate: averages of Poisson samples approach normality
let sample_sizes = [1, 2, 5, 10, 30, 100]

each(sample_sizes, |n| {
  # Take 2000 means of n Poisson(5) values
  let means = map(range(0, 2000), |i| {
    let sample = rpois(n, 5.0)
    mean(sample)
  })
  let m = mean(means)
  let s = stdev(means)
  println("n=" + to_string(n) + ": mean=" + to_string(round(m, 3)) + ", SD="
+ to_string(round(s, 3)))
})

# Even n=5 gives reasonably normal-looking means
# By n=30, the CLT is working well for most distributions
```

```
# The CLT has limits!
# It fails or is slow to converge when:
# 1. The underlying distribution has very heavy tails (e.g., Cauchy)
# 2. The sample size is too small (n < 5 for skewed data)
# 3. The data are not independent (correlated observations)

# Demonstrate heavy tails: Cauchy distribution
# (ratio of two normals — no finite mean or variance!)
let cauchy_means = map(range(0, 2000), |i| {
  let sample = map(range(0, 30), |j| {
    let num = rnorm(1, 0.0, 1.0) |> take(1)
    let denom = rnorm(1, 0.0, 1.0) |> take(1)
    if abs(denom) > 0.001 then num / denom else 0.0
  })
  mean(sample)
})

println("Means of Cauchy samples (n=30):")
println("  Mean: " + to_string(round(mean(cauchy_means), 2)))
println("  SD: " + to_string(round(stdev(cauchy_means), 2)))
println("  These are highly variable — CLT does NOT apply!")
```

Allele Frequency Distributions

```
# In population genetics, allele frequencies follow specific patterns
# The site frequency spectrum (SFS) describes the distribution of
# allele frequencies in a population

# Under neutral evolution, the expected number of variants at
# frequency p in a sample of n chromosomes is proportional to 1/p
# This is the neutral SFS

# Simulate a neutral SFS for a sample of 100 chromosomes
let n_chrom = 100
let sfs = map(range(1, n_chrom), |k| {
  # Expected count proportional to 1/k (Watterson's result)
  let expected = 1000.0 / k  # scale factor of 1000
  record(
    allele_count: k,
    frequency: round(k / n_chrom, 3),
    expected_variants: round(expected, 1)
  )
})

println("Neutral site frequency spectrum (first 20 bins):")
each(take(sfs, 20), |s| {
  println("  AC=" + to_string(s.allele_count) + " (freq=" +
to_string(s.frequency) + "): ~" + to_string(s.expected_variants) + "
variants")
})

# Key observation: most variants are RARE
# There are far more singletons (AC=1) than common variants
# This 1/f pattern is a prediction of neutral evolution theory
```

```
# Hardy-Weinberg Equilibrium uses the binomial distribution
# For a biallelic site with allele freq p:
# P(AA) = p^2, P(Aa) = 2p(1-p), P(aa) = (1-p)^2

# Example: variant with minor allele frequency (MAF) = 0.3
let p = 0.3
let q = 1.0 - p

let expected_AA = p * p
let expected_Aa = 2.0 * p * q
let expected_aa = q * q

println("Hardy-Weinberg expectations for MAF = " + to_string(p) + ":")
println("  P(AA) = " + to_string(round(expected_AA, 4)))
println("  P(Aa) = " + to_string(round(expected_Aa, 4)))
println("  P(aa) = " + to_string(round(expected_aa, 4)))
println("  Sum = " + to_string(round(expected_AA + expected_Aa + expected_aa,
4)))

# In a sample of 1000 individuals:
let n_sample = 1000
println("Expected genotype counts in n=" + to_string(n_sample) + ":")
println("  AA: " + to_string(round(expected_AA * n_sample, 1)))
println("  Aa: " + to_string(round(expected_Aa * n_sample, 1)))
println("  aa: " + to_string(round(expected_aa * n_sample, 1)))

# Deviations from HWE can indicate:
# 1. Genotyping errors (common)
# 2. Population stratification
# 3. Selection (rare for a single test)
# 4. Inbreeding (excess homozygosity)
```

Read Depth Distribution

```
# The distribution of read depth across the genome is important
# for variant calling and quality assessment

# In an ideal world, depth follows Poisson(lambda) where lambda
# is the average coverage
# In reality, GC bias, mappability, and duplicates cause deviations

# Simulate realistic read depth distribution
# Model: mixture of Poisson (unique regions) and inflated values (duplicates)
let mean_depth = 30.0
let n_positions = 10000

let depths = map(range(0, n_positions), |i| {
  # 80% of positions: Poisson with some GC-bias variation
  # 20% of positions: lower coverage (GC-extreme or low-mappability)
  let is_normal = rbinom(1, 1, 0.8) |> take(1)
  if is_normal == 1 then {
    # GC-bias: modulate the rate
    let gc_factor = rnorm(1, 1.0, 0.15) |> take(1)
    rpois(1, mean_depth * max(gc_factor, 0.1)) |> take(1)
  } else {
    # Low-coverage region
    rpois(1, mean_depth * 0.3) |> take(1)
  }
})

println("Simulated read depth distribution:")
println("  Mean: " + to_string(round(mean(depths), 1)))
println("  Median: " + to_string(round(median(depths), 1)))
println("  SD: " + to_string(round(stdev(depths), 1)))
println("  Min: " + to_string(min(depths)))
println("  Max: " + to_string(max(depths)))

# Variance/Mean ratio > 1 indicates overdispersion (deviation from Poisson)
let var_mean_ratio = variance(depths) / mean(depths)
println("  Variance/Mean ratio: " + to_string(round(var_mean_ratio, 2)) + "
(Poisson would be 1.0)")

histogram(depths, "plots/read_depth_distribution.svg")
  |> save_plot("plots/read_depth_distribution.svg")
```

```
# Positions with very low depth are problematic for variant calling
# Positions with very high depth may indicate duplicates or CNVs

let very_low = len(filter(depths, |d| d < 10))
let low = len(filter(depths, |d| d >= 10 && d < 20))
let normal = len(filter(depths, |d| d >= 20 && d <= 50))
let high = len(filter(depths, |d| d > 50))

println("Depth categories:")
println("  Very low (<10x): " + to_string(very_low) + " (" +
to_string(round(100.0 * very_low / n_positions, 1)) + "%)")
println("  Low (10-19x): " + to_string(low) + " (" + to_string(round(100.0 *
low / n_positions, 1)) + "%)")
println("  Normal (20-50x): " + to_string(normal) + " (" +
to_string(round(100.0 * normal / n_positions, 1)) + "%)")
println("  High (>50x): " + to_string(high) + " (" + to_string(round(100.0 *
high / n_positions, 1)) + "%)")

# For variant calling, a typical minimum depth threshold is 10x
# The fraction of the genome below this threshold determines
# the "callable" fraction
let callable_frac = 1.0 - (very_low / n_positions)
println("Callable fraction (>= 10x): " + to_string(round(100.0 *
callable_frac, 1)) + "%")
```

Distribution Mixtures in Real Data

```
# Real genomic data often follows MIXTURE distributions
# — a combination of two or more distributions

# Example: variant quality scores
# True variants: high quality (normal distribution centered at 30)
# Artifacts: low quality (normal distribution centered at 10)

let n_true = 8000
let n_artifact = 2000

let true_quals = rnorm(n_true, 30.0, 5.0)
let artifact_quals = rnorm(n_artifact, 10.0, 4.0)
let all_quals = concat(true_quals, artifact_quals)

println("Mixed quality score distribution:")
println("  Mean: " + to_string(round(mean(all_quals), 2)))
println("  Median: " + to_string(round(median(all_quals), 2)))
println("  SD: " + to_string(round(stdev(all_quals), 2)))

histogram(all_quals, "plots/mixed_quality_scores.svg")
  |> save_plot("plots/mixed_quality_scores.svg")

# The histogram should show a bimodal distribution
# A quality filter at Q=20 would separate most true variants
# from artifacts

let above_20 = len(filter(all_quals, |q| q >= 20.0))
let below_20 = len(filter(all_quals, |q| q < 20.0))
println("Q >= 20: " + to_string(above_20) + " (" + to_string(round(100.0 *
above_20 / len(all_quals), 1)) + "%)")
println("Q < 20: " + to_string(below_20) + " (" + to_string(round(100.0 *
below_20 / len(all_quals), 1)) + "%)")

# Estimate false positive and false negative rates at Q=20 cutoff
let fp_at_20 = len(filter(artifact_quals, |q| q >= 20.0))
let fn_at_20 = len(filter(true_quals, |q| q < 20.0))
println("At Q=20 threshold:")
println("  False positives (artifacts passing): " + to_string(fp_at_20) + " /
" + to_string(n_artifact))
println("  False negatives (true variants failing): " + to_string(fn_at_20) +
" / " + to_string(n_true))
```

ClinVar Variant Length Distribution

```
# Analyze the distribution of variant sizes in ClinVar
# For deletions and insertions, the size distribution is important

let deletions = filter_by(variants, col("Type") == "Deletion")
println("Total deletions in ClinVar: " + to_string(len(deletions)))

# Calculate deletion sizes from Start and Stop positions
let del_sizes = map(range(0, min(len(deletions), 5000)), |i| {
  let row = slice(deletions, i, i + 1)
  let start = col_values(row, "Start") |> take(1) |> to_float()
  let stop = col_values(row, "Stop") |> take(1) |> to_float()
  abs(stop - start) + 1.0
})

# Filter out any invalid sizes
let valid_sizes = filter(del_sizes, |s| s > 0.0 && s < 1000000.0)

println("Deletion size distribution:")
println("  Mean: " + to_string(round(mean(valid_sizes), 1)) + " bp")
println("  Median: " + to_string(round(median(valid_sizes), 1)) + " bp")
println("  Min: " + to_string(round(min(valid_sizes), 0)) + " bp")
println("  Max: " + to_string(round(max(valid_sizes), 0)) + " bp")

# Size categories
let small = len(filter(valid_sizes, |s| s <= 50.0))
let medium = len(filter(valid_sizes, |s| s > 50.0 && s <= 1000.0))
let large = len(filter(valid_sizes, |s| s > 1000.0))
let total_valid = len(valid_sizes)

println("Size categories:")
println("  Small (<= 50 bp): " + to_string(small) + " (" +
to_string(round(100.0 * small / total_valid, 1)) + "%)")
println("  Medium (51-1000 bp): " + to_string(medium) + " (" +
to_string(round(100.0 * medium / total_valid, 1)) + "%)")
println("  Large (> 1000 bp): " + to_string(large) + " (" +
to_string(round(100.0 * large / total_valid, 1)) + "%)")
```

```
# Log-transform the sizes for better visualization
let log_sizes = map(valid_sizes, |s| log10(s))
histogram(log_sizes, "plots/deletion_size_log10.svg")
  |> save_plot("plots/deletion_size_log10.svg")

# The deletion size distribution is typically heavy-tailed
# with most deletions being small but a long tail of large ones
```

Interpretation

The distributions we have explored each tell us something fundamental about the data-generating process in genomics:

Normal distribution appears when many small, independent effects combine additively. Log-transformed expression values are approximately normal because gene regulation involves many multiplicative factors, and log transforms multiplication into addition. The CLT ensures that means and test statistics are approximately normal even when individual observations are not.

Poisson distribution appears when events occur independently at a constant rate. This is a reasonable first approximation for read counts at a locus, but it underestimates the variance because it ignores biological variability.

Negative binomial is the correct model for RNA-seq counts because it accounts for both technical (Poisson) and biological variability. The dispersion parameter captures the extent to which biological variance exceeds what Poisson predicts. Using Poisson when data are negative binomial leads to massive false positive inflation.

Binomial distribution describes allele sampling in genotyping. At a heterozygous site, each read independently samples one of two alleles with equal probability. Deviations from binomial expectations signal allelic imbalance, copy number changes, or genotyping errors.

Uniform distribution is what p-values look like when the null hypothesis is true. Any deviation from uniformity in your null p-values indicates a problem with your statistical test or your assumptions.

Decision

When choosing a distribution for your analysis:

Data Type	First Choice	Alternative	Red Flag
Log expression	Normal	Nonparametric	Heavy tails in QQ-plot
Raw read counts	Negative binomial	Poisson	Var/Mean >> 1
Allele counts	Binomial	Beta-binomial	Overdispersion
Coverage depth	Neg. binomial	Poisson	Multimodal histogram
P-values (null)	Uniform	—	Not flat histogram
Variant sizes	Log-normal	Power law	—

```
# Quick distribution diagnostic pipeline
# Run this on any new dataset to identify the distribution family

let data = rnorm(1000, 10.0, 3.0)  # replace with your data

let m = mean(data)
let med = median(data)
let s = stdev(data)
let v = variance(data)

println("Distribution diagnostics:")
println("  Mean: " + to_string(round(m, 3)))
println("  Median: " + to_string(round(med, 3)))
println("  Mean/Median: " + to_string(round(m / med, 3)) + " (1.0 =
symmetric)")
println("  Var/Mean: " + to_string(round(v / m, 3)) + " (1.0 = Poisson)")
println("  CV: " + to_string(round(s / m, 3)))

# Decision rules:
# Mean/Median ~ 1 and QQ-plot straight: Normal
# Mean/Median > 1.5: Right-skewed, try log transform
# Var/Mean ~ 1: Poisson
# Var/Mean >> 1: Negative binomial or overdispersed
# Var/Mean << 1: Underdispersed (unusual, check for truncation)
# Bimodal histogram: Mixture model needed
```

Pitfalls

Pitfall 1: Using Poisson for RNA-seq

```
# This is the most common distributional error in genomics
# Poisson assumes Var = Mean
# RNA-seq data has Var >> Mean
# Result: p-values are far too small, massive false positives

# Demonstrate the damage
let true_mean = 100.0
let true_disp = 0.15

# Generate NB data (realistic RNA-seq)
let group1 = map(range(0, 50), |i| {
  let rate = rnorm(1, true_mean, true_mean * sqrt(true_disp)) |> take(1)
  rpois(1, max(rate, 1.0)) |> take(1) |> to_float()
})
let group2 = map(range(0, 50), |i| {
  let rate = rnorm(1, true_mean, true_mean * sqrt(true_disp)) |> take(1)
  rpois(1, max(rate, 1.0)) |> take(1) |> to_float()
})

# These groups have the SAME mean — no real difference
# Test with t-test (approximately correct)
let ttest_result = ttest(group1, group2)
println("t-test p-value (correct): " + to_string(round(ttest_result.p_value,
4)))

# The variance is much larger than Poisson would predict
println("Group 1 Var/Mean: " + to_string(round(variance(group1) /
mean(group1), 2)))
println("Group 2 Var/Mean: " + to_string(round(variance(group2) /
mean(group2), 2)))
```

Pitfall 2: Ignoring zero inflation

```
# Many genomic datasets have excess zeros
# - Single-cell RNA-seq: most genes are zero in most cells (dropout)
# - Rare variant counts: most genes have zero rare variants
# - Methylation: some CpGs are completely unmethylated
#
# Standard distributions (normal, Poisson, NB) do not model excess zeros
# You need zero-inflated versions or hurdle models
#
# How to detect: compare the observed zero fraction to
# the expected zero fraction under your assumed distribution

let lambda_estimate = 2.5
let n_obs = 1000
let observed_zeros_frac = 0.45  # example: 45% of observations are zero
let expected_zeros_frac = dpois(0, lambda_estimate)

println("Expected zero fraction (Poisson, lambda=2.5): " +
to_string(round(expected_zeros_frac, 4)))
println("Observed zero fraction: " + to_string(observed_zeros_frac))

if observed_zeros_frac > 2.0 * expected_zeros_frac then {
  println("WARNING: Zero-inflated! Standard Poisson/NB is inappropriate")
}
```

Pitfall 3: Assuming normality for small samples

```
# With n < 10, you cannot reliably assess normality
# The QQ-plot will look fine for almost any distribution
# The Shapiro-Wilk test has low power
#
# Solution: use nonparametric tests (Wilcoxon, KS) for small samples
# Or use the t-test but interpret results cautiously

let small_sample = rnorm(5, 0.0, 1.0)
println("Small sample (n=5): " + to_string(small_sample))
println("Can we assess normality from this? No — too few points.")
println("Use Wilcoxon test or bootstrap for inference.")
```

Pitfall 4: Confusing the p-value distribution shape

```
# When analyzing a genome-wide experiment, the p-value histogram
# provides critical diagnostic information:
#
# GOOD shapes:
# - Flat (uniform): all null, no real effects — unexpected in genomics
# - Flat + spike at 0: mixture of nulls + true effects — typical
#
# BAD shapes:
# - U-shaped (spikes at 0 AND 1): anti-conservative test, possible
#   confounding or population stratification
# - Spike only at 1: overly conservative test or wrong test direction
# - Wavy/bumpy: batch effects or violated assumptions
#
# If you see a bad shape, DO NOT proceed with multiple testing correction
# Fix the underlying problem first
```

Where Next?

Understanding distributions is the foundation for choosing the right statistical test. With this knowledge:

- You can diagnose problems in your data by checking distributional assumptions (QQ-plots, variance/mean ratios, p-value histograms).
- You know why RNA-seq requires the negative binomial, not Poisson.
- You can interpret allele frequency distributions in population genetics.
- You know that p-values should be uniform under the null, and you can detect when they are not.

In **Chapter 4: Experimental Design and Power**, we will use these distributional insights to calculate sample size requirements. The variance of your chosen distribution directly determines how many samples you need to detect effects of a given size.

Exercises

```
# Exercise 1: Generate 10,000 Poisson(5) samples and 10,000 NB samples
# with the same mean but dispersion 0.2. Plot both histograms.
# Can you visually distinguish them? What about Var/Mean ratio?

# Exercise 2: Take the ClinVar data and examine the distribution of
# variants across chromosomes. Is it proportional to chromosome length?
# Use a chi-square test to check.

# Exercise 3: Simulate allele sampling at a heterozygous site with
# 30x coverage. What is the probability of seeing an allele ratio
# more extreme than 80:20? (Use the binomial distribution.)

# Exercise 4: Generate 1000 p-values from t-tests where the true
# effect size is d=0.5 and n=20 per group. What fraction are
# significant at alpha=0.05? This is the power.

# Exercise 5: Create a QQ-plot of the deletion sizes from ClinVar
# against a log-normal distribution. Do they match?

# Exercise 6: Simulate the effect of GC-bias on coverage distribution.
# Generate coverage from Poisson(30) but modulate lambda by a
# GC-content factor drawn from Normal(1, 0.2). How does the
# resulting coverage distribution differ from Poisson?

# Exercise 7: For a gene with mean count 50 and dispersion 0.1,
# calculate P(count > 100) under both Poisson and negative binomial.
# How much does the distributional choice matter?
```

Quick reference: distribution cheat sheet

Distribution	Parameters	Mean	Variance	Genomics Use
Normal	mu, sigma	mu	sigma^2	Log-expression, errors
Poisson	lambda	lambda	lambda	Idealized counts
Neg. Binomial	mu, alpha	mu	mu + alpha*mu^2	RNA-seq counts
Binomial	n, p	np	np(1-p)	Allele counts
Uniform	a, b	(a+b)/2	(b-a)^2/12	Null p-values
Log-normal	mu, sigma	exp(mu+s^2/2)	complex	Variant sizes

Experimental Design and Power

The Problem

Dr. Sarah Nguyen's lab wants to run an RNA-seq experiment. They plan to compare gene expression in treated versus untreated cell lines to identify transcriptional responses to a new drug candidate. The budget allows for 12 sequencing libraries.

The question that will determine whether this experiment succeeds or fails: how should those 12 libraries be allocated?

Option A: 2 conditions x 6 biological replicates = 12 libraries Option B: 2 conditions x 3 biological replicates x 2 technical replicates = 12 Option C: 4 conditions x 3 biological replicates = 12 libraries Option D: 2 conditions x 2 biological replicates x 3 time points = 12 libraries

Each option answers a different question with different statistical power. Option B is almost always wrong (technical replicates add little value). Option A maximizes power for a single comparison. Option C enables multiple comparisons but with less power for each. Option D enables a time course but is severely underpowered.

This chapter is about making these decisions rigorously, using real variance estimates from pilot data and explicit power calculations. We will show that a poorly designed experiment wastes money, time, and biological material — and produces results that cannot be trusted.

Why It Matters

The cost of an underpowered experiment is not just statistical. It is ethical. Animals are sacrificed, patient samples are consumed, years of work are invested, and papers are published with results that do not replicate. A 30-minute power analysis before the experiment can prevent all of this.

Here is what happens when experiments are underpowered:

1. **Wasted resources.** If your experiment has 20% power, you have an 80% chance of finding nothing — even when a real effect exists. The money,

samples, and time are gone.

2. **Winner's curse.** The effects that DO reach significance in underpowered studies are systematically overestimated. Follow-up studies will show smaller effects, creating the illusion that the original finding "failed to replicate."

3. **Publication bias.** Only "significant" results get published, creating a literature full of inflated effects from underpowered studies. This distorts the field's understanding of effect sizes.

4. **Ethical concerns.** Using biological samples (especially patient tissue or animal models) in experiments too small to yield reliable conclusions is ethically questionable.

The solution is to calculate the required sample size BEFORE the experiment, using realistic effect size and variance estimates from pilot data or the literature.

The Data

We will use two datasets for this chapter:

1. **GEO GSE183947** — breast cancer RNA-seq FPKM values. We will extract real variance estimates to use in power calculations for RNA-seq experiments.

2. **gnomAD constraint metrics** — gene-level constraint scores from the Genome Aggregation Database, useful for variant detection power analysis.

```
# Download breast cancer RNA-seq data for variance estimation
let expr_url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let expr_file = download(expr_url, "data/GSE183947_fpkm.csv.gz")
let expr = read_csv(expr_file)

println("Expression dataset: " + to_string(len(expr)) + " genes x " +
to_string(ncols(expr)) + " columns")

# Download gnomAD constraint metrics
let gnomad_url = "https://storage.googleapis.com/gcp-public-data--
gnomad/release/4.1/constraint/gnomad.v4.1.constraint_metrics.tsv"
let gnomad_file = download(gnomad_url, "data/gnomad_constraint.tsv")
let gnomad = read_tsv(gnomad_file)

println("gnomAD constraint dataset: " + to_string(len(gnomad)) + " genes x "
+ to_string(ncols(gnomad)) + " columns")
println("gnomAD columns:")
each(columns(gnomad), |c| println("  " + c))
```

Exploration

Estimating variance from pilot data

The most critical input to any power calculation is the variance. For RNA-seq, we need the within-group variance of log-transformed expression values.

```
# Extract expression columns (skip the gene ID column)
let all_cols = columns(expr)
let gene_col = all_cols[0]
let sample_cols = filter(all_cols, |c| c != gene_col)

println("Gene ID column: " + gene_col)
println("Sample columns: " + to_string(len(sample_cols)))
each(sample_cols, |s| println("  " + s))

# For each gene, calculate the within-group variance
# Use the first 3 samples as one group (for pilot estimation)
let pilot_cols = slice(sample_cols, 0, 3)

let gene_variances = map(range(0, min(len(expr), 5000)), |i| {
  let row = slice(expr, i, i + 1)
  let vals = map(pilot_cols, |c| {
    col_values(row, c) |> head(1) |> to_float()
  })
  # Log-transform (adding pseudocount)
  let log_vals = map(vals, |x| log2(x + 1.0))
  let m = mean(log_vals)
  let v = variance(log_vals)
  let s = stdev(log_vals)
  record(mean_expr: m, variance: v, stdev: s)
})

# Summary of variance estimates
let all_vars = map(gene_variances, |g| g.variance)
let all_sds = map(gene_variances, |g| g.stdev)
let all_means = map(gene_variances, |g| g.mean_expr)

# Filter to expressed genes (mean log2 expression > 1)
let expressed = filter(gene_variances, |g| g.mean_expr > 1.0)
let expr_vars = map(expressed, |g| g.variance)
let expr_sds = map(expressed, |g| g.stdev)

println("Variance estimates for expressed genes (n=" +
to_string(len(expressed)) + "):")
println("  Mean variance: " + to_string(round(mean(expr_vars), 4)))
println("  Median variance: " + to_string(round(median(expr_vars), 4)))
println("  Mean SD: " + to_string(round(mean(expr_sds), 3)))
println("  Median SD: " + to_string(round(median(expr_sds), 3)))
println("  Q25 SD: " + to_string(round(quantile(expr_sds, 0.25), 3)))
println("  Q75 SD: " + to_string(round(quantile(expr_sds, 0.75), 3)))
println("  Q90 SD: " + to_string(round(quantile(expr_sds, 0.90), 3)))
```

```
# The distribution of variance across genes is crucial
# It tells us what effect sizes we can detect at different power levels

histogram(expr_sds, "plots/gene_sd_distribution.svg")
  |> save_plot("plots/gene_sd_distribution.svg")

# Plot variance vs mean expression
# Lowly expressed genes tend to have higher variance
scatter(all_means, all_sds, "plots/mean_vs_sd.svg")
  |> save_plot("plots/mean_vs_sd.svg")

println("Mean-variance relationship:")
let high_expr = filter(gene_variances, |g| g.mean_expr > 5.0)
let low_expr = filter(gene_variances, |g| g.mean_expr > 1.0 && g.mean_expr <=
5.0)
println("  High expression (>5 log2): median SD = " +
to_string(round(median(map(high_expr, |g| g.stdev)), 3)))
println("  Low expression (1-5 log2): median SD = " +
to_string(round(median(map(low_expr, |g| g.stdev)), 3)))
```

Understanding effect sizes in RNA-seq

```
# Effect size in RNA-seq is typically measured as log2 fold change
# Cohen's d = log2FC / pooled_SD

# What log2 fold changes are biologically meaningful?
# |log2FC| = 0.5 means ~1.4-fold change (small)
# |log2FC| = 1.0 means ~2-fold change (moderate)
# |log2FC| = 2.0 means ~4-fold change (large)
# |log2FC| = 3.0 means ~8-fold change (very large)

let fc_table = [0.5, 1.0, 1.5, 2.0, 3.0]
println("Log2FC to fold-change conversion:")
each(fc_table, |lfc| {
  let fc = pow(2.0, lfc)
  println("  log2FC=" + to_string(lfc) + " -> " + to_string(round(fc, 2)) +
"-fold change")
})

# Using the median SD from our pilot data, compute Cohen's d
let pilot_sd = median(expr_sds)
println("")
println("Pilot data median SD: " + to_string(round(pilot_sd, 3)))
println("Cohen's d for different fold changes:")
each(fc_table, |lfc| {
  let d = lfc / pilot_sd
  println("  log2FC=" + to_string(lfc) + ": d=" + to_string(round(d, 2)))
})
```

The Analysis

Power calculation from first principles

Power is the probability of rejecting the null hypothesis when it is false. It depends on four quantities:

- Effect size (how big the difference is)
- Sample size (how many observations per group)
- Significance level (alpha, usually 0.05)
- Variance (how noisy the data are)

```
# Power for a two-sample t-test
# The non-centrality parameter is: delta = d * sqrt(n/2)
# where d = Cohen's d, n = sample size per group
#
# Power = P(|T| > t_crit | delta)
# Using the normal approximation:
# Power approximately = Phi(delta - z_alpha/2)
#   where z_alpha/2 = 1.96 for alpha=0.05 (two-sided)

# Calculate power for different sample sizes and effect sizes
let alpha = 0.05
let z_alpha = qnorm(1.0 - alpha / 2.0, 0.0, 1.0)

let sample_sizes = [3, 4, 5, 6, 8, 10, 12, 15, 20, 25, 30, 50]
let effect_sizes = [0.5, 0.8, 1.0, 1.5, 2.0]

println("Power table (alpha=" + to_string(alpha) + ", two-sided t-test):")
println("")

# Header
let header = "  n/group"
each(effect_sizes, |d| {
  let h = header + "  d=" + to_string(d)
})
println("n/group | d=0.5  d=0.8  d=1.0  d=1.5  d=2.0")
println("--------|-------------------------------------")

each(sample_sizes, |n| {
  let powers = map(effect_sizes, |d| {
    let delta = d * sqrt(n / 2.0)
    let power = pnorm(delta - z_alpha, 0.0, 1.0)
    round(power * 100, 1)
  })
  let line = to_string(n)
  let power_strs = map(powers, |p| to_string(p) + "%")
  println("  " + to_string(n) + "     | " + join(power_strs, "  "))
})
```

Power curves

```
# Plot power as a function of sample size for different effect sizes

let n_range = range(3, 51)

# For d = 1.0 (a 2-fold change with typical variance)
let power_d1 = map(n_range, |n| {
  let delta = 1.0 * sqrt(n / 2.0)
  pnorm(delta - z_alpha, 0.0, 1.0)
})

# For d = 0.5 (a moderate effect)
let power_d05 = map(n_range, |n| {
  let delta = 0.5 * sqrt(n / 2.0)
  pnorm(delta - z_alpha, 0.0, 1.0)
})

# For d = 1.5 (a large effect)
let power_d15 = map(n_range, |n| {
  let delta = 1.5 * sqrt(n / 2.0)
  pnorm(delta - z_alpha, 0.0, 1.0)
})

scatter(n_range, power_d1, "plots/power_curve_d1.svg")
  |> save_plot("plots/power_curve_d1.svg")

# Find the sample size needed for 80% power
println("Sample size for 80% power:")
each([0.5, 0.8, 1.0, 1.5, 2.0], |d| {
  let n_needed = 3
  let found = false
  each(range(3, 200), |n| {
    if !found then {
      let delta = d * sqrt(n / 2.0)
      let power = pnorm(delta - z_alpha, 0.0, 1.0)
      if power >= 0.80 then {
        println("  d=" + to_string(d) + ": n=" + to_string(n) + " per group")
        found = true
      }
    }
  })
})
```

RNA-seq specific power calculation

```
# For RNA-seq, we use the variance estimates from our pilot data
# The typical workflow:
# 1. Get variance estimates from pilot data or similar published data
# 2. Decide on the minimum fold-change you want to detect
# 3. Calculate power across sample sizes
# 4. Choose the smallest n that gives >= 80% power

# Use the real variance estimates from GSE183947
let typical_sd = median(expr_sds)
let high_var_sd = quantile(expr_sds, 0.75)
let low_var_sd = quantile(expr_sds, 0.25)

println("Variance estimates from pilot data:")
println("  Low-variance genes (Q25 SD): " + to_string(round(low_var_sd, 3)))
println("  Typical genes (median SD): " + to_string(round(typical_sd, 3)))
println("  High-variance genes (Q75 SD): " + to_string(round(high_var_sd,
3)))

# Power for detecting 2-fold change (log2FC = 1)
let target_lfc = 1.0
println("")
println("Power to detect 2-fold change (log2FC=1):")
println("n/group | Low-var  Typical  High-var")
println("--------|---------------------------")

each([3, 4, 5, 6, 8, 10, 12, 15, 20], |n| {
  let powers = map([low_var_sd, typical_sd, high_var_sd], |sd| {
    let d = target_lfc / sd
    let delta = d * sqrt(n / 2.0)
    let power = pnorm(delta - z_alpha, 0.0, 1.0)
    to_string(round(power * 100, 1)) + "%"
  })
  println("  " + to_string(n) + "     | " + join(powers, "    "))
})
```

What fraction of genes can you detect at each sample size?

```
# For a fixed fold-change threshold, different genes have different
# power because they have different variances
# This is a more realistic way to think about RNA-seq power:
# "What fraction of truly DE genes will I detect?"

let target_lfcs = [1.0, 1.5, 2.0]
let test_ns = [3, 5, 8, 12, 20]

println("Fraction of expressed genes detectable at 80% power:")
println("(assuming all DE genes have the specified log2FC)")
println("")

each(target_lfcs, |lfc| {
  println("log2FC = " + to_string(lfc) + " (" + to_string(round(pow(2.0,
lfc), 1)) + "-fold):")
  each(test_ns, |n| {
    # For each gene, calculate whether power >= 0.80
    let detectable = filter(expr_sds, |sd| {
      let d = lfc / sd
      let delta = d * sqrt(n / 2.0)
      let power = pnorm(delta - z_alpha, 0.0, 1.0)
      power >= 0.80
    })
    let pct = round(100.0 * len(detectable) / len(expr_sds), 1)
    println("  n=" + to_string(n) + ": " + to_string(pct) + "% of genes
detectable")
  })
  println("")
})
```

The cost of underpowered studies

```
# Simulate the consequences of different sample sizes
# on a genome-wide study

# Set up: 1000 genes, 100 truly DE (10%)
# True DE genes have log2FC = 1.5
let n_genes = 1000
let n_de = 100
let n_null = n_genes - n_de
let true_lfc = 1.5
let gene_sd = typical_sd

println("Simulation: " + to_string(n_genes) + " genes, " + to_string(n_de) +
" truly DE (log2FC=" + to_string(true_lfc) + ")")
println("")

each([3, 5, 8, 12, 20], |n| {
  # Simulate null genes
  let null_pvals = map(range(0, n_null), |i| {
    let g1 = rnorm(n, 8.0, gene_sd)
    let g2 = rnorm(n, 8.0, gene_sd)
    let res = ttest(g1, g2)
    res.p_value
  })

  # Simulate DE genes
  let de_pvals = map(range(0, n_de), |i| {
    let g1 = rnorm(n, 8.0, gene_sd)
    let g2 = rnorm(n, 8.0 + true_lfc, gene_sd)
    let res = ttest(g1, g2)
    res.p_value
  })

  # Apply BH correction
  let all_pvals = concat(null_pvals, de_pvals)
  let all_adjusted = p_adjust(all_pvals, "BH")

  # Split back into null and DE
  let null_adj = slice(all_adjusted, 0, n_null)
  let de_adj = slice(all_adjusted, n_null, n_genes)

  let fp = len(filter(null_adj, |p| p < 0.05))
  let tp = len(filter(de_adj, |p| p < 0.05))
  let fn_count = n_de - tp
  let total_sig = fp + tp

  let fdr = if total_sig > 0 then round(100.0 * fp / total_sig, 1) else 0.0
  let power = round(100.0 * tp / n_de, 1)
  let sensitivity = power

  println("n=" + to_string(n) + " per group:")
  println("  True positives: " + to_string(tp) + "/" + to_string(n_de) + "
(power=" + to_string(power) + "%)")
  println("  False positives: " + to_string(fp) + "/" + to_string(n_null))
  println("  False negatives: " + to_string(fn_count) + "/" +
to_string(n_de))
  println("  Total significant: " + to_string(total_sig))
  println("  Observed FDR: " + to_string(fdr) + "%")
  println("")
})
```

Biological vs technical replicates

```
# This is the most important design concept in genomics
# and the most commonly violated

# BIOLOGICAL REPLICATES: independent biological samples
# - Different patients, different mice, different cell cultures
# - Capture biological variability
# - THESE ARE WHAT COUNT FOR SAMPLE SIZE

# TECHNICAL REPLICATES: repeated measurements of the same sample
# - Same RNA extracted twice, same library sequenced twice
# - Capture technical variability
# - DO NOT increase your effective sample size for biological questions

# Demonstrate why:
# Scenario 1: 6 biological replicates, 1 technical rep each = n=6
# Scenario 2: 3 biological replicates, 2 technical reps each = n=3 (NOT n=6!)

let bio_sd = 1.5   # biological variability (between individuals)
let tech_sd = 0.3   # technical variability (between replicates)

# Total SD with biological replicates:
# Each observation is independent, SD = sqrt(bio_sd^2 + tech_sd^2)
let total_sd = sqrt(bio_sd * bio_sd + tech_sd * tech_sd)
println("Standard deviation components:")
println("  Biological SD: " + to_string(bio_sd))
println("  Technical SD: " + to_string(tech_sd))
println("  Total SD: " + to_string(round(total_sd, 3)))
println("")
println("Proportion of variance from biology: " + to_string(round(100.0 *
bio_sd * bio_sd / (total_sd * total_sd), 1)) + "%")
println("Proportion of variance from technology: " + to_string(round(100.0 *
tech_sd * tech_sd / (total_sd * total_sd), 1)) + "%")

# Averaging technical replicates reduces tech_sd but NOT bio_sd
# With k technical reps per biological sample:
# SD of average = sqrt(bio_sd^2 + tech_sd^2/k)
each([1, 2, 3, 5, 10], |k| {
  let avg_sd = sqrt(bio_sd * bio_sd + tech_sd * tech_sd / k)
  let reduction = round(100.0 * (1.0 - avg_sd / total_sd), 1)
  println("  k=" + to_string(k) + " tech reps: SD=" + to_string(round(avg_sd,
3)) + " (" + to_string(reduction) + "% reduction)")
})

println("")
println("Conclusion: doubling technical reps barely helps because")
println("biology dominates the variance. Spend the budget on more")
println("biological replicates instead.")
```

```
# Compare the two allocation strategies
# Budget: 12 libraries total

# Strategy A: 6 bio reps x 1 tech rep per group = n=6
let n_bio_a = 6
let sd_a = total_sd
let d_a = 1.0 / sd_a
let delta_a = d_a * sqrt(n_bio_a / 2.0)
let power_a = pnorm(delta_a - z_alpha, 0.0, 1.0)

# Strategy B: 3 bio reps x 2 tech reps per group = n=3 (with averaged tech
reps)
let n_bio_b = 3
let sd_b = sqrt(bio_sd * bio_sd + tech_sd * tech_sd / 2.0)
let d_b = 1.0 / sd_b
let delta_b = d_b * sqrt(n_bio_b / 2.0)
let power_b = pnorm(delta_b - z_alpha, 0.0, 1.0)

println("Comparison for detecting 2-fold change:")
println("Strategy A (6 bio reps, 1 tech rep):")
println("  Effective SD: " + to_string(round(sd_a, 3)))
println("  Power: " + to_string(round(power_a * 100, 1)) + "%")
println("")
println("Strategy B (3 bio reps, 2 tech reps):")
println("  Effective SD: " + to_string(round(sd_b, 3)))
println("  Power: " + to_string(round(power_b * 100, 1)) + "%")
println("")
println("Strategy A has " + to_string(round(power_a * 100 - power_b * 100,
1)) + " percentage points more power")
println("for the SAME total cost (12 libraries)")
```

Paired vs unpaired designs

```
# Paired designs (before/after treatment on the same individual)
# remove inter-individual variability, dramatically increasing power

# In a paired design:
# We test the DIFFERENCES within each pair
# The variance of the difference = 2 * within_individual_variance
# NOT 2 * total_variance (which includes between-individual variance)

let between_ind_sd = 2.0  # large variability between individuals
let within_ind_sd = 0.5   # small variability within individuals (over time)
let unpaired_sd = sqrt(between_ind_sd * between_ind_sd + within_ind_sd *
within_ind_sd)
let paired_diff_sd = sqrt(2.0) * within_ind_sd  # SD of paired differences

println("Paired vs Unpaired design:")
println("  Between-individual SD: " + to_string(between_ind_sd))
println("  Within-individual SD: " + to_string(within_ind_sd))
println("  Unpaired test SD: " + to_string(round(unpaired_sd, 3)))
println("  Paired test SD (of differences): " +
to_string(round(paired_diff_sd, 3)))
println("")

# Power comparison
let target = 1.0  # detect log2FC = 1
each([5, 8, 10, 15, 20], |n| {
  let d_unpaired = target / unpaired_sd
  let d_paired = target / paired_diff_sd

  let delta_unp = d_unpaired * sqrt(n / 2.0)
  let delta_pr = d_paired * sqrt(n)  # paired test uses n, not n/2

  let power_unp = pnorm(delta_unp - z_alpha, 0.0, 1.0)
  let power_pr = pnorm(delta_pr - z_alpha, 0.0, 1.0)

  println("  n=" + to_string(n) + " per group: Unpaired=" +
to_string(round(power_unp * 100, 1)) + "%, Paired=" +
to_string(round(power_pr * 100, 1)) + "%")
})

println("")
println("Paired designs are dramatically more powerful when")
println("between-individual variability is high")
```

Blocking and randomization

```
# Blocking: grouping experimental units to control known sources
# of variability
#
# Example: processing RNA-seq libraries in two batches
# BAD: all treated in batch 1, all control in batch 2
#   -> confounds treatment with batch
# GOOD: half treated + half control in each batch
#   -> batch effect is orthogonal to treatment

# Randomization within blocks:
# Within each batch, randomly assign samples to conditions
# This prevents unconscious bias

# Example design for 12 samples (6 treated, 6 control) in 2 batches:
println("Balanced block design:")
println("")
println("Batch 1:")
println("  Treated:  T1, T2, T3")
println("  Control:  C1, C2, C3")
println("")
println("Batch 2:")
println("  Treated:  T4, T5, T6")
println("  Control:  C4, C5, C6")
println("")
println("Each batch has equal numbers of treated and control")
println("Batch effect cancels out in the treatment comparison")

# The variance reduction from blocking:
# Total variance = treatment_variance + batch_variance + error_variance
# With blocking: we REMOVE batch_variance from the error term
# Without blocking: batch_variance inflates the error term

let batch_sd = 0.8
let error_sd = 1.0
let total_unblocked = sqrt(batch_sd * batch_sd + error_sd * error_sd)
let total_blocked = error_sd

println("")
println("Variance with and without blocking:")
println("  Unblocked SD: " + to_string(round(total_unblocked, 3)))
println("  Blocked SD: " + to_string(round(total_blocked, 3)))
println("  Variance reduction: " + to_string(round(100.0 * (1.0 -
total_blocked * total_blocked / (total_unblocked * total_unblocked)), 1)) +
"%")
```

Pseudoreplication: the most common design mistake

```
# Pseudoreplication occurs when non-independent observations are
# treated as independent replicates
#
# Common examples in genomics:
#
# 1. CELLS from the same individual treated as biological replicates
#    If you grow 3 wells from the same cell line and call them n=3,
#    you have pseudoreplication. The true n=1 (one cell line).
#
# 2. MULTIPLE measurements per mouse
#    Taking 3 tissue samples from 1 mouse does not give n=3.
#    It gives n=1 with 3 technical replicates.
#
# 3. SINGLE-CELL data
#    1000 cells from one donor is NOT n=1000 for between-donor
#    comparisons. It is n=1 with 1000 cells per donor.
#
# 4. SEQUENCING LANES
#    Running the same library on 2 lanes gives 2 FASTQ files
#    but n=1. Merge and analyze as a single sample.

println("Pseudoreplication example:")
println("")
println("A study reports: 'n=30 cells per condition'")
println("But all 30 cells came from ONE donor per condition")
println("")
println("Truth: n=1 per condition (one donor)")
println("Claim: n=30 per condition (treating cells as replicates)")
println("")
println("Consequence: variance is underestimated by ~30x")
println("All p-values are far too small")
println("Almost everything appears 'significant'")

# Demonstrate the damage
let donor_mean_treated = 8.0
let donor_mean_control = 8.0  # no real difference
let between_donor_sd = 1.5
let within_donor_sd = 0.3     # cell-to-cell within one donor

# Correct analysis: compare donor means (n=1 vs n=1)
# No test is possible with n=1 per group!

# Pseudoreplicated analysis: treat cells as independent
let pseudo_treated = rnorm(30, donor_mean_treated, within_donor_sd)
let pseudo_control = rnorm(30, donor_mean_control, within_donor_sd)
let pseudo_result = ttest(pseudo_treated, pseudo_control)
println("")
println("Pseudoreplicated t-test (WRONG):")
println("  p-value: " + to_string(round(pseudo_result.p_value, 4)))
println("  This p-value is meaningless because cells are not independent")
```

Power for variant detection

```
# Different question: How many samples do I need to detect a variant
# at a given allele frequency?

# For a rare variant with frequency f in n individuals:
# The number of copies follows Binomial(2n, f) for diploid organisms
# We need at least 1 copy to detect the variant
# P(detect) = 1 - P(0 copies) = 1 - (1-f)^(2n)

let frequencies = [0.01, 0.005, 0.001, 0.0005, 0.0001]
let cohort_sizes = [100, 500, 1000, 5000, 10000, 50000, 100000]

println("Probability of observing at least one copy of a variant:")
println("(by allele frequency and cohort size)")
println("")
println("Cohort   | f=1%    f=0.5%  f=0.1%  f=0.05% f=0.01%")
println("---------|-------------------------------------------")

each(cohort_sizes, |n| {
  let probs = map(frequencies, |f| {
    let p_detect = 1.0 - pow(1.0 - f, 2.0 * n)
    to_string(round(p_detect * 100, 1)) + "%"
  })
  println("  " + to_string(n) + " | " + join(probs, "  "))
})
```

Using gnomAD constraint data for design

```
# gnomAD constraint metrics tell us which genes are intolerant
# to loss-of-function (LoF) mutations
# This is relevant for study design: constrained genes have fewer
# variants, requiring larger samples to assess

# Explore the constraint data
println("gnomAD constraint metrics columns:")
each(head(columns(gnomad), 20), |c| println("  " + c))

head(gnomad, 5)

# Key metrics:
# oe_lof: observed/expected ratio for LoF variants
# pLI: probability of being LoF-intolerant
# LOEUF: LoF observed/expected upper bound fraction (95% CI upper bound)

# A gene with LOEUF < 0.35 is considered LoF-constrained
# These genes are likely to be haploinsufficient
```

```
# Analyze the distribution of constraint metrics
let loeuf_vals = col_values(gnomad, "oe_lof_upper")
let loeuf_clean = filter(loeuf_vals, |x| !is_none(x))
let loeuf_float = map(loeuf_clean, |x| to_float(x))
let loeuf_valid = filter(loeuf_float, |x| x > 0.0 && x < 10.0)

println("LOEUF distribution (n=" + to_string(len(loeuf_valid)) + " genes):")
println("  Mean: " + to_string(round(mean(loeuf_valid), 3)))
println("  Median: " + to_string(round(median(loeuf_valid), 3)))
println("  Q10: " + to_string(round(quantile(loeuf_valid, 0.10), 3)))
println("  Q90: " + to_string(round(quantile(loeuf_valid, 0.90), 3)))

let constrained = filter(loeuf_valid, |x| x < 0.35)
let unconstrained = filter(loeuf_valid, |x| x >= 0.35)
println("  Constrained (LOEUF < 0.35): " + to_string(len(constrained)) + "
genes")
println("  Unconstrained (LOEUF >= 0.35): " + to_string(len(unconstrained)) +
" genes")

histogram(loeuf_valid, "plots/loeuf_distribution.svg")
  |> save_plot("plots/loeuf_distribution.svg")
```

```
# For rare disease studies: if your candidate gene is LoF-constrained,
# you need fewer affected individuals to be confident that a de novo
# LoF variant is pathogenic (because such variants are depleted in
# the general population)

# Expected number of de novo LoF variants per gene per generation:
# Approximately 1.5e-8 per nucleotide per generation
# For a gene with ~1000 coding nucleotides: ~1.5e-5 de novo LoFs expected

let mutation_rate_per_bp = 1.5e-8
let coding_length = 1000.0
let de_novo_rate = mutation_rate_per_bp * coding_length

println("De novo LoF variant rate per gene per generation:")
println("  " + to_string(de_novo_rate))

# In a cohort of N trios (affected child + parents):
# Expected de novo LoF variants in gene G = N * de_novo_rate
# Finding >= 2 de novo LoF variants in the same gene is significant
# if the gene is constrained

each([100, 500, 1000, 5000, 10000], |n_trios| {
  let expected = n_trios * de_novo_rate
  let p_ge_2 = 1.0 - ppois(1, expected)
  println("  " + to_string(n_trios) + " trios: E[de novo]=" +
to_string(round(expected, 4)) + ", P(>=2)=" + to_string(round(p_ge_2, 6)))
})
```

GWAS power calculation

```
# For GWAS, the effect sizes are much smaller than RNA-seq
# A typical odds ratio for a common variant is 1.1-1.3
# This requires thousands to hundreds of thousands of samples

# Power for a GWAS association test (simplified):
# Uses a chi-square approximation

# Key parameters:
# - Minor allele frequency (MAF)
# - Effect size (odds ratio or beta)
# - Sample size
# - Alpha (typically 5e-8 for genome-wide significance)

let gwas_alpha = 5e-8  # genome-wide significance threshold
let z_gwas = qnorm(1.0 - gwas_alpha / 2.0, 0.0, 1.0)

println("GWAS power analysis:")
println("Genome-wide significance threshold: p < " + to_string(gwas_alpha))
println("Required Z-score: " + to_string(round(z_gwas, 2)))
println("")

# For a quantitative trait:
# Power depends on variance explained (R^2)
# R^2 = 2 * MAF * (1-MAF) * beta^2

let mafs = [0.05, 0.10, 0.20, 0.30, 0.40]
let betas = [0.05, 0.10, 0.20]  # effect size in SD units

println("Minimum sample size for 80% power in GWAS:")
println("(quantitative trait, genome-wide significance)")
println("")

each(betas, |beta| {
  println("Effect size beta = " + to_string(beta) + " SD:")
  each(mafs, |maf| {
    let r2 = 2.0 * maf * (1.0 - maf) * beta * beta
    # Approximate n for 80% power:
    # n approximately = (z_alpha + z_beta)^2 / r2
    # where z_beta = 0.842 for 80% power
    let z_beta = 0.842
    let n_needed = ceil((z_gwas + z_beta) * (z_gwas + z_beta) / r2)
    println("  MAF=" + to_string(maf) + ": n=" + to_string(round(n_needed,
0)))
  })
  println("")
})
```

Interpretation

The power analyses in this chapter reveal several important truths:

1. n = 3 gives 20-30% power for moderate effects. This means you miss 70-80% of real biology. For RNA-seq, n = 6 per group is a practical minimum for discovery; n = 12 is needed for robust detection of 2-fold changes.

2. Biological replicates dominate technical replicates. When biological variance accounts for 95%+ of total variance (as is typical), adding technical replicates is almost worthless. Always maximize biological replicates.

3. Paired designs can be dramatically more powerful. When between-individual variability is high (which it usually is), a paired design effectively removes it, often doubling or tripling the effective sample size.

4. GWAS requires enormous samples. Effect sizes for common variants are tiny (explaining < 1% of variance per variant), requiring thousands to hundreds of thousands of samples for genome-wide significance.

5. Power is not a property of the study design alone. It depends on the specific gene's variance, the effect size for that gene, and the significance threshold. Some genes are always easy to detect; others are always hard.

Decision

```
# Decision framework for experimental design

# Step 1: Define your primary question
# "Which genes are DE between treated and control?"
# "Is there a GWAS signal for trait X at locus Y?"
# "Does this drug reduce tumor growth?"

# Step 2: Estimate effect size
# From pilot data, literature, or biological reasoning
# Be conservative — real effects are usually smaller than pilot data suggests

# Step 3: Estimate variance
# From pilot data (use median across genes for RNA-seq)
# Or from publicly available data (GTEx, ENCODE, GEO)

# Step 4: Choose your power target
# Standard: 80% (you detect 4 out of 5 real effects)
# High: 90% (preferable for confirmatory studies)
# Minimum: 60% (only acceptable for very exploratory work)

# Step 5: Calculate required sample size
# Use the formulas from this chapter
# Or simulation for complex designs

# Step 6: Can you afford it?
# If yes: proceed
# If no: consider:
#   a) Reducing the number of conditions
#   b) Using a paired design (if possible)
#   c) Pooling with collaborators
#   d) Using a more sensitive assay
#   e) Accepting lower power (document this limitation)
#   f) Focusing on larger effect sizes

# Step 7: Randomize and block
# Assign samples to conditions randomly
# Balance batches across conditions
# Document the randomization procedure

# Step 8: Pre-register your analysis plan
# Specify the statistical test, significance threshold, and
# effect size threshold BEFORE looking at the data
# This prevents p-hacking and HARKing

println("Sample size recommendations summary:")
println("")
println("| Study Type              | Min n/group | Target n/group | Notes
|")
println("|-------------------------|-------------|----------------|-----------
--------------------|")
println("| RNA-seq discovery       | 3           | 6-12           | Use
empirical Bayes methods   |")
println("| RNA-seq confirmation    | 6           | 12-20          | Pre-
registered analysis       |")
println("| ATAC-seq                | 2           | 3-5            | Lower
variance than RNA-seq   |")
println("| ChIP-seq                | 2           | 3-5            | Need input
controls           |")
println("| GWAS (common variant)   | 1000        | 10000+         | Effect
sizes are tiny         |")
println("| scRNA-seq               | 3 donors    | 6+ donors      | Cells are
NOT replicates      |")
```

```
println("| Rare variant burden    | 500         | 5000+          | Depends on
gene size          |")
println("| Clinical trial         | 30          | 100+           | Depends on
endpoint variance  |")
```

Sample size calculation template

```
# Template for RNA-seq power calculation
# Fill in your own values

# YOUR PARAMETERS (modify these)
let my_target_lfc = 1.0       # minimum log2 fold change to detect
let my_sd = 1.2               # estimated within-group SD (from pilot)
let my_alpha = 0.05           # significance level
let my_power_target = 0.80    # desired power

# CALCULATION
let my_z_alpha = qnorm(1.0 - my_alpha / 2.0, 0.0, 1.0)
let my_z_power = qnorm(my_power_target, 0.0, 1.0)
let my_d = my_target_lfc / my_sd

# Sample size per group (approximate formula)
let n_needed = ceil(2.0 * pow((my_z_alpha + my_z_power) / my_d, 2.0))

println("=== RNA-seq Power Calculation ===")
println("Target log2FC: " + to_string(my_target_lfc))
println("Within-group SD: " + to_string(my_sd))
println("Cohen's d: " + to_string(round(my_d, 3)))
println("Alpha: " + to_string(my_alpha))
println("Target power: " + to_string(my_power_target))
println("")
println("REQUIRED SAMPLE SIZE: n = " + to_string(round(n_needed, 0)) + " per
group")
println("TOTAL LIBRARIES: " + to_string(round(2.0 * n_needed, 0)))
println("")

# Verify with power calculation
let verify_delta = my_d * sqrt(n_needed / 2.0)
let verify_power = pnorm(verify_delta - my_z_alpha, 0.0, 1.0)
println("Verification: power at n=" + to_string(round(n_needed, 0)) + " is "
+ to_string(round(verify_power * 100, 1)) + "%")
```

Pitfalls

Pitfall 1: Using pilot effect sizes for power calculation

```
# Pilot studies overestimate effect sizes because only
# "significant" results from pilots are used for planning
# This is the "winner's curse" applied to study design
#
# If your pilot found log2FC = 2.5 for your gene of interest,
# the true effect is probably closer to 1.5-2.0
#
# Solution: use the LOWER BOUND of the confidence interval
# for effect size from the pilot, not the point estimate

let pilot_lfc = 2.5
let pilot_se = 0.8  # standard error of the pilot estimate

# The 80% CI lower bound is a better estimate for power calculation
let conservative_lfc = pilot_lfc - 1.28 * pilot_se  # 80% CI lower bound
println("Pilot log2FC: " + to_string(pilot_lfc))
println("Conservative estimate (80% CI lower): " +
to_string(round(conservative_lfc, 2)))
println("Use " + to_string(round(conservative_lfc, 2)) + " for power
calculation, not " + to_string(pilot_lfc))
```

Pitfall 2: Ignoring multiple testing in power calculations

```
# If you test 20,000 genes, your per-test alpha is NOT 0.05
# After BH correction, the effective per-test alpha is approximately
# alpha * (number of true positives / total tests)
#
# For a rough correction: use alpha / (number of tests) as your
# per-test alpha in the power calculation (Bonferroni approximation)
# This is conservative but gives a floor for the required n

let n_tests = 20000
let bonf_alpha = 0.05 / n_tests
let z_bonf = qnorm(1.0 - bonf_alpha / 2.0, 0.0, 1.0)

println("Single test alpha: 0.05")
println("Bonferroni-corrected alpha: " + to_string(bonf_alpha))
println("Required Z-score: " + to_string(round(z_bonf, 2)) + " (vs 1.96 for
single test)")

# This substantially increases the required sample size
let d_example = 1.0
let z_power = 0.842

let n_single = ceil(2.0 * pow((1.96 + z_power) / d_example, 2.0))
let n_bonf = ceil(2.0 * pow((z_bonf + z_power) / d_example, 2.0))

println("")
println("Required n per group for d=1.0:")
println("  Single test: " + to_string(round(n_single, 0)))
println("  After Bonferroni for 20000 tests: " + to_string(round(n_bonf, 0)))
println("  (BH correction requires somewhat less than Bonferroni)")
```

Pitfall 3: Pseudoreplication in single-cell studies

```
# Single-cell RNA-seq creates a massive pseudoreplication temptation
# You might have 10,000 cells per condition
# But if all cells come from one donor per condition, n = 1
#
# The correct approach:
# 1. Summarize per-donor (average or pseudobulk)
# 2. Test between donors (the biological replicates)
# 3. Report the number of DONORS, not cells
#
# The cells provide information about WITHIN-DONOR heterogeneity
# The donors provide information about BETWEEN-DONOR differences
# Only the latter matters for generalizing to the population

println("Single-cell study design:")
println("")
println("WRONG: 2 donors x 5000 cells = '10000 replicates'")
println("RIGHT: 2 donors x 5000 cells = '2 biological replicates'")
println("")
println("To achieve n=6 per group in a scRNA-seq experiment,")
println("you need 6 DONORS per group (12 donors total)")
println("The number of cells per donor affects resolution")
println("but NOT the sample size for between-group comparisons")
```

Pitfall 4: Post-hoc power analysis

```
# NEVER compute "observed power" after seeing the results
# This is circular and provides no useful information
#
# If you found p = 0.06 and compute observed power = 48%,
# this does NOT mean "you almost had enough power"
# The observed power is a deterministic function of the p-value
# It tells you nothing beyond what the p-value already tells you
#
# The right time for power analysis is BEFORE the experiment
# After the experiment, report effect sizes with confidence intervals
# instead of power

println("Post-hoc power analysis is NEVER appropriate")
println("")
println("Before experiment: Power analysis guides design")
println("After experiment: Report effect sizes and CIs")
println("")
println("The observed power is:")
println("  High when p is small (you already know this from p)")
println("  Low when p is large (you already know this from p)")
println("  It adds no new information")
```

Pitfall 5: Confusing precision with power

```
# More data gives you more PRECISION (narrower confidence intervals)
# AND more POWER (probability of detecting an effect)
# These are related but not the same
#
# You might have enough power to detect a 2-fold change
# but not enough precision to estimate the fold change accurately
#
# For estimation, the CI width matters:
# CI width approximately = 2 * z * SD / sqrt(n)

let target_precision = 0.5  # want CI width < 0.5 log2FC units
let example_sd = 1.2

# Required n for this precision
let n_for_precision = ceil(pow(2.0 * 1.96 * example_sd / target_precision,
2.0))
println("Required n for CI width < " + to_string(target_precision) + "
log2FC:")
println("  n = " + to_string(round(n_for_precision, 0)) + " per group")
println("")
println("This may be MORE than needed for 80% power")
println("Design for the more demanding requirement")
```

Where Next?

This chapter has given you the tools to design experiments that can actually answer your questions. The key principles:

1. **Calculate power before the experiment.** Use real variance estimates from pilot data or public datasets. Be conservative about effect sizes.
2. **Maximize biological replicates.** Technical replicates provide negligible benefit when biological variance dominates (which it always does).
3. **Consider paired designs.** If you can measure the same individual before and after treatment, the power gain is enormous.
4. **Block known confounders.** Balance batches, sexes, ages, and other covariates across conditions.
5. **Avoid pseudoreplication.** Cells are not donors. Lanes are not libraries. Technical replicates are not biological replicates.
6. **Adjust for multiple testing.** Genome-wide studies require either larger samples or acceptance of lower power per gene.
7. **Pre-register your analysis.** Decide your statistical approach before looking at the data.

With solid foundations in statistical thinking (Chapter 1), exploratory analysis (Chapter 2), distributional knowledge (Chapter 3), and experimental design (this chapter), you are equipped to design, execute, and analyze genomics experiments that produce reliable, reproducible results.

Exercises

```
# Exercise 1: Using the GSE183947 variance estimates, calculate the
# sample size needed to detect a 1.5-fold change (log2FC = 0.585)
# with 80% power for the median-variance gene.

# Exercise 2: A lab has 24 library prep slots. They want to compare
# 3 drug doses vs control. Design the experiment (allocation of
# samples to conditions and batches). Calculate power for each
# pairwise comparison.

# Exercise 3: You have a paired design (before/after treatment)
# with within-individual SD = 0.4 and between-individual SD = 2.0.
# How many individuals do you need for 90% power to detect
# log2FC = 0.5?

# Exercise 4: Using the gnomAD constraint data, find the 10 most
# constrained genes (lowest LOEUF). For each, estimate the sample
# size needed to observe at least 2 de novo LoF variants by chance
# (assuming the neutral mutation rate).

# Exercise 5: A GWAS plans to test 1 million SNPs for association
# with a quantitative trait. Using genome-wide significance
# (p < 5e-8), calculate the required sample size to detect a
# variant explaining 0.1% of trait variance with 80% power.

# Exercise 6: Repeat the biological vs technical replicate comparison
# but now assume technical variance is 50% of biological variance
# (a more noisy assay). How does the optimal allocation change?

# Exercise 7: Design a scRNA-seq experiment to compare gene expression
# between patients with disease A vs healthy controls. You want to
# detect log2FC = 0.8 with 80% power. The between-patient SD is 1.0.
# How many patients per group? How many cells per patient is sufficient?
```

Summary formulas

Quantity	Formula	Notes
Cohen’s d	effect / SD	Standardized effect size
Power (approx)	Phi(d * sqrt(n/2) - z_alpha)	Two-sample t-test
n per group	2 * ((z_alpha + z_beta) / d)^2	For target power
Paired power	Phi(d * sqrt(n) - z_alpha)	d uses paired SD
GWAS n	(z_alpha + z_beta)^2 / R^2	R^2 = variance explained
Variant detection	1 - (1-f)^(2n)	f = allele frequency
CI width	2 * z * SD / sqrt(n)	For estimation precision

Comparing Two Groups

> *"The difference between a significant result and a non-significant result is not itself statistically significant."* — Andrew Gelman & Hal Stern

You have gene expression measurements from tumor tissue and normal tissue. The question seems simple: which genes are different? The answer is anything but.

This chapter teaches you to compare two groups properly — with the right test, the right assumptions check, the right effect size measure, and the right visualization. We will work with real breast cancer expression data and discover that p-values alone are dangerously incomplete.

The Problem

A breast cancer researcher has RNA-seq expression data from tumor and adjacent normal tissue. For each of ~20,000 genes, she has FPKM values in both conditions. She wants to identify genes that are differentially expressed between tumor and normal.

Her first instinct: run a t-test on every gene, keep the ones with $p < 0.05$.

This approach has at least five problems. By the end of this chapter, you will understand all of them and know how to do it correctly.

Why It Matters

Comparing two groups is the most common statistical task in biology. Tumor vs normal. Treated vs untreated. Responder vs non-responder. Wild-type vs knockout. Case vs control.

Getting this comparison wrong has real consequences:

- **False positives** waste years of follow-up experiments on genes that are not truly different

- **False negatives** miss real biological signals — the drug target that was actually there
- **Ignoring effect size** means you chase genes with tiny differences that are statistically significant only because of large sample size
- **Ignoring assumptions** means the p-values you report are not calibrated — they do not mean what you think they mean

The t-test was invented in 1908 by William Sealy Gosset (publishing under the pseudonym “Student”) while working at the Guinness Brewery. He needed to compare small samples of barley yields. Over a century later, biologists still misuse his test daily.

The Data

We use GEO dataset GSE183947, which contains FPKM expression values from breast cancer (tumor) and adjacent normal tissue samples. This is a real RNA-seq dataset with paired tumor-normal samples.

Downloading the Data

```
# Download the breast cancer expression dataset
let url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let raw_file = download(url, "GSE183947_fpkm.csv.gz")
println("Downloaded: " + raw_file)
```

Loading and Inspecting

```
# Load the expression matrix
let expr = read_csv("GSE183947_fpkm.csv.gz")

# Basic dimensions
println("Rows: " + to_string(len(expr)))
println("Columns: " + to_string(ncols(expr)))
println("Column names:")
let cols = columns(expr)
cols |> each(|c| println("  " + c))
```

```
# Preview the first few rows
take(expr, 5)
```

The data has genes in rows and samples in columns. Tumor samples and normal samples are identified by their column names. We need to separate them.

Preparing Tumor vs Normal Groups

```
# Identify tumor and normal sample columns
# In GSE183947, tumor samples contain "tumor" and normal contain "normal"
let all_cols = columns(expr)

# Separate sample columns (skip the gene ID column)
let tumor_cols = all_cols |> filter(|c| contains(c, "tumor"))
let normal_cols = all_cols |> filter(|c| contains(c, "normal"))

println("Tumor samples: " + to_string(len(tumor_cols)))
println("Normal samples: " + to_string(len(normal_cols)))
```

```
# Extract expression values for a single gene to demonstrate
# Let us start with TP53 — a well-known tumor suppressor
let tp53_row = expr |> filter_by("gene", |g| contains(g, "TP53"))

let tp53_tumor = tumor_cols |> map(|c| col_values(tp53_row, c)) |> flatten()
let tp53_normal = normal_cols |> map(|c| col_values(tp53_row, c)) |>
flatten()

println("TP53 tumor values: " + to_string(tp53_tumor))
println("TP53 normal values: " + to_string(tp53_normal))
```

Exploration

Before running any test, **look at your data**. This is not optional. Skipping exploration is the single most common source of wrong conclusions in genomics.

Summary Statistics

```
# Descriptive statistics for TP53 in both groups
println("=== TP53 Expression ===")
println("Tumor:")
println("  n     = " + to_string(len(tp53_tumor)))
println("  mean  = " + to_string(round(mean(tp53_tumor), 3)))
println("  median= " + to_string(round(median(tp53_tumor), 3)))
println("  sd    = " + to_string(round(stdev(tp53_tumor), 3)))
println("  min   = " + to_string(round(min(tp53_tumor), 3)))
println("  max   = " + to_string(round(max(tp53_tumor), 3)))

println("Normal:")
println("  n     = " + to_string(len(tp53_normal)))
println("  mean  = " + to_string(round(mean(tp53_normal), 3)))
println("  median= " + to_string(round(median(tp53_normal), 3)))
println("  sd    = " + to_string(round(stdev(tp53_normal), 3)))
println("  min   = " + to_string(round(min(tp53_normal), 3)))
println("  max   = " + to_string(round(max(tp53_normal), 3)))
```

Already you can see something important: are the standard deviations similar? If one group is much more variable than the other, we need Welch's t-test, not Student's t-test.

Visualizing the Distributions

```
# Histogram of TP53 expression in both groups
histogram(tp53_tumor, "TP53 Tumor Expression (FPKM)")
  |> save_plot("tp53_tumor_hist.svg")

histogram(tp53_normal, "TP53 Normal Expression (FPKM)")
  |> save_plot("tp53_normal_hist.svg")
```

```
# Side-by-side boxplot
let tp53_data = table(
  "expression", concat(tp53_tumor, tp53_normal),
  "group", concat(
    tp53_tumor |> map(|_| "Tumor"),
    tp53_normal |> map(|_| "Normal")
  )
)

boxplot(tp53_data, "group", "expression")
  |> save_plot("tp53_boxplot.svg")
```

The Log-Transform Question

Gene expression data is almost always right-skewed — a few genes have very high expression, most are moderate or low. This violates the normality assumption of the t-test.

```
# Raw expression is typically right-skewed
# Check skewness by comparing mean vs median
let skew_ratio = mean(tp53_tumor) / median(tp53_tumor)
println("Mean/median ratio (tumor): " + to_string(round(skew_ratio, 3)))
# If ratio >> 1, the data is right-skewed
```

The standard solution is to log-transform:

```
# Log2-transform with pseudocount to avoid log(0)
let log_tp53_tumor = tp53_tumor |> map(|x| log2(x + 1.0))
let log_tp53_normal = tp53_normal |> map(|x| log2(x + 1.0))

println("Log2 tumor mean:   " + to_string(round(mean(log_tp53_tumor), 3)))
println("Log2 tumor median: " + to_string(round(median(log_tp53_tumor), 3)))
println("Log2 normal mean:  " + to_string(round(mean(log_tp53_normal), 3)))
println("Log2 normal median:" + to_string(round(median(log_tp53_normal), 3)))
```

```
# QQ plot to check normality after log-transform
qq_plot(log_tp53_tumor, "TP53 Tumor (log2)")
  |> save_plot("tp53_tumor_qq.svg")

qq_plot(log_tp53_normal, "TP53 Normal (log2)")
  |> save_plot("tp53_normal_qq.svg")
```

A QQ plot plots the quantiles of your data against the quantiles of a normal distribution. If the points fall on a straight line, the data is approximately normal. Deviations at the tails indicate heavy or light tails.

Rule of thumb: Log-transformed expression data is usually close enough to normal for the t-test. Always check with a QQ plot.

The Analysis

Step 1: Student's t-Test — The Classic

Student's t-test assumes:

1. Both groups are drawn from normal distributions
2. Both groups have **equal variance** (homoscedasticity)
3. Observations are independent

```
# Student's t-test on log-transformed TP53
let result = ttest(log_tp53_tumor, log_tp53_normal)
println("=== Student's t-test (TP53 log2-FPKM) ===")
println("t-statistic: " + to_string(round(result.statistic, 4)))
println("p-value:     " + to_string(result.p_value))
println("df:          " + to_string(round(result.df, 1)))
```

But wait — did we check the equal variance assumption?

```
# Check variance ratio (F-ratio)
let var_tumor = variance(log_tp53_tumor)
let var_normal = variance(log_tp53_normal)
let f_ratio = var_tumor / var_normal
println("Variance tumor:  " + to_string(round(var_tumor, 4)))
println("Variance normal: " + to_string(round(var_normal, 4)))
println("F-ratio:         " + to_string(round(f_ratio, 4)))
# If F-ratio is far from 1 (say > 2 or < 0.5), variances are unequal
```

Step 2: Welch's t-Test — The Safer Choice

Welch's t-test does **not** assume equal variance. It adjusts the degrees of freedom based on the sample variances. In practice, Welch's test is almost always preferred because:

- When variances are equal, Welch's gives nearly identical results to Student's
- When variances are unequal, Student's gives wrong p-values but Welch's is correct
- There is almost no cost to using Welch's and a real cost to using Student's incorrectly

```
# Welch's t-test (default in most modern software)
# In BioLang, ttest performs Welch's by default
let welch = ttest(log_tp53_tumor, log_tp53_normal)
println("=== Welch's t-test (TP53 log2-FPKM) ===")
println("t-statistic: " + to_string(round(welch.statistic, 4)))
println("p-value:     " + to_string(welch.p_value))
println("df:          " + to_string(round(welch.df, 1)))
```

Key insight: The degrees of freedom in Welch's test are often not an integer. This is correct — it is the Welch-Satterthwaite approximation. If you see df = 14.7 instead of df = 18, that is Welch's adjustment working.

Step 3: When to Use Paired Tests

If each tumor sample has a matched normal sample from the same patient, you should use a **paired** t-test. Pairing removes patient-to-patient variability and dramatically increases statistical power.

```
# If samples are paired (tumor-normal from same patient):
# The paired test operates on DIFFERENCES within each pair
let differences = zip(log_tp53_tumor, log_tp53_normal)
  |> map(|pair| pair[0] - pair[1])

println("Mean of differences: " + to_string(round(mean(differences), 4)))
println("SD of differences:   " + to_string(round(stdev(differences), 4)))

# The paired t-test tests whether the mean difference is zero
let paired_result = ttest_paired(log_tp53_tumor, log_tp53_normal)
println("=== Paired t-test (TP53 log2-FPKM) ===")
println("t-statistic: " + to_string(round(paired_result.statistic, 4)))
println("p-value:     " + to_string(paired_result.p_value))
```

When to pair: If the samples are naturally matched (same patient, same litter, same batch processed together), ALWAYS use the paired test. The unpaired test ignores this information and throws away statistical power.

When NOT to pair: If the samples are independent (different patients, no natural matching), the paired test is inappropriate and can give wrong results.

Step 4: Effect Size — Cohen's d

A p-value tells you whether a difference is statistically distinguishable from zero. It does NOT tell you how large the difference is. With a large enough sample size, even a biologically meaningless 0.01-FPKM difference becomes "statistically significant."

Cohen's d measures effect size in units of standard deviation:

```
# Cohen's d for independent samples
# d = (mean1 - mean2) / pooled_sd
let mean_diff = mean(log_tp53_tumor) - mean(log_tp53_normal)
let n1 = to_float(len(log_tp53_tumor))
let n2 = to_float(len(log_tp53_normal))
let pooled_var = ((n1 - 1.0) * variance(log_tp53_tumor) + (n2 - 1.0) *
variance(log_tp53_normal)) / (n1 + n2 - 2.0)
let pooled_sd = sqrt(pooled_var)
let cohens_d = mean_diff / pooled_sd

println("=== Effect Size ===")
println("Mean difference (log2): " + to_string(round(mean_diff, 4)))
println("Pooled SD:              " + to_string(round(pooled_sd, 4)))
println("Cohen's d:              " + to_string(round(cohens_d, 4)))
```

Interpreting Cohen's d:

- |d| < 0.2: negligible effect
- |d| ~ 0.2: small effect
- |d| ~ 0.5: medium effect
- |d| ~ 0.8: large effect
- |d| > 1.0: very large effect

```
# Effect size interpretation
let d_abs = abs(cohens_d)
let interpretation = if d_abs < 0.2 then "negligible"
  else if d_abs < 0.5 then "small"
  else if d_abs < 0.8 then "medium"
  else "large"

println("Effect size interpretation: " + interpretation)
```

Critical point: A gene with p = 0.001 and d = 0.1 is probably not biologically interesting. A gene with p = 0.06 and d = 1.2 probably is. Report both.

Step 5: Confidence Intervals

A confidence interval gives you a range of plausible values for the true difference. Unlike a p-value, it tells you about the magnitude of the effect.

```
# 95% confidence interval for the difference in means
let se_diff = sqrt(variance(log_tp53_tumor) / n1 + variance(log_tp53_normal)
/ n2)
let t_crit = qnorm(0.975)  # approximate for large samples
let ci_lower = mean_diff - t_crit * se_diff
let ci_upper = mean_diff + t_crit * se_diff

println("=== 95% Confidence Interval ===")
println("Difference in means (log2): " + to_string(round(mean_diff, 4)))
println("SE of difference:           " + to_string(round(se_diff, 4)))
println("95% CI: [" + to_string(round(ci_lower, 4)) + ", " +
to_string(round(ci_upper, 4)) + "]")
```

If the confidence interval contains zero, the difference is not statistically significant at the 0.05 level. But notice how much MORE information the CI provides compared to a binary yes/no p-value.

Step 6: The Fold Change

In genomics, the fold change is the ratio of expression between conditions. On log2-transformed data, the fold change is simply the difference:

```
# Log2 fold change
let log2fc = mean(log_tp53_tumor) - mean(log_tp53_normal)
println("log2 fold change: " + to_string(round(log2fc, 4)))

# Convert back to fold change
let fc = pow(2.0, abs(log2fc))
let direction = if log2fc > 0.0 then "upregulated in tumor" else
"downregulated in tumor"
println("Fold change: " + to_string(round(fc, 2)) + "x " + direction)
```

Step 7: Genome-Wide Analysis

Now let us scale up to all genes. This is where things get interesting.

```
# Run t-tests for every gene across the expression matrix
let gene_col = columns(expr) |> take(1)  # first column is gene ID
let genes = col_values(expr, gene_col[0])

# Build results table gene by gene
let results = genes |> map(|gene| {
  let row = expr |> filter_by(gene_col[0], |g| g == gene)

  # Extract tumor and normal values
  let tumor_vals = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
  let normal_vals = normal_cols |> map(|c| col_values(row, c)) |> flatten()

  # Log2 transform
  let log_tumor = tumor_vals |> map(|x| log2(x + 1.0))
  let log_normal = normal_vals |> map(|x| log2(x + 1.0))

  # Calculate statistics
  let log2fc = mean(log_tumor) - mean(log_normal)
  let test_result = ttest(log_tumor, log_normal)

  # Cohen's d
  let n_t = to_float(len(log_tumor))
  let n_n = to_float(len(log_normal))
  let pvar = ((n_t - 1.0) * variance(log_tumor) + (n_n - 1.0) *
variance(log_normal)) / (n_t + n_n - 2.0)
  let d = if pvar > 0.0 then log2fc / sqrt(pvar) else 0.0

  record(
    "gene", gene,
    "log2fc", round(log2fc, 4),
    "t_stat", round(test_result.statistic, 4),
    "p_value", test_result.p_value,
    "cohens_d", round(d, 4),
    "mean_tumor", round(mean(log_tumor), 4),
    "mean_normal", round(mean(log_normal), 4)
  )
}) |> to_list()

let results_table = table(results)
println("Tested " + to_string(len(results_table)) + " genes")
```

```
# Sort by p-value to see top hits
let top_genes = results_table
  |> arrange("p_value")
  |> take(20)

println("=== Top 20 Genes by P-value ===")
top_genes
```

Step 8: The Volcano Plot

The volcano plot is the signature visualization of differential expression analysis. It plots log2 fold change (x-axis) against -log10 p-value (y-axis). Genes that are both statistically significant AND have large effect size appear in the upper corners.

```
# Create the volcano plot
let volcano_data = results_table
  |> mutate("neg_log10_p", |row| -log10(row.p_value + 1e-300))

volcano(volcano_data, "log2fc", "neg_log10_p")
  |> save_plot("breast_cancer_volcano.svg")
```

```
# Annotated volcano with significance thresholds
# Genes with |log2FC| > 1 AND p < 0.05 are "significant"
let sig_up = results_table
  |> filter_by("log2fc", |fc| fc > 1.0)
  |> filter_by("p_value", |p| p < 0.05)

let sig_down = results_table
  |> filter_by("log2fc", |fc| fc < -1.0)
  |> filter_by("p_value", |p| p < 0.05)

println("Upregulated (FC>2, p<0.05):   " + to_string(len(sig_up)))
println("Downregulated (FC<0.5, p<0.05): " + to_string(len(sig_down)))
println("Not significant:                " + to_string(
  len(results_table) - len(sig_up) - len(sig_down)
))
```

Step 9: Mann-Whitney U (Wilcoxon Rank-Sum) as Alternative

When the normality assumption is questionable, the Wilcoxon rank-sum test is a non-parametric alternative. It tests whether one group tends to have larger values than the other, without assuming any particular distribution shape.

```
# Wilcoxon rank-sum test on TP53
let wilcox_result = wilcoxon(log_tp53_tumor, log_tp53_normal)
println("=== Wilcoxon Rank-Sum Test (TP53 log2-FPKM) ===")
println("W statistic: " + to_string(wilcox_result.statistic))
println("p-value:     " + to_string(wilcox_result.p_value))
```

Step 10: Head-to-Head — t-Test vs Wilcoxon

This is the real question: when do parametric and non-parametric tests disagree?

```
# Compare t-test and Wilcoxon across all genes
let comparison = genes |> map(|gene| {
  let row = expr |> filter_by(gene_col[0], |g| g == gene)
  let tumor_vals = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
  let normal_vals = normal_cols |> map(|c| col_values(row, c)) |> flatten()

  let log_tumor = tumor_vals |> map(|x| log2(x + 1.0))
  let log_normal = normal_vals |> map(|x| log2(x + 1.0))

  let t_result = ttest(log_tumor, log_normal)
  let w_result = wilcoxon(log_tumor, log_normal)

  record(
    "gene", gene,
    "p_ttest", t_result.p_value,
    "p_wilcoxon", w_result.p_value
  )
}) |> to_list()

let comp_table = table(comparison)
```

```
# Scatter plot: t-test p-values vs Wilcoxon p-values
let comp_log = comp_table
  |> mutate("log_p_t", |row| -log10(row.p_ttest + 1e-300))
  |> mutate("log_p_w", |row| -log10(row.p_wilcoxon + 1e-300))

scatter(comp_log, "log_p_t", "log_p_w")
  |> save_plot("ttest_vs_wilcoxon.svg")
```

```
# Find genes where the tests disagree
# Significant by t-test but not by Wilcoxon
let t_only = comp_table
  |> filter_by("p_ttest", |p| p < 0.05)
  |> filter_by("p_wilcoxon", |p| p >= 0.05)

# Significant by Wilcoxon but not by t-test
let w_only = comp_table
  |> filter_by("p_wilcoxon", |p| p < 0.05)
  |> filter_by("p_ttest", |p| p >= 0.05)

println("=== Disagreements ===")
println("Significant by t-test only:   " + to_string(len(t_only)))
println("Significant by Wilcoxon only: " + to_string(len(w_only)))
println("Agree on significance:        " + to_string(
  len(comp_table) - len(t_only) - len(w_only)
))
```

Why do they disagree?

The t-test is sensitive to outliers because it uses means and variances. A single extreme value can pull the mean and inflate the variance, changing the p-value substantially.

The Wilcoxon test uses ranks instead of raw values. An outlier that is 10x larger than the next value gets rank n regardless — its magnitude does not matter. This makes it robust to outliers but less powerful when the data truly is normal.

```
# Examine a gene where they disagree
let disagreement_genes = col_values(t_only, "gene")
if len(disagreement_genes) > 0 then {
  let check_gene = disagreement_genes[0]
  let row = expr |> filter_by(gene_col[0], |g| g == check_gene)
  let tumor_vals = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
  let normal_vals = normal_cols |> map(|c| col_values(row, c)) |> flatten()

  let log_tumor = tumor_vals |> map(|x| log2(x + 1.0))
  let log_normal = normal_vals |> map(|x| log2(x + 1.0))

  println("Gene: " + check_gene)
  println("Tumor values (log2): " + to_string(log_tumor))
  println("Normal values (log2): " + to_string(log_normal))

  # Check for outliers — values more than 3 SDs from mean
  let t_mean = mean(log_tumor)
  let t_sd = stdev(log_tumor)
  let outliers = log_tumor |> filter(|x| abs(x - t_mean) > 3.0 * t_sd)
  println("Outliers in tumor: " + to_string(len(outliers)))

  # Box plot to visualize
  let check_data = table(
    "expression", concat(log_tumor, log_normal),
    "group", concat(
      log_tumor |> map(|_| "Tumor"),
      log_normal |> map(|_| "Normal")
    )
  )
  boxplot(check_data, "group", "expression")
    |> save_plot("disagreement_gene_boxplot.svg")
}
```

Step 11: The One-Sample t-Test

Sometimes you want to test whether a single group differs from a known value. For instance: is the average log2 fold change across a pathway significantly different from zero?

```
# One-sample t-test: are the fold changes in a gene set different from zero?
# Let us test a set of known breast cancer genes
let bc_genes = ["BRCA1", "BRCA2", "TP53", "ERBB2", "ESR1", "PGR", "MKI67",
"CDH1"]

let bc_fcs = bc_genes |> map(|gene| {
  let row = results_table |> filter_by("gene", |g| contains(g, gene))
  if len(row) > 0 then col_values(row, "log2fc")[0] else 0.0
}) |> filter(|x| x != 0.0)

println("Fold changes of BC genes: " + to_string(bc_fcs))
println("Mean log2FC: " + to_string(round(mean(bc_fcs), 4)))

# Test: is the mean FC significantly different from 0?
let one_sample = ttest_one(bc_fcs, 0.0)
println("=== One-sample t-test ===")
println("t-statistic: " + to_string(round(one_sample.statistic, 4)))
println("p-value:     " + to_string(one_sample.p_value))
```

Interpretation

What the P-value Actually Means

The p-value is the probability of observing data as extreme as (or more extreme than) what was observed, **assuming the null hypothesis is true**. It is NOT:

- The probability that the null hypothesis is true
- The probability that you made an error
- The probability that the result will replicate

A p-value of 0.03 means: “If there were truly no difference between tumor and normal, data this extreme would occur about 3% of the time by chance.”

The Complete Report for a Gene

Never report just a p-value. A complete comparison includes:

```
# Complete report for TP53
let report_gene = "TP53"
let row = results_table |> filter_by("gene", |g| contains(g, report_gene))

println("=== Complete Report: " + report_gene + " ===")
println("Mean expression (log2 FPKM):")
println("  Tumor:  " + to_string(col_values(row, "mean_tumor")[0]))
println("  Normal: " + to_string(col_values(row, "mean_normal")[0]))
println("Log2 fold change: " + to_string(col_values(row, "log2fc")[0]))
println("Fold change: " + to_string(round(pow(2.0, abs(col_values(row,
"log2fc")[0])), 2)) + "x")
println("Direction: " + if col_values(row, "log2fc")[0] > 0.0 then "up in
tumor" else "down in tumor")
println("t-statistic: " + to_string(col_values(row, "t_stat")[0]))
println("p-value: " + to_string(col_values(row, "p_value")[0]))
println("Cohen's d: " + to_string(col_values(row, "cohens_d")[0]))
println("Note: p-value is NOT corrected for multiple testing (see Chapter
8)")
```

When the t-Test Finds Significance but the Biology Does Not

```
# Find genes with tiny fold changes but "significant" p-values
let stat_only = results_table
  |> filter_by("p_value", |p| p < 0.01)
  |> filter_by("log2fc", |fc| abs(fc) < 0.3)
  |> arrange("p_value")

println("=== Statistically Significant but Biologically Questionable ===")
println("Genes with p < 0.01 but |log2FC| < 0.3:")
println("(These are less than 1.23-fold changes)")
take(stat_only, 10)
```

These genes are statistically different but the difference is so small it is unlikely to be biologically meaningful. This is exactly why effect size matters.

When Biology Happens Without Statistical Significance

```
# Find genes with large fold changes but non-significant p-values
let bio_only = results_table
  |> filter_by("p_value", |p| p > 0.05)
  |> filter_by("log2fc", |fc| abs(fc) > 2.0)
  |> arrange("log2fc")

println("=== Biologically Large but Not Statistically Significant ===")
println("Genes with p > 0.05 but |log2FC| > 2 (4-fold change):")
take(bio_only, 10)
```

These genes have large fold changes but high variance or small sample sizes prevent the difference from being statistically significant. They may be worth

investigating with more samples.

Decision

What Test Should You Use?

```
# Decision framework encoded in code:

# 1. Are the samples paired (same patient)?
let paired = true  # set based on experimental design

# 2. Check normality of the data (or log-data)
let check_normality = true  # QQ plot looked reasonable

# 3. Choose the test
let test_choice = if paired then {
  if check_normality then "paired t-test"
  else "Wilcoxon signed-rank"
} else {
  if check_normality then "Welch's t-test"
  else "Wilcoxon rank-sum"
}

println("Recommended test: " + test_choice)
```

Summary Table of All Methods

```
# Build a decision summary
let summary_table = table(
  "scenario", [
    "Paired, normal data",
    "Paired, non-normal data",
    "Unpaired, normal data",
    "Unpaired, non-normal data",
    "Unpaired, unequal variance"
  ],
  "test", [
    "Paired t-test",
    "Wilcoxon signed-rank",
    "Student's t-test",
    "Wilcoxon rank-sum",
    "Welch's t-test"
  ],
  "biolang_function", [
    "ttest_paired(x, y)",
    "wilcoxon(x, y)",
    "ttest(x, y)",
    "wilcoxon(x, y)",
    "ttest(x, y)"
  ],
  "note", [
    "Most powerful when pairing exists",
    "Uses ranks, robust to outliers",
    "Assumes equal variance",
    "No distribution assumption",
    "Default — safe with unequal variance"
  ]
)

summary_table
```

The Recommended Workflow

```
# Complete workflow for comparing two groups in genomics:

# 1. Log-transform expression data
# 2. Check normality with QQ plots
# 3. Check variance equality (compare SDs)
# 4. Use paired test if samples are matched
# 5. Use Welch's t-test as default (not Student's)
# 6. Always compute Cohen's d
# 7. Always compute confidence intervals
# 8. Always make a volcano plot
# 9. Do NOT interpret raw p-values when testing many genes
#    (see Chapter 8 for multiple testing correction)
```

Pitfalls

Pitfall 1: Using Student's t-Test When Variances Are Unequal

```
# Simulate unequal variances
let group_a = rnorm(30, 5.0, 1.0)   # mean=5, sd=1
let group_b = rnorm(30, 5.5, 3.0)   # mean=5.5, sd=3

# Student's t-test ASSUMES equal variance
# Welch's does not
let t_student = ttest(group_a, group_b)
println("Welch's p-value: " + to_string(t_student.p_value))

# With unequal n AND unequal variance, Student's can be seriously wrong
let small_a = rnorm(10, 5.0, 1.0)
let large_b = rnorm(50, 5.5, 3.0)
let t_wrong = ttest(small_a, large_b)
println("Unequal n + unequal var: p = " + to_string(t_wrong.p_value))
```

Rule: Just use Welch's t-test. Always. There is no good reason to use Student's t-test in practice.

Pitfall 2: Testing Without Log-Transforming

```
# Demonstrate the danger of testing raw FPKM values
# Raw expression is heavily right-skewed
let raw_result = ttest(tp53_tumor, tp53_normal)
let log_result = ttest(log_tp53_tumor, log_tp53_normal)

println("=== Raw vs Log-transformed ===")
println("Raw FPKM t-test p-value: " + to_string(raw_result.p_value))
println("Log2 FPKM t-test p-value: " + to_string(log_result.p_value))
println("These can differ substantially!")
```

Pitfall 3: Confusing Statistical and Biological Significance

A gene with p = 1e-10 and fold change = 1.05 is statistically significant but biologically irrelevant. With enough samples, ANY non-zero difference becomes significant. Always report and filter by effect size.

Pitfall 4: Ignoring Multiple Testing

```
# Running 20,000 t-tests without correction
let n_tests = len(results_table)
let n_sig_raw = results_table
  |> filter_by("p_value", |p| p < 0.05)
  |> len()

let expected_false_pos = to_float(n_tests) * 0.05

println("=== The Multiple Testing Problem ===")
println("Total genes tested: " + to_string(n_tests))
println("Significant at p<0.05: " + to_string(n_sig_raw))
println("Expected false positives by chance: " +
to_string(round(expected_false_pos, 0)))
println("See Chapter 8 for correction methods!")
```

Pitfall 5: Applying Paired Test to Unpaired Data (or Vice Versa)

```
# Paired test on unpaired data gives WRONG results
# If samples are not truly paired, the differences are meaningless

# Simulate: generate two independent groups
let independent_a = rnorm(20, 10.0, 2.0)
let independent_b = rnorm(20, 11.0, 2.0)

# Wrong: treating independent data as paired
let wrong_paired = ttest_paired(independent_a, independent_b)

# Correct: unpaired test
let correct_unpaired = ttest(independent_a, independent_b)

println("Wrong (paired on unpaired data): p = " +
to_string(wrong_paired.p_value))
println("Correct (unpaired test):         p = " +
to_string(correct_unpaired.p_value))
```

Pitfall 6: Cherry-Picking Genes

If you look at a gene because it has a large fold change and then test it, you are not doing an honest hypothesis test. You are confirming what you already saw in the data. This inflates false positive rates. Either test all genes (and correct for multiple testing) or have a pre-specified hypothesis.

Pitfall 7: The Pseudocount Matters

```
# The choice of pseudocount affects log2 fold change for lowly expressed
genes
let low_expr = [0.0, 0.0, 0.1, 0.0, 0.2]

let log_pc1 = low_expr |> map(|x| log2(x + 1.0))
let log_pc01 = low_expr |> map(|x| log2(x + 0.1))
let log_pc001 = low_expr |> map(|x| log2(x + 0.01))

println("Pseudocount = 1:    " + to_string(log_pc1))
println("Pseudocount = 0.1:  " + to_string(log_pc01))
println("Pseudocount = 0.01: " + to_string(log_pc001))
println("The pseudocount dramatically affects near-zero genes!")
```

Putting It All Together

Here is the complete analysis pipeline for comparing tumor vs normal expression:

```
# ============================================================
# Complete Two-Group Comparison Pipeline
# ============================================================

# 1. Load data
let url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let expr = download(url, "GSE183947_fpkm.csv.gz") |> read_csv()

# 2. Identify groups
let all_cols = columns(expr)
let tumor_cols = all_cols |> filter(|c| contains(c, "tumor"))
let normal_cols = all_cols |> filter(|c| contains(c, "normal"))
let gene_col = all_cols[0]

# 3. Genome-wide comparison
let results = col_values(expr, gene_col) |> map(|gene| {
  let row = expr |> filter_by(gene_col, |g| g == gene)

  let tumor_vals = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
  let normal_vals = normal_cols |> map(|c| col_values(row, c)) |> flatten()

  # Log-transform
  let lt = tumor_vals |> map(|x| log2(x + 1.0))
  let ln = normal_vals |> map(|x| log2(x + 1.0))

  # Test and effect size
  let log2fc = mean(lt) - mean(ln)
  let t_res = ttest(lt, ln)
  let w_res = wilcoxon(lt, ln)

  let nt = to_float(len(lt))
  let nn = to_float(len(ln))
  let pv = ((nt - 1.0) * variance(lt) + (nn - 1.0) * variance(ln)) / (nt + nn
- 2.0)
  let d = if pv > 0.0 then log2fc / sqrt(pv) else 0.0

  record(
    "gene", gene,
    "log2fc", round(log2fc, 4),
    "p_ttest", t_res.p_value,
    "p_wilcoxon", w_res.p_value,
    "cohens_d", round(d, 4)
  )
}) |> to_list() |> table()

# 4. Volcano plot
let plot_data = results
  |> mutate("neg_log10_p", |row| -log10(row.p_ttest + 1e-300))

volcano(plot_data, "log2fc", "neg_log10_p")
  |> save_plot("complete_volcano.svg")

# 5. Summary
let sig = results
  |> filter_by("p_ttest", |p| p < 0.05)
  |> filter_by("log2fc", |fc| abs(fc) > 1.0)

println("=== Final Summary ===")
println("Total genes tested: " + to_string(len(results)))
println("Significant (p<0.05, |FC|>2): " + to_string(len(sig)))
println("WARNING: p-values are NOT adjusted for multiple testing")
println("See Chapter 8 before reporting these results!")
```

Where Next?

This chapter covered comparing **two** groups. But what happens when you have three, four, or five groups? You cannot simply run pairwise t-tests — that creates a multiple testing problem within a single comparison. Chapter 6 covers **multi-group comparisons** with ANOVA and Kruskal-Wallis.

We also noted repeatedly that testing 20,000 genes requires correction for multiple testing. Chapter 8 covers this critical topic — Bonferroni, Benjamini-Hochberg, and why the choice matters enormously.

And if your data is not normally distributed even after log-transformation, Chapter 7 covers **non-parametric methods** in detail.

Key Takeaways

1. **Use Welch's t-test** by default, not Student's
2. **Log-transform** expression data before testing
3. **Always compute effect size** (Cohen's d) alongside p-values
4. **Always compute confidence intervals** — they tell you more than p-values
5. **Visualize with volcano plots** — they combine fold change and significance
6. **Check your assumptions** — QQ plots for normality, variance comparison
7. **Use paired tests** when samples are naturally paired
8. **Consider Wilcoxon** when normality is questionable
9. **Never interpret raw p-values** from genome-wide tests (Chapter 8)
10. **Report the complete picture**: mean, SD, fold change, CI, p-value, effect size

Exercises

1. Download GSE183947 and identify the 10 most upregulated and 10 most downregulated genes. Report all six statistics (mean per group, log2FC, p-value, Cohen's d, 95% CI).

2. For the gene ERBB2 (HER2), compare the t-test result on raw FPKM vs log2-transformed FPKM. Which is more appropriate and why?

3. Pick 5 genes where the t-test and Wilcoxon disagree. Make boxplots for each. What pattern do you see? (Hint: look for outliers.)

4. Calculate Cohen's d for all 20,000 genes. What percentage have $|d| > 0.8$? How does this compare to the percentage with $p < 0.05$?
5. Simulate paired data: generate 20 pairs where each "normal" value is drawn from N(10, 2) and each "tumor" value is the paired normal value plus a random effect from N(1, 1). Run both paired and unpaired t-tests. Which is more powerful? Why?

Summary Table

Concept	Key Point
Student's t-test	Assumes equal variance — almost never use
Welch's t-test	Default choice — handles unequal variance
Paired t-test	Use when samples are matched — much more powerful
Wilcoxon rank-sum	Non-parametric alternative — robust to outliers
Cohen's d	Effect size in SD units — report alongside p-value
Confidence interval	Range of plausible effect sizes — more informative than p-value
Log-transformation	Essential for expression data — makes it approximately normal
Volcano plot	Combines fold change and significance — the signature genomics plot
Multiple testing	Testing 20,000 genes at $p<0.05$ yields ~1,000 false positives

---

Next: Chapter 6 — Multi-Group Comparisons

Multi-Group Comparisons

> *"If you torture the data long enough, it will confess to anything."* — Ronald Coase

You have expression data across five cancer subtypes. Or across 54 human tissues. Or across four treatment doses. The temptation is to run all pairwise t-tests and highlight the significant ones. This chapter explains why that approach is wrong and what to do instead.

The Problem

A researcher is studying tissue-specific gene expression using the GTEx (Genotype-Tissue Expression) dataset. She wants to know whether housekeeping genes like GAPDH truly have constant expression across tissues, and whether tissue-specific genes like albumin (ALB) show the expected patterns. She has median TPM values across 54 tissues.

Her first idea: compare every pair of tissues with a t-test. For 54 tissues, that is 54 * 53 / 2 = 1,431 pairwise comparisons. At alpha = 0.05, she expects about 72 false positives even if there is no real difference anywhere. And she is doing this for every gene.

She needs ANOVA.

Why It Matters

Multi-group comparisons arise constantly in biology:

- **Cancer subtypes**: Is gene X differentially expressed across Luminal A, Luminal B, HER2+, Basal, and Normal-like breast cancer?
- **Tissue types**: Does a drug candidate affect expression equally across liver, kidney, heart, and brain?
- **Dose responses**: Does the treatment effect change across 0, 1, 5, 10, and 50 mg/kg?
- **Developmental stages**: How does gene expression change across embryonic, neonatal, juvenile, and adult stages?

- **Species comparisons**: Is a pathway conserved across human, mouse, rat, and zebrafish?

Running pairwise t-tests inflates the false positive rate. With k groups, you have k*(k-1)/2 pairs. The probability of at least one false positive is:

```
# Multiple pairwise comparisons inflate false positives
# Probability of at least one false positive with alpha=0.05

let n_groups = [3, 5, 10, 20, 54]
n_groups |> each(|k| {
  let n_pairs = k * (k - 1) / 2
  let p_no_fp = pow(1.0 - 0.05, to_float(n_pairs))
  let p_at_least_one = 1.0 - p_no_fp
  println("k=" + to_string(k) + " groups: "
    + to_string(n_pairs) + " pairs, "
    + "P(>=1 false positive) = "
    + to_string(round(p_at_least_one * 100.0, 1)) + "%")
})
```

With 5 groups: 10 pairs, 40% chance of a false positive. With 10 groups: 45 pairs, 90% chance. With 54 tissues: 1,431 pairs, virtually certain to have many false positives.

ANOVA solves this by asking a single question: “Is there ANY difference among the groups?” — with a single test at a single alpha level.

The Data

We use the GTEx v8 gene median TPM dataset, which contains median expression values (in TPM) for every gene across 54 human tissues.

Downloading the Data

```
# Download GTEx median TPM by tissue
let url = "https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
let raw_file = download(url, "GTEx_v8_median_tpm.gct.gz")
println("Downloaded: " + raw_file)
```

Loading and Inspecting

GCT format has two header lines before the data matrix. We need to handle this:

```
# Read the GCT file (tab-separated with header rows)
# GCT files have 2 header lines, then column names, then data
let gtex = read_tsv("GTEx_v8_median_tpm.gct.gz")

println("Rows (genes): " + to_string(len(gtex)))
println("Columns: " + to_string(ncols(gtex)))

# Preview column names — these are tissue names
let cols = columns(gtex)
println("First 10 columns:")
cols |> take(10) |> each(|c| println("  " + c))
```

```
# List all tissue types
# First two columns are typically Name (gene ID) and Description (gene
symbol)
let tissue_cols = cols |> filter(|c| c != "Name" and c != "Description")
println("Number of tissues: " + to_string(len(tissue_cols)))
println("Tissues:")
tissue_cols |> enumerate() |> each(|pair| {
  println("  " + to_string(pair[0] + 1) + ". " + pair[1])
})
```

Understanding TPM

TPM (Transcripts Per Million) is a normalized expression measure. Unlike raw counts, TPM values are comparable across samples because they account for gene length and sequencing depth.

```
# Verify TPM normalization: should sum to ~1,000,000 per tissue
# (approximately, since these are median values across samples)
let first_tissue = tissue_cols[0]
let tissue_values = col_values(gtex, first_tissue)
let total_tpm = sum(tissue_values)
println("Total TPM in " + first_tissue + ": " + to_string(round(total_tpm,
0)))
```

Exploration

Housekeeping Genes — Should Be Constant

Housekeeping genes are expressed at relatively constant levels across all tissues. If ANOVA finds no significant difference for these genes across tissues, that is a good sign.

```
# Extract expression for classic housekeeping genes
let housekeeping = ["GAPDH", "ACTB", "B2M", "HPRT1", "RPL13A", "SDHA", "TBP",
"UBC"]

let hk_data = housekeeping |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let values = tissue_cols |> map(|t| col_values(row, t)[0])
    record("gene", gene, "values", values)
  } else {
    record("gene", gene, "values", [])
  }
})

# Summary stats for each housekeeping gene across tissues
hk_data |> each(|entry| {
  let vals = entry.values
  if len(vals) > 0 then {
    let cv = stdev(vals) / mean(vals) * 100.0  # coefficient of variation
    println(entry.gene + ":")
    println("  mean TPM = " + to_string(round(mean(vals), 2)))
    println("  SD       = " + to_string(round(stdev(vals), 2)))
    println("  CV       = " + to_string(round(cv, 1)) + "%")
    println("  range    = " + to_string(round(min(vals), 2)) + " - " +
to_string(round(max(vals), 2)))
  }
})
```

The coefficient of variation (CV = SD/mean * 100%) is useful here. A truly constant gene would have CV near 0%. In practice, housekeeping genes typically have CV of 30-80% across tissues — they are more stable than most genes but far from constant.

Tissue-Specific Genes — Should Vary

```
# Tissue-specific genes: should show high variance across tissues
let tissue_specific = ["ALB", "INS", "HBB", "KRT14", "MYH7", "SFTPC", "TTN",
"CDH5"]

tissue_specific |> each(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let values = tissue_cols |> map(|t| col_values(row, t)[0])

    # Find the tissue with maximum expression
    let max_val = max(values)
    let max_idx = values |> enumerate() |> filter(|p| p[1] == max_val)
    let max_tissue = if len(max_idx) > 0 then tissue_cols[max_idx[0][0]] else
"unknown"

    # What fraction of total expression is in the top tissue?
    let total = sum(values)
    let specificity = if total > 0.0 then max_val / total * 100.0 else 0.0

    println(gene + ":")
    println("  Top tissue: " + max_tissue + " (" + to_string(round(max_val,
1)) + " TPM)")
    println("  Specificity: " + to_string(round(specificity, 1)) + "% of
total in top tissue")
  }
})
```

Visualizing Expression Across Tissues

```
# Boxplot of GAPDH expression across tissues
let gapdh_row = gtex |> filter_by("Description", |d| d == "GAPDH")
let gapdh_values = tissue_cols |> map(|t| col_values(gapdh_row, t)[0])

let gapdh_tbl = table(
  "tissue", tissue_cols,
  "tpm", gapdh_values
)

boxplot(gapdh_tbl, "tissue", "tpm")
  |> save_plot("gapdh_across_tissues.svg")
```

```
# Bar chart of ALB (albumin) — should be almost exclusively in liver
let alb_row = gtex |> filter_by("Description", |d| d == "ALB")
let alb_values = tissue_cols |> map(|t| col_values(alb_row, t)[0])

let alb_tbl = table(
  "tissue", tissue_cols,
  "tpm", alb_values
) |> arrange("tpm") |> tail(15)  # top 15 tissues

bar_chart(alb_tbl, "tissue", "tpm")
  |> save_plot("albumin_tissue_expression.svg")
```

Log-Transforming for Analysis

```
# Log2(TPM + 1) is standard for GTEx analyses
# This makes the data more approximately normal

let gapdh_log = gapdh_values |> map(|x| log2(x + 1.0))
println("GAPDH (raw TPM):")
println("  mean = " + to_string(round(mean(gapdh_values), 2)))
println("  sd   = " + to_string(round(stdev(gapdh_values), 2)))

println("GAPDH (log2 TPM+1):")
println("  mean = " + to_string(round(mean(gapdh_log), 2)))
println("  sd   = " + to_string(round(stdev(gapdh_log), 2)))

# QQ plot to check normality of log-transformed data
qq_plot(gapdh_log, "GAPDH log2(TPM+1) across tissues")
  |> save_plot("gapdh_qq.svg")
```

The Analysis

Step 1: One-Way ANOVA — The Omnibus Test

ANOVA asks: “Is there at least one group mean that differs from the others?”

It does NOT tell you WHICH groups differ. That requires post-hoc tests (Step 3).

ANOVA works by partitioning the total variance into:

- **Between-group variance**: How much the group means differ from the overall mean
- **Within-group variance**: How much individual observations vary within each group

If the between-group variance is much larger than the within-group variance, the groups are probably different.

```
# ANOVA on GAPDH across a subset of tissues
# We need to structure data as groups for ANOVA

# Select 5 representative tissues
let selected_tissues = ["Brain - Cortex", "Heart - Left Ventricle",
                        "Liver", "Lung", "Whole Blood"]

# Extract GAPDH expression for each tissue
# In GTEx median data, each tissue column has one value per gene
# For ANOVA, we need multiple observations per group
# The median TPM file has one median per tissue — we will use
# multiple genes as our "observations" for a cross-tissue comparison

# For a proper ANOVA demonstration, let us compare multiple
# housekeeping genes across tissues
let hk_genes = ["GAPDH", "ACTB", "B2M", "HPRT1", "RPL13A",
                 "SDHA", "TBP", "UBC", "YWHAZ", "RPLP0",
                 "PPIA", "GUSB", "HMBS", "PGK1", "TFRC"]

# Build vectors of log2(TPM+1) for each tissue
let tissue_groups = selected_tissues |> map(|tissue| {
  hk_genes |> map(|gene| {
    let row = gtex |> filter_by("Description", |d| d == gene)
    if len(row) > 0 then {
      let val = col_values(row, tissue)[0]
      log2(val + 1.0)
    } else 0.0
  })
})

# Run ANOVA
let anova_result = anova(tissue_groups)
println("=== One-Way ANOVA: Housekeeping Genes Across 5 Tissues ===")
println("F-statistic: " + to_string(round(anova_result.statistic, 4)))
println("p-value:     " + to_string(anova_result.p_value))
println("df_between:  " + to_string(len(selected_tissues) - 1))
println("df_within:   " + to_string(len(hk_genes) * len(selected_tissues) -
len(selected_tissues)))
```

Interpreting the F-statistic: F is the ratio of between-group variance to within-group variance. F near 1 means groups are similar. F much larger than 1 means at least one group differs.

Step 2: Checking ANOVA Assumptions

ANOVA assumes:

1. **Independence**: Observations are independent
2. **Normality**: Data within each group is normally distributed
3. **Homogeneity of variance**: All groups have similar variance

```
# Check assumption 2: normality within each group
selected_tissues |> enumerate() |> each(|pair| {
  let idx = pair[0]
  let tissue = pair[1]
  let vals = tissue_groups[idx]
  qq_plot(vals, tissue + " (housekeeping genes)")
    |> save_plot("anova_qq_" + to_string(idx) + ".svg")
  println(tissue + ": mean=" + to_string(round(mean(vals), 2))
    + " sd=" + to_string(round(stdev(vals), 2)))
})
```

```
# Check assumption 3: homogeneity of variance
# Compare standard deviations across groups
let group_sds = tissue_groups |> map(|g| stdev(g))
let max_sd = max(group_sds)
let min_sd = min(group_sds)
let sd_ratio = max_sd / min_sd

println("=== Variance Homogeneity Check ===")
selected_tissues |> enumerate() |> each(|pair| {
  println(pair[1] + ": SD = " + to_string(round(group_sds[pair[0]], 4)))
})
println("Max/Min SD ratio: " + to_string(round(sd_ratio, 2)))
println("Rule of thumb: ratio > 2 suggests heterogeneity")
```

Step 3: Post-Hoc Pairwise Comparisons

ANOVA tells you that at least one group differs, but not which ones. Post-hoc tests identify the specific pairs that differ while controlling for multiple comparisons.

```
# Pairwise t-tests with correction for multiple comparisons
let n_tissues = len(selected_tissues)
let pairwise_results = []
let all_pvalues = []

# All pairwise comparisons
let pair_idx = 0
let pairs = []

# Generate all pairs
selected_tissues |> enumerate() |> each(|i_pair| {
  let i = i_pair[0]
  selected_tissues |> enumerate() |> each(|j_pair| {
    let j = j_pair[0]
    if j > i then {
      let t_result = ttest(tissue_groups[i], tissue_groups[j])
      let mean_diff = mean(tissue_groups[i]) - mean(tissue_groups[j])

      println(selected_tissues[i] + " vs " + selected_tissues[j] + ":")
      println("  mean diff = " + to_string(round(mean_diff, 3))
        + "  p = " + to_string(round(t_result.p_value, 6)))

      pairs = concat(pairs, [record(
        "group1", selected_tissues[i],
        "group2", selected_tissues[j],
        "mean_diff", round(mean_diff, 4),
        "p_value", t_result.p_value
      )])

      all_pvalues = concat(all_pvalues, [t_result.p_value])
    }
  })
})
```

```
# Apply Bonferroni correction to pairwise p-values
let bonf_adjusted = p_adjust(all_pvalues, "bonferroni")
let bh_adjusted = p_adjust(all_pvalues, "bh")

println("=== Pairwise Comparisons with Multiple Testing Correction ===")
println("Number of pairwise tests: " + to_string(len(all_pvalues)))
pairs |> enumerate() |> each(|entry| {
  let idx = entry[0]
  let pair = entry[1]
  println(pair.group1 + " vs " + pair.group2 + ":")
  println("  raw p      = " + to_string(round(pair.p_value, 6)))
  println("  Bonferroni = " + to_string(round(bonf_adjusted[idx], 6)))
  println("  BH (FDR)   = " + to_string(round(bh_adjusted[idx], 6)))
})
```

Step 4: Kruskal-Wallis — Non-Parametric Alternative

When the normality assumption is questionable, use the Kruskal-Wallis test instead of ANOVA. It is the multi-group generalization of the Wilcoxon rank-sum test.

```
# Kruskal-Wallis is the non-parametric version of one-way ANOVA
# It uses ranks instead of raw values, making it robust to
# non-normality and outliers

# For tissue-specific genes, the distribution is very non-normal
# (most tissues have near-zero expression, one has very high)

# Let us test ALB (albumin) across tissues
let alb_groups = selected_tissues |> map(|tissue| {
  # For each tissue, we will gather expression of liver-enriched genes
  let liver_genes = ["ALB", "TF", "SERPINA1", "HP", "FGA", "FGB", "FGG",
                     "APOA1", "APOB", "TTR", "ORM1", "CYP3A4", "CYP2E1"]
  liver_genes |> map(|gene| {
    let row = gtex |> filter_by("Description", |d| d == gene)
    if len(row) > 0 then {
      let val = col_values(row, tissue)[0]
      log2(val + 1.0)
    } else 0.0
  })
})

# ANOVA (parametric)
let anova_liver = anova(alb_groups)
println("=== Liver Genes Across Tissues ===")
println("ANOVA F-statistic: " + to_string(round(anova_liver.statistic, 4)))
println("ANOVA p-value:     " + to_string(anova_liver.p_value))
```

```
# Compare with Kruskal-Wallis approach
# Using Wilcoxon on each pair as non-parametric post-hoc
let np_pvalues = []
selected_tissues |> enumerate() |> each(|i_pair| {
  let i = i_pair[0]
  selected_tissues |> enumerate() |> each(|j_pair| {
    let j = j_pair[0]
    if j > i then {
      let w_result = wilcoxon(alb_groups[i], alb_groups[j])
      println(selected_tissues[i] + " vs " + selected_tissues[j]
        + ": Wilcoxon p = " + to_string(round(w_result.p_value, 6)))
      np_pvalues = concat(np_pvalues, [w_result.p_value])
    }
  })
})

# Correct non-parametric p-values too
let np_bh = p_adjust(np_pvalues, "bh")
println("BH-adjusted non-parametric p-values:")
np_bh |> enumerate() |> each(|pair| {
  println("  Pair " + to_string(pair[0] + 1) + ": " +
to_string(round(pair[1], 6)))
})
```

Step 5: Genome-Wide Tissue Comparisons

Now let us scale to all genes. For each gene, we test whether expression differs across our selected tissues.

```
# Genome-wide ANOVA across selected tissues
let all_genes = col_values(gtex, "Description")
  |> unique()
  |> filter(|g| len(g) > 0)

# Take a manageable subset for demonstration
let test_genes = all_genes |> take(2000)

let anova_results = test_genes |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let groups = selected_tissues |> map(|tissue| {
      let val = col_values(row, tissue)[0]
      log2(val + 1.0)
    })

    # For ANOVA we need multiple observations per group
    # With median data, we use a different approach:
    # compute an F-like ratio from the tissue medians

    let overall_mean = mean(groups |> flatten())
    let group_means = groups |> map(|g| mean(g))

    # Between-group variance
    let ss_between = group_means |> map(|m| pow(m - overall_mean, 2.0)) |>
sum()

    # Maximum fold difference
    let max_expr = max(groups |> flatten())
    let min_expr = min(groups |> flatten())
    let fold_range = max_expr - min_expr

    record(
      "gene", gene,
      "max_tissue_expr", round(max_expr, 4),
      "min_tissue_expr", round(min_expr, 4),
      "fold_range", round(fold_range, 4),
      "ss_between", round(ss_between, 4)
    )
  } else {
    record("gene", gene, "max_tissue_expr", 0.0, "min_tissue_expr", 0.0,
           "fold_range", 0.0, "ss_between", 0.0)
  }
}) |> to_list() |> table()

# Sort by between-group variance to find most tissue-specific genes
let most_variable = anova_results
  |> arrange("fold_range")
  |> tail(20)

println("=== Top 20 Most Tissue-Variable Genes ===")
most_variable
```

Step 6: Visualizing Multi-Group Comparisons

```
# Violin plot for a tissue-specific gene across all tissues
let gene_to_plot = "ALB"
let plot_row = gtex |> filter_by("Description", |d| d == gene_to_plot)
let plot_values = tissue_cols |> map(|t| col_values(plot_row, t)[0])

let violin_data = table(
  "tissue", tissue_cols,
  "log2_tpm", plot_values |> map(|x| log2(x + 1.0))
)

boxplot(violin_data, "tissue", "log2_tpm")
  |> save_plot("alb_violin_tissues.svg")
```

```
# Heatmap of housekeeping gene expression across tissues
let hk_matrix_rows = housekeeping |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    selected_tissues |> map(|t| {
      let val = col_values(row, t)[0]
      log2(val + 1.0)
    })
  } else {
    selected_tissues |> map(|_| 0.0)
  }
})

let hk_matrix = matrix(hk_matrix_rows)
heatmap(hk_matrix, housekeeping, selected_tissues)
  |> save_plot("housekeeping_heatmap.svg")
```

```
# Heatmap of tissue-specific genes
let ts_genes = ["ALB", "INS", "HBB", "KRT14", "MYH7",
                "SFTPC", "TTN", "CDH5", "GFAP", "SYN1"]

let ts_matrix_rows = ts_genes |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    selected_tissues |> map(|t| {
      let val = col_values(row, t)[0]
      log2(val + 1.0)
    })
  } else {
    selected_tissues |> map(|_| 0.0)
  }
})

let ts_matrix = matrix(ts_matrix_rows)
heatmap(ts_matrix, ts_genes, selected_tissues)
  |> save_plot("tissue_specific_heatmap.svg")
```

Step 7: The Multiple Comparison Problem in Post-Hoc Tests

When ANOVA is significant and you do post-hoc pairwise tests, you must correct for the number of pairwise comparisons. Here is a complete demonstration:

```
# Full post-hoc analysis for brain-enriched genes
let brain_tissues = ["Brain - Cortex", "Brain - Cerebellum",
                     "Brain - Hippocampus"]
let other_tissues = ["Liver", "Heart - Left Ventricle", "Lung"]
let all_compare = concat(brain_tissues, other_tissues)

let brain_genes = ["SYN1", "GFAP", "NEFL", "MBP", "SLC17A7",
                   "GAD1", "GAD2", "SLC6A3", "DRD1", "DRD2"]

# For each brain gene, compare brain vs non-brain tissues
brain_genes |> each(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let brain_vals = brain_tissues |> map(|t| {
      let val = col_values(row, t)[0]
      log2(val + 1.0)
    })
    let other_vals = other_tissues |> map(|t| {
      let val = col_values(row, t)[0]
      log2(val + 1.0)
    })

    let t_res = ttest(brain_vals, other_vals)
    println(gene + ": brain vs other, p = " + to_string(round(t_res.p_value,
6))
      + ", mean_brain = " + to_string(round(mean(brain_vals), 2))
      + ", mean_other = " + to_string(round(mean(other_vals), 2)))
  }
})
```

```
# Collect all brain gene p-values and adjust
let brain_pvals = brain_genes |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let brain_vals = brain_tissues |> map(|t| log2(col_values(row, t)[0] +
1.0))
    let other_vals = other_tissues |> map(|t| log2(col_values(row, t)[0] +
1.0))
    let t_res = ttest(brain_vals, other_vals)
    t_res.p_value
  } else 1.0
})

let brain_bh = p_adjust(brain_pvals, "bh")

println("=== Brain Gene P-values: Raw vs Adjusted ===")
brain_genes |> enumerate() |> each(|pair| {
  println(pair[1] + ": raw p = " + to_string(round(brain_pvals[pair[0]], 6))
    + ", BH adjusted = " + to_string(round(brain_bh[pair[0]], 6)))
})
```

Step 8: Interaction Effects

Sometimes the effect of one factor depends on another factor. For example, the tissue effect on expression might depend on whether the gene is a housekeeping gene or a tissue-specific gene. This is an interaction effect.

```
# Interaction: does the tissue effect depend on gene category?

# Compare expression patterns of housekeeping vs tissue-specific genes
# across selected tissues

let hk_by_tissue = selected_tissues |> map(|tissue| {
  housekeeping |> map(|gene| {
    let row = gtex |> filter_by("Description", |d| d == gene)
    if len(row) > 0 then log2(col_values(row, tissue)[0] + 1.0) else 0.0
  })
})

let ts_by_tissue = selected_tissues |> map(|tissue| {
  tissue_specific |> map(|gene| {
    let row = gtex |> filter_by("Description", |d| d == gene)
    if len(row) > 0 then log2(col_values(row, tissue)[0] + 1.0) else 0.0
  })
})

# For housekeeping genes, the variance across tissues should be LOW
# For tissue-specific genes, the variance should be HIGH
println("=== Variance Across Tissues ===")
println("Housekeeping genes:")
housekeeping |> enumerate() |> each(|pair| {
  let vals = selected_tissues |> enumerate() |> map(|t_pair| {
    hk_by_tissue[t_pair[0]][pair[0]]
  })
  println("  " + pair[1] + ": var = " + to_string(round(variance(vals), 4)))
})

println("Tissue-specific genes:")
tissue_specific |> enumerate() |> each(|pair| {
  let vals = selected_tissues |> enumerate() |> map(|t_pair| {
    ts_by_tissue[t_pair[0]][pair[0]]
  })
  println("  " + pair[1] + ": var = " + to_string(round(variance(vals), 4)))
})
```

```
# Visualize the interaction
# Build a table with gene category, tissue, and expression

let interaction_data = []
selected_tissues |> enumerate() |> each(|t_pair| {
  let tissue = t_pair[1]
  let t_idx = t_pair[0]

  # Housekeeping genes in this tissue
  let hk_mean = mean(hk_by_tissue[t_idx])
  interaction_data = concat(interaction_data, [record(
    "tissue", tissue,
    "category", "Housekeeping",
    "mean_log2tpm", round(hk_mean, 3)
  )])

  # Tissue-specific genes in this tissue
  let ts_mean = mean(ts_by_tissue[t_idx])
  interaction_data = concat(interaction_data, [record(
    "tissue", tissue,
    "category", "Tissue-Specific",
    "mean_log2tpm", round(ts_mean, 3)
  )])
})

let interaction_table = table(interaction_data |> to_list())
interaction_table

# The interaction is visible: housekeeping gene expression is flat
# across tissues, while tissue-specific gene expression varies dramatically
```

Step 9: Effect Size for Multi-Group Comparisons

Just as Cohen’s d measures effect size for two groups, eta-squared measures effect size for ANOVA:

```
# Eta-squared: proportion of total variance explained by group membership
# eta^2 = SS_between / SS_total

# Compute for several genes
let effect_genes = concat(housekeeping, tissue_specific)

let eta_results = effect_genes |> map(|gene| {
  let row = gtex |> filter_by("Description", |d| d == gene)
  if len(row) > 0 then {
    let values = selected_tissues |> map(|t| log2(col_values(row, t)[0] +
1.0))
    let grand_mean = mean(values)
    let ss_total = values |> map(|x| pow(x - grand_mean, 2.0)) |> sum()

    # Between-group SS (each tissue is a "group" of size 1 for medians)
    let ss_between = ss_total  # with 1 obs per group, ss_between = ss_total

    # For demonstration, compute CV as a proxy for tissue variation
    let cv = if grand_mean > 0.0 then stdev(values) / grand_mean * 100.0 else
0.0

    record(
      "gene", gene,
      "cv_percent", round(cv, 2),
      "mean_log2tpm", round(grand_mean, 3),
      "sd_log2tpm", round(stdev(values), 3)
    )
  } else {
    record("gene", gene, "cv_percent", 0.0, "mean_log2tpm", 0.0,
"sd_log2tpm", 0.0)
  }
}) |> to_list() |> table()

let eta_sorted = eta_results |> arrange("cv_percent")
println("=== Gene Variability Across Tissues (CV%) ===")
println("Low CV = housekeeping, High CV = tissue-specific")
eta_sorted
```

Interpretation

What ANOVA Tells You (and What It Does Not)

ANOVA answers ONE question: “Is there evidence that at least one group mean differs from the others?”

It does NOT tell you:

- Which groups differ
- How many groups differ
- The direction of the difference
- The magnitude of the difference

For those answers, you need post-hoc tests and effect sizes.

The F-Statistic in Context

```
# Understanding the F-statistic
# F = variance between groups / variance within groups

# Large F: groups are very different relative to within-group noise
# F near 1: groups are not distinguishable from noise
# F < 1: theoretically impossible, but rounding can cause it

# Rule of thumb for interpretation:
let f_values = [1.0, 2.0, 4.0, 8.0, 16.0, 32.0]
f_values |> each(|f| {
  # Approximate p-value for F with df1=4, df2=70
  # (5 groups, 75 total observations)
  let interpretation = if f < 2.0 then "not significant"
    else if f < 4.0 then "possibly significant"
    else if f < 8.0 then "significant"
    else "highly significant"
  println("F = " + to_string(f) + ": " + interpretation)
})
```

ANOVA vs Multiple t-Tests

```
# Demonstrate why ANOVA and multiple t-tests give different answers

# Generate 5 groups from the SAME distribution (no true difference)
let null_groups = [
  rnorm(20, 10.0, 2.0),
  rnorm(20, 10.0, 2.0),
  rnorm(20, 10.0, 2.0),
  rnorm(20, 10.0, 2.0),
  rnorm(20, 10.0, 2.0)
]

# ANOVA: single test
let null_anova = anova(null_groups)
println("ANOVA on null data: p = " + to_string(round(null_anova.p_value, 4)))

# Pairwise t-tests: 10 tests
let null_pvals = []
[0, 1, 2, 3, 4] |> each(|i| {
  [0, 1, 2, 3, 4] |> each(|j| {
    if j > i then {
      let t = ttest(null_groups[i], null_groups[j])
      null_pvals = concat(null_pvals, [t.p_value])
      if t.p_value < 0.05 then
        println("  Groups " + to_string(i) + " vs " + to_string(j)
          + ": p = " + to_string(round(t.p_value, 4)) + " *SIGNIFICANT*")
    }
  })
})

let n_false_pos = null_pvals |> filter(|p| p < 0.05) |> len()
println("Pairwise t-tests found " + to_string(n_false_pos)
  + " 'significant' differences in null data")
println("ANOVA correctly says: p = " + to_string(round(null_anova.p_value,
4)))
```

Decision

Which Method Should You Use?

```
# Decision tree for multi-group comparisons

# 1. How many groups?
#    - 2 groups → Chapter 5 (t-test / Wilcoxon)
#    - 3+ groups → continue here

# 2. Is the data approximately normal (or log-normal)?
#    - Yes → ANOVA
#    - No → Kruskal-Wallis

# 3. Is the ANOVA significant?
#    - No → stop, no evidence of group differences
#    - Yes → proceed to post-hoc tests

# 4. Post-hoc pairwise tests:
#    - Parametric → pairwise t-tests with BH correction
#    - Non-parametric → pairwise Wilcoxon with BH correction

# 5. Always report:
#    - The omnibus test (ANOVA / KW) F-statistic and p-value
#    - Which pairs differ (with corrected p-values)
#    - Effect sizes for the comparisons that matter
#    - A visualization (boxplot, violin, or heatmap)
```

Complete Multi-Group Pipeline

```
# ============================================================
# Complete Multi-Group Analysis Pipeline
# ============================================================

# 1. Load data
let url = "https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-
seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
let gtex = download(url, "GTEx_v8_median_tpm.gct.gz") |> read_tsv()

# 2. Define groups (tissues)
let tissues = ["Brain - Cortex", "Heart - Left Ventricle",
               "Liver", "Lung", "Whole Blood"]

# 3. For each gene of interest, run the full pipeline
let target_genes = ["ALB", "GAPDH", "HBB", "INS", "MYH7"]

target_genes |> each(|gene| {
  println("=== " + gene + " ===")
  let row = gtex |> filter_by("Description", |d| d == gene)

  if len(row) > 0 then {
    # Extract values
    let values = tissues |> map(|t| log2(col_values(row, t)[0] + 1.0))

    # Summary per tissue
    tissues |> enumerate() |> each(|pair| {
      println("  " + pair[1] + ": " + to_string(round(values[pair[0]], 2)) +
" log2(TPM+1)")
    })

    # Variation metric
    let cv = stdev(values) / mean(values) * 100.0
    println("  CV: " + to_string(round(cv, 1)) + "%")

    # Pairwise comparisons would go here with full sample data
    println("")
  }
})
```

Pitfalls

Pitfall 1: Running Pairwise t-Tests Without Correction

```
# The wrong way: 10 pairwise t-tests at alpha=0.05
# Expected false positives: 10 * 0.05 = 0.5 per gene
# Across 20,000 genes: 10,000 false positive pairs

# The right way: ANOVA first, then corrected post-hoc tests
# This controls the overall error rate
```

Pitfall 2: Unbalanced Groups

When group sizes are very different, both ANOVA and post-hoc tests lose power, and the assumption of equal variance becomes more critical.

```
# Unbalanced groups: 5 vs 50 observations
let small_group = rnorm(5, 10.0, 2.0)
let large_group = rnorm(50, 10.0, 2.0)

# The test has very low power to detect differences
let unbalanced = ttest(small_group, large_group)
println("Unbalanced test (n=5 vs n=50): p = " +
to_string(round(unbalanced.p_value, 4)))
println("Hard to detect differences with such unequal group sizes")
```

Pitfall 3: ANOVA on Non-Independent Observations

If your “observations” are technical replicates from the same biological sample, they are not independent. ANOVA requires independent observations. Technical replicates should be averaged before analysis.

Pitfall 4: Ignoring the Overall ANOVA Result

If the omnibus ANOVA is not significant ($p > 0.05$), you should NOT proceed to pairwise comparisons. Some pairs may appear significant by chance, but the overall test says there is no evidence of group differences.

Pitfall 5: Treating Ordinal Groups as Categorical

If your groups have a natural order (dose response, time points, developmental stages), one-way ANOVA ignores this ordering. You lose statistical power. A trend test or regression is more appropriate.

```
# Wrong: treating dose as categorical
let doses = ["0mg", "1mg", "5mg", "10mg", "50mg"]
# ANOVA treats these as 5 unordered groups

# Better: use regression with dose as a continuous variable
# This captures the trend and is more powerful
let dose_values = [0.0, 1.0, 5.0, 10.0, 50.0]
let responses = [2.1, 3.4, 5.8, 7.2, 8.9]

let dose_model = lm(dose_values, responses)
println("Dose-response regression:")
println("  slope = " + to_string(round(dose_model.coefficients[1], 4)))
println("  p = " + to_string(dose_model.p_value))
println("  R-squared = " + to_string(round(dose_model.r_squared, 4)))
```

Pitfall 6: Using Median Data for Hypothesis Testing

The GTEx median TPM dataset gives one value per tissue per gene. With only one observation per group, you cannot run a proper ANOVA. You need the per-sample data for formal hypothesis testing. The median dataset is useful for exploration and visualization, but not for inference about individual genes.

```
# Important caveat:
# The GTEx median file has 1 value per tissue per gene
# ANOVA requires multiple observations per group
# For proper inference, use the per-sample expression matrix:
# https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-
seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_tpm.gct.gz
# (Warning: this file is ~3 GB)
println("For formal tissue comparisons, use per-sample data from GTEx")
println("The median file is for exploration only")
```

Where Next?

This chapter covered multi-group comparisons under the assumption that your data is reasonably well-behaved (approximately normal, continuous). But biological data often violates these assumptions badly:

- Allele frequencies are bounded between 0 and 1
- Count data is discrete and non-negative
- Many genomic measurements are heavily skewed
- Outliers are common

Chapter 7 covers **non-parametric methods** — tests that make minimal assumptions about the data distribution. These are your fallback when ANOVA assumptions fail.

We also deferred the critical question of multiple testing correction to Chapter 8. When you run ANOVA on 20,000 genes, the 5% significance level gives 1,000 false positives. Chapter 8 explains Bonferroni, Benjamini-Hochberg, and other correction methods.

Key Takeaways

1. **Never run pairwise t-tests** without an omnibus test first
2. **ANOVA** is the parametric omnibus test for 3+ groups
3. **Kruskal-Wallis** is the non-parametric alternative
4. **Post-hoc pairwise tests** must be corrected for multiple comparisons
5. **Bonferroni** is conservative but simple; **BH** is usually preferred
6. **Eta-squared** measures effect size for multi-group comparisons
7. **Interaction effects** arise when one factor modifies another
8. **Heatmaps** and **violin plots** are natural visualizations for multi-group data
9. **Ordered groups** (dose, time) should use trend tests, not ANOVA
10. **One observation per group** is insufficient for hypothesis testing

Exercises

1. Using the GTEx median TPM data, identify the 20 most tissue-specific genes (highest CV across tissues). For each, name the tissue where it is most highly expressed.

2. Pick a set of 10 cardiac-specific genes. Compare their expression across Heart - Left Ventricle, Heart - Atrial Appendage, and three non-cardiac tissues. Run ANOVA and post-hoc tests.

3. For GAPDH, compare expression across all brain regions in GTEx. Is there significant variation even within the “housekeeping” label?

4. Simulate 5 groups of 20 observations each, all from N(0, 1). Run ANOVA 1000 times. How often is the ANOVA significant at $p < 0.05$? (Should be ~5%.)

5. Now simulate 5 groups where one group has mean = 1 and the others have mean = 0 (all SD = 1, n = 20). How often does ANOVA detect this? Try increasing n to 50 and 100. This is a power analysis.

Summary Table

Concept	Key Point
Pairwise t-tests	Inflates false positives — do not use alone
ANOVA	Omnibus test: “Is there ANY difference among groups?”
F-statistic	Between-group variance / within-group variance
Kruskal-Wallis	Non-parametric ANOVA — uses ranks
Post-hoc tests	Identify WHICH groups differ after significant ANOVA
Bonferroni	Most conservative correction — FWER control
Benjamini-Hochberg	Less conservative — FDR control, preferred in genomics
Eta-squared	Effect size for ANOVA — proportion of variance explained
Interaction	When the effect of one factor depends on another
Visualization	Boxplots, violin plots, heatmaps for multi-group data

---

Next: Chapter 7 — Non-Parametric Methods

Non-Parametric Methods

> *"All models are wrong, but some are useful."* — George E.P. Box

The t-test assumes normality. ANOVA assumes normality and equal variance. But allele frequencies cluster near 0 and 1. Count data is discrete. Pathogenicity scores are ordinal. Constraint metrics are heavily skewed. When the parametric assumptions fail, non-parametric methods step in — tests that work regardless of the underlying distribution.

This chapter teaches you when parametric assumptions break down, which non-parametric tests to use instead, and when the Central Limit Theorem rescues parametric methods despite non-normality.

The Problem

A human genetics researcher wants to compare constraint metrics between genes associated with disease and those that are not. Constraint metrics measure how intolerant a gene is to functional mutations — highly constrained genes are under strong purifying selection and are more likely to cause disease when mutated.

The constraint scores from gnomAD are notoriously non-normal. The loss-of-function observed/expected upper fraction (LOEUF) has a long right tail. The probability of loss-of-function intolerance (pLI) is bimodal — most genes have pLI near 0 or near 1, with few in between. No parametric test is appropriate for these distributions.

She also wants to compare the distribution of ClinVar pathogenic variants across gene categories. The count data is heavily zero-inflated — most genes have no pathogenic variants in ClinVar.

Why It Matters

Non-parametric methods matter because real biological data routinely violates parametric assumptions:

- **Allele frequencies** are bounded between 0 and 1 and often heavily skewed toward rare variants
- **Constraint scores** (pLI, LOEUF, missense Z-scores) have non-standard distributions
- **Count data** (variant counts, read counts at low coverage) is discrete and often zero-inflated
- **Pathogenicity scores** (CADD, REVEL, PolyPhen) have complex, non-normal distributions
- **Clinical scores** (Apgar, GCS, pain scales) are ordinal, not continuous
- **Survival times** are right-censored and typically right-skewed

Using a t-test on bimodal data gives meaningless p-values. The test statistic assumes the data has a single peak and symmetric spread — neither of which is true for pLI scores.

Non-parametric tests avoid these problems by working with **ranks** instead of raw values. The actual magnitude of the data does not matter — only the ordering. This makes them robust to outliers, skewness, and non-normality.

The trade-off: when the data truly IS normal, non-parametric tests are less powerful (they need more samples to detect the same effect). But the robustness is usually worth the cost.

The Data

We use two datasets in this chapter:

gnomAD Constraint Metrics

Gene-level constraint metrics from gnomAD v4.1, which quantify how intolerant each gene is to different classes of mutation.

```
# Download gnomAD constraint metrics
let constraint_url = "https://storage.googleapis.com/gcp-public-data--
gnomad/release/4.1/constraint/gnomad.v4.1.constraint_metrics.tsv"
let constraint_file = download(constraint_url, "gnomad_v4.1_constraint.tsv")
println("Downloaded: " + constraint_file)
```

ClinVar Variant Summary

Curated clinical variant annotations from ClinVar, the NCBI database linking human genetic variants to clinical significance.

```
# Download ClinVar variant summary
let clinvar_url =
"https://ftp.ncbi.nlm.nih.gov/pub/clinvar/tab_delimited/variant_summary.txt.g
z"
let clinvar_file = download(clinvar_url, "variant_summary.txt.gz")
println("Downloaded: " + clinvar_file)
```

Loading the Data

```
# Load gnomAD constraint data
let constraint = read_tsv("gnomad_v4.1_constraint.tsv")
println("Constraint rows: " + to_string(len(constraint)))
println("Constraint columns: " + to_string(ncols(constraint)))
columns(constraint) |> each(|c| println("  " + c))
```

```
# Preview the constraint data
take(constraint, 10)
```

```
# Load ClinVar data
let clinvar = read_tsv("variant_summary.txt.gz")
println("ClinVar rows: " + to_string(len(clinvar)))
println("ClinVar columns: " + to_string(ncols(clinvar)))
```

```
# Identify key columns in ClinVar
# We need GeneSymbol, ClinicalSignificance, and Assembly (GRCh38)
let clinvar_grch38 = clinvar
  |> filter_by("Assembly", |a| a == "GRCh38")

println("ClinVar variants (GRCh38): " + to_string(len(clinvar_grch38)))
```

Exploration

The Shape of Constraint Scores

```
# Look at the distribution of key constraint metrics
let oe_lof = col_values(constraint, "oe_lof") |> filter(|x| is_none(x) ==
false)
let oe_mis = col_values(constraint, "oe_mis") |> filter(|x| is_none(x) ==
false)
let oe_syn = col_values(constraint, "oe_syn") |> filter(|x| is_none(x) ==
false)

println("=== Constraint Score Distributions ===")
println("o/e LoF (loss of function):")
println("  n      = " + to_string(len(oe_lof)))
println("  mean   = " + to_string(round(mean(oe_lof), 4)))
println("  median = " + to_string(round(median(oe_lof), 4)))
println("  sd     = " + to_string(round(stdev(oe_lof), 4)))
println("  skew indicator (mean/median): " + to_string(round(mean(oe_lof) /
median(oe_lof), 3)))

println("o/e Missense:")
println("  n      = " + to_string(len(oe_mis)))
println("  mean   = " + to_string(round(mean(oe_mis), 4)))
println("  median = " + to_string(round(median(oe_mis), 4)))
println("  sd     = " + to_string(round(stdev(oe_mis), 4)))
```

```
# Histograms of constraint scores
histogram(oe_lof, "o/e Loss-of-Function Ratio")
  |> save_plot("oe_lof_histogram.svg")

histogram(oe_mis, "o/e Missense Ratio")
  |> save_plot("oe_mis_histogram.svg")

histogram(oe_syn, "o/e Synonymous Ratio")
  |> save_plot("oe_syn_histogram.svg")
```

Notice the distributions: o/e LoF is heavily right-skewed (most genes have values near 0-1, but some have very high values). o/e synonymous should be centered around 1.0 (synonymous variants are under little selection). These are NOT normal distributions.

QQ Plots to Diagnose Non-Normality

```
# QQ plots for constraint scores
qq_plot(oe_lof, "o/e LoF QQ Plot")
  |> save_plot("oe_lof_qq.svg")

qq_plot(oe_mis, "o/e Missense QQ Plot")
  |> save_plot("oe_mis_qq.svg")

qq_plot(oe_syn, "o/e Synonymous QQ Plot")
  |> save_plot("oe_syn_qq.svg")
```

If the QQ plot curves away from the diagonal, the data is not normally distributed. An S-curve indicates heavy tails. A curve bending up on the right indicates right skew. Points falling off at both ends indicate outliers.

How to read a QQ plot:

- Points on the diagonal line: data follows a normal distribution
- Points curving above the line at the right: right-skewed (long upper tail)
- Points curving below the line at the left: left-skewed (long lower tail)
- S-shaped curve: heavy tails (kurtosis)
- Isolated points far from the line: outliers

```
# Compare: log-transform and re-check normality
let log_oe_lof = oe_lof |> filter(|x| x > 0.0) |> map(|x| log2(x))

println("After log2 transform:")
println("  mean   = " + to_string(round(mean(log_oe_lof), 4)))
println("  median = " + to_string(round(median(log_oe_lof), 4)))
println("  mean/median ratio: " + to_string(round(mean(log_oe_lof) /
median(log_oe_lof), 3)))

qq_plot(log_oe_lof, "log2(o/e LoF) QQ Plot")
  |> save_plot("log_oe_lof_qq.svg")

# Better? Maybe. But still not great for bimodal distributions like pLI.
```

Disease Gene Classification

To compare constraint between disease and non-disease genes, we need to classify genes. We will use ClinVar: genes with pathogenic variants are “disease genes.”

```
# Count pathogenic variants per gene from ClinVar
let pathogenic = clinvar_grch38
  |> filter_by("ClinicalSignificance", |cs| contains(cs, "Pathogenic"))

let patho_by_gene = pathogenic
  |> group_by("GeneSymbol")
  |> count_by("GeneSymbol")

println("Genes with pathogenic variants: " + to_string(len(patho_by_gene)))
```

```
# Get top disease genes by number of pathogenic variants
let top_disease = patho_by_gene
  |> arrange("count")
  |> tail(20)

println("=== Top 20 Disease Genes (by ClinVar pathogenic variant count) ===")
top_disease
```

```
# Create disease gene set (genes with >= 5 pathogenic variants)
let disease_genes = patho_by_gene
  |> filter_by("count", |n| n >= 5)
  |> col_values("GeneSymbol")

println("Disease genes (>=5 pathogenic variants): " +
to_string(len(disease_genes)))
```

```
# Split constraint scores by disease status
let constraint_with_disease = constraint
  |> mutate("is_disease", |row| {
    let gene = row.gene
    if disease_genes |> contains(gene) then "Disease" else "Non-disease"
  })

let disease_lof = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Disease")
  |> col_values("oe_lof")
  |> filter(|x| is_none(x) == false)

let nondisease_lof = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Non-disease")
  |> col_values("oe_lof")
  |> filter(|x| is_none(x) == false)

println("=== o/e LoF by Disease Status ===")
println("Disease genes:")
println("  n      = " + to_string(len(disease_lof)))
println("  mean   = " + to_string(round(mean(disease_lof), 4)))
println("  median = " + to_string(round(median(disease_lof), 4)))
println("Non-disease genes:")
println("  n      = " + to_string(len(nondisease_lof)))
println("  mean   = " + to_string(round(mean(nondisease_lof), 4)))
println("  median = " + to_string(round(median(nondisease_lof), 4)))
```

```
# Boxplot comparison
let lof_comparison = table(
  "oe_lof", concat(disease_lof, nondisease_lof),
  "group", concat(
    disease_lof |> map(|_| "Disease"),
    nondisease_lof |> map(|_| "Non-disease")
  )
)

boxplot(lof_comparison, "group", "oe_lof")
  |> save_plot("oe_lof_disease_comparison.svg")
```

The Analysis

Step 1: Wilcoxon Rank-Sum Test (Mann-Whitney U)

The Wilcoxon rank-sum test compares two groups without assuming normality. It tests whether one group tends to produce larger values than the other.

How it works:

1. Combine all observations from both groups
2. Rank them from smallest to largest
3. Sum the ranks for each group
4. If one group has systematically higher ranks, the groups differ

```
# Wilcoxon rank-sum: disease vs non-disease genes by o/e LoF
let wilcox_lof = wilcoxon(disease_lof, nondisease_lof)
println("=== Wilcoxon Rank-Sum Test: o/e LoF ===")
println("W statistic: " + to_string(wilcox_lof.statistic))
println("p-value:     " + to_string(wilcox_lof.p_value))
```

```
# Compare with t-test on the same data
let ttest_lof = ttest(disease_lof, nondisease_lof)
println("=== t-test on Same Data ===")
println("t statistic: " + to_string(round(ttest_lof.statistic, 4)))
println("p-value:     " + to_string(ttest_lof.p_value))

println("")
println("Both tests should agree when the data is well-behaved.")
println("They may disagree when outliers or skewness dominate.")
```

```
# Now compare on missense constraint
let disease_mis = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Disease")
  |> col_values("oe_mis")
  |> filter(|x| is_none(x) == false)

let nondisease_mis = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Non-disease")
  |> col_values("oe_mis")
  |> filter(|x| is_none(x) == false)

let wilcox_mis = wilcoxon(disease_mis, nondisease_mis)
let ttest_mis = ttest(disease_mis, nondisease_mis)

println("=== Missense Constraint: Disease vs Non-disease ===")
println("Wilcoxon p: " + to_string(wilcox_mis.p_value))
println("t-test p:   " + to_string(ttest_mis.p_value))
println("Disease mean:     " + to_string(round(mean(disease_mis), 4)))
println("Non-disease mean: " + to_string(round(mean(nondisease_mis), 4)))
```

Step 2: Wilcoxon Signed-Rank Test (Paired Non-Parametric)

The signed-rank test is the paired version of the rank-sum test. It tests whether paired differences tend to be positive or negative.

```
# Paired comparison: o/e LoF vs o/e missense within the same gene
# These are naturally paired — same gene, different constraint types
let paired_lof = col_values(constraint, "oe_lof")
let paired_mis = col_values(constraint, "oe_mis")

# Keep only rows where both values are available
let valid_pairs = zip(paired_lof, paired_mis)
  |> filter(|pair| is_none(pair[0]) == false and is_none(pair[1]) == false)

let valid_lof = valid_pairs |> map(|p| p[0])
let valid_mis = valid_pairs |> map(|p| p[1])

println("Valid pairs: " + to_string(len(valid_lof)))

# Paired Wilcoxon signed-rank test
let paired_wilcox = wilcoxon(valid_lof, valid_mis)
println("=== Wilcoxon Signed-Rank: o/e LoF vs o/e Missense ===")
println("W statistic: " + to_string(paired_wilcox.statistic))
println("p-value:     " + to_string(paired_wilcox.p_value))

# Compare with paired t-test
let paired_t = ttest_paired(valid_lof, valid_mis)
println("Paired t-test p: " + to_string(paired_t.p_value))
```

```
# Distribution of differences
let diffs = zip(valid_lof, valid_mis)
  |> map(|p| p[0] - p[1])

println("Differences (LoF - Missense):")
println("  mean   = " + to_string(round(mean(diffs), 4)))
println("  median = " + to_string(round(median(diffs), 4)))
println("  % negative = " + to_string(round(
  to_float(diffs |> filter(|d| d < 0.0) |> len()) / to_float(len(diffs)) *
100.0, 1
)) + "%")

histogram(diffs, "o/e LoF minus o/e Missense")
  |> save_plot("lof_vs_mis_differences.svg")
```

Step 3: Kolmogorov-Smirnov Test

The KS test compares the entire distribution of two samples, not just a measure of central tendency. It asks: “Do these two samples come from the same distribution?”

The KS statistic is the maximum vertical distance between the two empirical cumulative distribution functions (ECDFs).

```
# KS test: disease vs non-disease constraint distributions
let ks_lof = ks_test(disease_lof, nondisease_lof)
println("=== Kolmogorov-Smirnov Test: o/e LoF ===")
println("KS statistic: " + to_string(round(ks_lof.statistic, 4)))
println("p-value:      " + to_string(ks_lof.p_value))
```

```
# KS test on missense constraint
let ks_mis = ks_test(disease_mis, nondisease_mis)
println("=== KS Test: o/e Missense ===")
println("KS statistic: " + to_string(round(ks_mis.statistic, 4)))
println("p-value:      " + to_string(ks_mis.p_value))
```

```
# KS test on synonymous constraint (should be less different)
let disease_syn = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Disease")
  |> col_values("oe_syn")
  |> filter(|x| is_none(x) == false)

let nondisease_syn = constraint_with_disease
  |> filter_by("is_disease", |d| d == "Non-disease")
  |> col_values("oe_syn")
  |> filter(|x| is_none(x) == false)

let ks_syn = ks_test(disease_syn, nondisease_syn)
println("=== KS Test: o/e Synonymous ===")
println("KS statistic: " + to_string(round(ks_syn.statistic, 4)))
println("p-value:      " + to_string(ks_syn.p_value))
println("(Synonymous variants are neutral — less difference expected)")
```

When to use KS vs Wilcoxon:

- Wilcoxon tests whether one group tends to have larger values (shift)
- KS tests whether the distributions differ in ANY way (shape, spread, location)
- KS is more general but less powerful for detecting simple shifts
- Wilcoxon is more powerful when the distributions differ only in location

Step 4: Spearman vs Pearson Correlation

Pearson correlation measures linear association and assumes both variables are normally distributed. Spearman correlation measures monotonic association using ranks — it works for any distribution shape.

```
# Pearson vs Spearman: o/e LoF vs o/e Missense
let pearson_r = cor(valid_lof, valid_mis)
let spearman_r = spearman(valid_lof, valid_mis)

println("=== Correlation: o/e LoF vs o/e Missense ===")
println("Pearson r:  " + to_string(round(pearson_r, 4)))
println("Spearman rho: " + to_string(round(spearman_r.statistic, 4)))
println("Spearman p:   " + to_string(spearman_r.p_value))
```

```
# Scatter plot to visualize the relationship
let cor_data = table(
  "oe_lof", valid_lof |> take(5000),
  "oe_mis", valid_mis |> take(5000)
)

scatter(cor_data, "oe_lof", "oe_mis")
  |> save_plot("lof_vs_mis_scatter.svg")
```

When do Pearson and Spearman disagree?

```
# Scenario 1: Perfect nonlinear relationship
let x_nonlinear = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]
let y_nonlinear = x_nonlinear |> map(|x| pow(x, 2.0))  # y = x^2

let pearson_nl = cor(x_nonlinear, y_nonlinear)
let spearman_nl = spearman(x_nonlinear, y_nonlinear)

println("=== Nonlinear Relationship (y = x^2) ===")
println("Pearson r:     " + to_string(round(pearson_nl, 4)))
println("Spearman rho:  " + to_string(round(spearman_nl.statistic, 4)))
println("Spearman is 1.0 because the relationship is perfectly monotonic")
println("Pearson < 1.0 because it is not perfectly linear")
```

```
# Scenario 2: Outlier sensitivity
let x_clean = rnorm(50, 0.0, 1.0)
let y_clean = x_clean |> map(|x| x * 0.8 + rnorm(1, 0.0, 0.5)[0])

# Add one extreme outlier
let x_outlier = concat(x_clean, [20.0])
let y_outlier = concat(y_clean, [-15.0])

let pearson_clean = cor(x_clean, y_clean)
let pearson_outlier = cor(x_outlier, y_outlier)
let spearman_clean = spearman(x_clean, y_clean)
let spearman_outlier = spearman(x_outlier, y_outlier)

println("=== Outlier Sensitivity ===")
println("Without outlier:")
println("  Pearson r:    " + to_string(round(pearson_clean, 4)))
println("  Spearman rho: " + to_string(round(spearman_clean.statistic, 4)))
println("With one outlier added:")
println("  Pearson r:    " + to_string(round(pearson_outlier, 4)))
println("  Spearman rho: " + to_string(round(spearman_outlier.statistic, 4)))
println("Pearson is heavily affected; Spearman is robust")
```

Step 5: Kendall’s Tau

Kendall’s tau is another rank-based correlation measure. It counts the number of concordant and discordant pairs. It is more robust than Spearman for small samples and easier to interpret: tau = (concordant - discordant) / total pairs.

```
# Kendall's tau: o/e LoF vs o/e Missense
let kendall_result = kendall(valid_lof |> take(1000), valid_mis |>
take(1000))
println("=== Kendall's Tau ===")
println("tau:     " + to_string(round(kendall_result.statistic, 4)))
println("p-value: " + to_string(kendall_result.p_value))

# Compare all three correlation measures
println("")
println("=== Correlation Comparison ===")
println("Pearson r:    " + to_string(round(pearson_r, 4)) + " (linear,
sensitive to outliers)")
println("Spearman rho: " + to_string(round(spearman_r.statistic, 4)) + "
(monotonic, rank-based)")
println("Kendall tau:  " + to_string(round(kendall_result.statistic, 4)) + "
(concordance, most robust)")
```

Step 6: Bootstrap Methods

When no standard test fits your data, bootstrap resampling lets you construct confidence intervals for ANY statistic by resampling your data with replacement.

```
# Bootstrap confidence interval for the median difference
# between disease and non-disease o/e LoF scores

let observed_median_diff = median(disease_lof) - median(nondisease_lof)
println("Observed median difference: " +
to_string(round(observed_median_diff, 4)))

# Bootstrap: resample with replacement and compute the statistic
let n_boot = 1000
let boot_diffs = range(0, n_boot) |> map(|_| {
  # Resample disease group
  let boot_disease = sample(disease_lof, len(disease_lof))
  let boot_nondisease = sample(nondisease_lof, len(nondisease_lof))
  median(boot_disease) - median(boot_nondisease)
})

# Bootstrap 95% CI
let sorted_boots = boot_diffs |> sort()
let ci_lower_idx = round(to_float(n_boot) * 0.025, 0)
let ci_upper_idx = round(to_float(n_boot) * 0.975, 0)

let boot_ci_lower = sorted_boots[to_int(ci_lower_idx)]
let boot_ci_upper = sorted_boots[to_int(ci_upper_idx)]

println("=== Bootstrap 95% CI for Median Difference ===")
println("Observed:  " + to_string(round(observed_median_diff, 4)))
println("Bootstrap CI: [" + to_string(round(boot_ci_lower, 4))
  + ", " + to_string(round(boot_ci_upper, 4)) + "]")
println("Bootstrap SE: " + to_string(round(stdev(boot_diffs), 4)))
```

```
# Bootstrap distribution
histogram(boot_diffs, "Bootstrap Distribution of Median Difference")
  |> save_plot("bootstrap_median_diff.svg")

# If the CI does not include 0, the difference is significant
let ci_contains_zero = boot_ci_lower <= 0.0 and boot_ci_upper >= 0.0
println("CI contains zero: " + to_string(ci_contains_zero))
println("Interpretation: " + if ci_contains_zero
  then "Cannot reject null (no significant difference)"
  else "Significant difference (CI excludes zero)")
```

```
# Bootstrap for Spearman correlation
let n_cor_boot = 1000
let boot_spearman = range(0, n_cor_boot) |> map(|_| {
  let indices = sample(range(0, len(valid_lof)), len(valid_lof))
  let boot_x = indices |> map(|i| valid_lof[i])
  let boot_y = indices |> map(|i| valid_mis[i])
  spearman(boot_x, boot_y).statistic
})

let sorted_sr = boot_spearman |> sort()
let sr_ci_lower = sorted_sr[to_int(round(to_float(n_cor_boot) * 0.025, 0))]
let sr_ci_upper = sorted_sr[to_int(round(to_float(n_cor_boot) * 0.975, 0))]

println("=== Bootstrap 95% CI for Spearman Correlation ===")
println("Point estimate: " + to_string(round(spearman_r.statistic, 4)))
println("Bootstrap CI: [" + to_string(round(sr_ci_lower, 4))
  + ", " + to_string(round(sr_ci_upper, 4)) + "]")
```

Step 7: When Parametric Tests Survive Non-Normality (CLT)

The Central Limit Theorem states that the sampling distribution of the mean approaches normality as the sample size grows, regardless of the underlying distribution. This means that for LARGE samples, the t-test can be valid even when the data is non-normal.

```
# Demonstrate CLT: even with skewed data, the t-test works for large n

# Highly skewed data: exponential-like distribution
let skewed = rnorm(1000, 0.0, 1.0) |> map(|x| exp(x))  # log-normal

println("Skewed data:")
println("  mean   = " + to_string(round(mean(skewed), 4)))
println("  median = " + to_string(round(median(skewed), 4)))
println("  ratio  = " + to_string(round(mean(skewed) / median(skewed), 2)))

histogram(skewed, "Skewed Distribution (exp of normal)")
  |> save_plot("skewed_distribution.svg")
```

```
# CLT simulation: sample means become normal
# Take many samples of size n from the skewed distribution
# and compute the mean of each sample

let sample_sizes = [5, 10, 30, 100]
sample_sizes |> each(|n| {
  let n_samples = 500
  let sample_means = range(0, n_samples) |> map(|_| {
    let s = sample(skewed, n)
    mean(s)
  })

  println("n = " + to_string(n) + ":")
  println("  mean of means = " + to_string(round(mean(sample_means), 4)))
  println("  SD of means   = " + to_string(round(stdev(sample_means), 4)))

  # QQ plot of sample means
  qq_plot(sample_means, "Sample Means (n=" + to_string(n) + ")")
    |> save_plot("clt_qq_n" + to_string(n) + ".svg")
})
```

The rule of thumb: With n > 30 per group, the t-test is usually robust to moderate non-normality. But this depends on HOW non-normal the data is:

- **Symmetric** non-normal: t-test is robust even for n = 15
- **Moderately skewed**: t-test needs n > 30
- **Heavily skewed or bimodal**: use non-parametric even with n > 100

```
# When does the CLT NOT save you?

# Bimodal data: pLI scores (near 0 or near 1)
# The mean is meaningless for bimodal data
let bimodal = concat(rnorm(500, 0.1, 0.05), rnorm(500, 0.9, 0.05))
  |> map(|x| max(0.0, min(1.0, x)))

println("Bimodal data (like pLI):")
println("  mean   = " + to_string(round(mean(bimodal), 4)))
println("  median = " + to_string(round(median(bimodal), 4)))
println("  The mean is ~0.5, but almost NO values are near 0.5!")
println("  The t-test on means is meaningless for bimodal data.")

histogram(bimodal, "Bimodal Distribution (like pLI)")
  |> save_plot("bimodal_distribution.svg")
```

Step 8: Comparing Distributions with ClinVar Data

```
# Compare pathogenic variant counts between gene categories

# Count pathogenic variants per gene
let patho_counts = pathogenic
  |> group_by("GeneSymbol")
  |> count_by("GeneSymbol")

# Merge with constraint data
let merged = left_join(constraint, patho_counts, "gene", "GeneSymbol")
  |> fill_na("count", 0)

println("Genes with constraint + ClinVar data: " + to_string(len(merged)))
```

```
# Split genes into constrained vs unconstrained
# Using LOEUF < 0.35 as the gnomAD recommended threshold for constrained
genes
let constrained = merged
  |> filter_by("oe_lof_upper", |x| is_none(x) == false and x < 0.35)
  |> col_values("count")

let unconstrained = merged
  |> filter_by("oe_lof_upper", |x| is_none(x) == false and x >= 0.35)
  |> col_values("count")

println("=== Pathogenic Variant Counts ===")
println("Constrained genes (LOEUF < 0.35):")
println("  n      = " + to_string(len(constrained)))
println("  mean   = " + to_string(round(mean(constrained), 2)))
println("  median = " + to_string(round(median(constrained), 1)))

println("Unconstrained genes (LOEUF >= 0.35):")
println("  n      = " + to_string(len(unconstrained)))
println("  mean   = " + to_string(round(mean(unconstrained), 2)))
println("  median = " + to_string(round(median(unconstrained), 1)))
```

```
# This data is count data — highly non-normal, zero-inflated
# Wilcoxon is appropriate, t-test is not
let wilcox_counts = wilcoxon(constrained, unconstrained)
let ttest_counts = ttest(constrained, unconstrained)

println("=== Pathogenic Variant Count Comparison ===")
println("Wilcoxon p: " + to_string(wilcox_counts.p_value))
println("t-test p:   " + to_string(ttest_counts.p_value))

# KS test on the same data
let ks_counts = ks_test(constrained, unconstrained)
println("KS test p:  " + to_string(ks_counts.p_value))
```

```
# Histogram of pathogenic variant counts — see the zero-inflation
histogram(constrained, "Pathogenic Variants per Constrained Gene")
  |> save_plot("patho_constrained_hist.svg")

histogram(unconstrained, "Pathogenic Variants per Unconstrained Gene")
  |> save_plot("patho_unconstrained_hist.svg")
```

Step 9: Multiple Constraint Metrics Comparison

```
# Compare all constraint metrics between disease and non-disease genes
let metrics = ["oe_lof", "oe_mis", "oe_syn"]
let metric_names = ["o/e LoF", "o/e Missense", "o/e Synonymous"]

let all_pvals = []

metrics |> enumerate() |> each(|pair| {
  let metric = pair[1]
  let name = metric_names[pair[0]]

  let disease_vals = constraint_with_disease
    |> filter_by("is_disease", |d| d == "Disease")
    |> col_values(metric)
    |> filter(|x| is_none(x) == false)

  let nondisease_vals = constraint_with_disease
    |> filter_by("is_disease", |d| d == "Non-disease")
    |> col_values(metric)
    |> filter(|x| is_none(x) == false)

  let w_res = wilcoxon(disease_vals, nondisease_vals)
  let ks_res = ks_test(disease_vals, nondisease_vals)

  println(name + ":")
  println("  Disease median:     " + to_string(round(median(disease_vals),
4)))
  println("  Non-disease median: " + to_string(round(median(nondisease_vals),
4)))
  println("  Wilcoxon p: " + to_string(w_res.p_value))
  println("  KS test p:  " + to_string(ks_res.p_value))

  all_pvals = concat(all_pvals, [w_res.p_value])
})

# Correct for multiple metrics tested
let adjusted = p_adjust(all_pvals, "bh")
println("=== BH-Adjusted P-values ===")
metrics |> enumerate() |> each(|pair| {
  println(metric_names[pair[0]] + ": " + to_string(round(adjusted[pair[0]],
6)))
})
```

Interpretation

Choosing Between Parametric and Non-Parametric

```
# Decision framework:

# 1. Check sample size
#    - n < 15 per group: non-parametric (CLT won't help)
#    - n = 15-30: depends on how non-normal
#    - n > 30: parametric usually OK (CLT) unless bimodal

# 2. Check distribution shape
#    - Symmetric with moderate tails: parametric OK
#    - Skewed: log-transform, then parametric
#    - Bimodal: non-parametric (no transform helps)
#    - Zero-inflated counts: non-parametric

# 3. Check for outliers
#    - Few outliers: consider removing or use non-parametric
#    - Systematic outliers: non-parametric

# 4. Check purpose
#    - Testing means: t-test (if assumptions met)
#    - Testing distributions: KS test
#    - Testing ranks/ordering: Wilcoxon
#    - Correlation: Spearman or Kendall
```

What the Tests Actually Test

Test	Null Hypothesis	Sensitive To
t-test	Means are equal	Location shift
Wilcoxon rank-sum	Distributions are identical	Location shift (via ranks)
KS test	Distributions are identical	Any difference (shape, location, spread)
Spearman	No monotonic association	Monotonic relationships
Kendall	No concordance	Concordance of pairs

The Power-Robustness Trade-Off

```
# Simulation: compare power of t-test vs Wilcoxon

# Normal data: t-test should be slightly more powerful
let n_sims = 500
let n_per_group = 20
let true_diff = 0.5  # small effect

let t_significant = 0
let w_significant = 0

range(0, n_sims) |> each(|_| {
  let group1 = rnorm(n_per_group, 0.0, 1.0)
  let group2 = rnorm(n_per_group, true_diff, 1.0)

  let t_p = ttest(group1, group2).p_value
  let w_p = wilcoxon(group1, group2).p_value

  if t_p < 0.05 then t_significant = t_significant + 1
  if w_p < 0.05 then w_significant = w_significant + 1
})

println("=== Power Comparison (Normal Data, d=0.5, n=20) ===")
println("t-test power:   " + to_string(round(to_float(t_significant) /
to_float(n_sims) * 100.0, 1)) + "%")
println("Wilcoxon power: " + to_string(round(to_float(w_significant) /
to_float(n_sims) * 100.0, 1)) + "%")
println("t-test wins by ~5% when data is truly normal")
```

```
# Skewed data: Wilcoxon should do better
let t_sig_skew = 0
let w_sig_skew = 0

range(0, n_sims) |> each(|_| {
  # Log-normal data (right-skewed)
  let group1 = rnorm(n_per_group, 0.0, 1.0) |> map(|x| exp(x))
  let group2 = rnorm(n_per_group, 0.3, 1.0) |> map(|x| exp(x))

  let t_p = ttest(group1, group2).p_value
  let w_p = wilcoxon(group1, group2).p_value

  if t_p < 0.05 then t_sig_skew = t_sig_skew + 1
  if w_p < 0.05 then w_sig_skew = w_sig_skew + 1
})

println("=== Power Comparison (Skewed Data, n=20) ===")
println("t-test power:   " + to_string(round(to_float(t_sig_skew) /
to_float(n_sims) * 100.0, 1)) + "%")
println("Wilcoxon power: " + to_string(round(to_float(w_sig_skew) /
to_float(n_sims) * 100.0, 1)) + "%")
println("Wilcoxon is more powerful on skewed data")
```

Decision

The Complete Non-Parametric Toolkit

```
let toolkit = table(
  "scenario", [
    "Two independent groups, non-normal",
    "Two paired groups, non-normal",
    "Compare two distributions",
    "Correlation, non-normal",
    "Correlation, small sample",
    "CI for any statistic",
    "Multi-group, non-normal"
  ],
  "test", [
    "Wilcoxon rank-sum",
    "Wilcoxon signed-rank",
    "KS test",
    "Spearman",
    "Kendall",
    "Bootstrap",
    "Kruskal-Wallis (Ch. 6)"
  ],
  "biolang_function", [
    "wilcoxon(x, y)",
    "wilcoxon(x, y)",
    "ks_test(x, y)",
    "spearman(x, y)",
    "kendall(x, y)",
    "sample() + quantile()",
    "See Chapter 6"
  ]
)

toolkit
```

When to NOT Use Non-Parametric Tests

Non-parametric tests are not always better:

1. **When the data IS normal**: you lose statistical power for no reason
2. **When you need to estimate parameters**: non-parametric tests give p-values but not estimates of the mean difference or its CI
3. **When you need to adjust for covariates**: regression handles covariates naturally; non-parametric tests do not
4. **When the sample is very large**: CLT makes parametric tests robust, and they are more informative

---

Pitfalls

Pitfall 1: Using Pearson Correlation on Non-Linear Relationships

```
# The quadratic relationship trap
let x = range(0, 100) |> map(|i| to_float(i) / 10.0 - 5.0)
let y = x |> map(|v| pow(v, 2.0) + rnorm(1, 0.0, 2.0)[0])

let pearson_trap = cor(x, y)
let spearman_trap = spearman(x, y)

println("=== Quadratic Relationship (y = x^2 + noise) ===")
println("Pearson r:  " + to_string(round(pearson_trap, 4)))
println("Spearman rho: " + to_string(round(spearman_trap.statistic, 4)))
println("Pearson is near 0 because the relationship is not linear")
println("But the variables ARE related!")
println("Lesson: ALWAYS scatter-plot before computing correlation")
```

Pitfall 2: Confusing Rank-Sum with Signed-Rank

The Wilcoxon rank-sum test is for **independent** groups. The signed-rank test is for **paired** groups. Using the wrong one gives incorrect p-values.

Pitfall 3: Interpreting KS Test with Very Large Samples

```
# KS test is too sensitive with large n
# Any tiny difference becomes "significant"
let large_a = rnorm(10000, 0.0, 1.0)
let large_b = rnorm(10000, 0.01, 1.0)  # mean differs by only 0.01

let ks_large = ks_test(large_a, large_b)
println("=== KS Test with n=10,000, mean diff=0.01 ===")
println("KS statistic: " + to_string(round(ks_large.statistic, 4)))
println("p-value:      " + to_string(ks_large.p_value))
println("Significant? Probably. Meaningful? Probably not.")
println("Always examine the KS statistic, not just the p-value.")
println("KS stat < 0.05 indicates very similar distributions.")
```

Pitfall 4: Bootstrap with Too Few Resamples

```
# Too few bootstrap samples give unstable CIs
let data = rnorm(50, 10.0, 3.0)

# Only 50 bootstrap samples — CI will be noisy
let few_boots = range(0, 50) |> map(|_| mean(sample(data, len(data))))
let few_sorted = few_boots |> sort()
println("50 bootstraps CI:   [" + to_string(round(few_sorted[1], 3))
  + ", " + to_string(round(few_sorted[48], 3)) + "]")

# 1000 bootstrap samples — CI will be stable
let many_boots = range(0, 1000) |> map(|_| mean(sample(data, len(data))))
let many_sorted = many_boots |> sort()
println("1000 bootstraps CI: [" + to_string(round(many_sorted[25], 3))
  + ", " + to_string(round(many_sorted[974], 3)) + "]")

println("Use at least 1000 bootstrap resamples (10000 is better)")
```

Pitfall 5: Thinking Non-Parametric Means No Assumptions

Non-parametric tests DO have assumptions:

- Independence of observations (all tests)
- Similar shape for both groups (Wilcoxon rank-sum tests for shift)
- Continuous data (Wilcoxon; ties reduce power)

They just do not assume a specific distribution shape.

Putting It All Together

```
# ============================================================
# Complete Non-Parametric Analysis Pipeline
# ============================================================

# Research question: Are genes associated with disease more
# constrained than genes not associated with disease?

# 1. Load constraint and ClinVar data
let constraint_url = "https://storage.googleapis.com/gcp-public-data--
gnomad/release/4.1/constraint/gnomad.v4.1.constraint_metrics.tsv"
let clinvar_url =
"https://ftp.ncbi.nlm.nih.gov/pub/clinvar/tab_delimited/variant_summary.txt.g
z"

let constraint = download(constraint_url, "gnomad_constraint.tsv") |>
read_tsv()
let clinvar = download(clinvar_url, "variant_summary.txt.gz") |> read_tsv()

# 2. Classify genes
let pathogenic = clinvar
  |> filter_by("Assembly", |a| a == "GRCh38")
  |> filter_by("ClinicalSignificance", |cs| contains(cs, "Pathogenic"))
  |> group_by("GeneSymbol")
  |> count_by("GeneSymbol")
  |> filter_by("count", |n| n >= 5)
  |> col_values("GeneSymbol")

# 3. Split constraint scores
let disease_scores = constraint
  |> filter_by("gene", |g| pathogenic |> contains(g))
  |> col_values("oe_lof")
  |> filter(|x| is_none(x) == false)

let other_scores = constraint
  |> filter_by("gene", |g| (pathogenic |> contains(g)) == false)
  |> col_values("oe_lof")
  |> filter(|x| is_none(x) == false)

# 4. Check normality
qq_plot(disease_scores, "Disease genes o/e LoF") |>
save_plot("qq_disease.svg")
qq_plot(other_scores, "Non-disease genes o/e LoF") |>
save_plot("qq_other.svg")

# 5. Non-parametric tests
let w = wilcoxon(disease_scores, other_scores)
let ks = ks_test(disease_scores, other_scores)

# 6. Bootstrap CI for median difference
let obs_diff = median(disease_scores) - median(other_scores)
let boots = range(0, 1000) |> map(|_| {
  median(sample(disease_scores, len(disease_scores)))
    - median(sample(other_scores, len(other_scores)))
}) |> sort()

# 7. Report
println("=== Disease Gene Constraint Analysis ===")
println("Disease genes: n=" + to_string(len(disease_scores))
  + ", median o/e LoF = " + to_string(round(median(disease_scores), 4)))
println("Other genes:   n=" + to_string(len(other_scores))
  + ", median o/e LoF = " + to_string(round(median(other_scores), 4)))
println("Wilcoxon p:    " + to_string(w.p_value))
```

```
  println("KS test p:     " + to_string(ks.p_value))
  println("Median diff:   " + to_string(round(obs_diff, 4)))
  println("Bootstrap 95% CI: [" + to_string(round(boots[25], 4))
    + ", " + to_string(round(boots[974], 4)) + "]")
```

Where Next?

You now have a complete toolkit for comparing groups with non-normal data. But we have been carefully avoiding the biggest problem in genomics: when you run these tests on 20,000 genes, you get 20,000 p-values. At $p < 0.05$, a thousand of them will be "significant" by chance alone.

Chapter 8 tackles the **multiple testing problem** head-on. You will learn Bonferroni, Benjamini-Hochberg, and why the histogram of your p-values tells you how many true discoveries you have.

Key Takeaways

1. **Check normality** with QQ plots before choosing a test
2. **Wilcoxon rank-sum** is the non-parametric alternative to the t-test
3. **KS test** compares entire distributions, not just central tendency
4. **Spearman** correlation is robust to non-linearity and outliers
5. **Kendall's tau** is the most robust correlation for small samples
6. **Bootstrap** gives CIs for any statistic without distribution assumptions
7. **CLT** rescues parametric tests for large samples ($n > 30$), but not always
8. **Bimodal data** defeats both CLT and log-transformation
9. **Large samples** make everything significant — focus on effect size
10. **Always scatter-plot** before computing correlation

Exercises

1. Download gnomAD constraint metrics. Compare LOEUF between genes on chromosome X vs autosomes using Wilcoxon. Is there a biological reason for any difference?

2. For the top 100 most constrained genes (lowest LOEUF), use the KS test to compare their o/e missense distribution against all other genes.

3. Compute Spearman and Kendall correlation between o/e LoF and o/e missense for all genes. Bootstrap 95% CIs for both. Do they agree?
4. Simulate data from a bimodal distribution (mixture of two normals). Run t-tests and Wilcoxon tests 1000 times each. Compare false positive rates at alpha = 0.05. Which test maintains the correct rate?
5. Pick 10 genes from ClinVar with the most pathogenic variants. Look up their gnomAD constraint metrics. Are they all constrained? Discuss any exceptions.

Summary Table

Concept	Key Point
Non-parametric tests	Work with ranks, not raw values — no distribution assumption
Wilcoxon rank-sum	Two independent groups — tests whether one tends to be larger
Wilcoxon signed-rank	Paired groups — tests whether differences are systematic
KS test	Tests whether two distributions are identical
Spearman correlation	Monotonic association using ranks
Kendall's tau	Concordance of pairs — most robust for small n
Bootstrap	Resampling-based CI for any statistic
CLT	Means are approximately normal for large n
Bimodal data	CLT fails, means are meaningless, use non-parametric
Power trade-off	Non-parametric is ~5% less powerful on normal data

---

Next: Chapter 8 — The Multiple Testing Problem

The Multiple Testing Problem

> *"If you look at enough things, you're guaranteed to find something interesting — whether or not anything truly interesting exists."* — John Tukey

This is the most important chapter in the book. If you take away one lesson from studying biostatistics, let it be this: when you test thousands of hypotheses simultaneously, your p-values lie. A p-value of 0.01 does not mean what you think it means when you have computed 20,000 of them.

The multiple testing problem is not a theoretical curiosity. It is the central statistical challenge of genomics. Every differential expression analysis, every GWAS, every eQTL study, every enrichment analysis faces this problem. Getting it wrong means publishing false discoveries. Getting it right means understanding the difference between a p-value and a q-value, between FWER and FDR, between Bonferroni and Benjamini-Hochberg.

The Problem

A researcher has RNA-seq expression data from breast cancer (tumor) vs adjacent normal tissue. She runs a t-test on each of 20,000 genes and declares any gene with $p < 0.05$ as "differentially expressed." Her results: 3,200 significant genes.

Is this a remarkable finding? Or is it exactly what you would expect from testing 20,000 genes even if NONE of them were truly different?

At alpha = 0.05, you expect 5% of your true null hypotheses to produce a "significant" result by chance. If 15,000 of the 20,000 genes are truly not different, you expect 15,000 * 0.05 = 750 false positives. Her 3,200 hits might include 750 genes that have nothing to do with cancer.

That is the multiple testing problem.

Why It Matters

The Scale of Modern Genomics

```
# How many tests do common genomic analyses involve?

let analyses = table(
  "analysis", [
    "Differential expression (RNA-seq)",
    "GWAS (genome-wide association)",
    "eQTL mapping",
    "Differential methylation",
    "ATAC-seq peak comparison",
    "Proteomics (mass spec)",
    "Single-cell marker genes",
    "ChIP-seq binding comparison"
  ],
  "typical_tests", [
    20000,
    1000000,
    200000000,
    450000,
    100000,
    10000,
    25000,
    50000
  ],
  "expected_fp_at_005", [
    1000,
    50000,
    10000000,
    22500,
    5000,
    500,
    1250,
    2500
  ]
)

analyses
println("Expected false positives = n_tests * 0.05")
```

For a GWAS with 1 million SNPs, you expect 50,000 false positives at $p < 0.05$. For eQTL mapping (testing every gene against every nearby variant), you expect 10 million false positives. Without correction, your results are meaningless.

The Jelly Bean Analogy

Imagine testing 20 colors of jelly beans for a link to acne. At alpha = 0.05, you expect 1 color to show $p < 0.05$ by chance. You publish: “Green jelly beans cause acne!” This is exactly what happens in genomics, but with 20,000 tests instead of 20.

The Data

We return to the GSE183947 breast cancer expression dataset from Chapter 5. This time, our focus is not on individual gene comparisons but on what happens when we test ALL genes simultaneously.

Loading and Preparing

```
# Download and load the breast cancer expression data
let url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let expr = download(url, "GSE183947_fpkm.csv.gz") |> read_csv()

println("Genes: " + to_string(len(expr)))
println("Samples: " + to_string(ncols(expr)))

# Identify tumor and normal samples
let all_cols = columns(expr)
let tumor_cols = all_cols |> filter(|c| contains(c, "tumor"))
let normal_cols = all_cols |> filter(|c| contains(c, "normal"))
let gene_col = all_cols[0]

println("Tumor samples: " + to_string(len(tumor_cols)))
println("Normal samples: " + to_string(len(normal_cols)))
```

Running Genome-Wide t-Tests

```
# Test every gene: tumor vs normal
let genes = col_values(expr, gene_col)

let results = genes |> map(|gene| {
  let row = expr |> filter_by(gene_col, |g| g == gene)

  let tumor_vals = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
  let normal_vals = normal_cols |> map(|c| col_values(row, c)) |> flatten()

  # Log2 transform
  let lt = tumor_vals |> map(|x| log2(x + 1.0))
  let ln = normal_vals |> map(|x| log2(x + 1.0))

  let log2fc = mean(lt) - mean(ln)
  let t_res = ttest(lt, ln)

  record(
    "gene", gene,
    "log2fc", round(log2fc, 4),
    "p_value", t_res.p_value,
    "mean_tumor", round(mean(lt), 4),
    "mean_normal", round(mean(ln), 4)
  )
}) |> to_list() |> table()

println("Total genes tested: " + to_string(len(results)))
```

Exploration

The Histogram of P-Values

This is the single most important diagnostic plot in genomics. Before applying any correction, look at the histogram of your raw p-values.

```
# Histogram of raw p-values
let pvals = col_values(results, "p_value")
histogram(pvals, "Distribution of Raw P-values (20,000 genes)")
  |> save_plot("pvalue_histogram.svg")
```

How to read a p-value histogram:

Under the null hypothesis (no genes are truly different), p-values should follow a uniform distribution — a flat histogram from 0 to 1. Every bin should have approximately the same height.

In reality, you see:

- A **spike near 0**: these are the true positives (genes that are actually differentially expressed). The spike is your signal.
- A **flat region** from about 0.3 to 1.0: these are the true nulls. The height of this flat region tells you what fraction of genes are truly null.
- A **dip or bump** in the middle: this can indicate problems (confounding, batch effects, wrong test).

```
# Count p-values in bins to see the shape
let bins = [0.0, 0.01, 0.05, 0.10, 0.20, 0.50, 1.0]
let bin_names = ["0-0.01", "0.01-0.05", "0.05-0.10", "0.10-0.20", "0.20-
0.50", "0.50-1.0"]

bins |> enumerate() |> each(|pair| {
  let i = pair[0]
  if i < len(bins) - 1 then {
    let lower = bins[i]
    let upper = bins[i + 1]
    let count = pvals |> filter(|p| p >= lower and p < upper) |> len()
    let pct = round(to_float(count) / to_float(len(pvals)) * 100.0, 1)
    println(bin_names[i] + ": " + to_string(count) + " genes (" +
to_string(pct) + "%)")
  }
})
```

```
# Compare to the expected uniform distribution
let n_genes = len(pvals)
println("=== Expected vs Observed Under Null ===")
println("If ALL genes were null (no true differences):")
println("  p < 0.01: expected " + to_string(round(to_float(n_genes) * 0.01,
0))
  + ", observed " + to_string(pvals |> filter(|p| p < 0.01) |> len()))
println("  p < 0.05: expected " + to_string(round(to_float(n_genes) * 0.05,
0))
  + ", observed " + to_string(pvals |> filter(|p| p < 0.05) |> len()))
println("  p < 0.10: expected " + to_string(round(to_float(n_genes) * 0.10,
0))
  + ", observed " + to_string(pvals |> filter(|p| p < 0.10) |> len()))
```

How Many Genes Are Significant Without Correction?

```
# Raw significance counts
let alpha_levels = [0.05, 0.01, 0.001, 0.0001, 0.00001]
println("=== Raw (Uncorrected) Significance ===")
alpha_levels |> each(|alpha| {
  let n_sig = pvals |> filter(|p| p < alpha) |> len()
  let expected_fp = round(to_float(n_genes) * alpha, 1)
  println("p < " + to_string(alpha) + ": "
    + to_string(n_sig) + " genes significant, "
    + to_string(expected_fp) + " expected by chance")
})
```

The Analysis

Step 1: Family-Wise Error Rate (FWER) — Bonferroni

The FWER is the probability of making at least one false positive. Bonferroni controls FWER by dividing alpha by the number of tests.

```
# Bonferroni correction
# Adjusted p-value = raw p-value * number of tests
# (capped at 1.0)
let bonferroni_pvals = p_adjust(pvals, "bonferroni")

let n_bonf_sig = bonferroni_pvals |> filter(|p| p < 0.05) |> len()
println("=== Bonferroni Correction ===")
println("Effective alpha per test: " + to_string(0.05 / to_float(n_genes)))
println("Significant at FWER < 0.05: " + to_string(n_bonf_sig) + " genes")
```

```
# Add Bonferroni-adjusted p-values to results
let results_bonf = results
  |> mutate("p_bonferroni", |row| {
    let idx = col_values(results, "gene")
      |> enumerate()
      |> filter(|pair| pair[1] == row.gene)
    if len(idx) > 0 then bonferroni_pvals[idx[0][0]] else 1.0
  })

# Top hits by Bonferroni
let bonf_hits = results_bonf
  |> filter_by("p_bonferroni", |p| p < 0.05)
  |> arrange("p_bonferroni")

println("Top Bonferroni-significant genes:")
take(bonf_hits, 20)
```

Why Bonferroni is too conservative for genomics:

Bonferroni guarantees that the probability of even ONE false positive is less than alpha. This is extremely strict. With 20,000 tests, you are dividing alpha by 20,000 — requiring p < 0.0000025 for significance. Many truly differentially expressed genes with moderate effects will be missed.

```
# Demonstrate Bonferroni's conservatism
println("=== Bonferroni Threshold ===")
println("Original alpha: 0.05")
println("Bonferroni alpha: " + to_string(0.05 / to_float(n_genes)))
println("That is " + to_string(round(to_float(n_genes), 0)) + "x more
stringent!")
println("")
println("With " + to_string(n_genes) + " tests at raw p<0.05: "
  + to_string(pvals |> filter(|p| p < 0.05) |> len()) + " significant")
println("With Bonferroni at FWER<0.05: "
  + to_string(n_bonf_sig) + " significant")
println("Bonferroni throws away " + to_string(
  (pvals |> filter(|p| p < 0.05) |> len()) - n_bonf_sig
) + " potentially real findings")
```

Step 2: Holm's Step-Down Procedure

Holm's method is a step-down version of Bonferroni that is uniformly more powerful (it always rejects at least as many hypotheses as Bonferroni) while still controlling FWER.

```
# Holm correction
let holm_pvals = p_adjust(pvals, "holm")

let n_holm_sig = holm_pvals |> filter(|p| p < 0.05) |> len()
println("=== Holm's Step-Down Correction ===")
println("Significant at FWER < 0.05: " + to_string(n_holm_sig) + " genes")
println("Bonferroni found: " + to_string(n_bonf_sig))
println("Holm is never worse than Bonferroni")
```

How Holm works:

1. Sort all p-values from smallest to largest
2. For the smallest p-value, compare to alpha/m (same as Bonferroni)
3. For the second smallest, compare to alpha/(m-1)
4. Continue until a p-value is not rejected — stop, reject nothing further

This is more powerful because later tests use less stringent thresholds.

Step 3: False Discovery Rate (FDR) — Benjamini-Hochberg

FDR is the expected proportion of false positives among all rejected hypotheses. Instead of controlling the probability of ANY false positive (FWER), FDR controls the RATE of false positives among your discoveries.

This is a fundamentally different question:

- **FWER**: "What is the probability I made even one mistake?"
- **FDR**: "Among the genes I call significant, what fraction are mistakes?"

For genomics, FDR is almost always more appropriate. If you call 500 genes significant at FDR = 0.05, you expect about 25 of them (5%) to be false positives. That is a tolerable error rate for most applications.

```
# Benjamini-Hochberg (BH) correction
let bh_pvals = p_adjust(pvals, "bh")

let n_bh_sig = bh_pvals |> filter(|p| p < 0.05) |> len()
println("=== Benjamini-Hochberg (FDR) Correction ===")
println("Significant at FDR < 0.05: " + to_string(n_bh_sig) + " genes")
println("Significant at FDR < 0.01: " + to_string(
  bh_pvals |> filter(|p| p < 0.01) |> len()
) + " genes")
println("Significant at FDR < 0.10: " + to_string(
  bh_pvals |> filter(|p| p < 0.10) |> len()
) + " genes")
```

```
# Compare all methods side by side
println("=== Comparison of Correction Methods ===")
println("Method          | Significant at 0.05 | Controls")
println("----------------|---------------------|---------")
println("None (raw)      | " + to_string(pvals |> filter(|p| p < 0.05) |>
len())
  + "               | Nothing")
println("Bonferroni      | " + to_string(n_bonf_sig)
  + "               | FWER")
println("Holm            | " + to_string(n_holm_sig)
  + "               | FWER")
println("BH (FDR)        | " + to_string(n_bh_sig)
  + "               | FDR")
```

How BH works:

1. Sort all m p-values from smallest to largest: p(1), p(2), ..., p(m)
2. For each p(i), compute the BH threshold: i/m * alpha
3. Find the largest i where p(i) <= i/m * alpha
4. Reject all hypotheses with p-values <= p(i)

```
# Demonstrate the BH procedure step by step
let sorted_pvals = pvals |> sort()
let m = len(sorted_pvals)
let alpha = 0.05

# Find the BH threshold
let bh_thresholds = sorted_pvals |> enumerate() |> map(|pair| {
  let i = pair[0] + 1  # 1-indexed rank
  let p = pair[1]
  let threshold = to_float(i) / to_float(m) * alpha
  record("rank", i, "p_value", p, "threshold", threshold,
         "reject", p <= threshold)
})

# Show the first 20 and last few before the cutoff
println("=== BH Procedure (first 20 genes) ===")
println("Rank | P-value      | Threshold    | Reject?")
bh_thresholds |> take(20) |> each(|row| {
  println(to_string(row.rank) + "    | "
    + to_string(row.p_value) + " | "
    + to_string(round(row.threshold, 8)) + " | "
    + to_string(row.reject))
})
```

Step 4: BY Correction for Correlated Tests

The BH procedure assumes independence between tests. But genes are correlated — co-regulated genes, pathway members, and genomic neighbors often have correlated expression. The Benjamini-Yekutieli (BY) correction handles arbitrary dependence structure.

```
# BY correction (for correlated tests)
let by_pvals = p_adjust(pvals, "by")

let n_by_sig = by_pvals |> filter(|p| p < 0.05) |> len()
println("=== Benjamini-Yekutieli (BY) Correction ===")
println("Significant at FDR < 0.05: " + to_string(n_by_sig) + " genes")
println("")
println("Comparison:")
println("  BH (assumes independence): " + to_string(n_bh_sig) + " genes")
println("  BY (allows correlation):   " + to_string(n_by_sig) + " genes")
println("  BY is more conservative than BH")
```

In practice, BH works well even with moderate correlation. BY is overly conservative for most genomics applications. Use BY only when you have strong prior reason to believe tests are heavily correlated.

Step 5: Independent Filtering — Improving Power

Here is a counterintuitive fact: you can INCREASE your power to detect true differences by REMOVING genes BEFORE testing. Specifically, removing genes with very low expression (low mean, low variance) removes true nulls that add noise to the multiple testing correction without contributing any signal.

```
# Independent filtering: remove low-expression genes before testing

# Step 1: compute mean expression across all samples
let mean_expr = genes |> map(|gene| {
  let row = expr |> filter_by(gene_col, |g| g == gene)
  let all_vals = concat(tumor_cols, normal_cols) |> map(|c| col_values(row,
c)) |> flatten()
  let log_mean = mean(all_vals |> map(|x| log2(x + 1.0)))
  record("gene", gene, "mean_expr", log_mean)
}) |> to_list() |> table()

# Step 2: filter out lowly expressed genes (mean log2 FPKM < 1)
let expressed = mean_expr
  |> filter_by("mean_expr", |m| m >= 1.0)

let n_expressed = len(expressed)
let n_filtered = len(mean_expr) - n_expressed

println("=== Independent Filtering ===")
println("Total genes: " + to_string(len(mean_expr)))
println("Expressed (mean log2 >= 1): " + to_string(n_expressed))
println("Filtered out: " + to_string(n_filtered))
```

```
# Re-run BH on only expressed genes
let expressed_genes = col_values(expressed, "gene")
let expressed_results = results
  |> filter_by("gene", |g| expressed_genes |> contains(g))

let expressed_pvals = col_values(expressed_results, "p_value")
let expressed_bh = p_adjust(expressed_pvals, "bh")

let n_expressed_sig = expressed_bh |> filter(|p| p < 0.05) |> len()

println("=== Effect of Independent Filtering on BH ===")
println("BH on ALL genes:       " + to_string(n_bh_sig) + " significant")
println("BH on expressed genes: " + to_string(n_expressed_sig) + "
significant")
println("Filtering INCREASED discoveries by: " + to_string(n_expressed_sig -
n_bh_sig))
println("")
println("This works because:")
println("  - Low-expression genes are rarely truly differential")
println("  - Removing them reduces the number of tests m")
println("  - The BH threshold i/m * alpha becomes larger (less strict)")
println("  - True positives that were borderline now pass the threshold")
```

Important: The filtering criterion must be INDEPENDENT of the test statistic. You cannot filter by p-value first and then re-correct — that is circular. Mean

expression is independent of the difference between groups, so it is a valid filter.

Step 6: Visualizing the Correction Effect

```
# Scatter: raw p-value vs BH-adjusted p-value
let pval_comparison = table(
  "raw_p", pvals |> map(|p| -log10(p + 1e-300)),
  "bh_p", bh_pvals |> map(|p| -log10(p + 1e-300))
)

scatter(pval_comparison, "raw_p", "bh_p")
  |> save_plot("raw_vs_bh_pvalues.svg")
```

```
# Volcano plots: raw vs corrected
# Raw p-values
let volcano_raw = results
  |> mutate("neg_log10_p", |row| -log10(row.p_value + 1e-300))

volcano(volcano_raw, "log2fc", "neg_log10_p")
  |> save_plot("volcano_raw.svg")

# BH-adjusted — add adjusted p-values to results
let results_with_bh = results
  |> mutate("bh_p", |row| {
    let idx = col_values(results, "gene") |> enumerate()
      |> filter(|pair| pair[1] == row.gene)
    if len(idx) > 0 then bh_pvals[idx[0][0]] else 1.0
  })
  |> mutate("neg_log10_bh", |row| -log10(row.bh_p + 1e-300))

volcano(results_with_bh, "log2fc", "neg_log10_bh")
  |> save_plot("volcano_bh.svg")
```

```
# Show genes that change status after correction
let sig_raw_only = results_with_bh
  |> filter_by("p_value", |p| p < 0.05)
  |> filter_by("bh_p", |p| p >= 0.05)

let sig_both = results_with_bh
  |> filter_by("p_value", |p| p < 0.05)
  |> filter_by("bh_p", |p| p < 0.05)

println("=== Correction Impact ===")
println("Significant by raw p < 0.05 only: " + to_string(len(sig_raw_only)))
println("Significant by both raw and BH:   " + to_string(len(sig_both)))
println("These " + to_string(len(sig_raw_only)) + " genes were likely false
positives")
```

Step 7: The Pi-Zero Concept

Pi-zero (pi_0) is the estimated proportion of true null hypotheses. If you test 20,000 genes and 15,000 are truly not different (pi_0 = 0.75), then 5,000 are truly different. Estimating pi_0 tells you how much signal is in your data.

```
# Estimating pi_0 from the p-value histogram
# The flat region of the p-value histogram estimates pi_0

# Under the null, p-values are uniform
# Under the alternative, p-values are concentrated near 0
# The height of the flat region (say p > 0.5) estimates pi_0

let n_total = len(pvals)
let n_above_05 = pvals |> filter(|p| p > 0.5) |> len()

# If p-values were uniform, half would be above 0.5
# The fraction above 0.5 estimates pi_0
let pi0_estimate = to_float(n_above_05) / to_float(n_total) * 2.0

println("=== Estimating pi_0 (proportion of true nulls) ===")
println("Total genes: " + to_string(n_total))
println("P-values > 0.5: " + to_string(n_above_05))
println("Estimated pi_0: " + to_string(round(pi0_estimate, 3)))
println("Estimated true positives: " + to_string(round(to_float(n_total) *
(1.0 - pi0_estimate), 0)))
println("Estimated true nulls: " + to_string(round(to_float(n_total) *
pi0_estimate, 0)))
```

```
# Use different cutoffs to verify pi_0 estimate stability
let cutoffs = [0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
println("Pi_0 estimates at different cutoffs:")
cutoffs |> each(|lambda| {
  let n_above = pvals |> filter(|p| p > lambda) |> len()
  let pi0 = to_float(n_above) / (to_float(n_total) * (1.0 - lambda))
  println("  lambda = " + to_string(lambda) + ": pi_0 = " +
to_string(round(pi0, 3)))
})
```

Step 8: Understanding What FDR = 0.05 Really Means

This is the most commonly misunderstood concept in genomics. When you set FDR = 0.05 and find 500 significant genes:

- It does NOT mean each gene has a 5% probability of being a false positive
- It DOES mean that among your 500 genes, approximately 25 (5%) are expected to be false positives
- It does NOT guarantee exactly 25 false positives — it is an expected value

```
# What FDR = 0.05 means in practice

let fdr_thresholds = [0.01, 0.05, 0.10, 0.20]
println("=== FDR Interpretation ===")
fdr_thresholds |> each(|fdr| {
  let n_sig = bh_pvals |> filter(|p| p < fdr) |> len()
  let expected_fp = round(to_float(n_sig) * fdr, 1)
  let expected_tp = round(to_float(n_sig) * (1.0 - fdr), 1)

  println("FDR < " + to_string(fdr) + ":")
  println("  Significant genes: " + to_string(n_sig))
  println("  Expected false positives: " + to_string(expected_fp)
    + " (" + to_string(round(fdr * 100.0, 0)) + "% of discoveries)")
  println("  Expected true positives: " + to_string(expected_tp))
  println("")
})
```

```
# The q-value: gene-specific FDR
# The BH-adjusted p-value for a gene IS its q-value
# q = 0.03 means: if you call all genes with q <= 0.03 significant,
# 3% of those calls are expected to be false

# Example genes
let example_genes = ["TP53", "BRCA1", "GAPDH", "ERBB2", "ESR1",
                     "MKI67", "CDH1", "PGR"]

println("=== Q-values for Known Genes ===")
example_genes |> each(|gene| {
  let row = results_with_bh
    |> filter_by("gene", |g| contains(g, gene))
  if len(row) > 0 then {
    let raw_p = col_values(row, "p_value")[0]
    let bh_q = col_values(row, "bh_p")[0]
    let fc = col_values(row, "log2fc")[0]
    println(gene + ": raw p = " + to_string(raw_p)
      + ", q-value = " + to_string(round(bh_q, 6))
      + ", log2FC = " + to_string(fc))
  }
})
```

Step 9: P-Value Histogram Diagnostics

The shape of the p-value histogram tells you about your data quality:

```
# Different p-value histogram shapes and what they mean

# Shape 1: Spike at 0 + flat = good data with real signal
println("Shape 1: Spike near 0 + flat elsewhere")
println("  Interpretation: True signal exists")
println("  Action: Apply BH correction and proceed")
println("")

# Shape 2: Completely flat = no signal (all null)
println("Shape 2: Completely uniform/flat")
println("  Interpretation: No genes are differentially expressed")
println("  Action: No correction needed — nothing is significant")
println("")

# Shape 3: Spike at 0 AND spike at 1
println("Shape 3: Spikes at both 0 and 1")
println("  Interpretation: Anti-conservative p-values near 1")
println("  Possible cause: Discrete test statistics, low counts")
println("  Action: Use a different test or filter low-count genes")
println("")

# Shape 4: U-shaped (high at both ends)
println("Shape 4: U-shaped distribution")
println("  Interpretation: Batch effects or confounders")
println("  Action: Check for technical artifacts BEFORE testing")
println("")

# Shape 5: Bump in the middle
println("Shape 5: Bump around 0.5")
println("  Interpretation: Correlation structure among tests")
println("  Action: Consider BY correction or decorrelation")
```

```
# Quantify the shape of our actual p-value distribution
let n_bins = 20
let bin_width = 1.0 / to_float(n_bins)

let bin_counts = range(0, n_bins) |> map(|i| {
  let lower = to_float(i) * bin_width
  let upper = lower + bin_width
  let count = pvals |> filter(|p| p >= lower and p < upper) |> len()
  record("bin", to_string(round(lower, 2)) + "-" + to_string(round(upper,
2)),
         "count", count,
         "expected", round(to_float(n_total) / to_float(n_bins), 0))
})

let bin_table = table(bin_counts |> to_list())
println("=== P-value Histogram (20 bins) ===")
bin_table
```

Step 10: Applying Corrections Genome-Wide

```
# Complete corrected results for all genes
let all_pvals = col_values(results, "p_value")

let bonf = p_adjust(all_pvals, "bonferroni")
let holm = p_adjust(all_pvals, "holm")
let bh = p_adjust(all_pvals, "bh")
let by = p_adjust(all_pvals, "by")

# Summary across all methods
let summary = table(
  "method", ["None", "Bonferroni", "Holm", "BH", "BY"],
  "controls", ["Nothing", "FWER", "FWER", "FDR", "FDR (correlated)"],
  "sig_005", [
    all_pvals |> filter(|p| p < 0.05) |> len(),
    bonf |> filter(|p| p < 0.05) |> len(),
    holm |> filter(|p| p < 0.05) |> len(),
    bh |> filter(|p| p < 0.05) |> len(),
    by |> filter(|p| p < 0.05) |> len()
  ],
  "sig_001", [
    all_pvals |> filter(|p| p < 0.01) |> len(),
    bonf |> filter(|p| p < 0.01) |> len(),
    holm |> filter(|p| p < 0.01) |> len(),
    bh |> filter(|p| p < 0.01) |> len(),
    by |> filter(|p| p < 0.01) |> len()
  ]
)

println("=== Correction Method Comparison ===")
summary
```

```
# Write complete results to file
let final_results = results
  |> mutate("p_bonferroni", |row| {
    let idx = genes |> enumerate() |> filter(|p| p[1] == row.gene)
    if len(idx) > 0 then bonf[idx[0][0]] else 1.0
  })
  |> mutate("p_holm", |row| {
    let idx = genes |> enumerate() |> filter(|p| p[1] == row.gene)
    if len(idx) > 0 then holm[idx[0][0]] else 1.0
  })
  |> mutate("p_bh", |row| {
    let idx = genes |> enumerate() |> filter(|p| p[1] == row.gene)
    if len(idx) > 0 then bh[idx[0][0]] else 1.0
  })
  |> mutate("p_by", |row| {
    let idx = genes |> enumerate() |> filter(|p| p[1] == row.gene)
    if len(idx) > 0 then by[idx[0][0]] else 1.0
  })

# Sort by BH-adjusted p-value
let final_sorted = final_results |> arrange("p_bh")

write_tsv(final_sorted, "differential_expression_results.csv")
println("Results written to differential_expression_results.csv")
```

Interpretation

When to Use Which Method

```
# Decision framework for multiple testing correction

let decision_table = table(
  "situation", [
    "GWAS (must avoid any false positive)",
    "Differential expression (tolerate some FP)",
    "Exploratory screen (maximize discoveries)",
    "Correlated tests (pathway genes)",
    "Follow-up study (few candidate genes)",
    "Publication requirement"
  ],
  "recommended", [
    "Bonferroni (FWER < 5e-8 by convention)",
    "BH (FDR < 0.05)",
    "BH (FDR < 0.10 or 0.20)",
    "BY (FDR for dependent tests)",
    "Bonferroni (small m makes it reasonable)",
    "BH is standard; report raw and adjusted"
  ],
  "why", [
    "False positives trigger expensive follow-up",
    "5% FP rate is acceptable for gene lists",
    "Cast a wide net, validate later",
    "BH assumes independence, BY does not",
    "With m < 50, Bonferroni is not too strict",
    "Reviewers expect FDR control"
  ]
)

decision_table
```

The GWAS Threshold

In genome-wide association studies, the convention is $p < 5e-8$, which corresponds to a Bonferroni correction for approximately 1 million independent tests (the effective number of independent SNPs in the genome after accounting for linkage disequilibrium).

```
# GWAS significance threshold
let gwas_threshold = 5e-8
let n_snps = 1000000

# This is approximately Bonferroni for 1M tests
let bonf_threshold = 0.05 / to_float(n_snps)
println("GWAS convention: p < 5e-8")
println("Bonferroni for 1M tests: p < " + to_string(bonf_threshold))
println("These are approximately equal")
println("")
println("This threshold has served the GWAS community well")
println("because false positives are very costly (replication studies)")
```

Reporting Results

```
# How to properly report multiple testing results

println("=== Reporting Checklist ===")
println("1. State the total number of tests performed")
println("2. State the correction method used (BH for FDR, etc.)")
println("3. Report BOTH raw and adjusted p-values")
println("4. State the significance threshold (e.g., FDR < 0.05)")
println("5. Report the number of significant results")
println("6. Show the p-value histogram as a supplementary figure")
println("7. If you filtered before testing, describe the filter")
println("8. Report effect sizes (log2FC, Cohen's d) alongside p-values")
println("9. Include a volcano plot (log2FC vs -log10 adjusted p)")
println("10. Acknowledge the expected number of false discoveries")
```

```
# Example results paragraph (fill in your numbers):
let n_tested = len(results)
let n_sig_bh = bh |> filter(|p| p < 0.05) |> len()
let n_up = final_sorted
  |> filter_by("p_bh", |p| p < 0.05)
  |> filter_by("log2fc", |fc| fc > 0.0)
  |> len()
let n_down = n_sig_bh - n_up

println("Example results text:")
println("'We tested " + to_string(n_tested) + " genes for differential")
println("expression between tumor and normal tissue using Welch's t-test")
println("on log2-transformed FPKM values. P-values were corrected for")
println("multiple testing using the Benjamini-Hochberg procedure to")
println("control the false discovery rate. At FDR < 0.05, we identified")
println(to_string(n_sig_bh) + " differentially expressed genes")
println("(" + to_string(n_up) + " upregulated, " + to_string(n_down)
  + " downregulated in tumor).'")
```

Decision

The Complete Multiple Testing Pipeline

```
# ============================================================
# Complete Multiple Testing Correction Pipeline
# ============================================================

# 1. Load and prepare data
let url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let expr = download(url, "GSE183947_fpkm.csv.gz") |> read_csv()

let all_cols = columns(expr)
let tumor_cols = all_cols |> filter(|c| contains(c, "tumor"))
let normal_cols = all_cols |> filter(|c| contains(c, "normal"))
let gene_col = all_cols[0]
let genes = col_values(expr, gene_col)

# 2. Pre-filter: remove lowly expressed genes (INDEPENDENT of test)
let keep_genes = genes |> filter(|gene| {
  let row = expr |> filter_by(gene_col, |g| g == gene)
  let all_vals = concat(tumor_cols, normal_cols)
    |> map(|c| col_values(row, c)) |> flatten()
  let log_mean = mean(all_vals |> map(|x| log2(x + 1.0)))
  log_mean >= 1.0
})

println("Genes before filter: " + to_string(len(genes)))
println("Genes after filter:  " + to_string(len(keep_genes)))

# 3. Test filtered genes
let filtered_results = keep_genes |> map(|gene| {
  let row = expr |> filter_by(gene_col, |g| g == gene)
  let lt = tumor_cols |> map(|c| col_values(row, c)) |> flatten()
    |> map(|x| log2(x + 1.0))
  let ln = normal_cols |> map(|c| col_values(row, c)) |> flatten()
    |> map(|x| log2(x + 1.0))
  let fc = mean(lt) - mean(ln)
  let t = ttest(lt, ln)
  record("gene", gene, "log2fc", round(fc, 4), "p_value", t.p_value)
}) |> to_list() |> table()

# 4. Correct for multiple testing
let raw_p = col_values(filtered_results, "p_value")
let adj_p = p_adjust(raw_p, "bh")

# 5. Inspect p-value histogram
histogram(raw_p, "P-value Distribution (filtered genes)")
  |> save_plot("pval_hist_filtered.svg")

# 6. Apply threshold and report
let n_sig = adj_p |> filter(|p| p < 0.05) |> len()
let expected_fp = round(to_float(n_sig) * 0.05, 0)

println("=== Final Results ===")
println("Genes tested: " + to_string(len(keep_genes)))
println("Significant (FDR < 0.05): " + to_string(n_sig))
println("Expected false discoveries: ~" + to_string(expected_fp))
```

```
# 7. Final volcano plot
let final_data = filtered_results
  |> mutate("neg_log10_fdr", |row| {
    let idx = col_values(filtered_results, "gene") |> enumerate()
      |> filter(|p| p[1] == row.gene)
    if len(idx) > 0 then -log10(adj_p[idx[0][0]] + 1e-300) else 0.0
  })

volcano(final_data, "log2fc", "neg_log10_fdr")
  |> save_plot("final_volcano.svg")
```

Pitfalls

Pitfall 1: Not Correcting at All

```
# The most common mistake: reporting raw p-values from genome-wide tests
println("If you test 20,000 genes and report raw p < 0.05:")
println("  You WILL have hundreds of false positives")
println("  Reviewers WILL reject your paper")
println("  The results CANNOT be trusted")
println("")
println("ALWAYS correct for multiple testing in genome-wide analyses")
```

Pitfall 2: Applying Bonferroni When BH Would Be Better

```
# Bonferroni is appropriate when:
# - You have few tests (< 50)
# - False positives are very costly (drug targets, clinical decisions)
# - You need a simple, universally accepted method

# Bonferroni is NOT appropriate when:
# - You have thousands of tests (genomics)
# - You can tolerate some false positives
# - You want to maximize discoveries for follow-up

println("With 20,000 genes:")
println("  Bonferroni threshold: p < " + to_string(0.05 / 20000.0))
println("  BH can identify 10x more genes at the same overall error rate")
println("  Use BH for discovery, Bonferroni for validation")
```

Pitfall 3: Filtering AFTER Testing

```
# WRONG: test all genes, then filter by expression level
# This biases the correction because you counted tests you then excluded

# RIGHT: filter by expression BEFORE testing
# The filter must be INDEPENDENT of the test statistic

# Common independent filters:
println("Valid independent filters:")
println("  - Mean expression across all samples")
println("  - Variance across all samples")
println("  - Gene length")
println("  - GC content")
println("")
println("INVALID filters (dependent on test):")
println("  - P-value (circular!)")
println("  - Fold change (correlated with test statistic)")
println("  - Expression in only one group")
```

Pitfall 4: Confusing FDR with Per-Gene Error Probability

```
# A common misinterpretation:
# "This gene has FDR = 0.03, so there's a 3% chance it's a false positive"
# WRONG!

# The correct interpretation:
# "If I call all genes with FDR <= 0.03 significant,
#  3% of those genes are expected to be false positives"

# FDR is a PROPERTY OF THE LIST, not of individual genes
println("FDR = 0.05 means:")
println("  Among ALL genes you call significant at this threshold,")
println("  5% are expected to be false discoveries.")
println("")
println("It does NOT mean:")
println("  Each individual gene has a 5% chance of being false.")
```

Pitfall 5: Multiple Testing Across Multiple Analyses

```
# You run DE analysis, then pathway analysis, then GO enrichment,
# then network analysis on the same dataset.
# Each analysis has its own multiple testing correction.
# But you are also testing multiple ANALYSES!

println("The meta-analysis problem:")
println("  1. Differential expression: 20,000 tests (BH corrected)")
println("  2. Pathway enrichment: 300 pathways (BH corrected)")
println("  3. GO enrichment: 5,000 terms (BH corrected)")
println("  4. Motif enrichment: 800 motifs (BH corrected)")
println("")
println("Each is individually corrected, but you ran 4 analyses.")
println("If each has a 5% FDR, the combined FDR is higher.")
println("")
println("In practice, this is usually acknowledged in the discussion")
println("rather than formally corrected. Pre-registration helps.")
```

Pitfall 6: The Replication Trap

```
# "We found 500 genes at FDR < 0.05 in dataset 1.
#  We then tested just these 500 genes in dataset 2.
#  We corrected for 500 tests (not 20,000)."
#
# This is actually CORRECT and is the standard approach.
# The replication study tests a pre-specified set of hypotheses.
# The multiple testing burden is only for the number of hypotheses tested.

println("Replication analysis:")
println("  Discovery: test 20,000 genes, correct for 20,000")
println("  Replication: test only the significant ones, correct for THOSE")
println("  This is valid because the hypotheses were pre-specified")
```

Pitfall 7: Thresholding-Induced Artifacts

```
# Using a hard FDR threshold creates artificial boundaries
# Gene A with FDR = 0.049 is "significant"
# Gene B with FDR = 0.051 is "not significant"
# But they are essentially identical

# Better approaches:
println("Instead of binary significant/not-significant:")
println("  1. Report the full ranked list with FDR values")
println("  2. Use gene set enrichment (which avoids thresholds)")
println("  3. Report results at multiple FDR thresholds")
println("  4. Focus on effect size (fold change) alongside FDR")
println("  5. Use volcano plots that show the continuous relationship")
```

Putting It All Together

```
# ============================================================
# Summary: The Multiple Testing Decision Tree
# ============================================================

println("1. Run your tests on all features (genes, variants, etc.)")
println("2. Plot the p-value histogram")
println("   - Spike at 0 + flat = good signal, proceed")
println("   - Flat = no signal, stop")
println("   - U-shaped = batch effects, fix first")
println("")
println("3. Consider independent filtering (remove noise features)")
println("   - Filter by mean expression, variance, or coverage")
println("   - Filter must be INDEPENDENT of the test")
println("")
println("4. Choose correction method:")
println("   - Few tests (< 50): Bonferroni")
println("   - Genomics (discovery): BH (FDR)")
println("   - Correlated tests: BY")
println("   - GWAS: Bonferroni at 5e-8 (convention)")
println("")
println("5. Report:")
println("   - Number of tests")
println("   - Correction method")
println("   - Both raw and adjusted p-values")
println("   - Number of discoveries at chosen FDR")
println("   - Expected false discoveries")
println("   - Effect sizes alongside p-values")
println("   - P-value histogram (supplementary)")
println("   - Volcano plot (main figure)")
```

Where Next?

This chapter covered the correction side of hypothesis testing. But the underlying tests (t-test, ANOVA, Wilcoxon) are all about comparing groups. Chapter 9 moves to **correlation and regression** — modeling relationships between continuous variables, which is the foundation for more complex analyses.

Chapters 10-12 build on regression to handle the specific challenges of genomics data: generalized linear models for count data (Chapter 10), survival analysis for time-to-event data (Chapter 11), and batch effects and confounders (Chapter 12).

Key Takeaways

1. **Testing 20,000 genes at p < 0.05 gives ~1,000 false positives** by chance
2. **FWER** controls the probability of ANY false positive (Bonferroni, Holm)
3. **FDR** controls the RATE of false positives among discoveries (BH, BY)

4. **Bonferroni is too conservative** for genomics — use BH instead
5. **BH is the workhorse** of genomics multiple testing correction
6. **Independent filtering** improves power without inflating false positives
7. **The p-value histogram** is the key diagnostic — always plot it
8. **Pi-zero** estimates the fraction of true nulls in your data
9. **FDR is a property of the list**, not of individual genes
10. **Report both raw and adjusted p-values** alongside effect sizes

Exercises

1. Download GSE183947 and run t-tests on all genes. Compare the number of significant genes using Bonferroni, Holm, BH, and BY at alpha = 0.05. Make a table comparing the methods.

2. Plot the p-value histogram with 50 bins. Estimate pi_0 using the flat region above 0.5. How many genes do you estimate are truly differentially expressed?

3. Apply independent filtering with three different mean expression thresholds (log2 FPKM > 0, > 1, > 2). How does each threshold affect the number of BH discoveries? Is there a sweet spot?

4. For the top 20 genes by BH-adjusted p-value, report: gene name, log2FC, raw p-value, BH q-value, Bonferroni-adjusted p-value. How many are significant by both methods?

5. Simulate: generate 20,000 p-values where 19,000 are from Uniform(0,1) and 1,000 are from Beta(0.1, 1) (enriched near 0). Apply BH at FDR = 0.05. How many true positives and false positives do you find? What is the actual FDR?

Summary Table

Concept	Key Point
Multiple testing	Testing many hypotheses inflates false positives
FWER	Probability of any false positive — controlled by Bonferroni
FDR	Expected false positive rate among discoveries — controlled by BH

Concept	Key Point
Bonferroni	Divide alpha by m — simple but very conservative
Holm	Step-down Bonferroni — always at least as powerful
Benjamini-Hochberg	FDR control — the standard in genomics
BY	FDR for correlated tests — more conservative than BH
Independent filtering	Remove noise features to improve BH power
P-value histogram	Key diagnostic — check shape before correction
Pi-zero	Proportion of true nulls — estimated from flat region
Q-value	BH-adjusted p-value — gene-specific FDR

Next: Chapter 9 — Correlation & Regression

Correlation & Regression

> "Correlation does not imply causation, but it does waggle its eyebrows suggestively and gesture furtively while mouthing 'look over there.'" — Randall Munroe

The Problem

You have RNA-seq expression data and clinical outcomes for over a thousand breast cancer patients from TCGA. A collaborator asks: "Does ESR1 expression predict survival? What about ERBB2? Can we build a model that uses expression of a handful of genes to predict patient outcomes?"

These are correlation and regression questions. Before you can answer them, you need to understand what correlation actually measures, when it misleads, and how regression lets you disentangle the effects of multiple variables acting simultaneously.

This chapter takes you from pairwise correlation to multiple regression, using real TCGA breast cancer data throughout.

Why It Matters

Clinical Decisions Depend on Associations

Breast cancer treatment has been revolutionized by molecular markers. The estrogen receptor (ER), progesterone receptor (PR), and HER2 (ERBB2) status determine treatment strategy. These markers were discovered through correlation studies: researchers noticed that expression of certain genes correlated with treatment response and survival.

But not every correlation is a useful biomarker. Thousands of genes correlate with survival in breast cancer — most because they correlate with subtype, which correlates with survival. The challenge is distinguishing direct, useful associations from indirect, confounded ones.

The Stakes of Getting It Wrong

If you report a “survival gene” that is actually correlated with survival only because it is a marker of ER status, you have not found a new biomarker — you have rediscovered ER status with extra steps. Worse, if you build a model that ignores confounders, it will fail when applied to a different cohort.

Regression Gives You Conditional Associations

Simple correlation tells you whether two variables move together. Regression tells you whether they move together *after accounting for other variables*. This is the tool that lets you ask: “Does gene X predict survival *after controlling for age, stage, and subtype*?”

The Data

We use two TCGA BRCA datasets from the Genomic Data Commons (GDC):

1. **Expression data**: FPKM-normalized RNA-seq expression for ~20,000 genes across ~1,200 samples.
2. **Clinical phenotype data**: Patient demographics, tumor characteristics, survival outcomes.

```
# Download TCGA BRCA expression data (FPKM)
let expr_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.htseq_fpkm.tsv.gz"
let expr_file = download(expr_url, "tcga_brca_fpkm.tsv.gz")

# Download clinical phenotype data
let clin_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.GDC_phenotype.tsv.gz"
let clin_file = download(clin_url, "tcga_brca_phenotype.tsv.gz")
```

Loading and Inspecting the Expression Matrix

```
# Read the expression matrix
let expr = read_tsv(expr_file)
println("Expression matrix dimensions:")
println("  Rows (genes): " + to_string(len(expr)))
println("  Columns (samples): " + to_string(ncols(expr)))

# Preview the first few rows and columns
head(expr, 5)
```

The expression matrix has genes as rows and samples as columns. Each value is the FPKM (Fragments Per Kilobase of transcript per Million mapped fragments) for that gene in that sample.

```
# Check column names — first column is gene ID, rest are sample barcodes
let columns = columns(expr)
println("First 5 columns:")
println(slice(columns, 0, 5))
```

Loading Clinical Data

```
# Read clinical phenotype data
let clinical = read_tsv(clin_file)
println("Clinical data dimensions:")
println("  Patients: " + to_string(len(clinical)))
println("  Variables: " + to_string(ncols(clinical)))

# What clinical variables do we have?
println("Clinical columns:")
println(columns(clinical))
```

The clinical data contains dozens of variables. The ones we care about most:

- `submitter_id.samples` — The sample barcode (links to expression columns)
- `age_at_initial_pathologic_diagnosis` — Patient age
- `pathologic_stage` — Tumor stage (Stage I through Stage IV)
- `OS` — Overall survival status (0 = alive/censored, 1 = dead)
- `OS.time` — Overall survival time in days
- `breast_carcinoma_estrogen_receptor_status` — ER status
- `breast_carcinoma_progesterone_receptor_status` — PR status
- `lab_proc_her2_neu_immunohistochemistry_receptor_status` — HER2 status

```
# Summary of key clinical variables
let age = col_values(clinical, "age_at_initial_pathologic_diagnosis")
let stage = col_values(clinical, "pathologic_stage")
let os_status = col_values(clinical, "OS")
let os_time = col_values(clinical, "OS.time")

println("Age summary:")
let age_numeric = age |> filter(|x| !is_none(x)) |> map(|x| to_float(x))
println("  Mean: " + to_string(round(mean(age_numeric), 1)))
println("  Median: " + to_string(median(age_numeric)))
println("  SD: " + to_string(round(stdev(age_numeric), 1)))
println("  Range: " + to_string(min(age_numeric)) + " - " +
to_string(max(age_numeric)))
```

Preparing a Joint Dataset

The expression matrix has samples as columns — we need to extract specific genes and join them with clinical data.

```
# Extract expression for key genes
# ESR1 (estrogen receptor), ERBB2 (HER2), TP53, MKI67 (Ki-67 proliferation
marker)
# PGR (progesterone receptor), BRCA1

let genes_of_interest = ["ESR1", "ERBB2", "TP53", "MKI67", "PGR", "BRCA1"]

# Filter expression matrix for our genes of interest
let gene_col = columns(expr) |> head(1)
let expr_subset = expr |> filter_by(gene_col, |g| contains(genes_of_interest,
g))

println("Extracted expression for " + to_string(len(expr_subset)) + " genes")
head(expr_subset, 6)
```

```
# Transpose so genes become columns and samples become rows
let expr_t = transpose(expr_subset)

# Merge expression with clinical data on sample ID
let merged = inner_join(expr_t, clinical, "sample_id",
"submitter_id.samples")

println("Merged dataset: " + to_string(len(merged)) + " patients with both
expression and clinical data")

# Drop rows with missing survival data
let analysis_data = merged |> drop_null("OS") |> drop_null("OS.time")
println("After removing missing survival: " + to_string(len(analysis_data)) +
" patients")
```

Exploration

Before computing any correlations, look at the data.

Distribution of Expression Values

```
# FPKM values are right-skewed — check ESR1
let esr1_vals = col_values(analysis_data, "ESR1") |> map(|x| to_float(x))

println("ESR1 FPKM summary:")
println(summary(esr1_vals))

histogram(esr1_vals, "ESR1 FPKM Distribution") |>
save_plot("esr1_fpkm_raw.svg")
```

The histogram will show a dramatically right-skewed distribution. Most samples have moderate ESR1 expression, but some ER-positive tumors have very high values. This is typical of gene expression data and matters for correlation.

```
# Log-transform for more normal distribution
# Add small constant to avoid log(0)
let esr1_log = esr1_vals |> map(|x| log2(x + 1.0))

println("log2(ESR1 + 1) summary:")
println(summary(esr1_log))

histogram(esr1_log, "ESR1 log2(FPKM+1) Distribution") |>
save_plot("esr1_fpkm_log.svg")
```

The log-transformed distribution is much closer to normal. This is important because Pearson correlation assumes linear relationships, which are more likely after log transformation of expression data.

Pairwise Scatter Plots

```
# ESR1 vs PGR — these should be strongly correlated (both ER pathway)
let pgr_log = col_values(analysis_data, "PGR") |> map(|x| log2(to_float(x) +
1.0))

scatter(esr1_log, pgr_log, "ESR1 vs PGR (log2 FPKM)")
    |> save_plot("esr1_pgr_scatter.svg")
```

```
# ESR1 vs ERBB2 — these should be negatively correlated
# (ER+ and HER2+ are largely distinct subtypes)
let erbb2_log = col_values(analysis_data, "ERBB2") |> map(|x|
log2(to_float(x) + 1.0))

scatter(esr1_log, erbb2_log, "ESR1 vs ERBB2 (log2 FPKM)")
    |> save_plot("esr1_erbb2_scatter.svg")
```

```
# MKI67 (proliferation) vs survival time
let mki67_log = col_values(analysis_data, "MKI67") |> map(|x|
log2(to_float(x) + 1.0))
let surv_time = col_values(analysis_data, "OS.time") |> map(|x| to_float(x))

scatter(mki67_log, surv_time, "Ki-67 vs Overall Survival Time")
    |> save_plot("ki67_survival_scatter.svg")
```

The Analysis

Part 1: Pearson Correlation

Pearson correlation (r) measures the strength and direction of the *linear* relationship between two continuous variables. It ranges from -1 (perfect negative linear relationship) to +1 (perfect positive linear relationship). Zero means no linear relationship.

The formula:

r = sum((x_i - mean_x)(y_i - mean_y)) / sqrt(sum((x_i - mean_x)^2) * sum((y_i - mean_y)^2))

Assumptions:

1. Both variables are continuous
2. The relationship is linear
3. Both variables are approximately normally distributed (for inference)
4. No extreme outliers (they inflate or deflate r dramatically)

```
# Pearson correlation between ESR1 and PGR
let r_esr1_pgr = cor(esr1_log, pgr_log)
println("Pearson r (ESR1, PGR): " + to_string(round(r_esr1_pgr, 4)))
```

Expected result: r around 0.65-0.75. ESR1 and PGR are co-regulated — the estrogen receptor drives PGR transcription. This is a biologically meaningful positive correlation.

```
# ESR1 vs ERBB2
let r_esr1_erbb2 = cor(esr1_log, erbb2_log)
println("Pearson r (ESR1, ERBB2): " + to_string(round(r_esr1_erbb2, 4)))
```

Expected: r around -0.20 to -0.35. The negative correlation reflects the biological reality that ER+ and HER2+ are largely distinct subtypes, though the correlation is moderate because some tumors are double-positive or double-negative.

```
# Full correlation matrix for all genes of interest
let gene_names = ["ESR1", "ERBB2", "TP53", "MKI67", "PGR", "BRCA1"]
let gene_data = gene_names |> map(|g|
    col_values(analysis_data, g) |> map(|x| log2(to_float(x) + 1.0))
)

let gene_matrix = matrix(gene_data)
let cor_mat = cor_matrix(gene_matrix)

println("Gene-gene correlation matrix:")
println(cor_mat)

heatmap(cor_mat, gene_names, gene_names, "Gene Expression Correlation
Matrix")
    |> save_plot("gene_correlation_heatmap.svg")
```

Interpreting the Correlation Matrix

The heatmap reveals the structure of breast cancer biology:

- **ESR1-PGR**: Strong positive. Co-regulated by estrogen signaling.
- **ESR1-ERBB2**: Negative. Different subtypes.
- **MKI67-ESR1**: Negative. High-proliferation tumors tend to be ER-negative.
- **MKI67-ERBB2**: Positive. HER2+ tumors tend to be more proliferative.
- **TP53-MKI67**: Positive. TP53 mutations associate with high proliferation.
- **BRCA1-ESR1**: Negative. BRCA1-related tumors are often triple-negative.

Each of these correlations has a biological explanation. But correlation alone does not tell you which is cause and which is effect. ESR1 drives PGR transcription (causal), but ESR1 and ERBB2 are negatively correlated because they mark different cell lineages (not causal — subtype is the common cause).

Part 2: Correlation Is Not Causation

This point deserves a detailed treatment because genomics is full of misleading correlations.

Example 1: The Subtype Confounder

```
# Separate ER-positive and ER-negative patients
let er_status = col_values(analysis_data,
"breast_carcinoma_estrogen_receptor_status")

let er_pos_idx = er_status |> map(|x| x == "Positive")
let er_neg_idx = er_status |> map(|x| x == "Negative")

# Correlation between MKI67 and ERBB2 in ALL patients
let mki67_all = col_values(analysis_data, "MKI67") |> map(|x|
log2(to_float(x) + 1.0))
let erbb2_all = col_values(analysis_data, "ERBB2") |> map(|x|
log2(to_float(x) + 1.0))
let r_all = cor(mki67_all, erbb2_all)
println("MKI67-ERBB2 correlation (all patients): " + to_string(round(r_all,
4)))

# Now separate by ER status
let data_er_pos = analysis_data |>
filter_by("breast_carcinoma_estrogen_receptor_status", |x| x == "Positive")
let data_er_neg = analysis_data |>
filter_by("breast_carcinoma_estrogen_receptor_status", |x| x == "Negative")

let mki67_pos = col_values(data_er_pos, "MKI67") |> map(|x| log2(to_float(x)
+ 1.0))
let erbb2_pos = col_values(data_er_pos, "ERBB2") |> map(|x| log2(to_float(x)
+ 1.0))
let r_pos = cor(mki67_pos, erbb2_pos)

let mki67_neg = col_values(data_er_neg, "MKI67") |> map(|x| log2(to_float(x)
+ 1.0))
let erbb2_neg = col_values(data_er_neg, "ERBB2") |> map(|x| log2(to_float(x)
+ 1.0))
let r_neg = cor(mki67_neg, erbb2_neg)

println("MKI67-ERBB2 correlation (ER+): " + to_string(round(r_pos, 4)))
println("MKI67-ERBB2 correlation (ER-): " + to_string(round(r_neg, 4)))
```

This demonstrates a critical phenomenon. The overall correlation between MKI67 and ERBB2 may be positive, but within each ER subgroup, the correlation can be much weaker or even reversed. The overall correlation was driven by ER status acting as a confounder — ER-negative tumors tend to have both higher MKI67 and higher ERBB2 on average, creating an apparent positive correlation.

This is Simpson's paradox in action: the trend in the combined data does not hold within the subgroups.

Example 2: Batch Effects Create Spurious Correlation

```
# Suppose two genes are uncorrelated biologically but both are affected
# by library size. Samples processed in batch A have larger libraries.
# Both genes appear "upregulated" in batch A — creating a correlation
# that is purely technical.

# Simulate this to demonstrate the principle
let batch_a_lib = rnorm(50, 100.0, 10.0)  # larger libraries
let batch_b_lib = rnorm(50, 60.0, 10.0)   # smaller libraries
let all_lib = concat(batch_a_lib, batch_b_lib)

# Gene X: no real difference, but expression scales with library size
let gene_x = all_lib |> map(|lib| lib * 0.5 + rnorm(1, 0.0, 5.0) |> head(1))
let gene_x_flat = gene_x |> flatten()

# Gene Y: also no real difference, also scales with library size
let gene_y = all_lib |> map(|lib| lib * 0.3 + rnorm(1, 0.0, 5.0) |> head(1))
let gene_y_flat = gene_y |> flatten()

let r_spurious = cor(gene_x_flat, gene_y_flat)
println("Spurious correlation from batch effect: " +
to_string(round(r_spurious, 4)))
# This will be strongly positive despite no biological relationship
```

Part 3: Spearman Correlation for Non-Linear Relationships

Pearson correlation measures linear relationships. But many biological relationships are monotonic but non-linear: as X increases, Y increases, but not at a constant rate.

Spearman correlation works on ranks rather than raw values. It measures whether the rank order of X matches the rank order of Y.

```
# Spearman correlation for ESR1 vs PGR
let sp_esr1_pgr = spearman(esr1_log, pgr_log)
println("Spearman rho (ESR1, PGR): " + to_string(round(sp_esr1_pgr.rho, 4)))
println("Spearman p-value: " + to_string(sp_esr1_pgr.p_value))
```

```
# Compare Pearson and Spearman for all gene pairs
println("Comparison of Pearson r and Spearman rho:")
println("Gene Pair            Pearson    Spearman")
println("─────────────────    ───────    ────────")

let pairs = [
    ["ESR1", "PGR"], ["ESR1", "ERBB2"], ["ESR1", "MKI67"],
    ["ERBB2", "MKI67"], ["TP53", "MKI67"], ["BRCA1", "ESR1"]
]

pairs |> each(|pair|
    let v1 = col_values(analysis_data, pair[0]) |> map(|x| log2(to_float(x) +
1.0))
    let v2 = col_values(analysis_data, pair[1]) |> map(|x| log2(to_float(x) +
1.0))
    let r = cor(v1, v2)
    let sp = spearman(v1, v2)
    let label = pair[0] + "-" + pair[1]
    println(label + "    " + to_string(round(r, 4)) + "    " +
to_string(round(sp.rho, 4)))
)
```

When Pearson and Spearman agree closely, the relationship is close to linear. When Spearman is notably larger than Pearson, the relationship is monotonic but non-linear — common with expression data where a few extreme values distort the linear fit.

Kendall's Tau

Kendall's tau is another rank-based correlation. It counts concordant and discordant pairs. It is more robust than Spearman with small sample sizes and ties.

```
# Kendall's tau for ESR1 vs PGR
let kt = kendall(esr1_log, pgr_log)
println("Kendall tau (ESR1, PGR): " + to_string(round(kt.tau, 4)))
println("Kendall p-value: " + to_string(kt.p_value))
```

Kendall's tau is always smaller in magnitude than Spearman's rho for the same data. This is normal — they have different scales. The p-values, however, are usually similar.

Part 4: Simple Linear Regression

Correlation tells you that two variables are associated. Regression goes further: it fits a line that predicts one variable from the other and quantifies the prediction.

The model: Y = beta_0 + beta_1 * X + epsilon

- beta_0: intercept (predicted Y when X = 0)
- beta_1: slope (change in Y per unit change in X)
- epsilon: residual error (what the model cannot explain)

```
# Does ESR1 expression predict overall survival time?
let esr1_expr = col_values(analysis_data, "ESR1") |> map(|x| log2(to_float(x)
+ 1.0))
let survival_days = col_values(analysis_data, "OS.time") |> map(|x|
to_float(x))

# Remove any remaining nulls
let valid_idx = zip(esr1_expr, survival_days)
    |> filter(|pair| !is_none(pair[0]) && !is_none(pair[1]))
let esr1_clean = valid_idx |> map(|pair| pair[0])
let surv_clean = valid_idx |> map(|pair| pair[1])

# Fit simple linear regression
let model_simple = lm(surv_clean, esr1_clean)
println("Simple Linear Regression: Survival ~ ESR1")
println("  Intercept (beta_0): " + to_string(round(model_simple.intercept,
2)))
println("  Slope (beta_1): " + to_string(round(model_simple.coefficients[0],
2)))
println("  R-squared: " + to_string(round(model_simple.r_squared, 4)))
println("  P-value (slope): " + to_string(model_simple.p_values[0]))
```

Interpreting the Simple Regression

The **slope** (beta_1) tells you: for each unit increase in log2(ESR1 FPKM+1), the predicted survival time changes by beta_1 days. If it is positive, higher ESR1 expression is associated with longer survival; if negative, shorter.

The **R-squared** tells you what fraction of the variance in survival time is explained by ESR1 expression alone. In genomics, R-squared for a single gene predicting a clinical outcome is usually very small (0.01-0.05). This does not mean the association is unimportant — survival is determined by many factors, and any single gene explains only a small fraction.

The **p-value** tells you whether the slope is significantly different from zero. With 1,000+ patients, even tiny effects can be significant. This is a case where statistical significance and practical significance diverge.

```
# Visualize the regression
scatter(esr1_clean, surv_clean, "ESR1 Expression vs Survival Time")
    |> save_plot("esr1_survival_regression.svg")

# QQ plot of residuals to check normality assumption
let residuals = zip(surv_clean, esr1_clean) |> map(|pair|
    pair[0] - (model_simple.intercept + model_simple.coefficients[0] *
pair[1])
)
qq_plot(residuals, "Residuals QQ Plot") |> save_plot("esr1_survival_qq.svg")
```

The QQ plot is crucial. If the residuals are normally distributed, the points fall along the diagonal. Survival data is notoriously non-normal (right-skewed, censored), so the QQ plot will likely show deviation. This means the p-values from linear regression are approximate at best, which is one reason survival analysis (Chapter 11) uses specialized methods.

Part 5: Multiple Regression — Controlling for Confounders

Simple regression of survival on ESR1 ignores everything else about the patient. Older patients have shorter survival regardless of gene expression. Higher-stage tumors have shorter survival. Multiple regression lets us ask: “Does ESR1 predict survival *after controlling for age and stage*?”

The model: Y = beta_0 + beta_1*ESR1* + *beta_2*Age + beta_3*Stage + epsilon

```
# Prepare variables for multiple regression
let age_vals = col_values(analysis_data,
"age_at_initial_pathologic_diagnosis")
    |> map(|x| to_float(x))

# Encode stage as numeric (simplified)
# Stage I = 1, Stage II = 2, Stage III = 3, Stage IV = 4
let stage_vals = col_values(analysis_data, "pathologic_stage") |> map(|s|
    if contains(to_string(s), "IV") then 4.0
    else if contains(to_string(s), "III") then 3.0
    else if contains(to_string(s), "II") then 2.0
    else if contains(to_string(s), "I") then 1.0
    else 0.0  # unknown/missing
)

# Build predictor matrix: [ESR1, Age, Stage]
let predictors = [esr1_clean, age_vals, stage_vals]
let pred_matrix = matrix(predictors)

# Fit multiple regression
let model_multi = lm(surv_clean, pred_matrix)
println("Multiple Regression: Survival ~ ESR1 + Age + Stage")
println("  Intercept: " + to_string(round(model_multi.intercept, 2)))
println("  ESR1 coefficient: " + to_string(round(model_multi.coefficients[0],
2)))
println("  Age coefficient: " + to_string(round(model_multi.coefficients[1],
2)))
println("  Stage coefficient: " +
to_string(round(model_multi.coefficients[2], 2)))
println("  R-squared: " + to_string(round(model_multi.r_squared, 4)))
println("  Adjusted R-squared: " + to_string(round(model_multi.adj_r_squared,
4)))
```

Interpreting Multiple Regression Coefficients

Each coefficient is now a **partial regression coefficient** — it represents the effect of that variable *holding the other variables constant*.

- **ESR1 coefficient**: Change in predicted survival per unit increase in log2(ESR1), *holding age and stage constant*. This is the adjusted effect.
- **Age coefficient**: Change in predicted survival per year of age, *holding ESR1 and stage constant*. Expected to be negative (older patients have shorter survival).
- **Stage coefficient**: Change in predicted survival per stage increase, *holding ESR1 and age constant*. Expected to be strongly negative (higher stage = shorter survival).

```
# Compare simple vs multiple regression ESR1 coefficient
println("ESR1 effect comparison:")
println("  Unadjusted (simple): " +
to_string(round(model_simple.coefficients[0], 2)))
println("  Adjusted (multiple): " +
to_string(round(model_multi.coefficients[0], 2)))
```

If the ESR1 coefficient changes substantially between simple and multiple regression, this tells you that age and stage were confounders. The unadjusted effect was partially attributable to correlations between ESR1, age, and stage.

Part 6: Residual Analysis

The validity of regression depends on its assumptions. Residual analysis checks those assumptions.

Assumptions of linear regression:

1. **Linearity**: The relationship between X and Y is linear
2. **Independence**: Residuals are independent of each other
3. **Homoscedasticity**: Residuals have constant variance
4. **Normality**: Residuals are normally distributed

```
# Calculate residuals from multiple regression
let predicted = zip(esr1_clean, zip(age_vals, stage_vals)) |> map(|pair|
    let esr1_val = pair[0]
    let age_val = pair[1][0]
    let stage_val = pair[1][1]
    model_multi.intercept
        + model_multi.coefficients[0] * esr1_val
        + model_multi.coefficients[1] * age_val
        + model_multi.coefficients[2] * stage_val
)

let resid_multi = zip(surv_clean, predicted) |> map(|pair| pair[0] - pair[1])

# 1. Residuals vs fitted values (check linearity and homoscedasticity)
scatter(predicted, resid_multi, "Residuals vs Fitted Values")
    |> save_plot("residuals_vs_fitted.svg")

# 2. QQ plot of residuals (check normality)
qq_plot(resid_multi, "Multiple Regression Residuals") |>
save_plot("multi_reg_qq.svg")

# 3. Residual summary statistics
println("Residual summary:")
println(summary(resid_multi))
println("Residual standard deviation: " + to_string(round(stdev(resid_multi),
2)))
```

If the residuals-vs-fitted plot shows a funnel shape (wider on one side), you have heteroscedasticity. If it shows a curve, the linearity assumption is violated. For survival data, both problems are common because survival times are bounded below by zero and right-skewed.

Part 7: Gene-Gene Correlation Networks

Beyond pairwise correlations, we can build a network of gene-gene correlations. This reveals modules of co-expressed genes, which often correspond to biological pathways.

```
# Select top variable genes for correlation network
# First, compute variance across samples for each gene
let expr_data = read_tsv(expr_file)

# Get a subset of known cancer-related genes
let cancer_genes = [
    "ESR1", "ERBB2", "TP53", "MKI67", "PGR", "BRCA1",
    "EGFR", "CDH1", "PTEN", "PIK3CA", "AKT1", "MTOR",
    "MYC", "CCND1", "CDK4", "RB1", "VEGFA", "KRAS",
    "BRAF", "MAP2K1"
]

# Extract expression for these genes across all samples
let cancer_expr = expr_data |> filter_by("Ensembl_ID", |g|
contains(cancer_genes, g))

# Build correlation matrix
let cancer_vals = cancer_genes |> map(|g|
    col_values(analysis_data, g) |> map(|x| log2(to_float(x) + 1.0))
)
let cancer_matrix = matrix(cancer_vals)
let cancer_cor = cor_matrix(cancer_matrix)

# Heatmap of the correlation network
heatmap(cancer_cor, cancer_genes, cancer_genes, "Cancer Gene Correlation
Network")
    |> save_plot("cancer_gene_network.svg")
```

```
# Build a graph where edges represent strong correlations
let gene_network = graph()

# Add nodes
cancer_genes |> each(|g| add_node(gene_network, g))

# Add edges for correlations above threshold
let threshold = 0.3
let n_genes = len(cancer_genes)

let edge_count = 0
let i = 0
cancer_genes |> each(|g1|
    let j = i + 1
    slice(cancer_genes, j, n_genes) |> each(|g2|
        let v1 = col_values(analysis_data, g1) |> map(|x| log2(to_float(x) +
1.0))
        let v2 = col_values(analysis_data, g2) |> map(|x| log2(to_float(x) +
1.0))
        let r_val = abs(cor(v1, v2))
        if r_val > threshold then
            add_edge(gene_network, g1, g2)
            edge_count = edge_count + 1
        end
    )
    i = i + 1
)

println("Gene correlation network:")
println("  Nodes: " + to_string(len(cancer_genes)))
println("  Edges (|r| > " + to_string(threshold) + "): " +
to_string(edge_count))

# Find connected components — these are co-expression modules
let components = connected_components(gene_network)
println("  Connected components: " + to_string(len(components)))

# Show the degree of each gene (number of strong correlations)
cancer_genes |> each(|g|
    let deg = degree(gene_network, g)
    println("  " + g + ": " + to_string(deg) + " strong correlations")
)
```

Genes with high degree are “hub” genes — they correlate with many other genes. In cancer biology, hub genes often correspond to transcription factors or signaling pathway components that regulate many downstream targets.

Part 8: Regression to the Mean

Regression to the mean is one of the most misunderstood phenomena in biostatistics. It was discovered by Francis Galton (who also coined the term “regression” — the method is named after this phenomenon).

The trap: If you select patients with the highest gene expression at time point 1 and measure them again at time point 2, their expression will appear to decrease — even if nothing has changed. This is because extreme values include

measurement error, and that error is unlikely to be equally extreme at the next measurement.

```
# Demonstrate regression to the mean with simulated data
# True expression is constant, but measurement adds noise

let n_patients = 200
let true_expr = rnorm(n_patients, 10.0, 2.0)  # true expression

# Measurement 1: true + noise
let noise1 = rnorm(n_patients, 0.0, 1.5)
let measure1 = zip(true_expr, noise1) |> map(|pair| pair[0] + pair[1])

# Measurement 2: true + different noise
let noise2 = rnorm(n_patients, 0.0, 1.5)
let measure2 = zip(true_expr, noise2) |> map(|pair| pair[0] + pair[1])

# Select the top 20% from measurement 1
let sorted_m1 = measure1 |> sort() |> to_list()
let cutoff = quantile(measure1, 0.80)
let top_idx = measure1 |> map(|x| x >= cutoff)

let top_m1 = zip(measure1, top_idx) |> filter(|pair| pair[1]) |> map(|pair|
pair[0])
let top_m2 = zip(measure2, top_idx) |> filter(|pair| pair[1]) |> map(|pair|
pair[0])

println("Regression to the mean demonstration:")
println("  Mean of top 20% at time 1: " + to_string(round(mean(top_m1), 2)))
println("  Mean of same patients at time 2: " + to_string(round(mean(top_m2),
2)))
println("  Apparent decrease: " + to_string(round(mean(top_m1) -
mean(top_m2), 2)))
println("  (This 'decrease' is purely statistical, not biological)")
```

This is devastating for pre-post studies. If you select patients with the highest tumor marker levels and give them a treatment, the marker will likely decrease at follow-up even if the treatment does nothing. Regression to the mean masquerades as a treatment effect.

The solution: Use a control group. If both treatment and control groups are selected for high values, both will regress to the mean. Only the *difference* between groups reflects a true treatment effect.

```
# Show the same phenomenon with a scatter plot
scatter(measure1, measure2, "Measurement 1 vs Measurement 2")
    |> save_plot("regression_to_mean.svg")

# The slope of the regression line is less than 1.0
let rtm_model = lm(measure2, measure1)
println("Regression slope (should be < 1 if regression to mean): "
    + to_string(round(rtm_model.coefficients[0], 4)))
```

A slope less than 1.0 means that extreme values on X are associated with less extreme values on Y. This is regression to the mean. The more measurement noise, the stronger the effect.

Part 9: Building a Practical Prediction Model

Now let us combine everything into a practical prediction model for breast cancer outcomes.

```
# Build a multi-gene model predicting survival
# Include: ESR1, ERBB2, MKI67, Age, Stage

let esr1_v = col_values(analysis_data, "ESR1") |> map(|x| log2(to_float(x) +
1.0))
let erbb2_v = col_values(analysis_data, "ERBB2") |> map(|x| log2(to_float(x)
+ 1.0))
let mki67_v = col_values(analysis_data, "MKI67") |> map(|x| log2(to_float(x)
+ 1.0))
let age_v = col_values(analysis_data, "age_at_initial_pathologic_diagnosis")
    |> map(|x| to_float(x))
let stage_v = col_values(analysis_data, "pathologic_stage") |> map(|s|
    if contains(to_string(s), "IV") then 4.0
    else if contains(to_string(s), "III") then 3.0
    else if contains(to_string(s), "II") then 2.0
    else if contains(to_string(s), "I") then 1.0
    else 0.0
)
let surv_v = col_values(analysis_data, "OS.time") |> map(|x| to_float(x))

# Fit the full model
let pred_full = matrix([esr1_v, erbb2_v, mki67_v, age_v, stage_v])
let model_full = lm(surv_v, pred_full)

println("Full Model: Survival ~ ESR1 + ERBB2 + MKI67 + Age + Stage")
println("  R-squared: " + to_string(round(model_full.r_squared, 4)))
println("  Adjusted R-squared: " + to_string(round(model_full.adj_r_squared,
4)))
println("")
println("  Coefficients:")
let coef_names = ["ESR1", "ERBB2", "MKI67", "Age", "Stage"]
zip(coef_names, model_full.coefficients) |> each(|pair|
    println("    " + pair[0] + ": " + to_string(round(pair[1], 4))
        + "  (p = " + to_string(round(model_full.p_values[0], 4)) + ")")
)
```

```
# Compare models: simple vs multiple
println("Model comparison:")
println("  ESR1 only:        R^2 = " +
to_string(round(model_simple.r_squared, 4)))
println("  ESR1+Age+Stage:   R^2 = " + to_string(round(model_multi.r_squared,
4)))
println("  Full (5 vars):    R^2 = " + to_string(round(model_full.r_squared,
4)))
```

The improvement from adding more variables shows you how much additional variance each variable explains. But beware: adding more variables always increases R-squared (even random variables will increase it). The adjusted R-squared penalizes for the number of predictors and is a better guide.

```
# Normalize coefficients to compare effect sizes
# Standardize each predictor to mean 0, SD 1
let esr1_z = normalize(esr1_v)
let erbb2_z = normalize(erbb2_v)
let mki67_z = normalize(mki67_v)
let age_z = normalize(age_v)
let stage_z = normalize(stage_v)

let pred_std = matrix([esr1_z, erbb2_z, mki67_z, age_z, stage_z])
let model_std = lm(surv_v, pred_std)

println("Standardized coefficients (effect size comparison):")
zip(coef_names, model_std.coefficients) |> each(|pair|
    println("  " + pair[0] + ": " + to_string(round(pair[1], 2)))
)
```

Standardized coefficients are directly comparable: the variable with the largest absolute standardized coefficient has the strongest effect on survival. Typically, stage dominates, followed by age, with individual gene effects being smaller but still meaningful.

Interpretation

What We Found

1. **ESR1 and PGR are strongly positively correlated** (r ~ 0.7), reflecting the biology of estrogen receptor signaling.
2. **ESR1 and ERBB2 are negatively correlated** (r ~ -0.25), reflecting distinct breast cancer subtypes.
3. **ESR1 expression predicts survival** in simple regression, but the effect is partially confounded by age and stage.
4. **Multiple regression reveals the adjusted effect**: after controlling for clinical variables, the independent contribution of ESR1 to survival prediction is smaller than the unadjusted effect suggests.
5. **Stage is the dominant predictor** of survival time, as expected.

6. **Gene-gene correlation networks** reveal modules of co-expressed genes corresponding to known biological pathways.

What We Cannot Conclude

- We **cannot** say ESR1 expression *causes* longer survival. ER-positive tumors are a distinct subtype with different biology, different treatment (tamoxifen, aromatase inhibitors), and different prognosis.
- We **cannot** use this linear regression model for clinical prediction of survival. Survival data requires specialized methods (Chapter 11) because of censoring.
- We **cannot** conclude that low-correlation genes are biologically unrelated. Correlation measures linear (or rank-based) association. Non-linear relationships, threshold effects, and interactions are invisible to correlation.

Decision

When to Report a Correlation

Report correlations when:

- The sample size is large enough for the confidence interval to be narrow
- You have checked that the relationship is not driven by outliers
- You have considered and addressed potential confounders
- You report the confidence interval, not just the point estimate

When to Use Multiple Regression

Use multiple regression when:

- You want the effect of X on Y *adjusting for* other variables
- You suspect confounders (age, batch, clinical variables)
- You want to compare the relative importance of multiple predictors
- The outcome variable is continuous and approximately normal

When to Use Something Else

- **Binary outcome** (alive/dead, mutated/wild-type): Use logistic regression (Chapter 10)
- **Survival time with censoring**: Use Cox regression (Chapter 11)
- **Count data** (read counts, variant counts): Use Poisson or negative binomial regression (Chapter 10)
- **Non-linear relationships**: Consider spline regression or generalized additive models

Pitfalls

Pitfall 1: Ignoring Non-Linearity

```
# Pearson correlation can be zero even with a strong non-linear relationship
# Example: quadratic relationship
let x_vals = rnorm(200, 0.0, 3.0)
let y_quad = x_vals |> map(|x| pow(x, 2.0) + rnorm(1, 0.0, 1.0) |> head(1))
let y_flat = y_quad |> flatten()

let r_quad = cor(x_vals, y_flat)
println("Pearson r for quadratic relationship: " + to_string(round(r_quad,
4)))
println("(Near zero despite perfect quadratic relationship)")

# Always plot your data!
scatter(x_vals, y_flat, "Quadratic Relationship (r ~ 0)")
    |> save_plot("quadratic_relationship.svg")
```

Pitfall 2: Correlation of Ratios

In genomics, we often correlate ratios or normalized values. This creates spurious correlations because ratios share denominators.

If X = A/C and Y = B/C, then X and Y will be correlated even if A and B are independent — because both depend on C. This affects:

- FPKM values (normalized by library size)
- Compositional data (proportions that sum to 1)
- Fold changes relative to the same reference

```
# Demonstrate spurious ratio correlation
let a_vals = rnorm(100, 50.0, 10.0)  # independent
let b_vals = rnorm(100, 50.0, 10.0)  # independent
let c_vals = rnorm(100, 50.0, 10.0)  # shared denominator

let ratio_x = zip(a_vals, c_vals) |> map(|pair| pair[0] / pair[1])
let ratio_y = zip(b_vals, c_vals) |> map(|pair| pair[0] / pair[1])

let r_ratio = cor(ratio_x, ratio_y)
println("Correlation of A/C and B/C (A, B independent): " +
to_string(round(r_ratio, 4)))
println("(Positive despite A and B being independent)")
```

Pitfall 3: Extrapolation

Linear regression fits a line through the observed range of data. Predicting outside that range (extrapolation) is unreliable because the relationship may not be linear beyond the data.

If your TCGA patients range from age 25 to 90, do not use the model to predict survival for a 15-year-old. The linear relationship between age and survival may not hold at extreme ages.

Pitfall 4: Multicollinearity

When predictors in multiple regression are highly correlated with each other, the individual coefficients become unstable. This is multicollinearity.

```
# ESR1 and PGR are highly correlated — including both may cause instability
let pred_collinear = matrix([esr1_v, pgr_log, age_v])
let model_collinear = lm(surv_v, pred_collinear)

println("Model with collinear predictors (ESR1 + PGR + Age):")
println("  ESR1 coefficient: " +
to_string(round(model_collinear.coefficients[0], 4)))
println("  PGR coefficient: " +
to_string(round(model_collinear.coefficients[1], 4)))
println("  R-squared: " + to_string(round(model_collinear.r_squared, 4)))

# Compare to models with only one of the two
let model_esr1_only = lm(surv_v, matrix([esr1_v, age_v]))
let model_pgr_only = lm(surv_v, matrix([pgr_log, age_v]))

println("ESR1 + Age only R^2: " + to_string(round(model_esr1_only.r_squared,
4)))
println("PGR + Age only R^2: " + to_string(round(model_pgr_only.r_squared,
4)))
```

When two predictors carry largely the same information, adding both does not improve prediction and makes individual coefficients unreliable. Choose one or use dimensionality reduction.

Pitfall 5: Overfitting

With enough predictors, you can fit any dataset perfectly. But the model will not generalize to new data.

Rule of thumb: You need at least 10-15 observations per predictor variable. With 1,000 patients, a model with 5 predictors is fine. A model with 200 gene predictors needs 2,000-3,000 patients.

```
# Demonstrate with adjusted R-squared
println("Adjusted R-squared penalizes for number of predictors:")
println("  Simple (1 predictor): adj R^2 = " +
to_string(round(model_simple.adj_r_squared, 4)))
println("  Multiple (3 predictors): adj R^2 = " +
to_string(round(model_multi.adj_r_squared, 4)))
println("  Full (5 predictors): adj R^2 = " +
to_string(round(model_full.adj_r_squared, 4)))
println("If adj R^2 stops increasing or decreases, added variables are not
helping.")
```

Pitfall 6: P-value Fishing

With 20,000 genes, testing each one for correlation with survival will produce ~1,000 “significant” results at $p < 0.05$ by chance alone. Always correct for multiple testing when scanning many genes.

```
# Suppose we tested 100 genes for correlation with survival
# Collect p-values and apply correction
let test_genes = slice(cancer_genes, 0, 10)  # using our subset for
demonstration

let p_values_raw = test_genes |> map(|g|
    let gv = col_values(analysis_data, g) |> map(|x| log2(to_float(x) + 1.0))
    let sp = spearman(gv, surv_v)
    sp.p_value
)

let p_adj = p_adjust(p_values_raw, "BH")

println("Gene          Raw P         BH-adjusted P")
println("————————      ————          ——————————————")
zip(test_genes, zip(p_values_raw, p_adj)) |> each(|pair|
    println(pair[0] + "    " + to_string(round(pair[1][0], 6))
        + "    " + to_string(round(pair[1][1], 6)))
)
```

Where Next?

Immediate Extensions

- **Logistic regression**: When the outcome is binary (Chapter 10). Use GLM with logit link to predict ER status from gene expression.
- **Cox regression**: When the outcome is survival time with censoring (Chapter 11). The correct approach for the survival question we explored here with imperfect linear regression.
- **Partial correlation**: Correlation between X and Y after removing the effect of Z. Computable by correlating the residuals from regressing X on Z and Y on Z.

Advanced Topics

- **Ridge and Lasso regression**: When you have more predictors than samples (the genomics default). Regularization prevents overfitting.
- **Principal component regression**: Use PCA to reduce thousands of genes to a handful of components, then regress survival on the components.
- **Weighted gene co-expression network analysis (WGCNA)**: Scale gene-gene correlation networks to the entire transcriptome. Identify modules and

relate them to clinical traits.

Practice Problems

1. **Hormone receptor genes**: Compute the correlation matrix for all known hormone receptor genes (ESR1, ESR2, PGR, AR) and their downstream targets in the TCGA BRCA dataset. Which pairs have the strongest correlations?

2. **Confounding by subtype**: Run simple regression of MKI67 on ERBB2 in all patients, then separately within PAM50 subtypes. Does the relationship change? This demonstrates that subtype acts as a confounder.

3. **Effect of log transformation**: Compare Pearson correlations between ESR1 and PGR on raw FPKM values vs log2-transformed values. How much does the transformation change the result? Plot both.

4. **Regression to the mean**: Using two replicate measurements from the same samples (if available), demonstrate regression to the mean by selecting extreme samples from replicate 1 and measuring their values in replicate 2.

5. **Multicollinearity**: Build a regression model using 10 genes from the same pathway (e.g., the PI3K-AKT pathway). Examine how unstable the individual coefficients are. Then use PCA on the 10 genes and regress survival on the first 2 principal components instead.

Key takeaways from this chapter:

- Correlation measures association, not causation. Always consider confounders.
- Always visualize the data before computing correlations — scatter plots reveal non-linearity, outliers, and subgroups that numbers hide.
- Multiple regression adjusts for confounders but assumes linearity and independence.
- Spearman and Kendall correlations handle non-linear monotonic relationships.
- Regression to the mean traps pre-post studies without control groups.
- Multicollinearity makes individual coefficients in multiple regression unreliable when predictors are correlated.
- Gene-gene correlation networks reveal co-expression modules, but edges are not causal relationships.
- With thousands of genes, always correct for multiple testing.

Generalized Linear Models

> "All models are wrong, but some are useful." — George Box

The Problem

ClinVar contains over 2 million variant records classified by clinical significance — pathogenic, benign, uncertain, and others. You want to build a model that predicts whether a variant is pathogenic based on measurable properties: allele frequency in the population, gene constraint (tolerance to loss of function), variant type, and review status.

This is a classification problem with a binary outcome: pathogenic or not. If you try to use ordinary linear regression, you will get predicted probabilities outside the [0, 1] range and biased estimates. You need a generalized linear model (GLM) — specifically, logistic regression.

But the GLM framework is much broader than logistic regression. In this chapter, we also cover Poisson regression for count data (how many pathogenic variants does each gene have?) and negative binomial regression for overdispersed counts (why RNA-seq differential expression tools use NB, not Poisson). By the end, you will understand the GLM family well enough to choose the right model for any outcome type.

Why It Matters

Variant Classification Is a Prediction Problem

As clinical genomics scales to millions of patients, we encounter millions of variants of uncertain significance (VUS). Machine learning approaches like REVEL, CADD, and AlphaMissense predict pathogenicity from sequence features. Understanding logistic regression is essential because:

1. It is the foundation of all these tools — even complex ensemble methods include logistic regression components.
2. It produces interpretable odds ratios that clinicians can understand.

3. It provides proper uncertainty estimates (confidence intervals on predictions).
4. Regulatory bodies (FDA, EMA) require interpretable models for clinical decision support.

Count Data Is Everywhere in Genomics

RNA-seq produces read counts. Variant calling produces variant counts per gene. ChIP-seq produces peak counts per region. WGS produces coverage counts per base. All of these are count data, and count data violates the assumptions of ordinary linear regression (counts are non-negative integers, variance often increases with the mean).

Poisson and negative binomial regression — both GLMs — handle count data correctly. DESeq2, edgeR, and every major RNA-seq tool uses negative binomial GLMs under the hood.

The GLM Framework Unifies Everything

The GLM framework covers:

- **Linear regression**: Continuous outcome, identity link, normal errors
- **Logistic regression**: Binary outcome, logit link, binomial errors
- **Poisson regression**: Count outcome, log link, Poisson errors
- **Negative binomial regression**: Overdispersed count outcome, log link, NB errors

Once you understand the framework, choosing the right model is straightforward: look at your outcome variable, choose the appropriate distribution and link function, and fit.

The Data

ClinVar Variant Summary

ClinVar is NCBI's public archive of relationships between human genetic variants and phenotypes. The variant summary file contains one row per variant with clinical significance, gene, variant type, and review status.

```
# Download ClinVar variant summary
let clinvar_url =
"https://ftp.ncbi.nlm.nih.gov/pub/clinvar/tab_delimited/variant_summary.txt.g
z"
let clinvar_file = download(clinvar_url, "variant_summary.txt.gz")

let clinvar = read_tsv(clinvar_file)
println("ClinVar records: " + to_string(len(clinvar)))
println("Columns: " + to_string(ncols(clinvar)))
println(columns(clinvar))
```

```
# Examine clinical significance categories
let clin_sig = col_values(clinvar, "ClinicalSignificance")
let sig_counts = value_counts(clin_sig)
println("Clinical significance distribution:")
println(sig_counts |> arrange("count", "desc") |> head(15))
```

gnomAD Constraint Metrics

gnomAD provides gene-level constraint metrics: how tolerant each gene is to loss-of-function (LoF) variants. Genes under strong constraint (intolerant of LoF) are more likely to harbor disease-causing variants.

```
# Download gnomAD v4.1 constraint metrics
let gnomad_url = "https://storage.googleapis.com/gcp-public-data--
gnomad/release/4.1/constraint/gnomad.v4.1.constraint_metrics.tsv"
let gnomad_file = download(gnomad_url, "gnomad_constraint.tsv")

let constraint = read_tsv(gnomad_file)
println("gnomAD constraint: " + to_string(len(constraint)) + " genes")
println("Columns:")
println(columns(constraint))
```

```
# Key constraint columns:
# - gene: gene symbol
# - transcript: canonical transcript
# - oe_lof: observed/expected ratio for LoF variants (lower = more
constrained)
# - oe_lof_upper: upper confidence bound
# - pLI: probability of LoF intolerance (higher = more constrained)
# - oe_mis: observed/expected for missense
# - lof.z: Z-score for LoF constraint

let pli_vals = col_values(constraint, "pLI")
    |> filter(|x| !is_none(x))
    |> map(|x| to_float(x))

println("pLI summary:")
println(summary(pli_vals))

histogram(pli_vals, "gnomAD pLI Distribution") |>
save_plot("pli_distribution.svg")
```

The pLI distribution is bimodal: most genes cluster near 0 (LoF tolerant) or near 1 (LoF intolerant), with relatively few in between. This reflects the biology — genes are either essential (constraint) or dispensable (tolerance).

Preparing the Analysis Dataset

```
# Filter ClinVar to single-gene variants with clear pathogenic/benign status
# Restrict to GRCh38 assembly
let clinvar_clean = clinvar
    |> filter_by("Assembly", |a| a == "GRCh38")
    |> filter_by("ClinicalSignificance", |s|
        s == "Pathogenic" || s == "Benign" || s == "Likely pathogenic" || s
== "Likely benign"
    )

# Create binary outcome: pathogenic (1) vs benign (0)
let clinvar_binary = clinvar_clean |> mutate("is_pathogenic", |row|
    if contains(row["ClinicalSignificance"], "athogenic") then 1.0
    else 0.0
)

println("Analysis dataset: " + to_string(len(clinvar_binary)) + " variants")
let outcome = col_values(clinvar_binary, "is_pathogenic")
let path_count = outcome |> filter(|x| x == 1.0) |> len()
let benign_count = outcome |> filter(|x| x == 0.0) |> len()
println("  Pathogenic: " + to_string(path_count))
println("  Benign: " + to_string(benign_count))
println("  Prevalence: " + to_string(round(to_float(path_count) /
to_float(len(outcome)) * 100.0, 1)) + "%")
```

```
# Join with gnomAD constraint data to get gene-level features
let analysis = inner_join(clinvar_binary, constraint, "GeneSymbol", "gene")
println("After joining with gnomAD: " + to_string(len(analysis)) + "
variants")
```

Exploration

Why Linear Regression Fails for Binary Outcomes

Before learning logistic regression, we need to understand why ordinary linear regression is inappropriate for binary outcomes.

```
# Attempt linear regression on binary outcome
let y_binary = col_values(analysis, "is_pathogenic") |> map(|x| to_float(x))
let pli = col_values(analysis, "pLI")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))

let bad_model = lm(y_binary, pli)
println("Linear regression on binary outcome:")
println("  Intercept: " + to_string(round(bad_model.intercept, 4)))
println("  pLI slope: " + to_string(round(bad_model.coefficients[0], 4)))
println("  R-squared: " + to_string(round(bad_model.r_squared, 4)))

# Compute predicted values
let predicted_linear = pli |> map(|x| bad_model.intercept +
bad_model.coefficients[0] * x)

# Check for predictions outside [0, 1]
let below_zero = predicted_linear |> filter(|p| p < 0.0) |> len()
let above_one = predicted_linear |> filter(|p| p > 1.0) |> len()
println("  Predictions below 0: " + to_string(below_zero))
println("  Predictions above 1: " + to_string(above_one))
println("  (These are impossible probabilities!)")
```

Linear regression on a binary outcome produces predictions that can be negative or greater than 1. These cannot be interpreted as probabilities. The model also violates the homoscedasticity assumption — the variance of a binary variable depends on its mean (var = p * (1-p)), so residuals are inherently heteroscedastic.

```
# Scatter plot shows the problem visually
scatter(pli, y_binary, "Linear Regression on Binary Outcome (Wrong!)")
    |> save_plot("linear_binary_wrong.svg")
```

The scatter plot shows points clustered at 0 and 1 with a straight line trying to fit through them. The line inevitably extends beyond the valid range. We need a

model that maps any linear combination of predictors to the [0, 1] interval.

Exploring Predictor-Outcome Relationships

```
# Distribution of pLI by pathogenicity status
let pli_pathogenic = analysis
    |> filter_by("is_pathogenic", |x| to_float(x) == 1.0)
let pli_benign = analysis
    |> filter_by("is_pathogenic", |x| to_float(x) == 0.0)

let pli_path_vals = col_values(pli_pathogenic, "pLI")
    |> filter(|x| !is_none(x)) |> map(|x| to_float(x))
let pli_ben_vals = col_values(pli_benign, "pLI")
    |> filter(|x| !is_none(x)) |> map(|x| to_float(x))

println("pLI in pathogenic variants: mean = " +
to_string(round(mean(pli_path_vals), 4)))
println("pLI in benign variants: mean = " +
to_string(round(mean(pli_ben_vals), 4)))

boxplot([pli_path_vals, pli_ben_vals], ["Pathogenic", "Benign"], "pLI by
Clinical Significance")
    |> save_plot("pli_by_significance.svg")
```

Pathogenic variants should be enriched in constrained genes (high pLI). If a gene is intolerant of loss-of-function variants, losing one copy is more likely to cause disease.

```
# Variant type distribution
let type_path = col_values(pli_pathogenic, "Type") |> value_counts()
let type_ben = col_values(pli_benign, "Type") |> value_counts()

println("Variant types in pathogenic:")
println(type_path |> arrange("count", "desc") |> head(10))
println("Variant types in benign:")
println(type_ben |> arrange("count", "desc") |> head(10))
```

The Analysis

Part 1: Logistic Regression from First Principles

Logistic regression models the log-odds of the outcome as a linear function of predictors.

The logistic function (sigmoid):

p(Y=1) = 1 / (1 + exp(-(beta_0 + beta_1*X)))

This function maps any real number to the (0, 1) interval, solving the problem of impossible probabilities.

Equivalently, the model says:

log(p / (1-p)) = beta_0 + beta_1*X

The left side is the **log-odds** (also called the logit). The logit transformation is the **link function** that connects the linear predictor to the probability.

```
# Visualize the logistic function
let x_range = []
let i = -6.0
# Build a sequence from -6 to 6
let x_range = rnorm(100, 0.0, 3.0) |> sort()
let sigmoid_vals = x_range |> map(|x| 1.0 / (1.0 + exp(-x)))

scatter(x_range, sigmoid_vals, "The Logistic (Sigmoid) Function")
    |> save_plot("sigmoid_function.svg")
```

The S-shaped curve shows the key property: probability approaches 0 for very negative values and 1 for very positive values, with the steepest change around 0. This matches our intuition for how risk factors work — low exposure means low risk, high exposure means high risk, with the biggest change in the middle.

Part 2: Fitting Logistic Regression in BioLang

```
# Simple logistic regression: pathogenicity ~ pLI
let y = col_values(analysis, "is_pathogenic") |> map(|x| to_float(x))
let x_pli = col_values(analysis, "pLI")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))

let logit_simple = glm(y, x_pli, "binomial")

println("Logistic Regression: Pathogenicity ~ pLI")
println("  Intercept: " + to_string(round(logit_simple.intercept, 4)))
println("  pLI coefficient: " + to_string(round(logit_simple.coefficients[0],
4)))
println("  p-value (pLI): " + to_string(logit_simple.p_values[0]))
```

Part 3: Odds Ratios — The Key to Interpretation

In logistic regression, we do not interpret coefficients directly the way we do in linear regression. Instead, we exponentiate them to get **odds ratios**.

Odds: p / (1-p). If the probability is 0.75, the odds are 3:1 (3 in favor for every 1 against).

Odds ratio: The factor by which the odds change for a one-unit increase in the predictor. OR = exp(beta).

```
# Calculate odds ratios
let or_pli = exp(logit_simple.coefficients[0])
println("Odds Ratio for pLI: " + to_string(round(or_pli, 4)))

# Interpretation:
# If OR > 1: Higher pLI increases the odds of pathogenicity
# If OR < 1: Higher pLI decreases the odds of pathogenicity
# If OR = 1: No association

if or_pli > 1.0 then
    println("Interpretation: Each unit increase in pLI multiplies the odds
of")
    println("  pathogenicity by " + to_string(round(or_pli, 2)))
else
    println("Interpretation: Each unit increase in pLI divides the odds of")
    println("  pathogenicity by " + to_string(round(1.0 / or_pli, 2)))
end
```

Reading odds ratios in context:

- OR = 1.0: No association
- OR = 1.5: 50% increase in odds per unit increase in predictor
- OR = 2.0: Doubling of odds (strong effect)
- OR = 0.5: Halving of odds (strong protective effect)
- OR = 10: Very strong effect (common in rare variant studies)

For pLI, since it ranges from 0 to 1, the OR represents the change in odds when going from a completely LoF-tolerant gene (pLI=0) to a completely LoF-intolerant gene (pLI=1). This is a biologically meaningful contrast.

Part 4: Multiple Logistic Regression

```
# Add more predictors: LoF O/E ratio, missense O/E, LoF Z-score
let x_oe_lof = col_values(analysis, "oe_lof")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))
let x_oe_mis = col_values(analysis, "oe_mis")
    |> map(|x| if is_none(x) then 1.0 else to_float(x))
let x_lof_z = col_values(analysis, "lof.z")
    |> map(|x| if is_none(x) then 0.0 else to_float(x))

# Build predictor matrix
let pred_mat = matrix([x_pli, x_oe_lof, x_oe_mis, x_lof_z])

let logit_multi = glm(y, pred_mat, "binomial")

println("Multiple Logistic Regression:")
println("  Pathogenicity ~ pLI + oe_lof + oe_mis + lof_z")
println("")
let pred_names = ["pLI", "oe_lof", "oe_mis", "lof_z"]
println("  Variable     Coef       OR        p-value")
println("  _________    ____       __        _______")
zip(pred_names, logit_multi.coefficients) |> each(|pair|
    let coef = pair[1]
    let or_val = exp(coef)
    println("  " + pair[0] + "    "
        + to_string(round(coef, 4)) + "    "
        + to_string(round(or_val, 4)))
)
```

Interpreting Multiple Logistic Regression

Each odds ratio is now **adjusted** — it represents the effect of that predictor holding all other predictors constant.

- **pLI OR**: Effect of gene constraint on pathogenicity, adjusting for observed/expected ratios
- **oe_lof OR**: Lower O/E ratio (more constrained) should increase odds of pathogenicity. But the sign might flip from the simple model because pLI and oe_lof are correlated (multicollinearity)
- **oe_mis OR**: Missense constraint — genes intolerant of missense variants may be more likely to harbor pathogenic missense variants
- **lof_z OR**: Z-score for LoF constraint — another measure of the same underlying biology

```
# Check for multicollinearity in constraint metrics
let constraint_cor = cor_matrix(pred_mat)
println("Correlation among predictors:")
println("         pLI    oe_lof  oe_mis  lof_z")
let idx = 0
pred_names |> each(|name|
    println("  " + name)
    idx = idx + 1
)
heatmap(constraint_cor, pred_names, pred_names, "Predictor Correlation")
    |> save_plot("predictor_correlation.svg")
```

If predictors are highly correlated (e.g., pLI and lof_z both measure LoF constraint), their individual coefficients become unstable. This does not affect the model's overall predictive performance, but it makes individual coefficient interpretation unreliable. Choose one representative per concept, or use the model purely for prediction.

Part 5: The GLM Framework

Logistic regression is one member of the GLM family. The framework has three components:

1. **Random component**: The probability distribution of the outcome
    - Normal → linear regression
    - Binomial → logistic regression
    - Poisson → Poisson regression
    - Negative binomial → NB regression
2. **Systematic component**: The linear predictor
    - eta = beta_0 + beta_1*X_1* + ... + *beta_p*X_p
3. **Link function**: Maps the expected value to the linear predictor
    - Identity: E(Y) = eta (linear regression)
    - Logit: log(E(Y)/(1-E(Y))) = eta (logistic)
    - Log: log(E(Y)) = eta (Poisson, NB)

```
# The three link functions visualized

# 1. Identity link (linear regression)
let x_vals = rnorm(100, 0.0, 2.0) |> sort()

# 2. Logit link (logistic regression) — inverse is sigmoid
let logistic_vals = x_vals |> map(|x| 1.0 / (1.0 + exp(-x)))

# 3. Log link (Poisson regression) — inverse is exponential
let poisson_vals = x_vals |> map(|x| exp(x))

scatter(x_vals, logistic_vals, "Inverse Logit (Sigmoid) — For Probabilities")
    |> save_plot("link_logit.svg")

scatter(x_vals, poisson_vals, "Inverse Log (Exponential) — For Counts")
    |> save_plot("link_log.svg")
```

Part 6: Poisson Regression for Count Data

How many pathogenic variants does each gene have? This is a count: 0, 1, 2, 3, etc. Poisson regression models counts as a function of predictors.

The model: log(E(count)) = beta_0 + beta_1*X

The log link ensures predicted counts are always non-negative (exp of anything is positive).

```
# Count pathogenic variants per gene
let variants_per_gene = analysis
    |> filter_by("is_pathogenic", |x| to_float(x) == 1.0)
    |> group_by("GeneSymbol")
    |> count_by("GeneSymbol")

println("Pathogenic variants per gene:")
println(describe(variants_per_gene))

let counts = col_values(variants_per_gene, "count") |> map(|x| to_float(x))
println("Count summary:")
println(summary(counts))

histogram(counts, "Pathogenic Variants per Gene") |>
save_plot("variants_per_gene.svg")
```

```
# Join gene-level counts with constraint data
let gene_counts = inner_join(variants_per_gene, constraint, "GeneSymbol",
"gene")

let count_y = col_values(gene_counts, "count") |> map(|x| to_float(x))
let count_pli = col_values(gene_counts, "pLI")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))

# Fit Poisson regression: variant_count ~ pLI
let poisson_model = glm(count_y, count_pli, "poisson")

println("Poisson Regression: Pathogenic Variant Count ~ pLI")
println("  Intercept: " + to_string(round(poisson_model.intercept, 4)))
println("  pLI coefficient: " +
to_string(round(poisson_model.coefficients[0], 4)))
println("  Rate Ratio (exp(coef)): " +
to_string(round(exp(poisson_model.coefficients[0]), 4)))
```

In Poisson regression, exp(beta) is the **rate ratio** — the factor by which the expected count changes for a unit increase in the predictor. If the rate ratio for pLI is 2.5, then fully constrained genes (pLI=1) have 2.5 times as many pathogenic variants as unconstrained genes (pLI=0).

Part 7: The Overdispersion Problem

Poisson regression assumes that the mean equals the variance. Real count data in genomics almost always violates this: the variance exceeds the mean. This is **overdispersion**.

```
# Check for overdispersion
let count_mean = mean(count_y)
let count_var = variance(count_y)
let dispersion = count_var / count_mean

println("Checking Poisson assumption (mean = variance):")
println("  Mean count: " + to_string(round(count_mean, 2)))
println("  Variance: " + to_string(round(count_var, 2)))
println("  Dispersion ratio (var/mean): " + to_string(round(dispersion, 2)))

if dispersion > 2.0 then
    println("  OVERDISPERSED: Variance >> Mean")
    println("  Poisson model underestimates standard errors")
    println("  Use negative binomial regression instead")
else if dispersion > 1.5 then
    println("  MILDLY OVERDISPERSED: Consider negative binomial")
else
    println("  Poisson assumption approximately met")
end
```

Why overdispersion matters: When the data is overdispersed and you use Poisson regression anyway, the standard errors are too small. This means p-

values are too small, confidence intervals are too narrow, and you declare significance too easily. The model is overconfident.

Part 8: Negative Binomial Regression

The negative binomial distribution adds an extra parameter (dispersion) that allows the variance to exceed the mean. This is why DESeq2 and edgeR use negative binomial models for RNA-seq.

```
# Fit negative binomial regression
let nb_model = glm(count_y, count_pli, "negative_binomial")

println("Negative Binomial Regression: Count ~ pLI")
println("  Intercept: " + to_string(round(nb_model.intercept, 4)))
println("  pLI coefficient: " + to_string(round(nb_model.coefficients[0],
4)))
println("  Rate Ratio: " + to_string(round(exp(nb_model.coefficients[0]),
4)))
println("  p-value: " + to_string(nb_model.p_values[0]))

# Compare with Poisson
println("")
println("Comparison:")
println("  Poisson pLI coef: " +
to_string(round(poisson_model.coefficients[0], 4)))
println("  NB pLI coef: " + to_string(round(nb_model.coefficients[0], 4)))
println("  (Coefficients similar, but NB has larger standard errors)")
```

The negative binomial and Poisson coefficients will be similar — the interpretation is the same (rate ratios). But the NB standard errors are larger, properly reflecting the extra variability in the data. This means NB p-values are larger (more conservative), which is appropriate when the data is overdispersed.

Part 9: Why RNA-seq Uses Negative Binomial

This is worth a dedicated section because every bioinformatician encounters this design choice.

```
# Simulate RNA-seq-like count data to illustrate
# Gene with mean count 100 across 20 samples
# Under Poisson: variance = 100, so SD ~ 10
# Under NB: variance = 100 + 100^2/theta, so SD >> 10

let poisson_counts = rpois(20, 100.0)
let nb_counts = rpois(20, 100.0)  # simplified; real NB has extra variance

println("Poisson simulation (mean=100):")
println("  Mean: " + to_string(round(mean(poisson_counts |> map(|x|
to_float(x))), 1)))
println("  Variance: " + to_string(round(variance(poisson_counts |> map(|x|
to_float(x))), 1)))

# In real RNA-seq, biological replicates show much more variation
# than Poisson predicts. This "biological variation" is the overdispersion.
# Technical replicates (same library sequenced twice) follow Poisson.
# Biological replicates (different mice/patients) add extra variance.

println("")
println("Key insight for RNA-seq:")
println("  Technical replicates: Poisson (var ~ mean)")
println("  Biological replicates: Negative Binomial (var >> mean)")
println("  DESeq2 and edgeR use NB because experiments have biological
replicates")
```

The biological variation between individuals (or between mice, or between cell cultures) creates overdispersion. Two mice with the same genotype and treatment still have different gene expression levels. This biological noise is on top of the technical noise (Poisson sampling during sequencing). The negative binomial captures both sources of variation.

Part 10: Model Selection — AIC and BIC

When you have multiple candidate models, how do you choose? Adjusted R-squared works for linear regression, but GLMs use different criteria.

AIC (Akaike Information Criterion): Balances fit against complexity. Lower is better. Penalizes by 2 * number of parameters.

BIC (Bayesian Information Criterion): Similar to AIC but with a stronger penalty for complexity. Penalizes by log(n) * number of parameters, where n is sample size. BIC favors simpler models when n is large.

```
# Compare models with different predictor sets
# Model 1: pLI only
let m1 = glm(y, x_pli, "binomial")

# Model 2: pLI + oe_lof
let m2 = glm(y, matrix([x_pli, x_oe_lof]), "binomial")

# Model 3: pLI + oe_lof + oe_mis
let m3 = glm(y, matrix([x_pli, x_oe_lof, x_oe_mis]), "binomial")

# Model 4: all four predictors
let m4 = glm(y, pred_mat, "binomial")

println("Model Comparison:")
println("  Model                  AIC         BIC")
println("  ─────                  ───         ───")
println("  pLI only               " + to_string(round(m1.aic, 1))
    + "    " + to_string(round(m1.bic, 1)))
println("  pLI + oe_lof           " + to_string(round(m2.aic, 1))
    + "    " + to_string(round(m2.bic, 1)))
println("  pLI + oe_lof + oe_mis  " + to_string(round(m3.aic, 1))
    + "    " + to_string(round(m3.bic, 1)))
println("  All four               " + to_string(round(m4.aic, 1))
    + "    " + to_string(round(m4.bic, 1)))
```

The model with the lowest AIC (or BIC) is preferred. If adding a predictor reduces AIC, the information gained by including it outweighs the complexity penalty. If AIC increases, the predictor is not adding enough to justify its inclusion.

Important: AIC and BIC can only compare models fitted to the same dataset. If you subset the data differently for different models, the comparison is invalid.

Part 11: Predicting Pathogenicity — A Practical Pipeline

```
# Build a practical pathogenicity prediction model
# Prepare features at the variant level

# Encode variant type as numeric features
let clinvar_features = analysis |> mutate("is_snv", |row|
    if row["Type"] == "single nucleotide variant" then 1.0 else 0.0
)
let clinvar_features = clinvar_features |> mutate("is_deletion", |row|
    if row["Type"] == "Deletion" then 1.0 else 0.0
)
let clinvar_features = clinvar_features |> mutate("is_insertion", |row|
    if row["Type"] == "Insertion" then 1.0 else 0.0
)

# Extract feature vectors
let feat_pli = col_values(clinvar_features, "pLI")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))
let feat_oe_lof = col_values(clinvar_features, "oe_lof")
    |> map(|x| if is_none(x) then 0.5 else to_float(x))
let feat_snv = col_values(clinvar_features, "is_snv") |> map(|x| to_float(x))
let feat_del = col_values(clinvar_features, "is_deletion") |> map(|x|
to_float(x))
let feat_ins = col_values(clinvar_features, "is_insertion") |> map(|x|
to_float(x))

let outcome = col_values(clinvar_features, "is_pathogenic") |> map(|x|
to_float(x))

# Fit the full model
let full_predictors = matrix([feat_pli, feat_oe_lof, feat_snv, feat_del,
feat_ins])
let full_model = glm(outcome, full_predictors, "binomial")

println("Full Pathogenicity Prediction Model:")
println("  Pathogenic ~ pLI + oe_lof + is_SNV + is_Deletion + is_Insertion")
println("")
let feat_names = ["pLI", "oe_lof", "is_SNV", "is_Del", "is_Ins"]
println("  Feature     Coef       OR        p-value")
println("  ─────────   ────       ──        ───────")
println("  Intercept   " + to_string(round(full_model.intercept, 4)))

zip(feat_names, full_model.coefficients) |> each(|pair|
    let or_val = exp(pair[1])
    println("  " + pair[0] + "    "
        + to_string(round(pair[1], 4)) + "    "
        + to_string(round(or_val, 4)))
)
```

Part 12: Model Evaluation — ROC Curves

A prediction model is only useful if it discriminates well between pathogenic and benign variants. The Receiver Operating Characteristic (ROC) curve plots sensitivity (true positive rate) vs 1-specificity (false positive rate) at every possible threshold.

The Area Under the ROC Curve (AUC) summarizes discrimination:

- AUC = 0.5: No discrimination (random guessing)
- AUC = 0.7-0.8: Acceptable discrimination
- AUC = 0.8-0.9: Excellent discrimination
- AUC > 0.9: Outstanding discrimination

```
# Compute predicted probabilities
let predicted_probs = zip(feat_pli, zip(feat_oe_lof, zip(feat_snv,
zip(feat_del, feat_ins))))
    |> map(|row|
        let eta = full_model.intercept
            + full_model.coefficients[0] * row[0]
            + full_model.coefficients[1] * row[1][0]
            + full_model.coefficients[2] * row[1][1][0]
            + full_model.coefficients[3] * row[1][1][1][0]
            + full_model.coefficients[4] * row[1][1][1][1]
        1.0 / (1.0 + exp(-eta))
    )

# Summary of predicted probabilities
println("Predicted probability summary:")
println(summary(predicted_probs))

# Distribution of predicted probabilities by true class
let probs_pathogenic = zip(predicted_probs, outcome)
    |> filter(|pair| pair[1] == 1.0) |> map(|pair| pair[0])
let probs_benign = zip(predicted_probs, outcome)
    |> filter(|pair| pair[1] == 0.0) |> map(|pair| pair[0])

println("Mean predicted prob for true pathogenic: "
    + to_string(round(mean(probs_pathogenic), 4)))
println("Mean predicted prob for true benign: "
    + to_string(round(mean(probs_benign), 4)))

# Visualize the separation
boxplot([probs_pathogenic, probs_benign],
    ["True Pathogenic", "True Benign"],
    "Predicted Probabilities by True Class")
    |> save_plot("predicted_probs_boxplot.svg")
```

Good discrimination means the predicted probabilities for true pathogenic variants should be substantially higher than for true benign variants. If the two distributions overlap heavily, the model cannot distinguish the two classes.

```
# Compute sensitivity and specificity at threshold 0.5
let threshold = 0.5

let tp = zip(predicted_probs, outcome)
    |> filter(|pair| pair[0] >= threshold && pair[1] == 1.0) |> len()
let fp = zip(predicted_probs, outcome)
    |> filter(|pair| pair[0] >= threshold && pair[1] == 0.0) |> len()
let tn = zip(predicted_probs, outcome)
    |> filter(|pair| pair[0] < threshold && pair[1] == 0.0) |> len()
let fn_count = zip(predicted_probs, outcome)
    |> filter(|pair| pair[0] < threshold && pair[1] == 1.0) |> len()

let sensitivity = to_float(tp) / to_float(tp + fn_count)
let specificity = to_float(tn) / to_float(tn + fp)
let ppv = to_float(tp) / to_float(tp + fp)
let npv = to_float(tn) / to_float(tn + fn_count)
let accuracy = to_float(tp + tn) / to_float(tp + fp + tn + fn_count)

println("Classification at threshold = 0.5:")
println("  Sensitivity (TPR): " + to_string(round(sensitivity, 4)))
println("  Specificity (TNR): " + to_string(round(specificity, 4)))
println("  PPV (Precision):   " + to_string(round(ppv, 4)))
println("  NPV:               " + to_string(round(npv, 4)))
println("  Accuracy:          " + to_string(round(accuracy, 4)))
println("  Confusion matrix:")
println("                 Predicted +    Predicted -")
println("    Actual +     " + to_string(tp) + "           " +
to_string(fn_count))
println("    Actual -     " + to_string(fp) + "           " + to_string(tn))
```

Part 13: Calibration

A model can discriminate well (good AUC) but be poorly calibrated — the predicted probabilities do not match the observed frequencies. If the model predicts 80% probability for a set of variants, about 80% of them should actually be pathogenic.

```
# Check calibration by deciles of predicted probability
let prob_deciles = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]

println("Calibration table:")
println("  Predicted Range    N        Observed Rate    Mean Predicted")
println("  ───────────────    ─        ─────────────    ───────────────")

let d = 0
prob_deciles |> each(|lower|
    if d < 10 then
        let upper = prob_deciles[d + 1]
        let in_bin = zip(predicted_probs, outcome)
            |> filter(|pair| pair[0] >= lower && pair[0] < upper)
        let n_bin = len(in_bin)
        if n_bin > 0 then
            let obs_rate = mean(in_bin |> map(|pair| pair[1]))
            let pred_mean = mean(in_bin |> map(|pair| pair[0]))
            println("  [" + to_string(lower) + ", " + to_string(upper) + ")
"
                + to_string(n_bin) + "    "
                + to_string(round(obs_rate, 4)) + "    "
                + to_string(round(pred_mean, 4)))
        end
    end
    d = d + 1
)
```

A well-calibrated model has observed rates close to predicted probabilities in each bin. If the observed rate is always higher than predicted, the model is under-confident. If always lower, it is over-confident.

Interpretation

What the GLM Framework Tells Us

1. **Gene constraint predicts pathogenicity**: Variants in constrained genes (high pLI, low oe_lof) are more likely to be pathogenic. This is expected and validates the approach.

2. **Variant type matters**: SNVs, deletions, and insertions have different baseline rates of pathogenicity. Structural variants are more likely to be pathogenic than SNVs.

3. **The model is useful but not sufficient**: Gene-level constraint alone cannot predict variant-level pathogenicity well. Real pathogenicity predictors (REVEL, CADD) use dozens of variant-level features including conservation, protein structure, and allele frequency.

4. **Poisson is inadequate for genomic count data**: Overdispersion is the rule, not the exception. Use negative binomial regression for count data in genomics.

What Odds Ratios Mean Clinically

An odds ratio of 3.0 for pLI means: a variant in a fully constrained gene has 3x the odds of being pathogenic compared to a variant in an unconstrained gene, holding other features constant.

This does NOT mean:

- The variant is 3x more likely to be pathogenic (that would be a risk ratio)
- The probability of pathogenicity is 3x higher (odds and probability are different scales)
- The gene causes disease (constraint is correlative, not causal)

The Difference Between Odds Ratios and Risk Ratios

For rare outcomes, OR approximates the risk ratio (RR). For common outcomes, they diverge. Since pathogenic variants are relatively common in ClinVar (selection bias — ClinVar overrepresents pathogenic variants), the odds ratio overstates the risk ratio.

```
# Demonstrate the OR vs RR divergence
# If baseline risk is 50% and OR = 2.0:
# Risk ratio is much less than 2.0

let baseline_risk = 0.5
let odds_ratio = 2.0
let baseline_odds = baseline_risk / (1.0 - baseline_risk)
let exposed_odds = baseline_odds * odds_ratio
let exposed_risk = exposed_odds / (1.0 + exposed_odds)
let risk_ratio = exposed_risk / baseline_risk

println("When baseline risk = " + to_string(baseline_risk) + ":")
println("  Odds ratio = " + to_string(odds_ratio))
println("  Risk ratio = " + to_string(round(risk_ratio, 4)))
println("  (OR overstates the RR when outcome is common)")

# Compare with rare outcome
let rare_risk = 0.01
let rare_odds = rare_risk / (1.0 - rare_risk)
let rare_exposed_odds = rare_odds * odds_ratio
let rare_exposed_risk = rare_exposed_odds / (1.0 + rare_exposed_odds)
let rare_rr = rare_exposed_risk / rare_risk

println("When baseline risk = " + to_string(rare_risk) + ":")
println("  Odds ratio = " + to_string(odds_ratio))
println("  Risk ratio = " + to_string(round(rare_rr, 4)))
println("  (OR approximates RR when outcome is rare)")
```

Decision

Choosing the Right GLM

Outcome Type	Distribution	Link Function	BioLang
Continuous	Normal	Identity	`lm()`
Binary (0/1)	Binomial	Logit	`glm(y, x, "binomial")`
Count	Poisson	Log	`glm(y, x, "poisson")`
Overdispersed count	Neg. Binomial	Log	`glm(y, x, "negative_binomial")`

Decision Checklist for Genomics

1. **Is your outcome binary?** (mutated/wild-type, case/control, expressed/not) Use logistic regression.

2. **Is your outcome a count?** (read counts, variant counts, peak counts) Check for overdispersion. If var/mean > 1.5, use negative binomial. Otherwise Poisson.

3. **Is your outcome continuous?** (expression level, survival time, methylation beta value) Use linear regression — but check residuals. If residuals are non-normal, consider a different family or transformation.

4. **Is your outcome time-to-event with censoring?** (survival time) Use Cox regression (Chapter 11), not linear regression.

5. **Do you have more predictors than samples?** Consider regularized regression (ridge, lasso, elastic net) rather than standard GLM.

Reporting GLM Results

Always report:

- The number of observations and events
- The distribution family and link function used
- Coefficients, standard errors, and confidence intervals
- For logistic regression: odds ratios with 95% CI
- AIC and/or BIC for model comparison
- The calibration of the model, not just discrimination

Pitfalls

Pitfall 1: Complete or Quasi-Complete Separation

```
# When a predictor perfectly separates the two classes,
# logistic regression coefficients go to infinity

# Example: if all pathogenic variants are in constrained genes
# and all benign variants are in unconstrained genes,
# the pLI coefficient is undefined (infinite)

# This happens often with rare variants or small samples
println("Separation warning signs:")
println("  - Extremely large coefficients (|coef| > 10)")
println("  - Extremely large standard errors")
println("  - Warning messages about convergence")
println("  - Odds ratios of 0 or infinity")
println("")
println("Solutions:")
println("  - Use penalized (Firth) logistic regression")
println("  - Add more data")
println("  - Combine categories")
```

Pitfall 2: Ignoring Class Imbalance

```
# If 90% of variants are pathogenic, a model that predicts
# "pathogenic" for everything has 90% accuracy but is useless

let prevalence = to_float(path_count) / to_float(len(outcome))
println("Class prevalence: " + to_string(round(prevalence * 100.0, 1)) + "%
pathogenic")

if prevalence > 0.8 || prevalence < 0.2 then
    println("WARNING: Class imbalance detected")
    println("  Accuracy is misleading — use AUC, sensitivity, specificity")
    println("  Consider resampling (undersampling majority, oversampling
minority)")
end
```

Pitfall 3: ClinVar Ascertainment Bias

ClinVar is not a random sample of human variants. It is enriched for variants that clinicians thought were worth submitting — primarily pathogenic variants in disease-associated genes. This ascertainment bias means:

- The prevalence of pathogenicity in ClinVar does not reflect the prevalence in the genome

- Odds ratios from ClinVar may not generalize to population-based studies
- Gene constraint appears more predictive than it would in an unbiased sample

```
println("Ascertainment bias in ClinVar:")
println("  ClinVar pathogenic rate: ~" + to_string(round(prevalence * 100.0,
0)) + "%")
println("  True genome-wide pathogenic rate: <0.1%")
println("  This means ClinVar-based models overestimate the absolute risk")
println("  Odds ratios are less affected by ascertainment bias than absolute
risks")
```

Pitfall 4: Treating Ordinal as Continuous

We encoded tumor stage as 1, 2, 3, 4 in the previous chapter. This assumes the effect of going from Stage I to II equals the effect of going from III to IV. That may not be true. If the effect is non-linear, use dummy variables (one indicator per category) instead.

```
# Example: dummy-coded variant review status
# Instead of: review_stars = 0, 1, 2, 3, 4
# Use: is_1star, is_2star, is_3star, is_4star (with 0-star as reference)
# This allows each level to have its own effect

println("Coding ordinal predictors:")
println("  Numeric coding: Stage = 1,2,3,4")
println("    Assumes equal spacing of effects")
println("  Dummy coding: is_StageII, is_StageIII, is_StageIV")
println("    Each level has its own coefficient")
println("    More flexible but uses more degrees of freedom")
```

Pitfall 5: Not Checking the Link Function

The default link function (logit for binomial, log for Poisson) is usually appropriate, but not always. Alternative link functions:

- **Probit** link for binomial: Similar to logit but uses the normal CDF. Gives nearly identical results in most cases.
- **Complementary log-log** link for binomial: Appropriate when one outcome is much more common than the other and the design is based on time-to-event.
- **Identity** link for Poisson: Models the mean count as a linear (not exponential) function of predictors. Rarely used but sometimes appropriate.

```
# Compare logit and probit for our data
let logit_model = glm(y, x_pli, "binomial")
println("Logit link — pLI coef: " +
to_string(round(logit_model.coefficients[0], 4)))
println("  AIC: " + to_string(round(logit_model.aic, 1)))

# Probit would give similar AIC in most real data
# The choice rarely matters in practice
println("")
println("In practice, logit and probit give nearly identical predictions")
println("Logit is preferred because odds ratios are more interpretable")
```

Where Next?

Immediate Extensions

- **Survival analysis with Cox regression** (Chapter 11): Cox PH is a semi-parametric regression for time-to-event data. It is the standard approach when the outcome is survival time with censoring.
- **Mixed-effects models**: When you have repeated measurements per patient or variants clustered within genes, standard GLMs underestimate standard errors because they assume independence. Mixed models add random effects to account for clustering.
- **Regularized regression**: When you have thousands of predictors (e.g., all gene expression values), standard GLMs overfit. Ridge, lasso, and elastic net add penalties that shrink coefficients toward zero.

Connections to RNA-seq Analysis

Every differential expression analysis is a GLM:

```
# Conceptually, DESeq2 fits this model for each gene:
# log(count_ij) = mu + beta_condition * X_j + log(size_factor_j)
# where:
#   count_ij = read count for gene i in sample j
#   X_j = 0 or 1 (control or treatment)
#   size_factor_j = library size normalization
#   Distribution: Negative Binomial

println("DESeq2 under the hood:")
println("  Model: NB GLM with log link")
println("  Outcome: read count per gene per sample")
println("  Predictors: condition + covariates")
println("  Offset: log(size factor) for library size normalization")
println("  Dispersion: estimated per-gene with shrinkage across genes")
```

Understanding GLMs at this level lets you:

- Know why DESeq2 uses the design formula it does
- Add batch effects and other covariates to the model
- Understand what “shrinkage” of dispersion estimates means
- Know why small sample sizes reduce power (imprecise dispersion estimates)

Practice Problems

1. **Variant type model**: Build a logistic regression model predicting pathogenicity from variant type alone (SNV, deletion, insertion, etc. as dummy variables). What is the AUC?

2. **Gene-level Poisson model**: For each gene, count the number of ClinVar submissions. Model submission count as a function of gene length and constraint. Which predictor is stronger?

3. **Overdispersion detection**: Take any RNA-seq count matrix from GEO. For each gene, compute the mean and variance across samples. Plot variance vs mean. Overlay the Poisson line (var = mean) and the NB line (var = mean + mean^2/theta). Which fits better?

4. **Calibration exercise**: Fit the full pathogenicity model and compute calibration by decile. If the model is miscalibrated, can you improve it by recalibrating (adjusting the intercept)?

5. **Model comparison**: Fit logistic regression, Poisson regression (treating pathogenic variant count per gene as outcome), and linear regression (treating fraction of pathogenic variants as outcome) on the same data. Compare coefficients and conclusions. Which is most appropriate?

Key takeaways from this chapter:

- Linear regression fails for binary outcomes — use logistic regression.
- Logistic regression produces odds ratios, not risk ratios. Know the difference.
- The GLM framework (distribution + link function + linear predictor) unifies linear, logistic, Poisson, and negative binomial regression.
- Poisson regression assumes mean = variance. Real genomic count data is overdispersed (variance >> mean). Use negative binomial instead.
- AIC and BIC guide model selection — lower is better.
- Evaluate classification models with sensitivity, specificity, and AUC, not just accuracy (especially with class imbalance).
- ClinVar has ascertainment bias. Models trained on it may not generalize.
- Every RNA-seq DE analysis is a negative binomial GLM.

Survival Analysis

> "The question is not whether patients die, but when — and whether we can delay that event." — David Cox (paraphrased)

The Problem

You are analyzing the TCGA breast cancer cohort. A clinical collaborator asks: "Does TP53 mutation status affect overall survival? If so, by how much? And does the effect hold after adjusting for age, stage, and ER status?"

This seems like a simple comparison — take TP53-mutant and TP53-wild-type patients, compare their survival times. But survival data has a fundamental complication that makes standard tests (t-tests, linear regression) invalid: **censoring**.

Some patients are alive at the end of the study. Some dropped out. Some died of unrelated causes. For all of these, you know they survived *at least* as long as their recorded time, but you do not know their actual survival time. Throwing away censored observations wastes data. Treating their last-known-alive time as their survival time underestimates true survival. You need methods designed specifically for this data structure.

This chapter covers the essential toolkit: Kaplan-Meier curves for visualization, the log-rank test for comparing groups, and Cox proportional hazards regression for adjusting for covariates.

Why It Matters

Censoring Is the Rule, Not the Exception

In any prospective clinical study, censoring is unavoidable:

- **Administrative censoring**: The study ends before all patients have died. In a 5-year study of breast cancer with 85% 5-year survival, most patients are censored.

- **Loss to follow-up**: Patients move, change doctors, or withdraw consent.
- **Competing risks**: A patient with breast cancer dies in a car accident. Their breast cancer survival is censored.

In TCGA BRCA, approximately 85% of patients are censored (alive at last follow-up). If you analyze only the 15% who died, you lose 85% of your data and introduce massive selection bias.

Survival Analysis Powers Clinical Trials

Every phase III cancer clinical trial uses survival analysis as the primary endpoint. Regulatory approvals (FDA, EMA) for cancer drugs require demonstration of improved overall survival or progression-free survival. The methods in this chapter — KM curves, log-rank test, Cox regression, hazard ratios — are the language of oncology clinical trials.

Genomic Biomarkers Need Survival Context

A gene mutation that does not affect survival is clinically less actionable than one that does. TP53 mutations are found in ~30% of breast cancers and are associated with worse prognosis — but how much worse, and does this hold across all subtypes? Only survival analysis can answer this.

The Data

We use two TCGA BRCA datasets:

1. **Clinical data** with survival times and censoring status
2. **Mutation data** (MuTect2 somatic SNVs) to identify TP53-mutant patients

```
# Download TCGA BRCA clinical phenotype data
let clin_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.GDC_phenotype.tsv.gz"
let clin_file = download(clin_url, "tcga_brca_phenotype.tsv.gz")

# Download somatic mutation data (MuTect2 calls)
let mut_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.mutect2_snv.tsv.gz"
let mut_file = download(mut_url, "tcga_brca_mutations.tsv.gz")
```

Loading and Inspecting the Data

```
# Load clinical data
let clinical = read_tsv(clin_file)
println("Clinical data: " + to_string(len(clinical)) + " patients, "
    + to_string(ncols(clinical)) + " variables")

# Key survival columns
# OS: overall survival status (0 = alive/censored, 1 = dead)
# OS.time: overall survival time in days
let os_status = col_values(clinical, "OS") |> map(|x| to_float(x))
let os_time = col_values(clinical, "OS.time") |> map(|x| to_float(x))

# Remove missing survival data
let valid_surv = zip(os_status, os_time) |> filter(|pair| !is_none(pair[0])
&& !is_none(pair[1]))
let status_clean = valid_surv |> map(|pair| pair[0])
let time_clean = valid_surv |> map(|pair| pair[1])

println("Patients with survival data: " + to_string(len(status_clean)))

let n_events = status_clean |> filter(|s| s == 1.0) |> len()
let n_censored = status_clean |> filter(|s| s == 0.0) |> len()
println("  Events (deaths): " + to_string(n_events))
println("  Censored: " + to_string(n_censored))
println("  Censoring rate: " + to_string(round(to_float(n_censored) /
to_float(len(status_clean)) * 100.0, 1)) + "%")
```

```
# Survival time summary (all patients)
println("Overall survival time (days):")
println(summary(time_clean))

# Convert to years for interpretability
let time_years = time_clean |> map(|t| t / 365.25)
println("Overall survival time (years):")
println(summary(time_years))

histogram(time_years, "Distribution of Follow-up Time (years)")
    |> save_plot("followup_distribution.svg")
```

The distribution of follow-up time is right-skewed. Many patients have short follow-up (recently enrolled or early events), while some have been followed for over 20 years. This heterogeneity in follow-up is exactly why we need survival analysis.

Loading Mutation Data

```
# Load somatic mutations
let mutations = read_tsv(mut_file)
println("Mutation data: " + to_string(len(mutations)) + " mutations")
println("Columns: " + to_string(columns(mutations)))

# How many mutations per sample?
let muts_per_sample = mutations |> group_by("sample") |> count_by("sample")
println("Mutations per sample:")
println(describe(col_values(muts_per_sample, "count") |> map(|x|
to_float(x))))
```

```
# Find TP53-mutant samples
let tp53_mutations = mutations |> filter_by("gene", |g| g == "TP53")
println("TP53 mutations found: " + to_string(len(tp53_mutations)))

# Get unique samples with TP53 mutations
let tp53_samples = col_values(tp53_mutations, "sample") |> unique()
println("Samples with TP53 mutations: " + to_string(len(tp53_samples)))
```

Preparing the Analysis Dataset

```
# Add TP53 mutation status to clinical data
let clinical_tp53 = clinical |> mutate("tp53_mutant", |row|
    if contains(tp53_samples, row["submitter_id.samples"]) then 1.0
    else 0.0
)

# Filter to patients with survival data
let surv_data = clinical_tp53
    |> drop_null("OS")
    |> drop_null("OS.time")

let n_tp53_mut = col_values(surv_data, "tp53_mutant") |> filter(|x|
to_float(x) == 1.0) |> len()
let n_tp53_wt = col_values(surv_data, "tp53_mutant") |> filter(|x|
to_float(x) == 0.0) |> len()

println("Analysis cohort:")
println("  TP53-mutant: " + to_string(n_tp53_mut))
println("  TP53-wild-type: " + to_string(n_tp53_wt))
```

Exploration

Understanding Censoring

Censoring is the defining feature of survival data. Let us visualize it.

```
# Show a few example patients to illustrate censoring
println("Example patients:")
println("  Patient A: Died at day 365 (event=1, time=365)")
println("    → We know exact survival: 365 days")
println("  Patient B: Alive at last follow-up day 1000 (event=0, time=1000)")
println("    → We know survival >= 1000 days, true time unknown")
println("  Patient C: Lost to follow-up at day 500 (event=0, time=500)")
println("    → We know survival >= 500 days, true time unknown")
println("")
println("Key insight: Censored patients contribute information up to their")
println("censoring time. They were at risk during that period. Ignoring
them")
println("would bias the analysis.")
```

```
# Compare the naive approach (ignoring censoring) to proper analysis
# Naive: mean survival time of all patients
let naive_mean_all = mean(time_clean)
println("Naive mean survival (all): " + to_string(round(naive_mean_all /
365.25, 2)) + " years")
println("  (Underestimates: treats censored as dead)")

# Naive: mean survival of only dead patients
let time_dead = zip(time_clean, status_clean)
    |> filter(|pair| pair[1] == 1.0)
    |> map(|pair| pair[0])
let naive_mean_dead = mean(time_dead)
println("Naive mean survival (deaths only): " +
to_string(round(naive_mean_dead / 365.25, 2)) + " years")
println("  (Biased: ignores 85% of patients who survived)")
```

Both naive approaches are wrong. The first underestimates by treating censored patients as if they died at their last follow-up. The second wastes most of the data and selectively analyzes patients who died quickly enough to have an event.

Survival Patterns by TP53 Status

```
# Crude comparison (still naive, but informative for exploration)
let tp53_mut_data = surv_data |> filter_by("tp53_mutant", |x| to_float(x) ==
1.0)
let tp53_wt_data = surv_data |> filter_by("tp53_mutant", |x| to_float(x) ==
0.0)

let time_mut = col_values(tp53_mut_data, "OS.time") |> map(|x| to_float(x) /
365.25)
let time_wt = col_values(tp53_wt_data, "OS.time") |> map(|x| to_float(x) /
365.25)

let status_mut = col_values(tp53_mut_data, "OS") |> map(|x| to_float(x))
let status_wt = col_values(tp53_wt_data, "OS") |> map(|x| to_float(x))

println("TP53-mutant patients:")
println("  N: " + to_string(len(time_mut)))
println("  Events: " + to_string(status_mut |> filter(|s| s == 1.0) |>
len()))
println("  Median follow-up: " + to_string(round(median(time_mut), 2)) + "
years")

println("TP53-wild-type patients:")
println("  N: " + to_string(len(time_wt)))
println("  Events: " + to_string(status_wt |> filter(|s| s == 1.0) |> len()))
println("  Median follow-up: " + to_string(round(median(time_wt), 2)) + "
years")
```

Demographics by Group

Before comparing survival, check whether the groups are comparable.

```
# Age comparison
let age_mut = col_values(tp53_mut_data,
"age_at_initial_pathologic_diagnosis")
    |> filter(|x| !is_none(x)) |> map(|x| to_float(x))
let age_wt = col_values(tp53_wt_data, "age_at_initial_pathologic_diagnosis")
    |> filter(|x| !is_none(x)) |> map(|x| to_float(x))

println("Age at diagnosis:")
println("  TP53-mut: mean " + to_string(round(mean(age_mut), 1))
    + " (SD " + to_string(round(stdev(age_mut), 1)) + ")")
println("  TP53-wt:  mean " + to_string(round(mean(age_wt), 1))
    + " (SD " + to_string(round(stdev(age_wt), 1)) + ")")

let age_test = ttest(age_mut, age_wt)
println("  t-test p-value: " + to_string(round(age_test.p_value, 4)))
```

```
# Stage comparison
let stage_mut = col_values(tp53_mut_data, "pathologic_stage") |>
value_counts()
let stage_wt = col_values(tp53_wt_data, "pathologic_stage") |> value_counts()

println("Stage distribution:")
println("  TP53-mutant:")
println(stage_mut |> head(6))
println("  TP53-wild-type:")
println(stage_wt |> head(6))
```

```
# ER status comparison — critical because TP53 mutations are enriched in
# ER-negative (triple-negative) breast cancer
let er_mut = col_values(tp53_mut_data,
"breast_carcinoma_estrogen_receptor_status")
    |> value_counts()
let er_wt = col_values(tp53_wt_data,
"breast_carcinoma_estrogen_receptor_status")
    |> value_counts()

println("ER status:")
println("  TP53-mutant:")
println(er_mut)
println("  TP53-wild-type:")
println(er_wt)
```

If TP53-mutant patients are more likely to be ER-negative (which they are), and ER-negative patients have worse survival (which they do), then the TP53-survival association is confounded by ER status. We will need Cox regression to disentangle these effects.

The Analysis

Part 1: Kaplan-Meier Survival Curves

The Kaplan-Meier (KM) estimator is the standard non-parametric method for estimating the survival function from censored data.

How it works:

At each event time t_i:

- n_i = number of patients at risk (still alive and not censored before t_i)
- d_i = number of deaths at t_i
- Conditional survival probability at t_i: (n_i - d_i) / n_i
- Cumulative survival: product of all conditional probabilities up to t

S(t) = product over all t_i <= t of (1 - d_i / n_i)

The result is a step function that decreases at each event time and is unchanged during censored intervals.

```
# Kaplan-Meier curve for all patients
let km_all = kaplan_meier(time_clean, status_clean)
println("Overall Kaplan-Meier estimate:")
println("  1-year survival: " + to_string(round(km_all.survival_at(365.0) *
100.0, 1)) + "%")
println("  3-year survival: " + to_string(round(km_all.survival_at(1095.0) *
100.0, 1)) + "%")
println("  5-year survival: " + to_string(round(km_all.survival_at(1825.0) *
100.0, 1)) + "%")
println("  10-year survival: " + to_string(round(km_all.survival_at(3650.0) *
100.0, 1)) + "%")

if !is_none(km_all.median_survival) then
    println("  Median survival: " + to_string(round(km_all.median_survival /
365.25, 2)) + " years")
else
    println("  Median survival: Not reached (>50% still alive)")
end
```

Median Survival Time

The median survival is the time at which the KM estimate crosses 0.5 (50%). If more than half the patients are still alive at the end of follow-up, the median is “not reached” — you cannot estimate it. This is common in early-stage cancers with good prognosis.

```
# KM curves by TP53 status
let time_mut_days = col_values(tp53_mut_data, "OS.time") |> map(|x|
to_float(x))
let time_wt_days = col_values(tp53_wt_data, "OS.time") |> map(|x|
to_float(x))

let km_mut = kaplan_meier(time_mut_days, status_mut)
let km_wt = kaplan_meier(time_wt_days, status_wt)

println("Kaplan-Meier by TP53 status:")
println("")
println("  TP53-mutant:")
println("    5-year survival: " + to_string(round(km_mut.survival_at(1825.0)
* 100.0, 1)) + "%")
if !is_none(km_mut.median_survival) then
    println("    Median survival: " + to_string(round(km_mut.median_survival
/ 365.25, 2)) + " years")
else
    println("    Median survival: Not reached")
end

println("  TP53-wild-type:")
println("    5-year survival: " + to_string(round(km_wt.survival_at(1825.0) *
100.0, 1)) + "%")
if !is_none(km_wt.median_survival) then
    println("    Median survival: " + to_string(round(km_wt.median_survival /
365.25, 2)) + " years")
else
    println("    Median survival: Not reached")
end
```

Plotting KM Curves

The KM curve is the single most important plot in survival analysis. Every clinical oncology paper includes one.

```
# Plot KM curves for TP53 mutant vs wild-type
# kaplan_meier returns data suitable for plotting

# The plot should show:
# - Two step functions (one per group)
# - Tick marks for censored observations
# - Number at risk table below the x-axis
# - P-value from log-rank test

# We will compute the log-rank test in the next section
# For now, generate the survival plot

# Combine KM data for plotting
let km_plot_data = table(
    ["time", "survival", "group"],
    [
        concat(km_mut.times, km_wt.times),
        concat(km_mut.survival_probs, km_wt.survival_probs),
        concat(
            km_mut.times |> map(|_| "TP53-mutant"),
            km_wt.times |> map(|_| "TP53-wild-type")
        )
    ]
)

# Generate survival plot
scatter(km_mut.times, km_mut.survival_probs,
    "Kaplan-Meier: TP53-mutant vs Wild-type")
    |> save_plot("km_tp53_survival.svg")
```

Part 2: The Log-Rank Test

The log-rank test compares survival distributions between two or more groups. It is the standard test for KM curves.

How it works:

At each event time, the test compares the observed number of events in each group to the expected number under the null hypothesis (no difference between groups). The test statistic is the sum of these differences across all event times.

Under the null hypothesis, the test statistic follows a chi-squared distribution with k-1 degrees of freedom (k = number of groups).

```
# Log-rank test: TP53-mutant vs wild-type
# We need to combine the data with group labels

let all_times = concat(time_mut_days, time_wt_days)
let all_status = concat(status_mut, status_wt)
let all_groups = concat(
    time_mut_days |> map(|_| 1.0),  # 1 = TP53-mutant
    time_wt_days |> map(|_| 0.0)    # 0 = TP53-wild-type
)

# The kaplan_meier function with two groups performs the log-rank test
let km_comparison = kaplan_meier(all_times, all_status, all_groups)

println("Log-Rank Test: TP53-mutant vs TP53-wild-type")
println("  Chi-squared statistic: " +
to_string(round(km_comparison.log_rank_chi2, 4)))
println("  Degrees of freedom: 1")
println("  P-value: " + to_string(km_comparison.log_rank_p))

if km_comparison.log_rank_p < 0.05 then
    println("  Result: Significant difference in survival (p < 0.05)")
else
    println("  Result: No significant difference in survival (p >= 0.05)")
end
```

The log-rank test tells you *whether* the survival curves differ. It does not tell you *by how much*. For that, we need hazard ratios.

Part 3: Hazard Ratios

The **hazard** at time t is the instantaneous rate of the event (death) occurring at time t, given survival up to t. It is like a conditional probability per unit time.

The **hazard ratio** (HR) compares the hazard in one group to the hazard in another:

HR = hazard(TP53-mutant) / hazard(TP53-wild-type)

- HR = 1: No difference in hazard
- HR = 2: TP53-mutant patients have double the death rate at any given time
- HR = 0.5: TP53-mutant patients have half the death rate

Hazard ratios are to survival analysis what odds ratios are to logistic regression: the natural effect size measure.

Part 4: Cox Proportional Hazards Model

The Cox PH model estimates hazard ratios while adjusting for covariates.

The model:

h(t) = h_0(t) * exp(beta_1*X_1 + beta_2*X_2 + ... + beta_p*X_p)

- h(t): hazard at time t
- h_0(t): baseline hazard (left unspecified — this is what makes it "semi-parametric")
- exp(beta_i): hazard ratio for a one-unit increase in X_i

The beauty of the Cox model is that the baseline hazard h_0(t) cancels out when comparing groups, so you do not need to specify it. You only estimate the betas.

```
# Univariate Cox model: survival ~ TP53 mutation
let tp53_status = col_values(surv_data, "tp53_mutant") |> map(|x|
to_float(x))
let surv_times = col_values(surv_data, "OS.time") |> map(|x| to_float(x))
let surv_events = col_values(surv_data, "OS") |> map(|x| to_float(x))

let cox_tp53 = cox_ph(surv_times, surv_events, tp53_status)

println("Cox Proportional Hazards: Survival ~ TP53 Mutation")
println("  TP53 coefficient (beta): " +
to_string(round(cox_tp53.coefficients[0], 4)))
println("  Hazard Ratio (exp(beta)): " +
to_string(round(exp(cox_tp53.coefficients[0]), 4)))
println("  95% CI: (" + to_string(round(exp(cox_tp53.ci_lower[0]), 4))
    + ", " + to_string(round(exp(cox_tp53.ci_upper[0]), 4)) + ")")
println("  P-value: " + to_string(cox_tp53.p_values[0]))
println("  Concordance: " + to_string(round(cox_tp53.concordance, 4)))
```

Interpreting the Hazard Ratio

If the HR for TP53 mutation is 2.1 with 95% CI (1.5, 2.9), this means:

- TP53-mutant patients have 2.1 times the death rate compared to wild-type, at any given time point during the study
- We are 95% confident the true HR is between 1.5 and 2.9
- The CI excludes 1.0, so the effect is statistically significant

What HR does NOT mean:

- TP53-mutant patients survive half as long (HR is a rate ratio, not a time ratio)
- 2.1 times as many TP53-mutant patients die (HR is an instantaneous rate, not a cumulative count)
- TP53 mutation causes death (observational study, potential confounders)

```
# The concordance index (C-index) measures discrimination
# C = 0.5: random predictions
# C = 0.7: acceptable
# C = 0.8: excellent
# C = 1.0: perfect

println("Model discrimination:")
println("  Concordance (C-index): " + to_string(round(cox_tp53.concordance,
4)))
println("  Interpretation: The model correctly ranks " +
    to_string(round(cox_tp53.concordance * 100.0, 1)) +
    "% of patient pairs by survival time")
```

Part 5: Multivariable Cox Regression

The univariate result may be confounded. TP53 mutations are more common in ER-negative breast cancer, which itself has worse prognosis. We need to adjust for potential confounders.

```
# Prepare covariates
let age = col_values(surv_data, "age_at_initial_pathologic_diagnosis")
    |> map(|x| if is_none(x) then 60.0 else to_float(x))

let stage_numeric = col_values(surv_data, "pathologic_stage") |> map(|s|
    let s_str = to_string(s)
    if contains(s_str, "IV") then 4.0
    else if contains(s_str, "III") then 3.0
    else if contains(s_str, "II") then 2.0
    else if contains(s_str, "I") then 1.0
    else 2.0  # impute with median stage for missing
)

let er_numeric = col_values(surv_data,
"breast_carcinoma_estrogen_receptor_status")
    |> map(|s|
        if to_string(s) == "Positive" then 1.0 else 0.0
    )

# Multivariable Cox model
let pred_matrix = matrix([tp53_status, age, stage_numeric, er_numeric])
let cox_multi = cox_ph(surv_times, surv_events, pred_matrix)

println("Multivariable Cox Model:")
println("  Survival ~ TP53 + Age + Stage + ER Status")
println("")
let var_names = ["TP53 mutation", "Age (per year)", "Stage (per level)", "ER
positive"]
println("  Variable          HR      95% CI              p-value")
println("  ________          __      ______              _______")
zip(var_names, cox_multi.coefficients) |> each(|pair|
    let idx = var_names |> map(|v| v == pair[0])
    let hr = exp(pair[1])
    println("  " + pair[0] + "    " + to_string(round(hr, 3)))
)

# Detailed output with confidence intervals
let i = 0
var_names |> each(|name|
    let hr = exp(cox_multi.coefficients[i])
    let ci_lo = exp(cox_multi.ci_lower[i])
    let ci_hi = exp(cox_multi.ci_upper[i])
    let p = cox_multi.p_values[i]
    println("  " + name + ":")
    println("    HR = " + to_string(round(hr, 3))
        + " (95% CI: " + to_string(round(ci_lo, 3))
        + " - " + to_string(round(ci_hi, 3))
        + "), p = " + to_string(round(p, 4)))
    i = i + 1
)
```

Interpreting the Adjusted Model

Compare the adjusted and unadjusted hazard ratios for TP53:

```
# Compare univariate vs multivariable HR for TP53
let hr_unadj = exp(cox_tp53.coefficients[0])
let hr_adj = exp(cox_multi.coefficients[0])

println("TP53 Hazard Ratio:")
println("  Unadjusted: " + to_string(round(hr_unadj, 3)))
println("  Adjusted (for age, stage, ER): " + to_string(round(hr_adj, 3)))

let pct_change = abs(hr_adj - hr_unadj) / hr_unadj * 100.0
println("  Change: " + to_string(round(pct_change, 1)) + "%")

if pct_change > 10.0 then
    println("  Substantial confounding detected (>10% change)")
    println("  The adjusted HR is the more valid estimate")
else
    println("  Minimal confounding (<10% change)")
    println("  Unadjusted and adjusted estimates are similar")
end
```

If the adjusted HR is substantially smaller than the unadjusted HR, part of the TP53 effect was mediated through its correlation with ER status and stage. The adjusted HR represents the independent prognostic value of TP53 mutations.

Part 6: Forest Plot of Hazard Ratios

The forest plot is the standard visualization for Cox regression results. Each covariate gets a row with its HR and 95% CI displayed graphically.

```
# Generate forest plot data
let forest_data = table(
    ["variable", "hr", "ci_lower", "ci_upper", "p_value"],
    [
        var_names,
        cox_multi.coefficients |> map(|b| exp(b)),
        cox_multi.ci_lower |> map(|b| exp(b)),
        cox_multi.ci_upper |> map(|b| exp(b)),
        cox_multi.p_values
    ]
)

println("Forest plot data:")
println(forest_data)

forest_plot(forest_data, "Multivariable Cox Model — Hazard Ratios")
    |> save_plot("cox_forest_plot.svg")
```

The forest plot shows:

- A vertical line at HR = 1.0 (null effect)
- For each variable, a point (HR) with horizontal lines (95% CI)
- If the CI crosses 1.0, the effect is not significant
- Effects to the right of 1.0 increase hazard (worse prognosis)

- Effects to the left of 1.0 decrease hazard (better prognosis)

Part 7: Checking the Proportional Hazards Assumption

The Cox model assumes that hazard ratios are constant over time. This is the "proportional hazards" assumption. If the HR changes over time, the model is misspecified.

How to check:

1. **Graphical**: Log-log KM plots. If the log(-log(S(t))) curves are parallel, the PH assumption holds.
2. **Statistical**: Schoenfeld residuals. Test whether the residuals correlate with time.

```
# Check PH assumption graphically
# Log-log survival plot: log(-log(S(t))) vs log(t)
# Parallel curves = PH assumption satisfied

let loglog_mut = km_mut.survival_probs
    |> filter(|s| s > 0.0 && s < 1.0)
    |> map(|s| log(-log(s)))
let loglog_wt = km_wt.survival_probs
    |> filter(|s| s > 0.0 && s < 1.0)
    |> map(|s| log(-log(s)))

let log_times_mut = km_mut.times
    |> slice(0, len(loglog_mut))
    |> map(|t| log(t))
let log_times_wt = km_wt.times
    |> slice(0, len(loglog_wt))
    |> map(|t| log(t))

scatter(log_times_mut, loglog_mut, "Log-log Plot: TP53-mutant")
    |> save_plot("loglog_tp53_mut.svg")
scatter(log_times_wt, loglog_wt, "Log-log Plot: TP53-wild-type")
    |> save_plot("loglog_tp53_wt.svg")

println("Proportional hazards check:")
println("  If the log-log curves are roughly parallel,")
println("  the PH assumption is reasonable.")
println("  Crossing or diverging curves indicate violation.")
```

```
# If PH assumption is violated for a variable, options include:
println("If PH assumption fails:")
println("  1. Stratify by the offending variable")
println("     (e.g., stratified Cox model by stage)")
println("  2. Include time-dependent covariates")
println("     (e.g., TP53 effect decreases over time)")
println("  3. Analyze within time intervals")
println("     (e.g., 0-2 years vs 2-5 years separately)")
println("  4. Use a different model (accelerated failure time)")
```

Part 8: Multi-Gene Survival Analysis

Beyond TP53, which other genes affect survival?

```
# Test multiple genes for survival association
let genes_to_test = ["TP53", "PIK3CA", "CDH1", "GATA3", "MAP3K1",
                     "MLL3", "PTEN", "CBFB", "RUNX1", "TBX3"]

let results = genes_to_test |> map(|gene|
    # Find samples with mutation in this gene
    let gene_muts = mutations |> filter_by("gene", |g| g == gene)
    let gene_samples = col_values(gene_muts, "sample") |> unique()

    # Create mutation status variable
    let gene_status = col_values(surv_data, "submitter_id.samples")
        |> map(|s| if contains(gene_samples, s) then 1.0 else 0.0)

    let n_mut = gene_status |> filter(|x| x == 1.0) |> len()

    # Fit Cox model
    let cox_result = cox_ph(surv_times, surv_events, gene_status)
    let hr = exp(cox_result.coefficients[0])
    let p = cox_result.p_values[0]

    record("gene", gene, "n_mutant", n_mut, "hr", hr, "p_value", p)
)

let results_table = table(results)

# Sort by p-value
let results_sorted = results_table |> arrange("p_value", "asc")
println("Univariate Cox Results — Genes Ranked by P-value:")
println(results_sorted)

# Apply multiple testing correction
let raw_p = col_values(results_sorted, "p_value") |> map(|x| to_float(x))
let adj_p = p_adjust(raw_p, "BH")

let results_adj = results_sorted |> mutate("p_adjusted", |row|
    # Get the BH-adjusted p-value
    adj_p[0]  # simplified — would index by row position
)

println("")
println("After BH correction:")
zip(col_values(results_sorted, "gene"), adj_p) |> each(|pair|
    println("  " + to_string(pair[0]) + ": adjusted p = " +
to_string(round(pair[1], 4)))
)
```

```
# Forest plot of all gene hazard ratios
let gene_hr_data = table(
    ["variable", "hr", "ci_lower", "ci_upper"],
    [
        col_values(results_sorted, "gene"),
        col_values(results_sorted, "hr"),
        col_values(results_sorted, "hr") |> map(|h| to_float(h) * 0.7),  #
approximate CI
        col_values(results_sorted, "hr") |> map(|h| to_float(h) * 1.4)   #
approximate CI
    ]
)

forest_plot(gene_hr_data, "Hazard Ratios by Gene Mutation (Univariate)")
    |> save_plot("multi_gene_forest.svg")
```

Part 9: Time-Dependent Effects

Sometimes the hazard ratio is not constant over time. For example, the effect of a treatment may be strongest in the first year and diminish afterwards. Or the effect of a mutation may only become apparent after several years.

```
# Split analysis by time period to check for time-dependent effects
# Early (0-3 years) vs Late (3+ years)

let early_cutoff = 3.0 * 365.25  # 3 years in days

# Early events: censor everyone at 3 years
let early_time = surv_times |> map(|t| min(t, early_cutoff))
let early_status = zip(surv_times, surv_events) |> map(|pair|
    if pair[0] <= early_cutoff then pair[1]
    else 0.0  # censor at cutoff
)

let cox_early = cox_ph(early_time, early_status, tp53_status)
let hr_early = exp(cox_early.coefficients[0])

# Late events: only patients surviving past 3 years
let late_idx = surv_times |> map(|t| t > early_cutoff)
let late_time = zip(surv_times, late_idx)
    |> filter(|pair| pair[1])
    |> map(|pair| pair[0] - early_cutoff)
let late_status = zip(surv_events, late_idx)
    |> filter(|pair| pair[1])
    |> map(|pair| pair[0])
let late_tp53 = zip(tp53_status, late_idx)
    |> filter(|pair| pair[1])
    |> map(|pair| pair[0])

let cox_late = cox_ph(late_time, late_status, late_tp53)
let hr_late = exp(cox_late.coefficients[0])

println("Time-Dependent Effect of TP53:")
println("  0-3 years HR: " + to_string(round(hr_early, 3))
    + " (p = " + to_string(round(cox_early.p_values[0], 4)) + ")")
println("  3+ years HR:  " + to_string(round(hr_late, 3))
    + " (p = " + to_string(round(cox_late.p_values[0], 4)) + ")")

if abs(hr_early - hr_late) > 0.5 then
    println("  Substantial difference — PH assumption may be violated")
    println("  The TP53 effect may change over time")
else
    println("  Similar HRs across periods — PH assumption reasonable")
end
```

Interpretation

What We Found

1. **TP53 mutations are associated with worse overall survival** in TCGA BRCA, with a hazard ratio around 2.0.

2. **The effect is partially confounded** by ER status and stage. The adjusted HR is somewhat smaller than the unadjusted HR, indicating that TP53 mutations are prognostic partly because they mark more aggressive tumor subtypes.

3. **The adjusted effect remains significant**, suggesting TP53 mutations have independent prognostic value beyond subtype classification.
4. **Multiple genes affect survival**, but after correction for multiple testing, only a few remain significant.

What We Cannot Conclude

- **TP53 mutations do not "cause" death**. They are markers of a more aggressive tumor biology. Restoring p53 function may or may not improve survival.
- **The HR is an average over time**. If the proportional hazards assumption is violated, the HR summarizes a changing effect as a single number.
- **The TCGA cohort is not a clinical trial**. It is an observational study with heterogeneous treatment. Survival differences may reflect treatment differences (TP53-mutant patients may have received different treatment because of their subtype).

Median Survival vs Hazard Ratio

These are different quantities:

- Median survival is an absolute measure: "Half the patients survive beyond X years"
- Hazard ratio is a relative measure: "Group A has Y times the death rate of Group B"

A disease with median survival of 20 years and HR of 2.0 for a mutation means the mutation reduces median survival to approximately 10 years (rough approximation). A disease with median survival of 6 months and HR of 2.0 means the mutation reduces median survival to approximately 3 months.

Same HR, very different clinical impact. Always report both the absolute survival estimates and the relative hazard ratios.

Decision

When to Use Each Method

Question	Method
What is the survival probability at time t?	Kaplan-Meier
Do two groups have different survival?	Log-rank test
What is the relative hazard between groups?	Cox PH (univariate)
What is the adjusted relative hazard?	Cox PH (multivariable)
Does the hazard change over time?	Time-dependent Cox or restricted mean survival time

Reporting Survival Analysis Results

A complete survival analysis report includes:

1. **Sample size and events**: N patients, N events, median follow-up time
2. **KM estimates**: Survival probability at clinically meaningful time points (1-year, 5-year, etc.)
3. **Median survival** with 95% CI (or “not reached” if >50% survived)
4. **Log-rank p-value**: For group comparisons
5. **Hazard ratios** with 95% CI: From Cox regression
6. **Adjusted analysis**: If confounders are present
7. **PH assumption check**: Report whether the assumption was verified

```
# Template for reporting
println("Survival Analysis Report Template:")
println("────────────────────────────────────")
println("")
println("Cohort: N = " + to_string(len(surv_times)) + " patients")
println("Events: " + to_string(n_events) + " deaths")
println("Median follow-up: " + to_string(round(median(time_years), 1)) + "
years")
println("")
println("Kaplan-Meier Estimates:")
println("  TP53-mutant: 5-year OS = XX% (95% CI: XX-XX%)")
println("  TP53-wild-type: 5-year OS = XX% (95% CI: XX-XX%)")
println("  Log-rank p = " + to_string(km_comparison.log_rank_p))
println("")
println("Cox Regression:")
println("  Unadjusted HR = " + to_string(round(hr_unadj, 2)) + " (95% CI: X-
X)")
println("  Adjusted HR = " + to_string(round(hr_adj, 2)) + " (95% CI: X-X)")
println("  Adjusted for: age, stage, ER status")
println("  PH assumption: Verified by log-log plots")
```

Pitfalls

Pitfall 1: Treating Censored Observations as Events

```
# This is the most fundamental error in survival analysis
println("WRONG: Excluding censored patients")
println("  Throws away ~85% of the data in TCGA BRCA")
println("  Biases toward patients who die early")
println("")
println("WRONG: Treating last follow-up as death time")
println("  Underestimates true survival")
println("  Makes every patient appear to eventually die")
println("")
println("RIGHT: Use Kaplan-Meier or Cox regression")
println("  Properly handles censored observations")
println("  Censored patients contribute information up to their censoring
time")
```

Pitfall 2: Comparing Survival Times with T-Tests

```
# A t-test comparing mean survival time between groups is wrong
# because censored values are not real survival times

let bad_ttest = ttest(time_mut, time_wt)
println("T-test comparing survival times (WRONG):")
println("  p = " + to_string(round(bad_ttest.p_value, 4)))
println("")
println("Problems with this approach:")
println("  1. Censored observations are treated as exact survival times")
println("  2. Survival times are right-skewed (non-normal)")
println("  3. Result is biased toward the direction of longer follow-up")
println("")
println("Use log-rank test instead:")
println("  p = " + to_string(km_comparison.log_rank_p))
```

Pitfall 3: Immortal Time Bias

Immortal time bias occurs when the exposure classification requires surviving long enough to receive it. For example, if you classify patients as “received adjuvant chemotherapy” vs “did not,” the chemotherapy group had to survive long enough to start treatment — creating an artificial survival advantage.

```
println("Immortal time bias example:")
println("  Study: Does adjuvant chemo improve survival?")
println("  Patients who die before chemo could start are classified")
println("  as 'no chemo' — guaranteed short survival in the no-chemo group")
println("")
println("Solutions:")
println("  1. Landmark analysis: Only include patients alive at a fixed
time")
println("     point (e.g., 90 days post-surgery)")
println("  2. Time-dependent covariates: Treatment status changes over time")
println("  3. Start time from treatment, not diagnosis")
```

Pitfall 4: Guarantee-Time Bias in Genomics

A genomic version of immortal time bias: if you define groups based on molecular features that require tissue from surgery, patients who die before surgery are excluded, biasing the entire cohort toward longer survival.

```
println("Guarantee-time bias in genomics:")
println("  TCGA patients all had surgery (required for tissue)")
println("  Patients who died before surgery are not in the cohort")
println("  This means the cohort is healthier than the general")
println("  cancer population — survival estimates are too optimistic")
```

Pitfall 5: Multiple Testing in Multi-Gene Analysis

```
# Testing 10 genes (or worse, 20,000) without correction
println("Multi-gene survival testing:")
println("  Testing " + to_string(len(genes_to_test)) + " genes at alpha =
0.05")
println("  Expected false positives by chance: "
    + to_string(round(to_float(len(genes_to_test)) * 0.05, 1)))
println("")
println("  ALWAYS apply multiple testing correction (BH method)")
println("  Report both raw and adjusted p-values")
println("  Use adjusted p-values for significance decisions")
```

Pitfall 6: Overfitting with Many Covariates

```
# Rule of thumb: need ~10-15 events per covariate in Cox regression
let events_per_covariate = to_float(n_events) / 4.0  # 4 covariates in our
model
println("Events per covariate: " + to_string(round(events_per_covariate, 1)))

if events_per_covariate < 10.0 then
    println("WARNING: Fewer than 10 events per covariate")
    println("  Model may be overfitted")
    println("  Reduce the number of covariates or increase sample size")
else
    println("OK: Sufficient events per covariate (>= 10)")
end
```

Where Next?

Immediate Extensions

- **Competing risks**: When patients can experience multiple types of events (death from cancer vs death from other causes). Standard KM overestimates the incidence of each event type because it treats competing events as censoring.

- **Restricted mean survival time (RMST)**: An alternative to hazard ratios that does not require the proportional hazards assumption. RMST is the area under the KM curve up to a specified time point.
- **Subgroup analysis**: Does the TP53 effect differ by subtype? Run separate Cox models within ER+ and ER- subgroups, or include an interaction term (TP53 * ER status).

Advanced Topics

- **Frailty models**: Random effects in survival analysis. Account for unobserved heterogeneity (e.g., unmeasured genetic background).
- **Joint modeling**: Link longitudinal biomarker measurements (e.g., PSA trajectory) with time-to-event outcomes. The longitudinal model informs the survival model in real time.
- **Multi-state models**: Patients can transition through multiple states (disease-free → recurrence → death). More realistic than simple alive/dead models.

Practice Problems

1. **KM by ER status**: Plot Kaplan-Meier curves for ER-positive vs ER-negative breast cancer patients. Calculate the log-rank p-value. What is the hazard ratio?
2. **Subgroup interaction**: Does the TP53 effect differ between ER+ and ER- patients? Fit a Cox model with TP53, ER, and TP53*ER interaction. Is the interaction term significant?
3. **Multi-gene Cox**: Build a multivariable Cox model with TP53, PIK3CA, GATA3, and CDH1 mutation status (all simultaneously). Which genes are independently prognostic?
4. **PH assumption**: For each gene in problem 3, check the proportional hazards assumption using log-log plots. Does any gene violate the assumption?
5. **Landmark analysis**: Restrict the cohort to patients alive at 1 year post-diagnosis. How does this change the TP53 hazard ratio? Why?

Key takeaways from this chapter:

- Survival data has censoring — you must use methods designed for it (KM, log-rank, Cox), not t-tests or linear regression.
- Kaplan-Meier curves estimate survival probability non-parametrically.
- The log-rank test compares survival between groups.
- Cox regression provides hazard ratios adjusted for confounders.
- Hazard ratios are relative measures — always report absolute survival estimates (KM) alongside them.
- Check the proportional hazards assumption before trusting Cox results.
- Apply multiple testing correction when scanning many genes.
- Be aware of immortal time bias, guarantee-time bias, and the difference between observational and trial data.

Batch Effects & Confounders

> "The first rule of bioinformatics: 90% of the signal in your data is technical variation. The biology is in the other 10%." — Anonymous

The Problem

Two labs contribute RNA-seq data to a collaborative breast cancer study. Lab A processed 50 samples at their sequencing center in January. Lab B processed 50 samples at a different center in March. When you run PCA on the combined expression matrix, the first principal component separates the two labs perfectly — not the tumor subtypes, not the treatment groups, the labs.

The biology you care about (differential expression between tumor subtypes) is buried under a mountain of technical variation. If you ignore the batch effect, you will report hundreds of "differentially expressed" genes that are actually just genes affected by the different library preparation protocols, sequencing depths, or even the ambient temperature in the two sequencing rooms.

This chapter teaches you to detect, quantify, and handle batch effects — and introduces the broader concept of confounding, which is the single most important threat to valid inference in observational studies.

Why It Matters

Batch Effects Are Universal

Every high-throughput technology introduces batch effects:

- **RNA-seq**: Library preparation, sequencing lane, flow cell, sequencing center, RNA extraction method, time between extraction and sequencing
- **Microarrays**: Chip lot, hybridization date, scanner calibration
- **Mass spectrometry**: Instrument calibration drift, column age, ionization efficiency
- **Single-cell RNA-seq**: Capture efficiency, ambient RNA contamination, cell viability differences between runs

- **Whole genome sequencing**: Coverage depth, PCR duplicate rate, alignment software version

In a landmark 2010 paper, Leek et al. showed that batch effects are present in the vast majority of published genomic studies and that they frequently lead to incorrect conclusions.

The Cost of Ignoring Batch Effects

- **False discoveries**: Genes that differ between batches are reported as biologically significant
- **Failed replication**: Results from one batch do not replicate in another
- **Incorrect clustering**: PCA and clustering algorithms group samples by batch, not biology
- **Wasted resources**: Follow-up experiments on batch-effect genes waste time and money
- **Retracted papers**: Several high-profile genomics papers have been retracted or corrected because batch effects were not properly handled

Confounding Is Even More Dangerous

Batch effects are a special case of confounding — when a technical or demographic variable is correlated with both the predictor and the outcome.

If Lab A processed mostly ER-positive tumors and Lab B processed mostly ER-negative tumors, you cannot tell whether gene expression differences are due to ER status or lab. The batch is **confounded** with the biology.

This is the most dangerous scenario because no statistical correction can fully separate the two effects. Confounding must be prevented at the design stage, not fixed in the analysis.

The Data

GTEx Tissue Expression Data

GTEx (Genotype-Tissue Expression) provides expression data from multiple tissue types across hundreds of donors. Samples were processed at different sites and in different batches, making it an excellent resource for studying batch effects.

```
# Download GTEx median TPM by tissue
let gtex_url = "https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-
seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
let gtex_file = download(gtex_url, "gtex_median_tpm.gct.gz")

# GCT format: first two rows are header, then gene_id, description, then
samples
let gtex = read_tsv(gtex_file)
println("GTEx expression data:")
println("  Genes: " + to_string(len(gtex)))
println("  Tissues: " + to_string(ncols(gtex) - 2))  # minus gene_id and
description

let tissue_names = columns(gtex) |> slice(2, ncols(gtex))
println("  Tissue types:")
tissue_names |> each(|t| println("    " + t))
```

GEO Dataset with Known Batch Structure

We also use a GEO dataset where batch information is available, allowing us to demonstrate detection and correction.

```
# Download GEO dataset GSE183947 — breast cancer RNA-seq with batch
information
let geo_url =
"https://ftp.ncbi.nlm.nih.gov/geo/series/GSE183nnn/GSE183947/suppl/GSE183947_
fpkm.csv.gz"
let geo_file = download(geo_url, "gse183947_fpkm.csv.gz")

let geo_expr = read_csv(geo_file)
println("GEO expression data:")
println("  Genes: " + to_string(len(geo_expr)))
println("  Samples: " + to_string(ncols(geo_expr) - 1))

head(geo_expr, 5)
```

Exploration

What Do Batch Effects Look Like?

Batch effects manifest as systematic differences in measured values between groups of samples processed differently. They can affect:

1. **Mean expression level**: All genes are shifted up or down (library size effect)
2. **Variance**: Some batches have more variable measurements

3. **Specific genes**: Certain genes are affected more than others (GC content bias, gene length bias)
4. **Correlation structure**: Gene-gene correlations change between batches

```
# Simulate a dataset with batch effects to demonstrate the principles
# 20 samples from each of 2 batches, measuring 1000 genes
# True biology: 2 groups (treatment/control), 10 samples each per batch

let n_genes = 1000
let n_samples = 40

# Generate true expression (same in both batches)
let true_expr = []
let g = 0
let gene_means = rnorm(n_genes, 8.0, 3.0)  # gene-level mean expression

# Batch effect: shift all genes by a constant
let batch_shift = 2.0  # log2 units

# Generate expression matrix
# Samples 1-20: Batch A, Samples 21-40: Batch B
# Samples 1-10 and 21-30: Control, Samples 11-20 and 31-40: Treatment

let batch_labels = concat(
    rnorm(20, 0.0, 0.0) |> map(|_| "Batch_A"),
    rnorm(20, 0.0, 0.0) |> map(|_| "Batch_B")
)
let group_labels = concat(
    concat(
        rnorm(10, 0.0, 0.0) |> map(|_| "Control"),
        rnorm(10, 0.0, 0.0) |> map(|_| "Treatment")
    ),
    concat(
        rnorm(10, 0.0, 0.0) |> map(|_| "Control"),
        rnorm(10, 0.0, 0.0) |> map(|_| "Treatment")
    )
)

println("Experimental design:")
println("  Batch A: 10 control + 10 treatment = 20 samples")
println("  Batch B: 10 control + 10 treatment = 20 samples")
println("  (Balanced design — batch is NOT confounded with group)")
```

```
# Simulate expression with batch effect
# For each gene, Batch B has expression shifted by batch_shift

let sim_data = gene_means |> map(|mu|
    let batch_a = rnorm(20, mu, 1.0)
    let batch_b = rnorm(20, mu + batch_shift, 1.0)
    concat(batch_a, batch_b)
)

let sim_matrix = matrix(sim_data)

println("Simulated expression matrix: "
    + to_string(len(gene_means)) + " genes x "
    + to_string(n_samples) + " samples")

# Overall mean by batch
let batch_a_means = sim_data |> map(|gene| mean(slice(gene, 0, 20)))
let batch_b_means = sim_data |> map(|gene| mean(slice(gene, 20, 40)))

println("Mean expression per gene:")
println("  Batch A: " + to_string(round(mean(batch_a_means), 2)))
println("  Batch B: " + to_string(round(mean(batch_b_means), 2)))
println("  Difference: " + to_string(round(mean(batch_b_means) -
mean(batch_a_means), 2)))
println("  (True batch effect was " + to_string(batch_shift) + ")")
```

Detecting Batch Effects with PCA

PCA is the most common method for detecting batch effects. If the first principal components separate samples by batch rather than by biology, you have a batch effect problem.

```
# PCA on simulated data
let pca_result = pca(sim_matrix)

println("PCA of simulated data:")
println("  PC1 variance explained: " +
to_string(round(pca_result.variance_explained[0] * 100.0, 1)) + "%")
println("  PC2 variance explained: " +
to_string(round(pca_result.variance_explained[1] * 100.0, 1)) + "%")

# Plot PCA colored by batch
pca_plot(pca_result, batch_labels, "PCA Colored by Batch (Before
Correction)")
    |> save_plot("pca_batch_before.svg")

# Plot PCA colored by biological group
pca_plot(pca_result, group_labels, "PCA Colored by Treatment Group (Before
Correction)")
    |> save_plot("pca_group_before.svg")
```

In the batch-colored plot, you will see two distinct clusters on PC1 — Batch A on one side, Batch B on the other. In the group-colored plot, the treatment and

control groups are mixed within each batch cluster. PC1 captures the batch effect, not the biology.

```
# Quantify the batch effect using PC1 scores
let pc1_scores = pca_result.scores |> map(|row| row[0])

let pc1_batch_a = slice(pc1_scores, 0, 20)
let pc1_batch_b = slice(pc1_scores, 20, 40)

let pc1_test = ttest(pc1_batch_a, pc1_batch_b)
println("PC1 vs Batch:")
println("  Batch A PC1 mean: " + to_string(round(mean(pc1_batch_a), 2)))
println("  Batch B PC1 mean: " + to_string(round(mean(pc1_batch_b), 2)))
println("  t-test p-value: " + to_string(pc1_test.p_value))

# Compare with PC1 vs treatment group
let pc1_control = concat(slice(pc1_scores, 0, 10), slice(pc1_scores, 20, 30))
let pc1_treatment = concat(slice(pc1_scores, 10, 20), slice(pc1_scores, 30,
40))

let pc1_group_test = ttest(pc1_control, pc1_treatment)
println("PC1 vs Treatment:")
println("  Control PC1 mean: " + to_string(round(mean(pc1_control), 2)))
println("  Treatment PC1 mean: " + to_string(round(mean(pc1_treatment), 2)))
println("  t-test p-value: " + to_string(pc1_group_test.p_value))
```

If PC1 is significantly associated with batch but not with treatment, the batch effect dominates the biology. This is the classic batch effect signature.

Detecting Batch Effects with Correlation

```
# Another detection method: correlation between expression and batch
# For each gene, compute correlation with batch indicator

let batch_numeric = batch_labels |> map(|b| if b == "Batch_A" then 0.0 else
1.0)
let group_numeric = group_labels |> map(|g| if g == "Control" then 0.0 else
1.0)

# How many genes correlate with batch vs treatment?
let batch_cors = sim_data |> map(|gene| abs(cor(gene, batch_numeric)))
let group_cors = sim_data |> map(|gene| abs(cor(gene, group_numeric)))

println("Absolute correlation with batch vs treatment:")
println("  Mean |cor| with batch: " + to_string(round(mean(batch_cors), 4)))
println("  Mean |cor| with treatment: " + to_string(round(mean(group_cors),
4)))

let high_batch_cor = batch_cors |> filter(|c| c > 0.3) |> len()
let high_group_cor = group_cors |> filter(|c| c > 0.3) |> len()
println("  Genes with |cor| > 0.3 with batch: " + to_string(high_batch_cor))
println("  Genes with |cor| > 0.3 with treatment: " +
to_string(high_group_cor))
```

The Analysis

Part 1: Library Size as a Confounder

The most basic batch effect in RNA-seq is library size — the total number of reads per sample. Samples with more reads have higher counts for every gene.

```
# Simulate library size variation
let lib_sizes_a = rnorm(20, 30.0, 5.0)   # millions of reads, Batch A
let lib_sizes_b = rnorm(20, 50.0, 5.0)   # millions of reads, Batch B
(deeper)
let lib_sizes = concat(lib_sizes_a, lib_sizes_b)

println("Library sizes (millions of reads):")
println("  Batch A mean: " + to_string(round(mean(lib_sizes_a), 1)))
println("  Batch B mean: " + to_string(round(mean(lib_sizes_b), 1)))
println("  Ratio: " + to_string(round(mean(lib_sizes_b) / mean(lib_sizes_a),
2)) + "x")

# Before normalization: raw counts correlate with library size
# This creates a spurious positive correlation between ALL genes
println("")
println("Before normalization:")
println("  Every gene appears 'upregulated' in Batch B")
println("  Correlation between any two genes is inflated")
println("  PCA separates batches perfectly")
```

```
# Library size normalization methods

# 1. CPM (Counts Per Million) — simplest
# CPM = count / total_counts * 1,000,000

let raw_counts = rnorm(40, 500.0, 200.0) |> map(|x| abs(x))
let total_counts = lib_sizes |> map(|x| x * 1000000.0)
let cpm_vals = zip(raw_counts, total_counts) |> map(|pair| pair[0] / pair[1]
* 1000000.0)

println("CPM normalization:")
println("  Divides each count by library size")
println("  Makes samples comparable in total counts")
println("  Does NOT account for compositional effects")

# 2. TPM (Transcripts Per Million) — accounts for gene length
println("")
println("TPM normalization:")
println("  Divides by gene length first (RPKM), then normalizes to sum to
1M")
println("  Makes samples comparable AND accounts for gene length")
println("  Better than FPKM for between-sample comparison")

# 3. DESeq2 size factors — median of ratios
println("")
println("DESeq2 normalization:")
println("  Uses median of ratios to a reference sample")
println("  Robust to outlier genes (e.g., one highly expressed gene)")
println("  Standard for differential expression analysis")
```

```
# Demonstrate CPM normalization on our simulated data
# Apply CPM to remove library size effect

let normalized_data = sim_data |> map(|gene|
    zip(gene, lib_sizes) |> map(|pair|
        pair[0] / pair[1] * 1.0  # simplified normalization
    )
)

# Re-run PCA after normalization
let norm_matrix = matrix(normalized_data)
let pca_norm = pca(norm_matrix)

println("PCA after library size normalization:")
println("  PC1 variance explained: " +
to_string(round(pca_norm.variance_explained[0] * 100.0, 1)) + "%")
println("  PC2 variance explained: " +
to_string(round(pca_norm.variance_explained[1] * 100.0, 1)) + "%")

pca_plot(pca_norm, batch_labels, "PCA After Normalization (Colored by
Batch)")
    |> save_plot("pca_batch_normalized.svg")
```

Library size normalization helps but may not remove all batch effects. Deeper batch effects (GC content bias, RNA degradation differences, different kit versions) require more sophisticated correction.

Part 2: ComBat-Style Batch Correction

ComBat (from the sva R package) is the most widely used batch correction method. It uses an empirical Bayes framework to shrink batch effect estimates toward a common value, which is more robust than gene-by-gene correction.

The principle:

For each gene, model expression as:

Y_ij = alpha + X*beta* + *batch_j*gamma + delta_j*epsilon

Where:

- alpha: overall mean
- X*beta: biological effects (treatment, etc.)
- gamma: additive batch effect (shift)
- delta: multiplicative batch effect (scaling)
- epsilon: residual

ComBat estimates gamma and delta for each gene, applies empirical Bayes shrinkage, then subtracts the batch effects from the data.

```
# Regression-based batch correction (simplified ComBat)
# For each gene, regress out the batch effect

let corrected_data = sim_data |> map(|gene|
    # Fit: expression ~ batch
    let model = lm(gene, batch_numeric)
    let batch_effect = model.coefficients[0]

    # Subtract batch effect from Batch B samples
    gene |> map(|val|
        val  # keep Batch A as-is
    )
    # More precisely: subtract fitted batch component
    zip(gene, batch_numeric) |> map(|pair|
        pair[0] - batch_effect * pair[1]
    )
)

let corrected_matrix = matrix(corrected_data)
let pca_corrected = pca(corrected_matrix)

println("PCA after regression-based batch correction:")
println("  PC1 variance explained: " +
to_string(round(pca_corrected.variance_explained[0] * 100.0, 1)) + "%")
println("  PC2 variance explained: " +
to_string(round(pca_corrected.variance_explained[1] * 100.0, 1)) + "%")

pca_plot(pca_corrected, batch_labels, "PCA After Batch Correction (Colored by
Batch)")
    |> save_plot("pca_batch_corrected.svg")

pca_plot(pca_corrected, group_labels, "PCA After Batch Correction (Colored by
Group)")
    |> save_plot("pca_group_corrected.svg")
```

After correction, the PCA should no longer separate samples by batch. The biological signal (treatment vs control) should now be more visible.

```
# Verify batch effect removal
let pc1_corrected = pca_corrected.scores |> map(|row| row[0])

let pc1_corr_a = slice(pc1_corrected, 0, 20)
let pc1_corr_b = slice(pc1_corrected, 20, 40)

let batch_test_after = ttest(pc1_corr_a, pc1_corr_b)
println("Batch effect in PC1 after correction:")
println("  Batch A mean: " + to_string(round(mean(pc1_corr_a), 4)))
println("  Batch B mean: " + to_string(round(mean(pc1_corr_b), 4)))
println("  p-value: " + to_string(round(batch_test_after.p_value, 4)))
println("  (Should be non-significant if correction worked)")

# Check that biological signal is preserved
let pc1_corr_ctrl = concat(slice(pc1_corrected, 0, 10), slice(pc1_corrected,
20, 30))
let pc1_corr_trt = concat(slice(pc1_corrected, 10, 20), slice(pc1_corrected,
30, 40))

let group_test_after = ttest(pc1_corr_ctrl, pc1_corr_trt)
println("Treatment effect in PC1 after correction:")
println("  p-value: " + to_string(round(group_test_after.p_value, 4)))
```

Part 3: Why You Should Include Batch as a Covariate Instead

There is a philosophical debate in genomics:

Approach A: Correct the data, then analyze. Remove batch effects from the expression matrix, then do differential expression analysis on the corrected data.

Approach B: Include batch as a covariate in the model. Leave the data uncorrected and add batch as a covariate in the statistical model (e.g., DESeq2 design formula: ~ batch + treatment).

Approach B is generally preferred for hypothesis testing because:

1. It preserves the full statistical model (uncertainty from batch estimation is included in the standard errors)
2. It does not alter the data (you analyze the original measurements)
3. It handles continuous confounders (you cannot “correct” for age the way you correct for batch)
4. It gives correct p-values and confidence intervals

Approach A is useful for:

- Visualization (PCA plots, heatmaps)
- Clustering (you want biology-driven clusters)
- Machine learning (you want batch-free features)

```
# Demonstrate the two approaches

# Approach A: Correct then analyze
# (Already done above — corrected_data)
let de_corrected = sim_data |> map(|gene|
    let corrected_gene = zip(gene, batch_numeric) |> map(|pair|
        pair[0] - lm(gene, batch_numeric).coefficients[0] * pair[1]
    )
    let ctrl = concat(slice(corrected_gene, 0, 10), slice(corrected_gene, 20,
30))
    let trt = concat(slice(corrected_gene, 10, 20), slice(corrected_gene, 30,
40))
    ttest(ctrl, trt).p_value
)

# Approach B: Include batch as covariate
let de_covariate = sim_data |> map(|gene|
    # Multiple regression: expression ~ treatment + batch
    let model = lm(gene, matrix([group_numeric, batch_numeric]))
    model.p_values[0]  # p-value for treatment effect
)

# Count significant genes with each approach
let sig_corrected = de_corrected |> filter(|p| p < 0.05) |> len()
let sig_covariate = de_covariate |> filter(|p| p < 0.05) |> len()

println("Differential expression results:")
println("  Approach A (correct then test): " + to_string(sig_corrected) + "
significant genes")
println("  Approach B (batch as covariate): " + to_string(sig_covariate) + "
significant genes")
println("")
println("  Both should give similar results when batch and biology are
orthogonal")
println("  Approach B is preferred for formal testing")
```

Part 4: Confounding — When Batch and Biology Are Entangled

The most dangerous scenario is when batch is correlated with the biological variable of interest. This happens when:

- All cases were processed in one batch and all controls in another
- One lab contributed mostly early-stage tumors and another mostly late-stage
- Different sequencing platforms were used for different groups

```
# Simulate a CONFOUNDED design
# Batch A: 20 controls
# Batch B: 20 treatments
# Now batch and treatment are perfectly confounded!

let confounded_batch = concat(
    rnorm(20, 0.0, 0.0) |> map(|_| "Batch_A"),
    rnorm(20, 0.0, 0.0) |> map(|_| "Batch_B")
)
let confounded_group = concat(
    rnorm(20, 0.0, 0.0) |> map(|_| "Control"),
    rnorm(20, 0.0, 0.0) |> map(|_| "Treatment")
)

let confounded_batch_num = confounded_batch |> map(|b| if b == "Batch_A" then
0.0 else 1.0)
let confounded_group_num = confounded_group |> map(|g| if g == "Control" then
0.0 else 1.0)

# These are perfectly correlated!
let confound_cor = cor(confounded_batch_num, confounded_group_num)
println("Correlation between batch and treatment: " +
to_string(confound_cor))
println("PERFECT CONFOUNDING — cannot separate batch from treatment")
println("")
println("Any gene that differs between batches also differs between groups")
println("Any gene that differs between groups also differs between batches")
println("NO statistical method can disentangle these effects")
```

```
# Show that regression fails with perfect confounding
let confounded_gene = concat(
    rnorm(20, 5.0, 1.0),  # Batch A / Control
    rnorm(20, 8.0, 1.0)   # Batch B / Treatment
)

# Is the difference due to batch or treatment?
# We literally cannot tell
let ttest_result = ttest(slice(confounded_gene, 0, 20),
slice(confounded_gene, 20, 40))
println("T-test p-value: " + to_string(ttest_result.p_value))
println("  Is this batch or biology? UNKNOWABLE")
println("")
println("Lesson: Confounding must be prevented at the DESIGN stage")
println("  Randomize samples across batches")
println("  If randomization is impossible, ensure batch and biology are")
println("  as uncorrelated as possible")
```

Part 5: Simpson’s Paradox in Genomics

Simpson’s paradox occurs when a trend that appears in grouped data reverses when the groups are combined. This is a direct consequence of confounding.

```
# Simpson's paradox example
# Gene X is upregulated in treatment in BOTH batches
# But overall, it appears DOWNregulated in treatment

# Batch A: low overall expression, treatment slightly higher
let batch_a_ctrl = rnorm(30, 3.0, 0.5)
let batch_a_trt = rnorm(10, 3.5, 0.5)

# Batch B: high overall expression, treatment slightly higher
let batch_b_ctrl = rnorm(10, 8.0, 0.5)
let batch_b_trt = rnorm(30, 8.5, 0.5)

# Within each batch: treatment > control
println("Within-batch effects:")
println("  Batch A: Control mean = " + to_string(round(mean(batch_a_ctrl),
2))
    + ", Treatment mean = " + to_string(round(mean(batch_a_trt), 2))
    + " (treatment higher)")
println("  Batch B: Control mean = " + to_string(round(mean(batch_b_ctrl),
2))
    + ", Treatment mean = " + to_string(round(mean(batch_b_trt), 2))
    + " (treatment higher)")

# Combined: control samples come mostly from high-expression Batch B
# treatment samples come mostly from low-expression Batch A
let all_ctrl = concat(batch_a_ctrl, batch_b_ctrl)
let all_trt = concat(batch_a_trt, batch_b_trt)

println("")
println("Combined effect (ignoring batch):")
println("  Control mean = " + to_string(round(mean(all_ctrl), 2))
    + " (n=" + to_string(len(all_ctrl)) + ")")
println("  Treatment mean = " + to_string(round(mean(all_trt), 2))
    + " (n=" + to_string(len(all_trt)) + ")")

let overall_test = ttest(all_ctrl, all_trt)
println("  T-test p-value: " + to_string(round(overall_test.p_value, 4)))
println("")
println("PARADOX: Treatment is HIGHER within each batch")
println("         But LOWER overall because batch confounds the comparison")
println("         More controls came from the high-expression batch")
```

```
# The fix: include batch in the model
let combined_expr = concat(concat(batch_a_ctrl, batch_a_trt),
                           concat(batch_b_ctrl, batch_b_trt))
let combined_group = concat(
    concat(rnorm(30, 0.0, 0.0) |> map(|_| 0.0),
           rnorm(10, 0.0, 0.0) |> map(|_| 1.0)),
    concat(rnorm(10, 0.0, 0.0) |> map(|_| 0.0),
           rnorm(30, 0.0, 0.0) |> map(|_| 1.0))
)
let combined_batch = concat(
    rnorm(40, 0.0, 0.0) |> map(|_| 0.0),
    rnorm(40, 0.0, 0.0) |> map(|_| 1.0)
)

let corrected_model = lm(combined_expr, matrix([combined_group,
combined_batch]))
println("Multiple regression (expression ~ treatment + batch):")
println("  Treatment coefficient: " +
to_string(round(corrected_model.coefficients[0], 4)))
println("  Batch coefficient: " +
to_string(round(corrected_model.coefficients[1], 4)))
println("  Treatment p-value: " +
to_string(round(corrected_model.p_values[0], 4)))
println("  (Now correctly shows treatment effect within batches)")
```

Part 6: Population Stratification in GWAS

Population stratification is a confounding problem specific to genetics. If cases and controls come from different ancestral populations, any allele that differs in frequency between populations will appear associated with the disease — even if it has no causal effect.

```
# Simulate population stratification
# Population A (European): higher frequency of allele, lower disease risk
# Population B (East Asian): lower frequency of allele, higher disease risk

let n_european = 500
let n_east_asian = 500

# Allele frequency differs between populations (not related to disease)
let af_european = 0.30
let af_east_asian = 0.15

# Disease prevalence differs between populations (for other reasons)
let disease_european = 0.05
let disease_east_asian = 0.10

# Generate genotypes
let geno_eur = rbinom(n_european, 2, af_european) |> map(|x| to_float(x))
let geno_eas = rbinom(n_east_asian, 2, af_east_asian) |> map(|x| to_float(x))

# Generate disease status (independent of genotype!)
let disease_eur = rbinom(n_european, 1, disease_european) |> map(|x|
to_float(x))
let disease_eas = rbinom(n_east_asian, 1, disease_east_asian) |> map(|x|
to_float(x))

let all_geno = concat(geno_eur, geno_eas)
let all_disease = concat(disease_eur, disease_eas)
let all_pop = concat(
    rnorm(n_european, 0.0, 0.0) |> map(|_| 0.0),
    rnorm(n_east_asian, 0.0, 0.0) |> map(|_| 1.0)
)

# Naive association (ignoring population)
let naive_chi2 = chi_square(
    [
        all_geno |> filter(|g| g > 0.0) |> len(),
        all_geno |> filter(|g| g == 0.0) |> len()
    ],
    [
        all_disease |> filter(|d| d == 1.0) |> len(),
        all_disease |> filter(|d| d == 0.0) |> len()
    ]
)

# Corrected association (including population as covariate)
let corrected_assoc = glm(all_disease, matrix([all_geno, all_pop]),
"binomial")

println("Population stratification example:")
println("  Allele has NO causal effect on disease")
println("  But populations differ in both allele frequency and disease risk")
println("")
println("  Naive test (ignoring population): p = " +
to_string(round(naive_chi2.p_value, 4)))
println("  Corrected test (adjusting for population): p = "
    + to_string(round(corrected_assoc.p_values[0], 4)))
println("")
if naive_chi2.p_value < 0.05 && corrected_assoc.p_values[0] > 0.05 then
    println("  The naive test gives a FALSE POSITIVE")
    println("  The corrected test correctly shows no association")
end
```

```
# GWAS solves this with principal components
# PCA of genome-wide genotypes captures population structure
# Include top PCs (typically 10-20) as covariates in the association model

println("GWAS population stratification correction:")
println("  1. Run PCA on genome-wide genotype data")
println("  2. Include top 10-20 PCs as covariates")
println("  3. Compute genomic inflation factor (lambda_GC)")
println("     lambda = 1.0 means no inflation (good)")
println("     lambda > 1.1 means residual stratification (adjust more)")
println("  4. Use QQ plot to check for systematic p-value inflation")
```

Part 7: Applying Corrections to Real Data

```
# Work with the GTEx data to show tissue effects
# GTEx has tissues as columns — these are not batch effects per se,
# but we can use the structure to demonstrate PCA-based analysis

let gtex_data = read_tsv(gtex_file)

# Extract numeric expression values (skip gene ID and description columns)
let tissue_cols = columns(gtex_data) |> slice(2, ncols(gtex_data))

# Get expression for a subset of genes (top variable genes)
let gtex_subset = gtex_data |> head(500)  # first 500 genes for speed

# Build a matrix of tissues x genes (transposed from the file)
# Each tissue is a "sample" — we are looking at tissue as a batch-like effect
println("GTEx analysis:")
println("  Using " + to_string(len(tissue_cols)) + " tissues as 'samples'")
println("  Using 500 genes for PCA")

# Extract expression values for each tissue
let tissue_data = tissue_cols |> map(|tissue|
    col_values(gtex_subset, tissue) |> map(|x| log2(to_float(x) + 1.0))
)

let tissue_matrix = matrix(tissue_data)

# PCA of tissues based on gene expression
let gtex_pca = pca(tissue_matrix)

println("GTEx PCA:")
println("  PC1 explains " + to_string(round(gtex_pca.variance_explained[0] *
100.0, 1)) + "% of variance")
println("  PC2 explains " + to_string(round(gtex_pca.variance_explained[1] *
100.0, 1)) + "% of variance")

pca_plot(gtex_pca, tissue_cols, "GTEx Tissues PCA") |>
save_plot("gtex_pca.svg")
```

In the GTEx PCA, brain tissues cluster together, muscle tissues cluster together, etc. This is not a batch effect — it is biology. The lesson is that PCA alone cannot distinguish batch effects from real biology. You need to know what the batches are and check whether PCA components correlate with them.

Part 8: Detecting Unknown Batches with Surrogate Variables

Sometimes batch information is not recorded or is incomplete. Surrogate Variable Analysis (SVA) uses the expression data itself to detect hidden sources of variation.

```
# The principle of surrogate variable analysis:
# 1. Fit a model with only the biological variable of interest
# 2. Examine the residuals — they contain batch effects + noise
# 3. Apply PCA/SVD to the residuals
# 4. The top components are "surrogate variables" — proxies for unknown
batches
# 5. Include surrogate variables as covariates in the final model

println("Surrogate Variable Analysis (SVA) principle:")
println("  1. Fit expression ~ treatment (ignoring batch)")
println("  2. Residuals = batch effects + noise")
println("  3. PCA of residuals → surrogate variables")
println("  4. Fit expression ~ treatment + SV1 + SV2 + ...")
println("  5. SVs capture unknown batch effects without needing batch
labels")
println("")
println("BioLang implementation:")

# Compute residuals from biology-only model
let residual_data = sim_data |> map(|gene|
    let model = lm(gene, group_numeric)
    # Residuals capture everything not explained by treatment
    zip(gene, group_numeric) |> map(|pair|
        pair[0] - (model.intercept + model.coefficients[0] * pair[1])
    )
)

let residual_matrix = matrix(residual_data)
let svd_result = pca(residual_matrix)

println("Surrogate variables from residual PCA:")
println("  SV1 variance explained: " +
to_string(round(svd_result.variance_explained[0] * 100.0, 1)) + "%")
println("  SV2 variance explained: " +
to_string(round(svd_result.variance_explained[1] * 100.0, 1)) + "%")

# SV1 should correlate with batch if batch is the dominant hidden effect
let sv1 = svd_result.scores |> map(|row| row[0])
let sv1_batch_cor = cor(sv1, batch_numeric |> map(|x| to_float(x)))
println("Correlation of SV1 with actual batch: " +
to_string(round(abs(sv1_batch_cor), 4)))
println("  (High correlation confirms SV1 captures the batch effect)")
```

Part 9: Before vs After — A Complete Workflow

```
# Complete batch correction workflow on the simulated data

println("=== COMPLETE BATCH CORRECTION WORKFLOW ===")
println("")

# Step 1: Detect batch effects
println("Step 1: Detect batch effects")
let raw_pca = pca(sim_matrix)
let pc1_batch_assoc = ttest(
    slice(raw_pca.scores |> map(|r| r[0]), 0, 20),
    slice(raw_pca.scores |> map(|r| r[0]), 20, 40)
)
println("  PC1 vs batch p-value: " + to_string(pc1_batch_assoc.p_value))
if pc1_batch_assoc.p_value < 0.05 then
    println("  BATCH EFFECT DETECTED in PC1")
end

# Step 2: Check for confounding
println("")
println("Step 2: Check for confounding")
let batch_group_cor = cor(
    batch_numeric |> map(|x| to_float(x)),
    group_numeric |> map(|x| to_float(x))
)
println("  Correlation(batch, treatment): " +
to_string(round(batch_group_cor, 4)))
if abs(batch_group_cor) < 0.3 then
    println("  LOW confounding — correction is feasible")
else if abs(batch_group_cor) < 0.7 then
    println("  MODERATE confounding — proceed with caution")
else
    println("  HIGH confounding — results may be unreliable")
end

# Step 3: Choose correction strategy
println("")
println("Step 3: Choose correction strategy")
println("  For hypothesis testing: include batch as covariate")
println("  For visualization: apply regression-based correction")

# Step 4: Apply correction for visualization
println("")
println("Step 4: Correct data for visualization")
let viz_corrected = sim_data |> map(|gene|
    let batch_effect = lm(gene, batch_numeric |> map(|x|
to_float(x))).coefficients[0]
    zip(gene, batch_numeric) |> map(|pair|
        pair[0] - batch_effect * to_float(pair[1])
    )
)

let viz_pca = pca(matrix(viz_corrected))
pca_plot(viz_pca, batch_labels, "PCA After Correction (Batch)") |>
save_plot("pca_corrected_batch.svg")
pca_plot(viz_pca, group_labels, "PCA After Correction (Group)") |>
save_plot("pca_corrected_group.svg")

# Step 5: DE analysis with batch as covariate
println("")
println("Step 5: Differential expression with batch covariate")
let sig_genes = 0
let de_results = sim_data |> map(|gene|
```

```
    let model = lm(gene, matrix([
        group_numeric |> map(|x| to_float(x)),
        batch_numeric |> map(|x| to_float(x))
    ]))
    record(
        "treatment_coef", model.coefficients[0],
        "batch_coef", model.coefficients[1],
        "treatment_p", model.p_values[0]
    )
)

let de_table = table(de_results)
let de_p = col_values(de_table, "treatment_p") |> map(|x| to_float(x))
let de_p_adj = p_adjust(de_p, "BH")

let sig_after_correction = de_p_adj |> filter(|p| p < 0.05) |> len()
println("  Significant genes (BH < 0.05): " +
to_string(sig_after_correction))

# Step 6: Verify correction
println("")
println("Step 6: Verify batch effect removal")
let pc1_after = ttest(
    slice(viz_pca.scores |> map(|r| r[0]), 0, 20),
    slice(viz_pca.scores |> map(|r| r[0]), 20, 40)
)
println("  PC1 vs batch p-value after correction: " +
to_string(round(pc1_after.p_value, 4)))
println("  (Should be non-significant)")
```

Interpretation

The Hierarchy of Batch Effect Solutions

From best to worst:

1. **Prevention**: Randomize samples across batches at the design stage. This makes batch orthogonal to biology. No correction needed.

2. **Covariate adjustment**: Include batch as a covariate in the statistical model. Preserves the data, gives correct p-values, handles both known and estimated (surrogate variable) batch effects.

3. **Data correction**: Remove batch effects from the data (ComBat, limma removeBatchEffect). Useful for visualization and clustering but can distort standard errors. Always analyze the uncorrected data with covariates for hypothesis testing.

4. **Ignoring batch effects**: Produces false discoveries, missed true signals, and irreproducible results. Never acceptable.

When Correction Fails

Correction fails when:

- Batch is perfectly confounded with biology (no statistical fix)
- The batch effect is non-linear (standard methods assume linearity)
- The batch effect is gene-specific and complex (standard methods assume simple additive/multiplicative effects)
- Sample sizes per batch are too small for reliable estimation

What We Learned from the Simulations

1. A batch shift of 2 log2-units dominates PCA, making the biological signal invisible
2. Library size normalization removes part but not all of the batch effect
3. Regression-based correction effectively removes the batch effect when batch and biology are orthogonal
4. When batch and biology are confounded, no correction can recover the biological signal
5. Simpson's paradox is a real threat — always check for confounders before interpreting overall trends

Decision

Checklist for Every Multi-Batch Study

Before analyzing any multi-batch dataset, answer these questions:

```
println("Batch Effect Checklist:")
println("──────────────────────")
println("")
println("1. [ ] Do you know all the batches?")
println("   (sequencing center, date, lane, operator, kit version)")
println("")
println("2. [ ] Is batch confounded with your biological variable?")
println("   Compute correlation(batch, biology)")
println("   If |r| > 0.3, you have a problem")
println("")
println("3. [ ] Does PCA separate samples by batch?")
println("   Color PCA by batch — is there clustering?")
println("   Test PC1-PC5 for association with batch")
println("")
println("4. [ ] Have you chosen a correction strategy?")
println("   For DE testing: batch as covariate in the model")
println("   For visualization: ComBat or regression correction")
println("   For both: correct for viz, covariate for testing")
println("")
println("5. [ ] Have you verified the correction?")
println("   PCA after correction should not separate by batch")
println("   Biological signal should be preserved or enhanced")
println("")
println("6. [ ] Have you documented the batch structure?")
println("   Report which samples were in which batch")
println("   Report the correction method used")
println("   Report before/after PCA plots")
```

When to Abort a Study

Sometimes batch effects are so severe or confounded that the study cannot produce reliable results. Consider abandoning the analysis when:

- Batch is perfectly confounded with the biological variable
- Batch effects are so large that biological signal is undetectable even after correction
- The number of samples per batch is too small (< 5) for reliable batch estimation
- Key clinical covariates are missing, making it impossible to check for confounding

In these cases, the honest conclusion is: “This study design does not support the intended analysis. New data with a balanced batch design is needed.”

Pitfalls

Pitfall 1: Over-Correction

```
# Batch correction can remove real biology if the biological effect
# is correlated with batch

println("Over-correction danger:")
println("  If 70% of ER+ samples are in Batch A and 70% of ER- in Batch B,")
println("  correcting for batch will partially remove the ER effect")
println("  Solution: Include ER status in the correction model")
println("  ComBat allows specifying biological variables to preserve")
println("")
println("  ComBat formula: correct(data, batch, model.matrix(~ biology))")
println("  This removes batch while preserving the biological signal")
```

Pitfall 2: Correcting Normalized and Raw Data Differently

```
println("Consistency in correction:")
println("  WRONG: Normalize data, then correct for batch")
println("         (normalization may interact with batch correction)")
println("  WRONG: Correct for batch, then normalize")
println("         (correction may break normalization assumptions)")
println("")
println("  RIGHT for visualization: Normalize → log transform → ComBat")
println("  RIGHT for DE testing: Use DESeq2/edgeR with ~batch+condition")
println("         (these tools normalize and model simultaneously)")
```

Pitfall 3: Applying Batch Correction Across Studies

```
println("Cross-study batch correction:")
println("  Combining data from different studies is extremely challenging")
println("  Each study may use different platforms, protocols, and
populations")
println("")
println("  Warning signs:")
println("  - PCA perfectly separates studies (always check this first)")
println("  - Effect sizes differ dramatically between studies")
println("  - Gene rankings change completely between studies")
println("")
println("  Better approach: meta-analysis (Chapter 15)")
println("  - Analyze each study separately")
println("  - Combine results (effect sizes) rather than data")
println("  - This avoids the need for cross-study batch correction")
```

Pitfall 4: Ignoring Sample-Level QC

```
# Individual samples can be outliers due to technical failure
# Remove these before batch correction

println("Sample-level QC before batch correction:")
println("  1. Check library size per sample — flag outliers")
println("  2. Check % mapped reads — low mapping suggests contamination")
println("  3. Check % ribosomal RNA — high rRNA suggests poor depletion")
println("  4. Check PCA — samples far from their expected cluster may be")
println("     mislabeled or contaminated")
println("  5. Remove outlier samples BEFORE batch correction")
println("     (outliers distort batch effect estimates)")
```

Pitfall 5: Not Reporting Batch Information

```
println("Reporting requirements:")
println("  Every genomics paper should report:")
println("  - How many batches/processing groups existed")
println("  - What variables define the batches")
println("  - Whether batch is confounded with biology (correlation)")
println("  - What correction method was used (if any)")
println("  - Before and after PCA plots showing batch effect and correction")
println("")
println("  Reviewers should reject papers that:")
println("  - Do not mention batch effects")
println("  - Combine multi-center data without discussion")
println("  - Show PCA plots without coloring by batch")
```

Pitfall 6: Forgetting About Non-Obvious Confounders

```
println("Non-obvious confounders in genomics:")
println("")
println("  RNA-seq:")
println("  - RNA integrity number (RIN) — degraded RNA affects gene-level
estimates")
println("  - Time from sample collection to freezing")
println("  - Amount of input RNA")
println("  - 3' bias from degradation")
println("")
println("  GWAS:")
println("  - Population stratification (ancestry)")
println("  - Relatedness (family structure)")
println("  - Genotyping array version")
println("  - Imputation reference panel")
println("")
println("  Clinical data:")
println("  - Age (confounder for almost everything)")
println("  - Sex (affects expression of ~1/3 of genes)")
println("  - BMI (correlates with inflammation gene expression)")
println("  - Medications (especially steroids, which affect gene
expression)")
println("  - Time of day of sample collection (circadian genes)")
```

Where Next?

Immediate Extensions

- **Dimensionality reduction** (Chapter 13): PCA is just the starting point. UMAP and t-SNE are commonly used for visualization in single-cell RNA-seq, where batch effects between experiments are a major challenge.
- **Clustering** (Chapter 14): Batch effects can create artificial clusters. Always check whether clusters correspond to biological groups or technical batches before interpreting them.
- **Meta-analysis** (Chapter 15): The principled alternative to combining raw data across studies. Analyze each study separately, then combine effect sizes. This avoids cross-study batch correction entirely.

Advanced Topics

- **Harmony and Scanorama**: Batch correction methods designed specifically for single-cell RNA-seq, where each experiment has different cell compositions and capture efficiencies.

- **Mutual nearest neighbors (MNN)**: A method that identifies shared cell types across batches and uses them as anchors for correction. More principled than global correction for heterogeneous samples.
- **Causal inference**: When you cannot randomize (observational studies), methods like propensity score matching, instrumental variables, and Mendelian randomization can help distinguish causal effects from confounding.

Practice Problems

1. **GTEx batch detection**: Download GTEx sample-level data (not just medians) and color PCA by sequencing center, ischemic time, and tissue type. Which technical variable explains the most variance?
2. **Simulated confounding**: Create a simulation where batch is 50% confounded with treatment (60% of treatments in Batch A, 40% in Batch B). How does partial confounding affect the DE results? How well does covariate adjustment work?
3. **Over-correction**: Simulate data where treatment causes a 2-fold change in 100 genes, with batch perfectly correlated with treatment. Apply batch correction and show that the biological signal is removed. Then simulate the balanced design and show that correction preserves the biology.
4. **Population stratification**: Using 1000 Genomes Project data, compute PCA of genotypes. Show that the top PCs separate continental populations. Simulate a GWAS with population-stratified cases and controls, and show that PC adjustment removes the inflation.
5. **Library size correlation**: Take any RNA-seq count matrix. For each gene, compute its correlation with total library size. How many genes are significantly correlated? Apply TPM or CPM normalization and repeat. How many genes are still correlated?

Key takeaways from this chapter:

- Batch effects are systematic technical variations that affect all samples processed together. They are present in virtually all genomic data.
- PCA is the primary tool for detecting batch effects. If PC1 separates batches, the batch effect dominates the biology.
- Confounding occurs when batch correlates with the biological variable. No statistical method can fix perfect confounding.

- For hypothesis testing, include batch as a covariate in the model rather than correcting the data.
- For visualization and clustering, use regression-based or ComBat correction on the data.
- Simpson's paradox can reverse the apparent direction of an effect when confounders are ignored.
- Population stratification is a form of confounding in GWAS, corrected by including ancestry PCs as covariates.
- Prevention (balanced experimental design) is always better than post-hoc correction.
- Report batch information, correction methods, and before/after PCA plots in every multi-batch study.

Chapter 13: Dimensionality Reduction

The Problem

You have a single-cell RNA-seq dataset: 2,700 peripheral blood mononuclear cells, each measured across roughly 20,000 genes. You want to visualize which cells are similar to each other. You want to discover cell types. You want to understand what gene programs drive the differences.

But you cannot scatter-plot 20,000 dimensions. You cannot compute distances meaningfully in a space this large. And most of those 20,000 genes are noise — housekeeping genes expressed at the same level in every cell, or dropout zeros where the measurement failed entirely.

How do you reduce 20,000 dimensions to 2 or 3 while preserving the meaningful biological structure?

This is the dimensionality reduction problem, and it is the gateway to every single-cell analysis, every bulk transcriptomics comparison, every clinical genomics quality check. Get it wrong, and you will see patterns that do not exist. Get it right, and entire cell atlases emerge from the noise.

Why It Matters

Dimensionality reduction is not just visualization. It is a fundamental analytical step that determines the success or failure of downstream analysis:

In single-cell RNA-seq, PCA followed by UMAP is the standard pipeline. Every cell atlas — the Human Cell Atlas, Tabula Sapiens, the Mouse Cell Atlas — depends on these methods to organize millions of cells into interpretable structures. If your PCA captures batch effects instead of biology, every cluster you find downstream is wrong.

In bulk RNA-seq, PCA is the primary quality control tool. Before running any differential expression analysis, you plot PC1 vs PC2. If samples do not group by biological condition — if instead they group by sequencing lane, RNA extraction date, or technician — you have a batch effect that must be addressed before any statistical test is valid.

In GWAS, PCA on genotype data reveals population structure. Failing to correct for population structure in a case-control study produces thousands of false positive associations. The top principal components of genotype data are routinely included as covariates in every GWAS regression model.

In clinical genomics, dimensionality reduction on tumor expression profiles reveals molecular subtypes that predict treatment response. The PAM50 breast cancer classifier, used clinically worldwide, is fundamentally a projection of expression data onto a low-dimensional subtype space.

The Data

We use two datasets in this chapter.

Dataset 1: PBMC 3k (single-cell RNA-seq)

The 3,000 PBMC dataset from 10x Genomics is the standard benchmark for single-cell methods. It contains peripheral blood mononuclear cells from a healthy donor, processed with the Chromium Single Cell 3' protocol.

Reference: https://www.10xgenomics.com/datasets/3-k-pbm-cs-from-a-healthy-donor-1-standard-1-1-0

For this chapter, we work with a pre-processed expression matrix where cells are rows and genes are columns — the standard format after Cell Ranger processing. We assume basic quality control has been performed: cells with fewer than 200 genes detected are removed, genes detected in fewer than 3 cells are removed, and cells with more than 5% mitochondrial reads are filtered out.

```
# Download the PBMC 3k filtered expression matrix
# The 10x Genomics dataset page provides the filtered gene-barcode matrix
# After Cell Ranger processing, we have a cells x genes table

# For this chapter we work with a pre-processed TSV
# (In practice you would use read_10x or similar to parse the HDF5/MEX
format)

let pbmc = read_tsv("pbmc3k_filtered.tsv")

println("PBMC dataset dimensions:")
println("  Cells: " + str(len(pbmc)))
println("  Genes: " + str(ncols(pbmc) - 1))  # subtract 1 for cell barcode
column
```

Dataset 2: GTEx median TPM (bulk RNA-seq)

The GTEx (Genotype-Tissue Expression) project provides gene expression data across 54 human tissues from nearly 1,000 donors.

URL: https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz

```
# Download GTEx v8 median TPM per tissue
let gtex_url = "https://storage.googleapis.com/adult-gtex/bulk-gex/v8/rna-
seq/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
let gtex_file = download(gtex_url, "gtex_median_tpm.gct.gz")

# GCT format: skip first 2 header lines, then tab-separated
# Columns: gene_id, gene_name, then one column per tissue
let gtex = read_tsv(gtex_file)

println("GTEx dimensions:")
println("  Genes: " + str(len(gtex)))
println("  Tissues: " + str(ncols(gtex) - 2))  # subtract gene_id and
gene_name
```

Exploration

Understanding the Curse of Dimensionality

Before applying any method, we need to understand why high-dimensional data is fundamentally different from low-dimensional data.

In high dimensions, distances become meaningless. In a 20,000-dimensional space, the ratio of the farthest point to the nearest point approaches 1 as dimensionality increases. Every point is approximately equidistant from every other point. This means that nearest-neighbor methods, distance-based clustering, and any visualization that relies on preserving distances will fail if applied directly to the raw gene expression matrix.

```
# Demonstrate the curse of dimensionality
# Compute pairwise distances in the full gene space

let all_genes = columns(pbmc) |> filter(|c| c != "barcode")

# Sample 100 cells for illustration
let sample_cells = sample(pbmc, 100)

# Extract expression values for all genes
let expr_matrix = sample_cells |> select(all_genes)

# Compute distance matrix in the full space
let full_dist = dist_matrix(expr_matrix, "euclidean")

# Extract all pairwise distances
let distances = []
let n = len(full_dist)
let i = 0
while i < n do
  let j = i + 1
  while j < n do
    distances = concat(distances, [full_dist[i][j]])
    j = j + 1
  end
  i = i + 1
end

# Look at the distribution of distances
let dist_stats = {
  mean: mean(distances),
  sd: stdev(distances),
  min: min(distances),
  max: max(distances),
  cv: stdev(distances) / mean(distances)
}

println("Distance statistics in full 20,000-D space:")
println("  Mean distance: " + str(round(dist_stats.mean, 2)))
println("  SD of distances: " + str(round(dist_stats.sd, 2)))
println("  CV (SD/mean): " + str(round(dist_stats.cv, 4)))
println("  Min/Max ratio: " + str(round(dist_stats.min / dist_stats.max, 4)))

# The CV will be very small — distances are concentrated
# This means "nearest neighbor" is barely distinguishable from "farthest
neighbor"
```

Looking at Variance Per Gene

Not all genes are informative. Most genes are either not expressed, expressed at a constant level, or dominated by technical noise. The first step in any dimensionality reduction is to identify the genes that carry biological signal.

```
# Compute variance for each gene across cells
let gene_names = columns(pbmc) |> filter(|c| c != "barcode")
let gene_variances = gene_names |> map(|g| {
  name: g,
  variance: variance(col_values(pbmc, g)),
  mean_expr: mean(col_values(pbmc, g))
})

# Sort by variance, descending
let sorted_genes = gene_variances |> sort(|a, b| b.variance - a.variance)

# Top 20 most variable genes
println("Top 20 most variable genes:")
sorted_genes |> head(20) |> each(|g|
  println("  " + g.name + ": var=" + str(round(g.variance, 2)) +
          " mean=" + str(round(g.mean_expr, 2)))
)

# How many genes have meaningful variance?
let high_var = sorted_genes |> filter(|g| g.variance > 1.0)
println("Genes with variance > 1.0: " + str(len(high_var)))
println("That is " + str(round(100.0 * len(high_var) / len(sorted_genes), 1))
+ "% of all genes")
```

The typical result: fewer than 2,000 genes (10% of the genome) carry most of the variance. The rest are noise. This is why dimensionality reduction works — the data actually lives on a much lower-dimensional manifold.

The Mean-Variance Relationship

In RNA-seq data, variance scales with the mean. Highly expressed genes have higher absolute variance simply because they have more counts. This is a problem for PCA, which finds directions of maximum variance — it would be dominated by highly expressed genes regardless of their biological relevance.

```
# Plot mean vs variance to see the relationship
let mean_var = gene_variances |> map(|g| {
  log_mean: log10(g.mean_expr + 0.01),
  log_var: log10(g.variance + 0.01)
})

let mv_table = table(mean_var)
scatter(mv_table, "log_mean", "log_var",
  title: "Mean-Variance Relationship (PBMC 3k)",
  x_label: "log10(Mean Expression)",
  y_label: "log10(Variance)")
  |> save_plot("mean_variance_relationship.svg")

# This is why we normalize and select highly variable genes (HVGs)
# before PCA — to remove the mean-variance confound
```

Normalization: Preparing for PCA

Before PCA, we must normalize the data. The standard single-cell preprocessing pipeline is: (1) normalize to total counts per cell, (2) log-transform, (3) select highly variable genes.

```
# Step 1: Library-size normalization
# Scale each cell to the same total count (e.g., 10,000)
let target_sum = 10000.0

let normalized = pbmc |> mutate(|row| {
  let total = gene_names |> map(|g| row[g]) |> sum()
  let scale = target_sum / total
  let result = { barcode: row.barcode }
  gene_names |> each(|g|
    result[g] = row[g] * scale
  )
  result
})

# Step 2: Log-transform (log1p to handle zeros)
let log_normalized = normalized |> mutate(|row| {
  let result = { barcode: row.barcode }
  gene_names |> each(|g|
    result[g] = log(1.0 + row[g])
  )
  result
})

# Step 3: Select highly variable genes (top 2000)
let hvg_variances = gene_names |> map(|g| {
  name: g,
  variance: variance(col_values(log_normalized, g))
})

let hvg_sorted = hvg_variances |> sort(|a, b| b.variance - a.variance)
let hvgs = hvg_sorted |> head(2000) |> map(|g| g.name)

println("Selected " + str(len(hvgs)) + " highly variable genes for PCA")

# Step 4: Center and scale (z-score each gene)
let scaled = log_normalized |> select(concat(["barcode"], hvgs))
let scaled_data = normalize(scaled |> select(hvgs), "zscore")

println("Prepared expression matrix: " + str(len(scaled_data)) + " cells x "
+
        str(ncols(scaled_data)) + " genes")
```

The Analysis

PCA: Finding the Axes of Maximum Variance

Principal Component Analysis finds orthogonal directions (principal components) that capture the maximum variance in the data. Each PC is a linear combination of the original genes. The first PC captures the most variance, the second captures the most variance orthogonal to the first, and so on.

```
# Run PCA on the scaled expression matrix
let pca_result = pca(scaled_data, n_components: 50)

# pca_result contains:
#   .components   — the PC coordinates for each cell (n_cells x n_components)
#   .variance     — variance explained by each component
#   .loadings     — gene weights for each component (n_genes x n_components)

println("PCA complete. Top 10 components — variance explained:")
let total_var = sum(pca_result.variance)
let cumulative = 0.0

pca_result.variance |> head(10) |> each(|v|
  let i = 0
  let pct = 100.0 * v / total_var
  cumulative = cumulative + pct
  println("  PC" + str(i + 1) + ": " + str(round(pct, 2)) + "% (cumulative: "
+
          str(round(cumulative, 2)) + "%)")
  i = i + 1
)
```

What PCA Actually Does — The Mathematics

PCA is an eigenvalue decomposition. For a centered data matrix X (cells x genes), PCA computes the covariance matrix C = X^T X / (n-1), then finds its eigenvectors and eigenvalues. The eigenvectors are the principal component directions. The eigenvalues are the variance explained by each direction.

But you don't need to compute the full covariance matrix (which would be 20,000 x 20,000). The singular value decomposition (SVD) of X gives the same result more efficiently: X = U S V^T, where the columns of V are the PC loadings (eigenvectors of the covariance matrix) and the diagonal of S squared gives the eigenvalues.

```
# BioLang's pca() uses SVD internally
# We can verify by computing SVD directly on a small example

let small_data = scaled_data |> head(100) |> select(hvgs |> head(50))
let small_matrix = matrix(small_data)

let svd_result = svd(small_matrix)
# svd_result.u — left singular vectors (cell space)
# svd_result.s — singular values
# svd_result.v — right singular vectors (gene space / loadings)

# Eigenvalues from SVD
let eigenvalues = svd_result.s |> map(|s| s * s / (len(small_matrix) - 1))

println("Top 5 eigenvalues from direct SVD:")
eigenvalues |> head(5) |> each(|e|
  println("  " + str(round(e, 4)))
)
```

Choosing the Number of PCs: The Elbow Plot

How many PCs should you keep? There is no single correct answer, but the elbow plot provides guidance. Plot the variance explained by each PC. Look for the “elbow” — the point where the curve flattens, indicating that additional PCs are capturing noise rather than signal.

```
# Create the elbow plot (scree plot)
let var_explained = pca_result.variance |> head(50)
let var_pct = var_explained |> map(|v| 100.0 * v / total_var)

let elbow_data = var_pct |> map(|v|
  let i = 0
  let result = { pc: i + 1, variance_pct: v }
  i = i + 1
  result
)

let elbow_table = table(elbow_data)
scatter(elbow_table, "pc", "variance_pct",
  title: "Scree Plot — Variance Explained by PC",
  x_label: "Principal Component",
  y_label: "% Variance Explained")
  |> save_plot("scree_plot.svg")

# Also plot cumulative variance
let cum_var = []
let running = 0.0
var_pct |> each(|v|
  running = running + v
  cum_var = concat(cum_var, [running])
)

let cum_data = cum_var |> map(|v|
  let i = 0
  let result = { pc: i + 1, cumulative_pct: v }
  i = i + 1
  result
)

let cum_table = table(cum_data)
scatter(cum_table, "pc", "cumulative_pct",
  title: "Cumulative Variance Explained",
  x_label: "Principal Component",
  y_label: "Cumulative % Variance")
  |> save_plot("cumulative_variance.svg")

# Typical single-cell: 10-30 PCs capture most biological signal
# A common heuristic: keep PCs until cumulative variance reaches 80-90%
# or until the scree plot elbow
```

In the PBMC 3k dataset, you will typically see an elbow around PC 10-15. The first 2-3 PCs separate major cell types (T cells from monocytes from B cells). PCs 4-15 capture finer distinctions (CD4+ vs CD8+ T cells, classical vs non-classical monocytes). Beyond PC 20, you are mostly fitting noise.

Interpreting PC Loadings — What Genes Drive Each Component

Each principal component is a linear combination of genes. The loadings (weights) tell you which genes contribute most to each PC. These loadings have biological meaning — they represent gene programs.

```
# Get the loadings for PC1 and PC2
# Loadings: which genes have the highest weights in each PC

let pc1_loadings = hvgs |> map(|g|
  let i = 0  # index into hvgs
  { gene: g, loading: pca_result.loadings[i][0] }
)

let pc2_loadings = hvgs |> map(|g|
  let i = 0
  { gene: g, loading: pca_result.loadings[i][1] }
)

# Sort by absolute loading to find the driving genes
let pc1_sorted = pc1_loadings |> sort(|a, b| abs(b.loading) - abs(a.loading))
let pc2_sorted = pc2_loadings |> sort(|a, b| abs(b.loading) - abs(a.loading))

println("Top 10 genes driving PC1:")
pc1_sorted |> head(10) |> each(|g|
  println("  " + g.gene + ": " + str(round(g.loading, 4)))
)

println("Top 10 genes driving PC2:")
pc2_sorted |> head(10) |> each(|g|
  println("  " + g.gene + ": " + str(round(g.loading, 4)))
)

# In PBMC data, PC1 typically separates lymphoid vs myeloid cells
# Key PC1 genes: LYZ, S100A9 (myeloid markers) vs NKG7, GNLY (lymphoid)
# PC2 often separates T cells from B cells and NK cells
```

Visualizing PCA: The Workhorse Plot

The PC1 vs PC2 scatter plot is the most common visualization in genomics. It reveals the global structure of your data at a glance.

```
# Create PC1 vs PC2 scatter plot
let pc_coords = pca_result.components

let pc_table = table(pc_coords |> map(|row| {
  PC1: row[0],
  PC2: row[1],
  PC3: row[2]
}))

scatter(pc_table, "PC1", "PC2",
  title: "PBMC 3k — PCA (PC1 vs PC2)",
  x_label: "PC1 (" + str(round(var_pct[0], 1)) + "% variance)",
  y_label: "PC2 (" + str(round(var_pct[1], 1)) + "% variance)")
  |> save_plot("pbmc_pca.svg")

# Also check PC1 vs PC3 — sometimes the third component
# reveals structure hidden in the first two
scatter(pc_table, "PC1", "PC3",
  title: "PBMC 3k — PCA (PC1 vs PC3)",
  x_label: "PC1 (" + str(round(var_pct[0], 1)) + "% variance)",
  y_label: "PC3 (" + str(round(var_pct[2], 1)) + "% variance)")
  |> save_plot("pbmc_pca_pc1_pc3.svg")
```

t-SNE: Preserving Local Neighborhoods

t-distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear method that excels at preserving local structure. Unlike PCA, it does not seek directions of maximum variance. Instead, it tries to place cells that are close in the high-dimensional space close together in 2D.

The algorithm:

1. Compute pairwise similarities in high-D using Gaussian kernels
2. Define pairwise similarities in 2D using a Student's t-distribution
3. Minimize the KL divergence between the two distributions

The t-distribution in step 2 (hence the name) has heavy tails, which helps prevent the "crowding problem" — in low dimensions, moderately distant points would all pile up in a narrow shell if Gaussian kernels were used.

```
# Run t-SNE on the PCA-reduced data (not on raw expression!)
# This is critical: t-SNE on 20,000 raw genes is slow and noisy
# Standard practice: PCA to 30-50 dims, then t-SNE on PCs

let pc_input = pca_result.components |> map(|row| row |> head(30))
let tsne_default = tsne(pc_input, perplexity: 30)

let tsne_table = table(tsne_default |> map(|row| {
  tSNE1: row[0],
  tSNE2: row[1]
}))

scatter(tsne_table, "tSNE1", "tSNE2",
  title: "PBMC 3k — t-SNE (perplexity=30)",
  x_label: "t-SNE 1",
  y_label: "t-SNE 2")
  |> save_plot("pbmc_tsne_default.svg")
```

The Perplexity Parameter: When t-SNE Lies

The perplexity parameter controls how many neighbors each point considers. It roughly corresponds to the expected number of neighbors. Low perplexity emphasizes very local structure; high perplexity captures more global patterns.

This is where t-SNE lies. Different perplexity values produce dramatically different plots. Cluster sizes and distances between clusters in a t-SNE plot are NOT meaningful. A cluster that appears twice as large as another may contain the same number of cells. Two clusters that appear far apart may be more similar than two clusters that appear close.

```
# Run t-SNE with different perplexity values to see the effect
let perplexities = [5, 15, 30, 50, 100]

perplexities |> each(|p|
  let result = tsne(pc_input, perplexity: p)
  let result_table = table(result |> map(|row| {
    tSNE1: row[0],
    tSNE2: row[1]
  }))

  scatter(result_table, "tSNE1", "tSNE2",
    title: "t-SNE perplexity=" + str(p),
    x_label: "t-SNE 1",
    y_label: "t-SNE 2")
    |> save_plot("tsne_perplexity_" + str(p) + ".svg")

  println("Perplexity " + str(p) + " complete")
)

# With perplexity=5: many tiny, tight clusters — overemphasis on local
structure
# With perplexity=30: balanced view, standard choice
# With perplexity=100: clusters merge, global structure dominates
# ALL of these are valid — which is why t-SNE is dangerous for quantitative
claims
```

UMAP: The Modern Alternative

Uniform Manifold Approximation and Projection (UMAP) has largely replaced t-SNE as the default visualization method in single-cell genomics. It is faster, better preserves global structure, and produces more reproducible layouts.

UMAP constructs a topological representation of the high-dimensional data using simplicial complexes, then finds a low-dimensional layout that preserves this topological structure. In practice, this means UMAP better preserves the relationships between clusters — clusters that are biologically related tend to appear near each other.

```
# Run UMAP on the PCA-reduced data
let umap_result = umap(pc_input, n_neighbors: 15, min_dist: 0.5)

let umap_table = table(umap_result |> map(|row| {
  UMAP1: row[0],
  UMAP2: row[1]
}))

scatter(umap_table, "UMAP1", "UMAP2",
  title: "PBMC 3k — UMAP",
  x_label: "UMAP 1",
  y_label: "UMAP 2")
  |> save_plot("pbmc_umap.svg")
```

UMAP Parameters: n_neighbors and min_dist

Like t-SNE's perplexity, UMAP has parameters that affect the visualization. Understanding them prevents misinterpretation.

n_neighbors controls how many neighbors define the local structure. Low values (5-15) emphasize fine local structure. High values (50-200) capture broader patterns. Default: 15.

min_dist controls how tightly points are packed. Low values (0.0-0.1) produce tighter clusters with more empty space between them. High values (0.5-1.0) produce a more uniform spread. Default: 0.1.

```
# Compare UMAP with different parameter settings
let umap_configs = [
  { nn: 5,  md: 0.1, label: "local-tight" },
  { nn: 15, md: 0.1, label: "default-tight" },
  { nn: 15, md: 0.5, label: "default-spread" },
  { nn: 50, md: 0.1, label: "global-tight" },
  { nn: 50, md: 0.5, label: "global-spread" }
]

umap_configs |> each(|cfg|
  let result = umap(pc_input, n_neighbors: cfg.nn, min_dist: cfg.md)
  let result_table = table(result |> map(|row| {
    UMAP1: row[0],
    UMAP2: row[1]
  }))

  scatter(result_table, "UMAP1", "UMAP2",
    title: "UMAP n_neighbors=" + str(cfg.nn) + " min_dist=" + str(cfg.md),
    x_label: "UMAP 1",
    y_label: "UMAP 2")
    |> save_plot("umap_" + cfg.label + ".svg")
)

# Key observation: all of these show the same major clusters
# but with very different visual appearance
# UMAP is more robust than t-SNE to parameter choice, but still varies
```

PCA vs t-SNE vs UMAP: When to Use Each

Each method has strengths and appropriate use cases:

Method	Preserves	Speed	Reproducible	Best For
PCA	Global variance structure	Fast	Yes (deterministic)	QC, batch detection, pre-processing

Method	Preserves	Speed	Reproducible	Best For
t-SNE	Local neighborhoods	Slow	No (stochastic)	Publication figures with clear clusters
UMAP	Local + some global	Fast	Mostly (seed-dependent)	Exploratory analysis, large datasets

Use PCA when: you need quantitative interpretation (loadings, variance explained), you are checking for batch effects, you need a deterministic result, or you are preparing input for downstream methods.

Use t-SNE when: you want to visualize clearly separated clusters in publication figures, and you understand that distances between clusters are not meaningful.

Use UMAP when: you are doing exploratory analysis, working with large datasets (>100k cells), or need to preserve relationships between clusters.

```
# Side-by-side comparison: color cells by a known marker gene

# Extract marker gene expression
let cd3d_expr = col_values(log_normalized, "CD3D")   # T cell marker
let cd14_expr = col_values(log_normalized, "CD14")    # Monocyte marker
let ms4a1_expr = col_values(log_normalized, "MS4A1")  # B cell marker (CD20)

# Create combined table with all three embeddings
let combined = table(pc_coords |> map(|row|
  let i = 0
  let result = {
    PC1: row[0],
    PC2: row[1],
    tSNE1: tsne_default[i][0],
    tSNE2: tsne_default[i][1],
    UMAP1: umap_result[i][0],
    UMAP2: umap_result[i][1],
    CD3D: cd3d_expr[i],
    CD14: cd14_expr[i],
    MS4A1: ms4a1_expr[i]
  }
  i = i + 1
  result
))

# In all three views, T cells (CD3D+), monocytes (CD14+), and B cells
(MS4A1+)
# should form distinct groups
# PCA shows them as broad clouds with some overlap
# t-SNE shows them as tight, well-separated clusters
# UMAP shows them as connected but distinct islands
```

PCA on GTEx: Tissues Separate by Biology

Now we apply PCA to the GTEx bulk RNA-seq data. This is a fundamentally different use case — instead of thousands of cells, we have tens of tissues, each summarized by a single median expression vector.

```
# Prepare GTEx data for PCA
# Columns are tissues, rows are genes
# We need to transpose: make tissues the observations

let tissue_names = columns(gtex) |> filter(|c| c != "Name" and c !=
"Description")

println("GTEx tissues: " + str(len(tissue_names)))

# Select the top 5000 most variable genes across tissues
let gene_vars = []
let gene_ids = col_values(gtex, "Name")
let gene_symbols = col_values(gtex, "Description")

tissue_names |> head(1) |> each(|t|
  println("Example tissue column: " + t)
)

# Compute variance across tissues for each gene
let var_per_gene = gene_ids |> map(|gid|
  let row_idx = 0  # would need to find the row for this gene
  let values = tissue_names |> map(|t| col_values(gtex, t)[row_idx])
  { gene: gid, variance: variance(values) }
)

# Select top 5000 most variable genes
let top_genes = var_per_gene |> sort(|a, b| b.variance - a.variance) |>
head(5000)
let selected_gene_ids = top_genes |> map(|g| g.gene)

# Build tissue x gene matrix
let tissue_data = tissue_names |> map(|t|
  let values = col_values(gtex, t)
  let result = { tissue: t }
  selected_gene_ids |> each(|gid|
    let idx = 0  # index of this gene
    result[gid] = log2(values[idx] + 1.0)
  )
  result
)

let tissue_table = table(tissue_data)

# Run PCA
let gene_cols = selected_gene_ids
let tissue_expr = tissue_table |> select(gene_cols)
let tissue_normalized = normalize(tissue_expr, "zscore")
let tissue_pca = pca(tissue_normalized, n_components: 10)

# Plot PC1 vs PC2 with tissue labels
let tissue_pc_table = table(tissue_names |> map(|t|
  let i = 0
  { tissue: t, PC1: tissue_pca.components[i][0], PC2:
tissue_pca.components[i][1] }
))

scatter(tissue_pc_table, "PC1", "PC2",
  title: "GTEx — PCA of Tissue Expression Profiles",
  x_label: "PC1",
  y_label: "PC2")
  |> save_plot("gtex_tissue_pca.svg")

println("GTEx PCA complete")
```

```
    println("PC1 likely separates brain tissues from all others")
    println("PC2 likely separates testis and other reproductive tissues")
```

In the GTEx data, PCA reveals beautiful biological structure:

- Brain tissues cluster tightly together, separated from all other tissues on PC1
- Testis is an extreme outlier on PC2 (unique transcriptional program)
- Blood and immune tissues cluster together
- Muscle tissues (skeletal, cardiac) cluster together
- Liver stands somewhat alone (unique metabolic program)

This is PCA working as a biological discovery tool. Nobody told the algorithm which tissues are similar — it recovered known biology purely from expression patterns.

PCA for Batch Effect Detection

One of the most critical applications of PCA in genomics is detecting batch effects. If your PC1 separates by sequencing lane rather than by biological condition, you have a problem that no downstream test can fix.

```
# Simulated batch effect detection example
# Suppose we have expression data with sample metadata

let expr_data = read_tsv("expression_with_batches.tsv")
let metadata = read_tsv("sample_metadata.tsv")

# Run PCA on expression
let expr_cols = columns(expr_data) |> filter(|c| c != "sample_id")
let expr_matrix = expr_data |> select(expr_cols)
let expr_scaled = normalize(expr_matrix, "zscore")
let batch_pca = pca(expr_scaled, n_components: 10)

# Add metadata to PC coordinates
let pc_with_meta = table(metadata |> map(|m|
  let i = 0
  {
    sample: m.sample_id,
    condition: m.condition,      # biological variable of interest
    batch: m.sequencing_batch,   # technical variable
    PC1: batch_pca.components[i][0],
    PC2: batch_pca.components[i][1]
  }
))

# Test: does PC1 correlate with batch or condition?
let pc1_values = col_values(pc_with_meta, "PC1")

# Group PC1 by condition
let cond_groups = pc_with_meta |> group_by("condition")
let cond_a = cond_groups |> filter(|g| g.key == "treatment") |> map(|g|
g.PC1)
let cond_b = cond_groups |> filter(|g| g.key == "control") |> map(|g| g.PC1)
let cond_test = ttest(cond_a, cond_b)

# Group PC1 by batch
let batch_groups = pc_with_meta |> group_by("batch")
let batch_1 = batch_groups |> filter(|g| g.key == "batch1") |> map(|g| g.PC1)
let batch_2 = batch_groups |> filter(|g| g.key == "batch2") |> map(|g| g.PC1)
let batch_test = ttest(batch_1, batch_2)

println("PC1 association with condition: p=" + str(round(cond_test.pvalue,
4)))
println("PC1 association with batch: p=" + str(round(batch_test.pvalue, 4)))

# If batch_test.pvalue << cond_test.pvalue, you have a batch effect
# The data must be corrected before differential expression
```

PCA as Pre-processing: The Standard Pipeline

In single-cell genomics, PCA is not the final step. It is the pre-processing step before clustering, UMAP visualization, and trajectory inference. The standard pipeline is:

1. Normalize and log-transform
2. Select highly variable genes
3. Scale (z-score)
4. PCA to 30-50 dimensions

5. Use PCs (not raw genes) for all downstream analysis

This is because: (a) PCA denoises the data by discarding low-variance components, (b) distances in PC space are more meaningful than in gene space, (c) it is computationally efficient — downstream algorithms work on 30-50 dimensions instead of 20,000.

```
# The standard single-cell pipeline in BioLang

# Step 1-3: Already done above (normalize, log, HVG selection, scaling)

# Step 4: PCA to 30 dimensions
let n_pcs = 30
let final_pca = pca(scaled_data, n_components: n_pcs)
let pc_embedding = final_pca.components |> map(|row| row |> head(n_pcs))

println("PC embedding: " + str(len(pc_embedding)) + " cells x " +
        str(n_pcs) + " dimensions")

# Step 5a: Cluster in PC space (covered in Chapter 14)
let clusters = leiden(pc_embedding, resolution: 1.0)
println("Found " + str(len(unique(clusters))) + " clusters")

# Step 5b: UMAP for visualization, using PC embedding as input
let final_umap = umap(pc_embedding, n_neighbors: 15, min_dist: 0.1)

# Combine everything
let cell_data = table(pbmc |> map(|row|
  let i = 0
  {
    barcode: row.barcode,
    cluster: clusters[i],
    UMAP1: final_umap[i][0],
    UMAP2: final_umap[i][1],
    PC1: pc_embedding[i][0],
    PC2: pc_embedding[i][1]
  }
))

# Plot UMAP colored by cluster
scatter(cell_data, "UMAP1", "UMAP2",
  color: "cluster",
  title: "PBMC 3k — UMAP colored by Leiden cluster",
  x_label: "UMAP 1",
  y_label: "UMAP 2")
  |> save_plot("pbmc_umap_clusters.svg")

# Count cells per cluster
let cluster_counts = cell_data |> count_by("cluster")
println("Cells per cluster:")
cluster_counts |> each(|c|
  println("  Cluster " + str(c.cluster) + ": " + str(c.n) + " cells")
)
```

Annotating PCs with Known Biology

Once we have clusters and PC embeddings, we can ask: which genes define each cluster? And do the PCs correspond to known biological programs?

```
# For each cluster, find the top marker genes
# A marker gene is one that is highly expressed in one cluster vs all others

let cluster_ids = unique(clusters) |> sort(|a, b| a - b)

cluster_ids |> each(|cid|
  println("Cluster " + str(cid) + " top markers:")

  # Get cell indices in this cluster vs all others
  let in_cluster = []
  let out_cluster = []
  let i = 0
  clusters |> each(|c|
    if c == cid then
      in_cluster = concat(in_cluster, [i])
    else
      out_cluster = concat(out_cluster, [i])
    end
    i = i + 1
  )

  # Test each HVG for differential expression
  let markers = hvgs |> head(500) |> map(|g|
    let vals = col_values(log_normalized, g)
    let in_vals = in_cluster |> map(|idx| vals[idx])
    let out_vals = out_cluster |> map(|idx| vals[idx])
    let test = wilcoxon(in_vals, out_vals)
    {
      gene: g,
      pvalue: test.pvalue,
      mean_in: mean(in_vals),
      mean_out: mean(out_vals),
      log_fc: mean(in_vals) - mean(out_vals)
    }
  )

  # Sort by p-value and show top 5
  let top_markers = markers
    |> filter(|m| m.log_fc > 0)  # upregulated in this cluster
    |> sort(|a, b| a.pvalue - b.pvalue)
    |> head(5)

  top_markers |> each(|m|
    println("  " + m.gene + ": logFC=" + str(round(m.log_fc, 2)) +
            " p=" + str(round(m.pvalue, 6)))
  )
)

# Expected markers in PBMC data:
# T cells:     CD3D, CD3E, IL7R (CD4+), CD8A, CD8B (CD8+)
# Monocytes:   LYZ, S100A8, S100A9, CD14
# B cells:     MS4A1 (CD20), CD79A, CD79B
# NK cells:    NKG7, GNLY, GZMB
# Dendritic:   FCER1A, CST3
# Platelets:   PPBP, PF4
```

The Eigenvalue Decomposition in Detail

To fully understand what PCA gives you, let us look at the eigenvalue decomposition directly.

```
# Compute the correlation matrix of the HVGs
let cor_mat = cor_matrix(scaled_data |> select(hvgs |> head(100)))

# The eigenvalues of the correlation matrix equal the variance
# explained by each PC (when data is scaled)
let eigenvals = eigenvalues(cor_mat)

println("Top 10 eigenvalues of the correlation matrix:")
eigenvals |> head(10) |> each(|e|
  println("  " + str(round(e, 4)))
)

# Eigenvalue > 1 means the PC explains more variance than any single gene
# This is the Kaiser criterion: keep PCs with eigenvalue > 1
let kaiser_count = eigenvals |> filter(|e| e > 1.0) |> len()
println("PCs with eigenvalue > 1 (Kaiser criterion): " + str(kaiser_count))

# Warning: the Kaiser criterion is a rough heuristic
# In genomics with thousands of genes, it often suggests too many PCs
# The elbow plot is more reliable for biological data
```

Dimensionality Reduction on Clinical Data

PCA is not just for expression data. In clinical genomics, you often have a mix of continuous and categorical variables — age, BMI, lab values, mutation status, copy number — and want to see if patients cluster into meaningful groups.

```
# Clinical data: tumor samples with mixed measurements
let clinical = read_tsv("clinical_data.tsv")

# Select numeric columns for PCA
let numeric_cols = ["age", "tumor_size", "ki67_pct", "mutation_burden",
                    "ctdna_fraction", "wbc_count", "albumin", "ldh"]

let clinical_numeric = clinical |> select(numeric_cols)
let clinical_scaled = normalize(clinical_numeric, "zscore")

# Handle missing values
let clinical_clean = drop_null(clinical_scaled)
println("Samples after dropping nulls: " + str(len(clinical_clean)))

# PCA on clinical variables
let clin_pca = pca(clinical_clean, n_components: 5)

# What clinical features drive PC1?
let clin_loadings = numeric_cols |> map(|col|
  let i = 0
  { feature: col, pc1_loading: clin_pca.loadings[i][0], pc2_loading:
clin_pca.loadings[i][1] }
)

println("Clinical feature loadings:")
clin_loadings |> sort(|a, b| abs(b.pc1_loading) - abs(a.pc1_loading)) |>
each(|l|
  println("  " + l.feature + ": PC1=" + str(round(l.pc1_loading, 3)) +
          " PC2=" + str(round(l.pc2_loading, 3)))
)

# Visualize clinical PCA
let clin_pc_table = table(clinical |> map(|row|
  let i = 0
  {
    sample: row.sample_id,
    stage: row.stage,
    PC1: clin_pca.components[i][0],
    PC2: clin_pca.components[i][1]
  }
))

scatter(clin_pc_table, "PC1", "PC2",
  color: "stage",
  title: "Clinical Data PCA — colored by tumor stage",
  x_label: "PC1",
  y_label: "PC2")
  |> save_plot("clinical_pca.svg")
```

Interpretation

What PCA Tells You (and What It Doesn’t)

PCA tells you:

- **How much of the variance is low-dimensional.** If 80% of variance is in the first 5 PCs, the data has strong structure.
- **Which genes co-vary.** The loadings reveal gene programs — groups of genes that vary together across cells or samples.
- **Whether your data has batch effects.** If technical variables correlate with the top PCs, you have a problem.
- **How many distinct groups exist.** The number of clear elbows in the scree plot gives a rough estimate.

PCA does NOT tell you:

- **What the groups are.** PCA is unsupervised — it does not use labels.
- **Whether nonlinear relationships exist.** PCA is strictly linear.
- **Whether small but important subpopulations exist.** PCA prioritizes large-variance directions, which may miss rare cell types.

What t-SNE and UMAP Tell You (and What They Don't)

t-SNE and UMAP tell you:

- **Whether distinct clusters exist.** Points that cluster together are genuinely similar in the original space.
- **Local neighborhood structure.** Nearby points in the plot are truly nearby in the original space.

They do NOT tell you:

- **Distances between clusters.** The spacing is an artifact of the algorithm.
- **Cluster sizes.** A large visual cluster may contain fewer cells than a small one.
- **Continuous gradients.** Both methods discretize continuous variation into apparent clusters.
- **Statistical significance.** Clusters in a UMAP plot are not automatically statistically significant cell types.

```
# Demonstrate that UMAP distances are not quantitative
# Compute actual distances between cluster centroids in PC space
# vs apparent distances in UMAP space

let centroids_pc = cluster_ids |> map(|cid|
  let mask = clusters |> map(|c| c == cid)
  let pc_vals = pc_embedding |> filter(|row| mask[0])  # simplified
  # In practice: average PC coordinates for cells in this cluster
  { cluster: cid, type: "PC_space" }
)

# Key point: the rank order of inter-cluster distances in PC space
# may not match the rank order in UMAP space
# Two clusters that are far apart in UMAP may be closer in PC space
# than two clusters that appear adjacent in UMAP
println("Warning: UMAP distances are NOT quantitative measures of
similarity")
println("Always verify apparent relationships by checking PC space
distances")
println("or by computing marker gene overlap between clusters")
```

Decision

Choosing Your Approach

Given your specific problem, here is the decision tree:

Exploratory analysis of single-cell data: PCA (30 PCs) -> UMAP -> Leiden clustering. This is the standard pipeline. Do not skip PCA.

Quality control of bulk RNA-seq: PCA only. Plot PC1 vs PC2. Color by biological condition AND by all technical variables (batch, lane, extraction date, operator). If any technical variable explains more variance than your biological question, correct the batch effect before proceeding.

Publication figure: Both PCA and UMAP/t-SNE. Show PCA for quantitative claims (variance explained, loadings). Show UMAP/t-SNE for visual clarity. Report the parameters you used.

Discovering gene programs: PCA loadings analysis. The genes with high absolute loadings on a PC form a biological program. Run gene ontology enrichment on these genes to interpret the program.

```
# Gene program discovery from PC loadings

# Get top 50 genes for PC1 (positive loadings)
let pc1_pos = pc1_sorted
  |> filter(|g| g.loading > 0)
  |> head(50)
  |> map(|g| g.gene)

# Get top 50 genes for PC1 (negative loadings)
let pc1_neg = pc1_sorted
  |> filter(|g| g.loading < 0)
  |> head(50)
  |> map(|g| g.gene)

println("PC1 positive gene program (myeloid):")
println("  " + str(pc1_pos |> head(10)))

println("PC1 negative gene program (lymphoid):")
println("  " + str(pc1_neg |> head(10)))

# You would run these through gene ontology enrichment to confirm:
# pc1_pos should enrich for innate immunity, phagocytosis
# pc1_neg should enrich for adaptive immunity, T cell receptor signaling
```

Reporting Dimensionality Reduction Results

When reporting dimensionality reduction in a paper or presentation:

1. **State the preprocessing**: normalization, gene selection, scaling
2. **State the number of PCs used** and how you chose it (elbow, cumulative variance threshold, or formal test)
3. **Show the scree plot** in supplementary materials
4. **For UMAP/t-SNE**: report all parameters (perplexity, n_neighbors, min_dist, random seed, number of iterations)
5. **Do not make distance claims** from UMAP/t-SNE plots
6. **Show PC loadings** when claiming biological programs

```
# Summary statistics for methods section

let var_explained_pct = pca_result.variance |> head(n_pcs) |> map(|v| 100.0 *
v / total_var)
let total_captured = sum(var_explained_pct)

println("=== Methods Section Statistics ===")
println("Expression matrix: " + str(len(pbmc)) + " cells x " + str(len(hvgs))
+ " HVGs")
println("Normalization: library-size to 10,000, log1p, z-score per gene")
println("PCA: " + str(n_pcs) + " components retained (" +
        str(round(total_captured, 1)) + "% total variance)")
println("UMAP: n_neighbors=15, min_dist=0.1, metric=euclidean")
println("Clustering: Leiden, resolution=1.0")
println("Clusters identified: " + str(len(unique(clusters))))
```

Pitfalls

Pitfall 1: Running t-SNE/UMAP on Raw Gene Expression

Never run t-SNE or UMAP directly on the raw gene expression matrix. The noise in 20,000 genes drowns out the signal. Always reduce to 30-50 PCs first. This denoising step is essential.

```
# DO NOT do this:
# let bad_tsne = tsne(raw_expression_matrix)

# DO this instead:
# let pca_reduced = pca(scaled_data, n_components: 30)
# let good_tsne = tsne(pca_reduced.components)

println("Always run PCA before t-SNE or UMAP")
println("PCA denoises the data by discarding low-variance components")
println("Without this step, nonlinear methods amplify noise")
```

Pitfall 2: Over-Interpreting t-SNE Cluster Separation

A common mistake is to claim that two cell types are “highly distinct” because they appear well-separated in a t-SNE plot. The visual separation is an artifact of the algorithm’s optimization. Two groups with 10% gene expression overlap can look as separated as two groups with 0% overlap.

Pitfall 3: Ignoring the Elbow Plot

Choosing too few PCs discards biological signal. Choosing too many includes noise dimensions that degrade clustering. Spend time on the scree plot.

```
# How the number of PCs affects clustering

let pc_counts = [5, 10, 15, 20, 30, 50]
pc_counts |> each(|n|
  let embedding = final_pca.components |> map(|row| row |> head(n))
  let cl = leiden(embedding, resolution: 1.0)
  let n_clusters = len(unique(cl))
  println("PCs=" + str(n) + " -> " + str(n_clusters) + " clusters")
)

# Too few PCs (5): may merge distinct cell types
# Too many PCs (50): may split cell types by noise
# Sweet spot: where the scree plot elbows (typically 10-30 for single-cell)
```

Pitfall 4: Not Checking for Confounders in PCA

If PC1 correlates with your variable of interest AND with a technical confounder, you cannot tell which one is driving the pattern. Always test PCs against all known technical variables.

```
# Check all PCs against technical variables
let technical_vars = ["sequencing_batch", "library_prep_date",
                      "rna_integrity_number", "percent_mito"]

println("Testing PCs against technical variables:")
technical_vars |> each(|var|
  # For continuous variables, use correlation
  # For categorical variables, use ANOVA
  println("  " + var + ": [test result here]")
)
```

Pitfall 5: Treating UMAP/t-SNE as Deterministic

Both t-SNE and UMAP are stochastic algorithms. Running them twice with different random seeds produces different layouts. If a biological claim depends on the specific layout of a UMAP plot, it is not robust.

```
# Demonstrate stochasticity
let umap_run1 = umap(pc_input, n_neighbors: 15, min_dist: 0.1)
let umap_run2 = umap(pc_input, n_neighbors: 15, min_dist: 0.1)

# The overall cluster structure should be the same
# But the rotation, reflection, and exact positions will differ
# If your conclusion changes between runs, it is not robust
println("UMAP is stochastic — always verify that conclusions are")
println("consistent across multiple random seeds")
```

Pitfall 6: PCA on Non-Centered Data

PCA assumes the data is centered (mean-subtracted). If you skip centering, PC1 will point toward the mean of the data, wasting your most informative component on a trivial direction.

```
# Centering matters
let uncentered = log_normalized |> select(hvgs)
let centered = normalize(uncentered, "zscore")

let pca_uncentered = pca(uncentered, n_components: 5)
let pca_centered = pca(centered, n_components: 5)

# Compare variance explained
println("Uncentered PCA — PC1 variance: " +
        str(round(100.0 * pca_uncentered.variance[0] /
sum(pca_uncentered.variance), 1)) + "%")
println("Centered PCA — PC1 variance: " +
        str(round(100.0 * pca_centered.variance[0] /
sum(pca_centered.variance), 1)) + "%")

# The uncentered PC1 will explain a huge amount of variance
# but it is trivially pointing toward the mean expression level
# The centered PC1 captures actual biological variation
```

Where Next?

Dimensionality reduction opens the door to several downstream analyses:

Chapter 14: Clustering & Classification — Once you have a PCA embedding, you can cluster cells or samples into groups. Leiden clustering on PCA coordinates is the standard approach in single-cell genomics. K-means on PCA is used for tumor subtyping.

Trajectory inference — If your cells represent a developmental process (stem cell differentiation, immune activation), dimensionality reduction reveals continuous trajectories rather than discrete clusters. Methods like diffusion maps and pseudotime ordering build on PCA/UMAP embeddings.

Data integration — When combining datasets from different labs, technologies, or species, PCA-based methods like Harmony, Scanorama, and BBKNN align the PC spaces across datasets to enable joint analysis.

Gene regulatory network inference — The co-variation structure captured by PCA can be used to infer which transcription factors regulate which genes. Genes with similar PC loading profiles are likely co-regulated.

Transfer learning — A PCA model trained on a large reference dataset (like the Human Cell Atlas) can be used to project new datasets into the same coordinate system. This enables automated cell type annotation by finding the nearest reference cell type in PC space.

```
# Example: project a new dataset into the PBMC reference PCA space
# This is the basis of automated cell type annotation

# Suppose we have a new sample
let new_sample = read_tsv("new_pbmc_sample.tsv")

# Normalize the same way as the reference
let new_normalized = normalize(new_sample |> select(hvgs), "zscore")

# Project into reference PC space using the saved loadings
# new_PCs = new_data %*% reference_loadings
let new_projected = new_normalized |> map(|row|
  let pc_coords = []
  let pc_idx = 0
  while pc_idx < n_pcs do
    let score = hvgs |> map(|g|
      let gene_idx = 0
      row[g] * pca_result.loadings[gene_idx][pc_idx]
    ) |> sum()
    pc_coords = concat(pc_coords, [score])
    pc_idx = pc_idx + 1
  end
  pc_coords
)

# Now new_projected is in the same coordinate system as the reference
# You can classify cells by nearest-neighbor in this space
println("Projected " + str(len(new_projected)) + " new cells into reference
PC space")
println("Ready for automated cell type annotation")
```

Summary

Concept	Key Takeaway
Curse of dimensionality	Distances become meaningless in high-D; reduce first
PCA	Linear, deterministic, quantitative; use for QC and preprocessing
Scree plot	Choose PCs at the elbow; typical range 10-30 for single-cell
PC loadings	Reveal gene programs; have biological interpretation
t-SNE	Good local structure; distances between clusters are meaningless
UMAP	Better global structure than t-SNE; faster; standard for single-cell
Perplexity/n_neighbors	Parameters matter; always try multiple values

Concept	Key Takeaway
Pre-processing	Normalize, log-transform, select HVGs, scale, THEN reduce
Batch effects	Check top PCs against technical variables before proceeding
Stochasticity	t-SNE and UMAP are non-deterministic; verify robustness

Dimensionality reduction is the bridge between raw high-dimensional measurements and biological understanding. It is not the destination — it is the foundation on which clustering, trajectory analysis, and biological interpretation are built. Master PCA first, understand its linearity limitations, then use UMAP/t-SNE for visualization while respecting their constraints.

Chapter 14: Clustering & Classification

The Problem

You have gene expression profiles from 1,097 breast cancer tumors in The Cancer Genome Atlas (TCGA). Clinicians treat breast cancer differently depending on the molecular subtype — Luminal A, Luminal B, HER2-enriched, Basal-like, and Normal-like. But these subtypes were originally discovered by unsupervised clustering of expression data. The question is: if you cluster the tumors without knowing the subtypes, do the clusters match the known biology?

More broadly: given a matrix of measurements (gene expression, protein levels, methylation, mutation profiles), how do you discover natural groups in the data? How do you decide how many groups exist? And how do you avoid the dangerous trap of finding patterns in noise?

This chapter covers unsupervised clustering — the art of discovering structure that you did not know existed. It also covers the critical discipline of validating that the structure is real.

Why It Matters

Clustering is not just an exploratory exercise. In genomics, clusters become clinical categories that determine treatment:

Breast cancer subtypes. The PAM50 classifier assigns each tumor to one of five molecular subtypes. Luminal A tumors have excellent prognosis and may avoid chemotherapy. Basal-like tumors are aggressive and require intensive treatment. This classification, which saves lives, was discovered by clustering gene expression profiles.

Single-cell atlas construction. Every cell atlas relies on clustering to define cell types. The Human Cell Atlas, which aims to map every cell in the human body, is fundamentally a clustering project. Each cluster becomes a cell type with a name, a marker gene signature, and a place in the developmental hierarchy.

Cancer of unknown primary (CUP). When a metastatic tumor is found but the primary site is unknown, expression-based classifiers assign it to the most likely

tissue of origin. These classifiers use clustering — the unknown tumor is assigned to the cluster of known tumors it most resembles.

Drug response prediction. Clustering patients by multi-omic profiles can reveal subgroups with different drug responses. The I-SPY 2 adaptive clinical trial uses expression-based clustering to match patients to experimental therapies.

The Data

TCGA Breast Cancer (BRCA) expression data

URL: https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-BRCA.htseq_fpkm.tsv.gz

This file contains FPKM-normalized gene expression for all TCGA-BRCA samples. Rows are genes (Ensembl IDs), columns are samples.

TCGA BRCA molecular subtypes

URL: https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-BRCA.molecular_subtype.tsv

This file contains the PAM50 molecular subtype classification for each sample.

```
# Download TCGA BRCA expression data
let expr_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.htseq_fpkm.tsv.gz"
let expr_file = download(expr_url, "tcga_brca_expr.tsv.gz")

let brca_expr = read_tsv(expr_file)
println("Expression matrix:")
println("  Genes: " + str(len(brca_expr)))
println("  Samples: " + str(ncols(brca_expr) - 1))

# Download molecular subtype labels
let subtype_url = "https://gdc-hub.s3.us-east-1.amazonaws.com/download/TCGA-
BRCA.molecular_subtype.tsv"
let subtype_file = download(subtype_url, "tcga_brca_subtypes.tsv")

let subtypes = read_tsv(subtype_file)
println("Subtype annotations: " + str(len(subtypes)) + " samples")

# Check subtype distribution
let subtype_counts = subtypes |> count_by("subtype")
println("Subtype distribution:")
subtype_counts |> each(|s|
  println("  " + s.subtype + ": " + str(s.n))
)
```

Exploration

Preparing the Data for Clustering

Before clustering, we must prepare the data the same way we did for PCA in Chapter 13: filter low-expression genes, log-transform, select the most variable features, and scale.

```
# Get sample IDs (column names, excluding the gene ID column)
let sample_ids = columns(brca_expr) |> filter(|c| c != "Ensembl_ID")
println("Total samples: " + str(len(sample_ids)))

# Filter genes: keep only those expressed in at least 10% of samples
# and with mean FPKM > 1
let gene_ids = col_values(brca_expr, "Ensembl_ID")

let gene_stats = gene_ids |> map(|g|
  let row_vals = sample_ids |> map(|s| col_values(brca_expr, s)[0])  #
simplified
  let expressed = row_vals |> filter(|v| v > 0) |> len()
  let pct_expressed = 100.0 * expressed / len(sample_ids)
  let mean_expr = mean(row_vals)
  { gene: g, pct_expressed: pct_expressed, mean_expr: mean_expr }
)

let expressed_genes = gene_stats
  |> filter(|g| g.pct_expressed > 10.0 and g.mean_expr > 1.0)
  |> map(|g| g.gene)

println("Genes after expression filter: " + str(len(expressed_genes)))

# Log2-transform (FPKM + 1)
# Then select top 5000 most variable genes
# Then z-score normalize across samples

# Build a samples x genes matrix (transpose the data)
let sample_data = sample_ids |> map(|s|
  let vals = col_values(brca_expr, s)
  let result = { sample: s }
  expressed_genes |> head(5000) |> each(|g|
    result[g] = log2(vals[0] + 1.0)  # simplified indexing
  )
  result
)

let sample_table = table(sample_data)
```

Selecting Features: The Most Variable Genes

For clustering, feature selection is critical. Too many features add noise. Too few may miss the signal. The standard approach is to select the most variable genes.

```
# Compute variance for each gene across samples
let gene_list = expressed_genes |> head(5000)
let gene_var = gene_list |> map(|g|
  let values = col_values(sample_table, g)
  { gene: g, variance: variance(values), mean: mean(values) }
)

# Select top 2000 most variable genes
let top_genes = gene_var
  |> sort(|a, b| b.variance - a.variance)
  |> head(2000)
  |> map(|g| g.gene)

println("Selected " + str(len(top_genes)) + " most variable genes for
clustering")

# Show the top 10
println("Top 10 most variable genes:")
gene_var |> sort(|a, b| b.variance - a.variance) |> head(10) |> each(|g|
  println("  " + g.gene + ": var=" + str(round(g.variance, 3)) +
          " mean=" + str(round(g.mean, 3)))
)

# Scale the selected genes (z-score across samples)
let cluster_data = sample_table |> select(top_genes)
let scaled_data = normalize(cluster_data, "zscore")

println("Clustering matrix: " + str(len(scaled_data)) + " samples x " +
        str(ncols(scaled_data)) + " genes")
```

Dimensionality Reduction Before Clustering

As we learned in Chapter 13, clustering in the full gene space is prone to the curse of dimensionality. We first reduce to PCA coordinates.

```
# PCA for denoising and dimensionality reduction
let pca_result = pca(scaled_data, n_components: 30)

# Scree plot to choose number of PCs
let total_var = sum(pca_result.variance)
let var_pct = pca_result.variance |> map(|v| 100.0 * v / total_var)

println("Variance explained by top PCs:")
let cum = 0.0
var_pct |> head(15) |> each(|v|
  let i = 0
  cum = cum + v
  println("  PC" + str(i + 1) + ": " + str(round(v, 2)) +
          "% (cumulative: " + str(round(cum, 1)) + "%)")
  i = i + 1
)

# Use 15 PCs for clustering (typical for bulk RNA-seq)
let n_pcs = 15
let pc_embedding = pca_result.components |> map(|row| row |> head(n_pcs))

println("PC embedding: " + str(len(pc_embedding)) + " samples x " +
str(n_pcs) + " PCs")
```

The Analysis

K-Means Clustering

K-means is the simplest and most widely used clustering algorithm. It partitions n observations into k clusters by minimizing the within-cluster sum of squared distances to the centroid.

The algorithm:

1. Choose k random points as initial centroids
2. Assign each observation to the nearest centroid
3. Recompute centroids as the mean of assigned observations
4. Repeat steps 2-3 until convergence

```
# K-means clustering with k=5 (to match the 5 PAM50 subtypes)
let km5 = kmeans(pc_embedding, k: 5)

# km5 contains:
#   .labels     — cluster assignment for each sample (0-indexed)
#   .centroids  — centroid coordinates

println("K-means (k=5) cluster sizes:")
let km5_counts = km5.labels |> value_counts()
km5_counts |> each(|c|
  println("  Cluster " + str(c.value) + ": " + str(c.count) + " samples")
)
```

Choosing K: The Elbow Method

But how do you know that k=5 is the right choice? What if the data contains 3 clusters, or 7? The elbow method runs k-means for a range of k values and plots the within-cluster sum of squares (WCSS, also called inertia) against k.

```
# Elbow method: run k-means for k=2 to k=10
let k_range = [2, 3, 4, 5, 6, 7, 8, 9, 10]
let elbow_results = k_range |> map(|k|
  let result = kmeans(pc_embedding, k: k)

  # Compute within-cluster sum of squares (WCSS)
  let wcss = 0.0
  let i = 0
  result.labels |> each(|label|
    let centroid = result.centroids[label]
    let point = pc_embedding[i]
    let dist_sq = point |> map(|p|
      let j = 0
      let d = p - centroid[j]
      j = j + 1
      d * d
    ) |> sum()
    wcss = wcss + dist_sq
    i = i + 1
  )

  println("k=" + str(k) + " WCSS=" + str(round(wcss, 1)))
  { k: k, wcss: wcss }
)

# Plot the elbow
let elbow_table = table(elbow_results)
scatter(elbow_table, "k", "wcss",
  title: "K-Means Elbow Plot",
  x_label: "Number of Clusters (k)",
  y_label: "Within-Cluster Sum of Squares")
  |> save_plot("kmeans_elbow.svg")

# The elbow is where the decrease in WCSS slows down sharply
# For TCGA BRCA, expect an elbow around k=4 or k=5
```

Choosing K: Silhouette Analysis

The silhouette score measures how similar each observation is to its own cluster compared to the nearest neighboring cluster. It ranges from -1 (wrong cluster) to +1 (well-matched to its cluster).

```
# Compute silhouette scores for different k values
let silhouette_results = k_range |> map(|k|
  let result = kmeans(pc_embedding, k: k)

  # Compute silhouette for each sample
  let sil_scores = []
  let i = 0
  result.labels |> each(|label|
    let point = pc_embedding[i]

    # a(i) = mean distance to same-cluster points
    let same_dists = []
    let j = 0
    result.labels |> each(|other_label|
      if other_label == label and j != i then
        let d = point |> map(|p|
          let dim = 0
          let diff = p - pc_embedding[j][dim]
          dim = dim + 1
          diff * diff
        ) |> sum() |> sqrt()
        same_dists = concat(same_dists, [d])
      end
      j = j + 1
    )
    let a_i = if len(same_dists) > 0 then mean(same_dists) else 0.0

    # b(i) = min over other clusters of mean distance to that cluster
    let other_clusters = unique(result.labels) |> filter(|c| c != label)
    let b_i = other_clusters |> map(|oc|
      let oc_dists = []
      let jj = 0
      result.labels |> each(|ol|
        if ol == oc then
          let d = point |> map(|p|
            let dim = 0
            let diff = p - pc_embedding[jj][dim]
            dim = dim + 1
            diff * diff
          ) |> sum() |> sqrt()
          oc_dists = concat(oc_dists, [d])
        end
        jj = jj + 1
      )
      mean(oc_dists)
    ) |> min()

    let s_i = if a_i < b_i then (b_i - a_i) / b_i else (b_i - a_i) / a_i
    sil_scores = concat(sil_scores, [s_i])
    i = i + 1
  )

  let mean_sil = mean(sil_scores)
  println("k=" + str(k) + " mean silhouette=" + str(round(mean_sil, 4)))
  { k: k, mean_silhouette: mean_sil }
)

let sil_table = table(silhouette_results)
scatter(sil_table, "k", "mean_silhouette",
  title: "Silhouette Score vs k",
  x_label: "Number of Clusters (k)",
  y_label: "Mean Silhouette Score")
  |> save_plot("silhouette_scores.svg")
```

```
# Higher silhouette = better clustering
# If k=4 and k=5 have similar scores, biological knowledge tips the balance
```

Hierarchical Clustering

Hierarchical clustering builds a tree (dendrogram) of nested clusters. Unlike k-means, it does not require you to specify k in advance. You choose k by cutting the dendrogram at a certain height.

There are two approaches:

- **Agglomerative (bottom-up)**: Start with each point as its own cluster. Merge the two closest clusters at each step. Repeat until one cluster remains.
- **Divisive (top-down)**: Start with all points in one cluster. Split recursively.

The key parameter is the linkage criterion — how you measure the distance between two clusters:

- **Single linkage**: Minimum distance between any pair of points
- **Complete linkage**: Maximum distance between any pair of points
- **Average linkage**: Mean distance between all pairs of points
- **Ward's method**: Minimizes the increase in total within-cluster variance

```
# Compute the distance matrix for hierarchical clustering
let dist_mat = dist_matrix(pc_embedding, "euclidean")

println("Distance matrix: " + str(len(dist_mat)) + " x " +
str(len(dist_mat[0])))

# Descriptive statistics of pairwise distances
let all_dists = []
let i = 0
while i < len(dist_mat) do
  let j = i + 1
  while j < len(dist_mat) do
    all_dists = concat(all_dists, [dist_mat[i][j]])
    j = j + 1
  end
  i = i + 1
end

let dist_stats = {
  mean: mean(all_dists),
  median: median(all_dists),
  sd: stdev(all_dists),
  min: min(all_dists),
  max: max(all_dists)
}

println("Pairwise distance statistics:")
println("  Mean: " + str(round(dist_stats.mean, 2)))
println("  Median: " + str(round(dist_stats.median, 2)))
println("  SD: " + str(round(dist_stats.sd, 2)))
println("  Range: [" + str(round(dist_stats.min, 2)) + ", " +
        str(round(dist_stats.max, 2)) + "]")
```

Heatmaps with Row/Column Clustering

The classic visualization for clustered expression data is a heatmap with dendrograms on both rows (genes) and columns (samples). This simultaneously shows the expression patterns and the cluster structure.

```
# Create a heatmap of the top variable genes across samples
# Use the k=5 clustering to order samples

let km_labels = km5.labels

# Order samples by cluster
let ordered_indices = [0, 1, 2, 3, 4] |> map(|cluster|
  let indices = []
  let i = 0
  km_labels |> each(|l|
    if l == cluster then
      indices = concat(indices, [i])
    end
    i = i + 1
  )
  indices
) |> flatten()

# Select a subset of top variable genes for the heatmap (top 100)
let heatmap_genes = top_genes |> head(100)
let heatmap_data = scaled_data |> select(heatmap_genes)

# Create the heatmap
heatmap(heatmap_data,
  title: "TCGA BRCA — Top 100 Variable Genes",
  x_label: "Samples (ordered by cluster)",
  y_label: "Genes")
  |> save_plot("brca_heatmap.svg")

println("Heatmap saved — samples ordered by k-means cluster")
println("Look for block-diagonal structure: genes that are high in one")
println("cluster and low in others define the cluster identity")
```

Leiden Community Detection

Leiden clustering is the standard method for single-cell RNA-seq. Unlike k-means, it does not require specifying the number of clusters. Instead, it builds a nearest-neighbor graph and finds communities of densely connected nodes.

The algorithm:

1. Build a k-nearest-neighbor (KNN) graph in PC space
2. Optimize the modularity function to find communities
3. The resolution parameter controls the granularity

```
# Leiden clustering at different resolutions
let resolutions = [0.2, 0.5, 0.8, 1.0, 1.5, 2.0]

let leiden_results = resolutions |> map(|res|
  let clusters = leiden(pc_embedding, resolution: res)
  let n_clusters = len(unique(clusters))
  println("Resolution " + str(res) + ": " + str(n_clusters) + " clusters")
  { resolution: res, n_clusters: n_clusters, labels: clusters }
)

# Resolution 0.2: 2-3 large clusters (Luminal vs Basal)
# Resolution 0.5: 4-5 clusters (approaching PAM50)
# Resolution 1.0: 6-8 clusters (may split subtypes further)
# Resolution 2.0: 10+ clusters (over-splitting)

# The resolution controls granularity — there is no single "correct" value
# Choose based on biological interpretability
```

Evaluating Clusters Against Known Subtypes

The TCGA BRCA dataset has a gold standard: the PAM50 molecular subtype classification. We can evaluate our unsupervised clustering against this reference.

```
# Match samples between our clustering and the known subtypes
let our_labels = km5.labels

# Create a contingency table: our clusters x PAM50 subtypes
let known_subtypes = col_values(subtypes, "subtype")
let sample_order = col_values(subtypes, "sample")

# Build the contingency table
let our_clusters = unique(our_labels) |> sort(|a, b| a - b)
let pam50_types = unique(known_subtypes) |> filter(|s| s != "NA")

println("Contingency table: Our clusters vs PAM50 subtypes")
println("")

# Header
let header = "Cluster"
pam50_types |> each(|t|
  header = header + "\t" + t
)
println(header)

# Rows
our_clusters |> each(|cl|
  let row = str(cl)
  pam50_types |> each(|t|
    let count = 0
    let i = 0
    our_labels |> each(|l|
      if l == cl and known_subtypes[i] == t then
        count = count + 1
      end
      i = i + 1
    )
    row = row + "\t" + str(count)
  )
  println(row)
)
```

Adjusted Rand Index

The Rand Index measures agreement between two clusterings. It counts the number of pairs of observations that are either in the same cluster in both clusterings or in different clusters in both. The Adjusted Rand Index (ARI) corrects for chance — an ARI of 0 means agreement no better than random, and 1 means perfect agreement.

```
# Compute the Adjusted Rand Index between our clustering and PAM50

# First, compute the raw Rand Index
let n = len(our_labels)
let a = 0  # pairs in same cluster in both
let b = 0  # pairs in different clusters in both
let c = 0  # same in ours, different in PAM50
let d = 0  # different in ours, same in PAM50

let i = 0
while i < n do
  let j = i + 1
  while j < n do
    let same_ours = our_labels[i] == our_labels[j]
    let same_pam50 = known_subtypes[i] == known_subtypes[j]

    if same_ours and same_pam50 then
      a = a + 1
    end
    if same_ours and !same_pam50 then
      c = c + 1
    end
    if !same_ours and same_pam50 then
      d = d + 1
    end
    if !same_ours and !same_pam50 then
      b = b + 1
    end

    j = j + 1
  end
  i = i + 1
end

let total_pairs = n * (n - 1) / 2
let rand_index = (a + b) / total_pairs

println("Raw Rand Index: " + str(round(rand_index, 4)))

# Adjusted Rand Index (corrects for chance)
# ARI = (RI - Expected_RI) / (Max_RI - Expected_RI)
# For k-means with k=5 on BRCA data, expect ARI around 0.4-0.7
# depending on preprocessing

# The ARI will not be 1.0 because:
# 1. K-means is a simple method — it finds spherical clusters
# 2. PAM50 subtypes have complex, non-spherical boundaries
# 3. Some subtypes (especially Normal-like) are poorly defined
# 4. Our feature selection may not match the PAM50 gene set
```

Comparing Clustering Methods

Different clustering methods find different structures. The right method depends on the shape of the clusters in your data.

```
# Compare k-means, hierarchical, and Leiden clustering

# K-means (already computed)
let km_labels = km5.labels
let km_n = len(unique(km_labels))

# Leiden at resolution chosen to give ~5 clusters
let leiden_labels = leiden(pc_embedding, resolution: 0.5)
let leiden_n = len(unique(leiden_labels))

println("Method comparison:")
println("  K-means (k=5): " + str(km_n) + " clusters")
println("  Leiden (res=0.5): " + str(leiden_n) + " clusters")

# For each method, show the cluster sizes
println("")
println("K-means cluster sizes:")
let km_vc = km_labels |> value_counts()
km_vc |> sort(|a, b| b.count - a.count) |> each(|c|
  println("  Cluster " + str(c.value) + ": " + str(c.count))
)

println("Leiden cluster sizes:")
let lei_vc = leiden_labels |> value_counts()
lei_vc |> sort(|a, b| b.count - a.count) |> each(|c|
  println("  Cluster " + str(c.value) + ": " + str(c.count))
)

# Agreement between methods
# If two methods find the same clusters, the structure is robust
# If they disagree, the data may not have clear clusters
```

Identifying Cluster-Defining Genes

Once clusters are found, the next question is: what genes define each cluster? These are the marker genes or signature genes.

```
# For each cluster, find genes with the highest differential expression
# compared to all other clusters

let cluster_markers = our_clusters |> map(|cl|
  println("Finding markers for cluster " + str(cl) + "...")

  let in_cluster = []
  let out_cluster = []
  let idx = 0
  our_labels |> each(|l|
    if l == cl then
      in_cluster = concat(in_cluster, [idx])
    else
      out_cluster = concat(out_cluster, [idx])
    end
    idx = idx + 1
  )

  # Test top genes for differential expression
  let markers = top_genes |> head(500) |> map(|g|
    let all_vals = col_values(scaled_data, g)
    let in_vals = in_cluster |> map(|i| all_vals[i])
    let out_vals = out_cluster |> map(|i| all_vals[i])

    let test_result = ttest(in_vals, out_vals)
    {
      gene: g,
      pvalue: test_result.pvalue,
      mean_in: mean(in_vals),
      mean_out: mean(out_vals),
      log_fc: mean(in_vals) - mean(out_vals)
    }
  )

  # Top 10 upregulated markers
  let top_markers = markers
    |> filter(|m| m.log_fc > 0)
    |> sort(|a, b| a.pvalue - b.pvalue)
    |> head(10)

  println("  Cluster " + str(cl) + " top markers:")
  top_markers |> each(|m|
    println("    " + m.gene + ": logFC=" + str(round(m.log_fc, 2)) +
            " p=" + str(round(m.pvalue, 6)))
  )

  { cluster: cl, markers: top_markers }
)

# Expected markers for breast cancer subtypes:
# Luminal A/B: ESR1 (estrogen receptor), GATA3, FOXA1
# HER2-enriched: ERBB2, GRB7
# Basal-like: KRT5, KRT17, EGFR
```

Visualization: UMAP Colored by Cluster and Subtype

```
# Create UMAP embedding for visualization
let umap_result = umap(pc_embedding, n_neighbors: 15, min_dist: 0.5)

# Plot colored by our clusters
let umap_cluster_table = table(sample_ids |> map(|s|
  let i = 0
  {
    sample: s,
    cluster: our_labels[i],
    UMAP1: umap_result[i][0],
    UMAP2: umap_result[i][1]
  }
))

scatter(umap_cluster_table, "UMAP1", "UMAP2",
  color: "cluster",
  title: "TCGA BRCA — UMAP colored by K-means cluster",
  x_label: "UMAP 1",
  y_label: "UMAP 2")
  |> save_plot("brca_umap_kmeans.svg")

# Plot colored by PAM50 subtype
let umap_subtype_table = table(sample_ids |> map(|s|
  let i = 0
  {
    sample: s,
    subtype: known_subtypes[i],
    UMAP1: umap_result[i][0],
    UMAP2: umap_result[i][1]
  }
))

scatter(umap_subtype_table, "UMAP1", "UMAP2",
  color: "subtype",
  title: "TCGA BRCA — UMAP colored by PAM50 subtype",
  x_label: "UMAP 1",
  y_label: "UMAP 2")
  |> save_plot("brca_umap_pam50.svg")

# Side-by-side comparison: do the unsupervised clusters
# align with the supervised subtypes?
println("Compare the two UMAP plots:")
println("  If clusters align with subtypes, the biology is strong")
println("  Misaligned regions indicate ambiguous or transitional tumors")
```

The Circular Analysis Trap

This is the single most dangerous pitfall in clustering:

You cluster the data using a set of genes. Then you test whether those same genes are differentially expressed between the clusters. Of course they are — you selected the clusters specifically to maximize differences on those genes.

This is circular reasoning. The p-values are meaningless because the hypothesis was generated from the same data used to test it.

```
# CIRCULAR ANALYSIS — DO NOT DO THIS

# Step 1: Cluster using all genes
# let clusters = kmeans(expression_data, k: 5)

# Step 2: Test each gene for differential expression between clusters
# let de_results = genes |> map(|g|
#   let groups = clusters |> map(|c| expression_data[c][g])
#   anova(groups...)
# )

# Step 3: Report "500 genes are significantly different between clusters"
# THIS IS MEANINGLESS — you are testing the same data you clustered on

# CORRECT APPROACH 1: Split-sample validation
# Cluster on half the samples, test on the other half

let n_samples = len(our_labels)
let half = n_samples / 2

# Random split
let indices = [0, 1, 2] |> map(|i| i)  # simplified: all sample indices
let train_idx = indices |> head(half)
let test_idx = indices |> tail(n_samples - half)

# Cluster on training set
let train_data = train_idx |> map(|i| pc_embedding[i])
let train_km = kmeans(train_data, k: 5)

# Assign test samples to nearest training centroid
let test_labels = test_idx |> map(|i|
  let point = pc_embedding[i]
  let min_dist = 999999.0
  let best_cluster = 0
  train_km.centroids |> each(|centroid|
    let c_idx = 0
    let d = point |> map(|p|
      let dim = 0
      let diff = p - centroid[dim]
      dim = dim + 1
      diff * diff
    ) |> sum() |> sqrt()
    if d < min_dist then
      min_dist = d
      best_cluster = c_idx
    end
    c_idx = c_idx + 1
  )
  best_cluster
)

# NOW test genes on the test set — this is valid
println("Split-sample validation:")
println("  Training set: " + str(len(train_idx)) + " samples")
println("  Test set: " + str(len(test_idx)) + " samples")
println("  Test set cluster sizes: " + str(test_labels |> value_counts()))

# CORRECT APPROACH 2: Use independent features
# Cluster on expression, validate with copy number or methylation data
println("")
println("Alternative: validate clusters using independent data modalities")
println("  Cluster on: gene expression")
println("  Validate on: copy number, methylation, survival, clinical
features")
```

Cluster Stability: Bootstrap Assessment

How do you know your clusters are real and not artifacts of noise? One approach is bootstrap resampling: repeatedly resample the data with replacement, cluster each resample, and check if the same clusters appear consistently.

```
# Bootstrap cluster stability
let n_bootstrap = 100
let n_samples_total = len(pc_embedding)

let stability_matrix = []  # n_samples x n_samples: fraction of times co-
clustered

# Initialize co-clustering counts
let co_cluster_count = []
let co_sampled_count = []
let i_init = 0
while i_init < n_samples_total do
  let row = []
  let row2 = []
  let j_init = 0
  while j_init < n_samples_total do
    row = concat(row, [0])
    row2 = concat(row2, [0])
    j_init = j_init + 1
  end
  co_cluster_count = concat(co_cluster_count, [row])
  co_sampled_count = concat(co_sampled_count, [row2])
  i_init = i_init + 1
end

# Run bootstrap iterations
let boot = 0
while boot < n_bootstrap do
  # Sample with replacement
  let boot_indices = []
  let b = 0
  while b < n_samples_total do
    let idx = round(rnorm(1)[0] * n_samples_total / 4 + n_samples_total / 2,
0)
    if idx < 0 then idx = 0 end
    if idx >= n_samples_total then idx = n_samples_total - 1 end
    boot_indices = concat(boot_indices, [idx])
    b = b + 1
  end

  # Cluster the bootstrap sample
  let boot_data = boot_indices |> map(|i| pc_embedding[i])
  let boot_km = kmeans(boot_data, k: 5)

  # Update co-clustering matrix
  # (simplified — full implementation would track all pairs)
  boot = boot + 1
end

println("Bootstrap stability assessment: " + str(n_bootstrap) + "
iterations")
println("Stable clusters should have co-clustering frequency > 0.8")
println("Unstable clusters have co-clustering frequency < 0.5")
```

Cluster Validation with Clinical Variables

The most convincing validation of biological clusters is their association with independent clinical outcomes — especially survival.

```
# Test whether clusters predict survival differently
# This is external validation — survival was not used for clustering

let clinical_data = read_tsv("tcga_brca_clinical.tsv")

# Merge cluster assignments with survival data
let cluster_survival = table(sample_ids |> map(|s|
  let i = 0
  {
    sample: s,
    cluster: our_labels[i],
    os_months: 0,       # would come from clinical_data
    os_status: 0        # 0=alive, 1=deceased
  }
))

# Kaplan-Meier by cluster
our_clusters |> each(|cl|
  let cl_data = cluster_survival |> filter_by("cluster", cl)
  let times = col_values(cl_data, "os_months")
  let events = col_values(cl_data, "os_status")

  let km = kaplan_meier(times, events)
  println("Cluster " + str(cl) + ": median survival = " +
          str(round(km.median_survival, 1)) + " months")
)

# Log-rank test between clusters (using cox_ph as proxy)
let all_times = col_values(cluster_survival, "os_months")
let all_events = col_values(cluster_survival, "os_status")
let all_clusters = col_values(cluster_survival, "cluster")

let cox_result = cox_ph(all_times, all_events, table({ cluster: all_clusters
}))
println("Cox PH test for cluster effect on survival:")
println("  p-value: " + str(round(cox_result.pvalue, 6)))

# If significant, the clusters have prognostic value
# This is the strongest validation that clustering found real biology
```

Supervised Classification: When Labels Are Available

When you have labels for some samples (like the PAM50 subtypes), you can train a classifier to predict the labels for new samples. This is supervised classification, and it complements unsupervised clustering.

```
# Simple nearest-centroid classifier for PAM50 subtypes

# Compute centroids for each subtype
let subtype_names = unique(known_subtypes) |> filter(|s| s != "NA")

let centroids = subtype_names |> map(|st|
  let st_indices = []
  let i = 0
  known_subtypes |> each(|s|
    if s == st then
      st_indices = concat(st_indices, [i])
    end
    i = i + 1
  )

  # Average PC coordinates for this subtype
  let centroid = []
  let pc = 0
  while pc < n_pcs do
    let vals = st_indices |> map(|i| pc_embedding[i][pc])
    centroid = concat(centroid, [mean(vals)])
    pc = pc + 1
  end

  { subtype: st, centroid: centroid, n_samples: len(st_indices) }
)

println("Subtype centroids computed:")
centroids |> each(|c|
  println("  " + c.subtype + ": " + str(c.n_samples) + " samples")
)

# Leave-one-out cross-validation
let correct = 0
let total = 0
let i = 0
known_subtypes |> each(|true_label|
  if true_label != "NA" then
    let point = pc_embedding[i]

    # Find nearest centroid (excluding this sample's contribution)
    let min_dist = 999999.0
    let predicted = ""
    centroids |> each(|c|
      let d = point |> map(|p|
        let dim = 0
        let diff = p - c.centroid[dim]
        dim = dim + 1
        diff * diff
      ) |> sum() |> sqrt()

      if d < min_dist then
        min_dist = d
        predicted = c.subtype
      end
    )

    if predicted == true_label then
      correct = correct + 1
    end
    total = total + 1
  end
  i = i + 1
)
```

```
let accuracy = 100.0 * correct / total
println("Nearest-centroid classification accuracy: " +
        str(round(accuracy, 1)) + "%")
println("(Leave-one-out cross-validation)")

# For PAM50 subtypes, expect ~70-80% accuracy with this simple method
# Misclassifications are mostly between Luminal A and Luminal B
# which are genuinely difficult to distinguish
```

Correlation Matrix and Heatmap

The correlation matrix between samples reveals the block structure that clustering exploits.

```
# Compute sample-sample correlation matrix
let cor_mat = cor_matrix(scaled_data)

# Visualize as a heatmap (ordered by cluster)
heatmap(cor_mat,
  title: "Sample-Sample Correlation Matrix (ordered by cluster)",
  x_label: "Samples",
  y_label: "Samples")
  |> save_plot("brca_correlation_heatmap.svg")

# The block-diagonal structure (high-correlation blocks along the diagonal)
# corresponds to clusters
# Off-diagonal high-correlation regions indicate similarity between clusters

# Summary statistics of within-cluster vs between-cluster correlations
our_clusters |> each(|cl|
  let indices = []
  let i = 0
  our_labels |> each(|l|
    if l == cl then indices = concat(indices, [i]) end
    i = i + 1
  )

  # Within-cluster correlations
  let within_cors = []
  let ii = 0
  while ii < len(indices) do
    let jj = ii + 1
    while jj < len(indices) do
      within_cors = concat(within_cors, [cor_mat[indices[ii]][indices[jj]]])
      jj = jj + 1
    end
    ii = ii + 1
  end

  println("Cluster " + str(cl) + ": mean within-cluster correlation = " +
          str(round(mean(within_cors), 3)) + " (n=" + str(len(indices)) + "
samples)")
)
```

Interpretation

What Clustering Found in TCGA BRCA

When you cluster TCGA breast cancer tumors by gene expression, you typically recover four to five major groups that correspond well to the PAM50 molecular subtypes:

1. **Luminal A**: Hormone receptor positive, low proliferation. Driven by ESR1, GATA3, FOXA1. Best prognosis.
2. **Luminal B**: Hormone receptor positive, high proliferation. Higher Ki67. More aggressive than Luminal A.
3. **HER2-enriched**: ERBB2 amplified. Distinct cluster driven by ERBB2 and nearby genes on chromosome 17q12.
4. **Basal-like**: Triple-negative (ER-, PR-, HER2-). Driven by basal keratins (KRT5, KRT17) and EGFR. Worst prognosis.
5. **Normal-like**: Resembles normal breast tissue. This subtype is controversial — it may represent contamination with normal tissue rather than a true tumor subtype.

The fact that unsupervised clustering recovers these biologically meaningful and clinically actionable groups is powerful evidence that the molecular subtypes are real, not arbitrary divisions.

When Clustering Finds Batch Effects

Sometimes clustering finds technical artifacts instead of biology. The most common scenario: you cluster samples from multiple institutions or sequencing centers, and the clusters separate by institution rather than by disease.

```
# Check if clusters correlate with technical variables

let institutions = col_values(subtypes, "institution")  # if available
let sequencing_platform = col_values(subtypes, "platform")  # if available

# Contingency table: cluster vs institution
println("Cluster vs Institution (checking for batch effects):")
our_clusters |> each(|cl|
  let cl_institutions = []
  let i = 0
  our_labels |> each(|l|
    if l == cl then
      cl_institutions = concat(cl_institutions, [institutions[i]])
    end
    i = i + 1
  )
  let inst_counts = cl_institutions |> value_counts()
  println("  Cluster " + str(cl) + ": " + str(inst_counts))
)

# If any cluster is dominated by a single institution,
# the clustering may be driven by batch effects
# Solution: batch correction before clustering (Chapter 12)
```

Comparing Clusterings Quantitatively

When you run multiple clustering methods, you need a way to compare them quantitatively. Beyond the ARI, the Normalized Mutual Information (NMI) is another popular metric.

```
# Compare all pairwise clusterings

let methods = [
  { name: "K-means", labels: km_labels },
  { name: "Leiden", labels: leiden_labels }
]

println("Clustering comparison:")
let m1_idx = 0
while m1_idx < len(methods) do
  let m2_idx = m1_idx + 1
  while m2_idx < len(methods) do
    let m1 = methods[m1_idx]
    let m2 = methods[m2_idx]

    # Compute agreement (simplified Rand Index)
    let agree = 0
    let total = 0
    let i = 0
    while i < len(m1.labels) do
      let j = i + 1
      while j < len(m1.labels) do
        let same1 = m1.labels[i] == m1.labels[j]
        let same2 = m2.labels[i] == m2.labels[j]
        if same1 == same2 then agree = agree + 1 end
        total = total + 1
        j = j + 1
      end
      i = i + 1
    end

    let ri = agree / total
    println("  " + m1.name + " vs " + m2.name + ": Rand Index = " +
            str(round(ri, 4)))

    m2_idx = m2_idx + 1
  end
  m1_idx = m1_idx + 1
end
```

Decision

When to Use Each Clustering Method

Method	Best For	Parameters	Cluster Shape
K-means	Large datasets, spherical clusters	k (number of clusters)	Spherical
Hierarchical	Small-medium datasets, dendrograms needed	Linkage method, cut height	Any (with right linkage)

Method	Best For	Parameters	Cluster Shape
Leiden	Single-cell, graph-based data	Resolution	Arbitrary

The Complete Clustering Pipeline

```
# The recommended pipeline for tumor subtype discovery:

# 1. Prepare data
# let expr = read_tsv("expression.tsv")
# let scaled = normalize(select_hvgs(expr, n: 2000), "zscore")

# 2. Reduce dimensions
# let pca_result = pca(scaled, n_components: 30)
# let pc_data = pca_result.components |> map(|r| r |> head(15))

# 3. Explore k range
# [2..10] |> each(|k| kmeans(pc_data, k: k) |> evaluate)

# 4. Cluster with chosen k
# let clusters = kmeans(pc_data, k: 5)

# 5. Validate externally
# - Compare to known subtypes (ARI)
# - Test against survival (Kaplan-Meier, Cox PH)
# - Check for batch effects
# - Split-sample or bootstrap stability

# 6. Characterize clusters
# - Find marker genes (differential expression)
# - Gene ontology enrichment on markers
# - Correlate with clinical variables

# 7. Visualize
# - UMAP colored by cluster
# - Heatmap with cluster dendrogram
# - Forest plot of cluster vs survival

println("Complete clustering pipeline defined")
println("Always validate clusters with independent data")
println("Never test significance on the same features used for clustering")
```

Pitfalls

Pitfall 1: Choosing K by “Looking at the UMAP”

Do not choose the number of clusters by counting the visual groups in a UMAP or t-SNE plot. These methods can split continuous variation into apparent discrete

clusters, and can merge distinct groups that happen to be close in the embedding. Use the elbow method, silhouette scores, or domain knowledge.

Pitfall 2: Assuming Clusters Are Cell Types

In single-cell data, a Leiden cluster is not automatically a cell type. It is a group of cells that are transcriptionally similar. Some clusters may represent cell states (activated vs resting T cells) rather than cell types. Some may represent doublets (two cells captured in one droplet). Some may represent low-quality cells. Biological validation with known marker genes is essential.

Pitfall 3: Ignoring Cluster Size Imbalance

K-means tends to produce clusters of roughly equal size, even when the true groups have very different sizes. If you have 800 Luminal A tumors and 50 Normal-like tumors, k-means may incorrectly split the large group to balance sizes. Leiden clustering handles imbalanced groups better.

```
# K-means can force equal-sized clusters

# Synthetic example: one large group and one small group
let big_group = rnorm(500) |> map(|x| [x, rnorm(1)[0]])
let small_group = rnorm(20) |> map(|x| [x + 10, rnorm(1)[0] + 10])
let all_points = concat(big_group, small_group)

let km2 = kmeans(all_points, k: 2)
let sizes = km2.labels |> value_counts()
println("True sizes: 500, 20")
println("K-means sizes: " + str(sizes))
# K-means may produce sizes like 260, 260 instead of 500, 20
```

Pitfall 4: Not Reporting Clustering Parameters

Every clustering result depends on specific parameter choices. Always report:

- The features used (which genes, how many)
- The preprocessing (normalization, scaling)
- The dimensionality reduction (number of PCs)
- The clustering method and parameters
- The number of clusters and how it was chosen
- The validation approach

Pitfall 5: Overclaiming Novel Subtypes

It is easy to increase k and find “novel subtypes.” But more clusters is not more discovery. Before claiming a new subtype, verify:

1. It is stable across bootstrap resamples
2. It has distinct marker genes
3. It has a distinct clinical phenotype
4. It is reproducible in an independent cohort

```
# Testing a "novel" cluster for independence

# Suppose cluster 6 is a potential novel subtype
# Test whether it is genuinely different from its nearest neighbor cluster

# let novel = samples_in_cluster_6
# let nearest = samples_in_nearest_cluster

# 1. Differential expression between novel and nearest cluster
# let de_genes = test_all_genes(novel, nearest)
# let sig_genes = de_genes |> filter(|g| g.adj_pvalue < 0.05 and
abs(g.log_fc) > 1)

# 2. If fewer than 50 genes are significant, the "novel" subtype
#    may just be a variant of the nearest cluster

# 3. Validate in an independent cohort
# let external_data = read_tsv("independent_cohort.tsv")
# let external_clusters = classify_by_centroids(external_data, centroids)
# Does cluster 6 appear in the external data?

println("Novel subtype claims require:")
println("  1. Distinct marker gene signature")
println("  2. Clinical or phenotypic distinction")
println("  3. Reproducibility in independent data")
println("  4. Bootstrap stability > 0.8")
```

Where Next?

Chapter 15: Meta-Analysis — When multiple studies have clustered the same disease, meta-analysis combines the evidence. Are the subtypes consistent across cohorts? Which clusters replicate and which are dataset-specific?

Multi-omic clustering — Instead of clustering on gene expression alone, integrate copy number, methylation, mutation, and protein data. Methods like iCluster and MOFA perform joint dimensionality reduction across data types before clustering. The TCGA Pan-Cancer Atlas used multi-omic clustering to define molecular subtypes across 33 cancer types.

Spatial transcriptomics — When you have spatial coordinates for each cell (from technologies like Visium, MERFISH, or Slide-seq), clustering must consider both expression similarity and spatial proximity. Spatially-aware clustering reveals tissue architecture.

Trajectory analysis — When cells form continuous developmental trajectories rather than discrete clusters, methods like pseudotime ordering (Monocle, Slingshot) trace paths through the PCA/UMAP space. The choice between clustering and trajectory analysis depends on the biology: differentiation is a trajectory, cell type identity is a cluster.

```
# Summary of clustering methods for genomics

println("=== Clustering Method Decision Guide ===")
println("")
println("Bulk RNA-seq tumor subtyping:")
println("  -> K-means or hierarchical on PCA, validate with survival")
println("")
println("Single-cell cell type discovery:")
println("  -> Leiden on PCA-derived KNN graph, validate with markers")
println("")
println("Multi-omic integration:")
println("  -> Joint dimensionality reduction (MOFA/iCluster), then cluster")
println("")
println("Time-series or developmental data:")
println("  -> Consider trajectory analysis instead of clustering")
println("")
println("Always validate. Always report parameters. Never trust a single
run.")
```

Summary

Concept	Key Takeaway
K-means	Simple, fast, requires k; finds spherical clusters
Hierarchical	No k needed; produces dendrogram; slow for large data
Leiden	Graph-based; handles non-spherical clusters; standard for single-cell
Elbow method	Plot WCSS vs k; look for the bend
Silhouette	Measures cluster cohesion vs separation; range -1 to +1
ARI	Compares two clusterings; corrected for chance; 0=random, 1=perfect
Circular analysis	Never test significance on features used for clustering

Concept	Key Takeaway
Split-sample	Cluster on half, validate on half
Bootstrap stability	Resample and re-cluster; stable clusters appear consistently
Clinical validation	The ultimate test: do clusters predict outcomes?

Chapter 15: Meta-Analysis

The Problem

Three genome-wide association studies have examined the same variant for its association with type 2 diabetes. Study A (n=5,000) reports an odds ratio of 1.35 with p=0.02. Study B (n=3,200) reports OR=1.18 with p=0.15. Study C (n=8,500) reports OR=1.28 with p=0.005. Is this variant truly associated with the disease?

Study A says yes. Study B says no. Study C says yes emphatically. A naive reading would conclude “two out of three studies agree.” But that ignores sample size, effect size magnitude, confidence intervals, and the possibility that the studies are measuring slightly different things.

Meta-analysis is the formal statistical framework for combining results across studies. It produces a single pooled estimate with proper uncertainty quantification, tests for consistency across studies, and can detect when studies disagree for systematic reasons.

Why It Matters

Individual genomics studies are almost always underpowered. A typical GWAS has 80% power to detect an odds ratio of 1.3 at genome-wide significance ($p < 5e-8$) only when the sample size exceeds 10,000 cases and 10,000 controls. Most studies are smaller. This means:

- True associations are missed (false negatives)
- Detected associations have inflated effect sizes (winner’s curse)
- Results vary dramatically between studies by chance

Meta-analysis solves these problems by combining evidence:

The GWAS Catalog at EBI (https://www.ebi.ac.uk/gwas/) contains over 500,000 variant-trait associations from thousands of studies. Many of the most important GWAS discoveries came from meta-analyses — the GIANT consortium for height (>5 million individuals), the PGC for psychiatric disorders, the DIAGRAM consortium for diabetes.

The GTEx consortium performs meta-analysis across tissues to identify expression quantitative trait loci (eQTLs) — genetic variants that affect gene expression. Their multi-tissue meta-analysis (METASOFT) identifies eQTLs that are shared across tissues vs tissue-specific.

Drug target validation increasingly relies on Mendelian randomization, which is itself a form of meta-analysis — combining effect sizes from multiple genetic instruments to estimate causal effects.

The Data

GWAS Catalog full download

URL: https://www.ebi.ac.uk/gwas/api/search/downloads/full

This file contains all published GWAS associations with effect sizes, p-values, sample sizes, and study information.

```
# Download the GWAS Catalog
let gwas_url = "https://www.ebi.ac.uk/gwas/api/search/downloads/full"
let gwas_file = download(gwas_url, "gwas_catalog_full.tsv")

let gwas = read_tsv(gwas_file)
println("GWAS Catalog entries: " + str(len(gwas)))
println("Columns: " + str(columns(gwas) |> len()))

# Examine the structure
println("Key columns:")
columns(gwas) |> head(20) |> each(|c| println("  " + c))
```

For the meta-analysis examples, we will focus on specific genes and traits with multiple independent studies reporting effect sizes.

```
# Find a well-studied variant-trait pair with multiple studies
# Look for SNPs associated with type 2 diabetes

let t2d_studies = gwas |> filter_by("DISEASE/TRAIT", "Type 2 diabetes")
println("Type 2 diabetes GWAS entries: " + str(len(t2d_studies)))

# Find SNPs reported by multiple studies
let snp_counts = t2d_studies |> count_by("SNPS")
let multi_study_snps = snp_counts |> filter(|s| s.n >= 3) |> arrange("n")

println("SNPs reported in 3+ T2D studies: " + str(len(multi_study_snps)))
multi_study_snps |> tail(10) |> each(|s|
  println("  " + s.SNPS + ": " + str(s.n) + " studies")
)
```

Exploration

Understanding Effect Sizes and Their Uncertainties

Before combining studies, we need to understand what each study reports. The key quantities are:

- **Effect size**: odds ratio (OR) for case-control studies, beta coefficient for quantitative traits
- **Standard error (SE)**: the uncertainty in the effect size
- **Confidence interval**: typically 95% CI = effect +/- 1.96 * SE
- **P-value**: significance of the effect

All meta-analysis methods work with effect sizes and their standard errors. If a study only reports a p-value (common in older GWAS), the effect size cannot be directly meta-analyzed — you must use p-value combination methods instead.

```
# Extract effect sizes for a specific SNP across studies
# Example: rs7903146 in TCF7L2 — the strongest T2D risk variant

let target_snp = "rs7903146"
let snp_studies = t2d_studies |> filter(|row|
  row.SNPS == target_snp or row.SNPS == target_snp
)

println("Studies for " + target_snp + ":")
println("  Number of studies: " + str(len(snp_studies)))

# Extract effect sizes and sample sizes
let study_data = snp_studies |> mutate(|row| {
  study: row.STUDY,
  first_author: row.FIRST_AUTHOR,
  year: row.DATE,
  or_beta: row.OR_BETA,
  ci_95: row.CI_95,
  pvalue: row.P_VALUE,
  sample_size: row.SAMPLE_SIZE,
  journal: row.JOURNAL
})

println("")
study_data |> each(|s|
  println("  " + str(s.first_author) + " (" + str(s.year) + "):")
  println("    OR/Beta: " + str(s.or_beta))
  println("    95% CI: " + str(s.ci_95))
  println("    p-value: " + str(s.pvalue))
  println("    Sample size: " + str(s.sample_size))
  println("")
)
```

Converting Between Effect Size Scales

Different studies may report effect sizes on different scales. Binary outcomes use odds ratios (OR), while quantitative traits use beta coefficients. For meta-analysis, all studies must be on the same scale.

```
# For odds ratios, meta-analysis is done on the log(OR) scale
# because log(OR) is approximately normally distributed

# Example: three studies report ORs for a diabetes variant
let studies = [
  { name: "Study A", or: 1.35, se_log_or: 0.12, n: 5000 },
  { name: "Study B", or: 1.18, se_log_or: 0.15, n: 3200 },
  { name: "Study C", or: 1.28, se_log_or: 0.08, n: 8500 }
]

# Convert to log(OR) scale
let log_or_studies = studies |> map(|s| {
  name: s.name,
  effect: log(s.or),              # log(OR)
  se: s.se_log_or,                # SE of log(OR)
  variance: s.se_log_or * s.se_log_or,
  weight: 1.0 / (s.se_log_or * s.se_log_or),  # inverse-variance weight
  n: s.n,
  or: s.or
})

println("Studies on log(OR) scale:")
log_or_studies |> each(|s|
  let ci_lo = exp(s.effect - 1.96 * s.se)
  let ci_hi = exp(s.effect + 1.96 * s.se)
  println("  " + s.name + ": log(OR)=" + str(round(s.effect, 4)) +
          " SE=" + str(round(s.se, 4)) +
          " OR=" + str(round(s.or, 3)) +
          " 95%CI=[" + str(round(ci_lo, 3)) + "," + str(round(ci_hi, 3)) +
"]")
)
```

Visualizing Study-Level Results

Before computing the pooled estimate, visualize each study's result with its confidence interval. This is the individual-study view that will become part of the forest plot.

```
# Plot individual study results with CIs
let forest_data = log_or_studies |> map(|s| {
  study: s.name,
  estimate: s.or,
  ci_lower: exp(s.effect - 1.96 * s.se),
  ci_upper: exp(s.effect + 1.96 * s.se),
  weight: s.weight
})

let forest_table = table(forest_data)

# Visual inspection: do the CIs overlap?
println("Individual study results (OR scale):")
forest_data |> each(|s|
  let ci_width = s.ci_upper - s.ci_lower
  println("  " + s.study + ": OR=" + str(round(s.estimate, 3)) +
          " [" + str(round(s.ci_lower, 3)) + ", " +
          str(round(s.ci_upper, 3)) + "]" +
          " CI width=" + str(round(ci_width, 3)))
)

# If CIs overlap substantially, studies agree
# If CIs barely overlap or don't overlap, there may be heterogeneity
```

The Analysis

Fixed-Effects Meta-Analysis (Inverse-Variance Weighting)

The simplest meta-analysis model is the fixed-effects model. It assumes that all studies are estimating the same true effect, and differences between studies are due to sampling variation alone.

The pooled estimate is the weighted average of the study effects, where each study is weighted by the inverse of its variance (i.e., more precise studies get more weight):

Pooled effect = sum(w_i * theta_i) / sum(w_i) SE of pooled = 1 / sqrt(sum(w_i))

where w_i = 1 / SE_i^2 and theta_i is the effect estimate from study i.

```
# Fixed-effects meta-analysis
let total_weight = log_or_studies |> map(|s| s.weight) |> sum()
let weighted_sum = log_or_studies |> map(|s| s.weight * s.effect) |> sum()

let pooled_effect = weighted_sum / total_weight
let pooled_se = 1.0 / sqrt(total_weight)
let pooled_or = exp(pooled_effect)
let pooled_ci_lo = exp(pooled_effect - 1.96 * pooled_se)
let pooled_ci_hi = exp(pooled_effect + 1.96 * pooled_se)
let pooled_z = pooled_effect / pooled_se
let pooled_p = 2.0 * (1.0 - pnorm(abs(pooled_z)))

println("=== Fixed-Effects Meta-Analysis ===")
println("Pooled log(OR): " + str(round(pooled_effect, 4)))
println("Pooled SE: " + str(round(pooled_se, 4)))
println("Pooled OR: " + str(round(pooled_or, 3)))
println("95% CI: [" + str(round(pooled_ci_lo, 3)) + ", " +
        str(round(pooled_ci_hi, 3)) + "]")
println("Z-score: " + str(round(pooled_z, 2)))
println("P-value: " + str(round(pooled_p, 8)))
println("")

# Weight contribution from each study
log_or_studies |> each(|s|
  let pct_weight = 100.0 * s.weight / total_weight
  println("  " + s.name + " contributes " + str(round(pct_weight, 1)) +
          "% of the weight (n=" + str(s.n) + ")")
)
```

Testing for Heterogeneity: The Q Statistic

The fixed-effects model assumes all studies estimate the same true effect. The Q statistic tests this assumption. Q is the weighted sum of squared deviations from the pooled effect:

Q = sum(w_i * (theta_i - pooled)^2)

Under the null hypothesis of no heterogeneity, Q follows a chi-squared distribution with k-1 degrees of freedom (where k is the number of studies).

```
# Cochran's Q test for heterogeneity
let q_stat = log_or_studies |> map(|s|
  let dev = s.effect - pooled_effect
  s.weight * dev * dev
) |> sum()

let k = len(log_or_studies)
let df_q = k - 1

# P-value from chi-squared distribution
# Using the approximation: for df=2, critical value at 0.05 is 5.991
let q_pvalue = 1.0 - pnorm(sqrt(2.0 * q_stat) - sqrt(2.0 * df_q - 1.0))

println("=== Heterogeneity Test ===")
println("Cochran's Q: " + str(round(q_stat, 3)))
println("Degrees of freedom: " + str(df_q))
println("P-value: " + str(round(q_pvalue, 4)))

if q_pvalue < 0.10 then
  println("-> Significant heterogeneity detected (p < 0.10)")
  println("   Consider random-effects model")
else
  println("-> No significant heterogeneity (p >= 0.10)")
  println("   Fixed-effects model is appropriate")
end
```

The I-Squared Statistic

While Q tests whether heterogeneity exists, I-squared quantifies how much of the variability in effect sizes is due to true heterogeneity (as opposed to sampling variation).

I^2 = max(0, (Q - df) / Q) * 100%

Interpretation:

- I^2 = 0%: No heterogeneity; all variability is sampling error
- I^2 = 25%: Low heterogeneity
- I^2 = 50%: Moderate heterogeneity
- I^2 = 75%: High heterogeneity

```
# I-squared statistic
let i_squared = max(0, (q_stat - df_q) / q_stat) * 100.0

println("I-squared: " + str(round(i_squared, 1)) + "%")

if i_squared < 25 then
  println("-> Low heterogeneity: studies are consistent")
  println("   Fixed-effects meta-analysis is appropriate")
end
if i_squared >= 25 and i_squared < 50 then
  println("-> Low-to-moderate heterogeneity")
  println("   Fixed-effects is likely still appropriate")
end
if i_squared >= 50 and i_squared < 75 then
  println("-> Moderate heterogeneity")
  println("   Random-effects model recommended")
  println("   Investigate sources of heterogeneity")
end
if i_squared >= 75 then
  println("-> High heterogeneity")
  println("   Random-effects model essential")
  println("   Pool estimate may not be meaningful without understanding why
studies differ")
end
```

Random-Effects Meta-Analysis (DerSimonian-Laird)

When heterogeneity is present, the fixed-effects model is too aggressive — it gives confidence intervals that are too narrow because it ignores the between-study variation. The random-effects model adds a between-study variance component (tau-squared) to account for this.

The DerSimonian-Laird method estimates tau-squared from the data:

tau^2 = max(0, (Q - df) / (sum(w_i) - sum(w_i^2)/sum(w_i)))

Then the random-effects weights become:

w_i* = 1 / (SE_i^2 + tau^2)

```
# DerSimonian-Laird random-effects meta-analysis

# Estimate tau-squared (between-study variance)
let sum_w = log_or_studies |> map(|s| s.weight) |> sum()
let sum_w2 = log_or_studies |> map(|s| s.weight * s.weight) |> sum()
let c = sum_w - sum_w2 / sum_w

let tau_sq = max(0, (q_stat - df_q) / c)

println("=== Random-Effects Meta-Analysis (DerSimonian-Laird) ===")
println("Tau-squared (between-study variance): " + str(round(tau_sq, 6)))
println("Tau (between-study SD): " + str(round(sqrt(tau_sq), 4)))

# Compute random-effects weights
let re_studies = log_or_studies |> map(|s| {
  name: s.name,
  effect: s.effect,
  se: s.se,
  re_variance: s.variance + tau_sq,
  re_weight: 1.0 / (s.variance + tau_sq),
  n: s.n
})

let re_total_weight = re_studies |> map(|s| s.re_weight) |> sum()
let re_weighted_sum = re_studies |> map(|s| s.re_weight * s.effect) |> sum()

let re_pooled_effect = re_weighted_sum / re_total_weight
let re_pooled_se = 1.0 / sqrt(re_total_weight)
let re_pooled_or = exp(re_pooled_effect)
let re_pooled_ci_lo = exp(re_pooled_effect - 1.96 * re_pooled_se)
let re_pooled_ci_hi = exp(re_pooled_effect + 1.96 * re_pooled_se)
let re_z = re_pooled_effect / re_pooled_se
let re_p = 2.0 * (1.0 - pnorm(abs(re_z)))

println("")
println("Pooled log(OR): " + str(round(re_pooled_effect, 4)))
println("Pooled SE: " + str(round(re_pooled_se, 4)))
println("Pooled OR: " + str(round(re_pooled_or, 3)))
println("95% CI: [" + str(round(re_pooled_ci_lo, 3)) + ", " +
        str(round(re_pooled_ci_hi, 3)) + "]")
println("P-value: " + str(round(re_p, 8)))

# Compare fixed vs random effects
println("")
println("=== Comparison ===")
println("Fixed-effects OR: " + str(round(pooled_or, 3)) +
        " [" + str(round(pooled_ci_lo, 3)) + ", " +
        str(round(pooled_ci_hi, 3)) + "]")
println("Random-effects OR: " + str(round(re_pooled_or, 3)) +
        " [" + str(round(re_pooled_ci_lo, 3)) + ", " +
        str(round(re_pooled_ci_hi, 3)) + "]")
println("")
println("Note: Random-effects CI is always wider than or equal to fixed-
effects CI")
println("The random-effects estimate is more conservative — it accounts for")
println("the possibility that the true effect varies between studies")

# Weight redistribution
println("")
println("Weight redistribution:")
re_studies |> each(|s|
  let fe_pct = 100.0 * (1.0 / s.se / s.se) / total_weight
  let re_pct = 100.0 * s.re_weight / re_total_weight
  println("  " + s.name + ": FE=" + str(round(fe_pct, 1)) +
          "% -> RE=" + str(round(re_pct, 1)) + "%")
```

```
)
println("Random-effects gives relatively more weight to small studies")
```

Forest Plot: The Essential Visualization

The forest plot is the standard visualization for meta-analysis. It shows each study's estimate with its confidence interval, the pooled estimate, and the relative weight of each study.

```
# Create the forest plot data

let fp_data = log_or_studies |> map(|s| {
  study: s.name + " (n=" + str(s.n) + ")",
  estimate: s.or,
  ci_lower: exp(s.effect - 1.96 * s.se),
  ci_upper: exp(s.effect + 1.96 * s.se),
  weight: round(100.0 * s.weight / total_weight, 1)
})

# Add the pooled estimates
let fp_pooled_fe = {
  study: "Fixed Effects (pooled)",
  estimate: pooled_or,
  ci_lower: pooled_ci_lo,
  ci_upper: pooled_ci_hi,
  weight: 100.0
}

let fp_pooled_re = {
  study: "Random Effects (pooled)",
  estimate: re_pooled_or,
  ci_lower: re_pooled_ci_lo,
  ci_upper: re_pooled_ci_hi,
  weight: 100.0
}

let all_fp_data = concat(fp_data, [fp_pooled_fe, fp_pooled_re])
let fp_table = table(all_fp_data)

forest_plot(fp_table, "study", "estimate", "ci_lower", "ci_upper",
  title: "Meta-Analysis: " + target_snp + " and Type 2 Diabetes",
  x_label: "Odds Ratio",
  null_value: 1.0)
  |> save_plot("meta_analysis_forest.svg")

println("Forest plot saved to meta_analysis_forest.svg")
println("The diamond represents the pooled estimate")
println("The vertical line at OR=1.0 represents the null (no effect)")
```

Combining P-Values: Fisher's Method

When effect sizes are not available (common with older studies), you can still combine p-values. Fisher's method uses the fact that under the null hypothesis, -2 * sum(log(p_i)) follows a chi-squared distribution with 2k degrees of freedom.

```
# Fisher's method for combining p-values

let pvalues = [0.02, 0.15, 0.005]

let fisher_stat = pvalues |> map(|p| -2.0 * log(p)) |> sum()
let fisher_df = 2 * len(pvalues)

# Chi-squared p-value approximation
# For large df, chi-squared is approximately normal
let fisher_z = (fisher_stat - fisher_df) / sqrt(2.0 * fisher_df)
let fisher_p = 1.0 - pnorm(fisher_z)

println("=== Fisher's Method for Combining P-Values ===")
println("Individual p-values: " + str(pvalues))
println("Test statistic: " + str(round(fisher_stat, 3)))
println("Degrees of freedom: " + str(fisher_df))
println("Combined p-value: " + str(round(fisher_p, 8)))
println("")

# Note: Fisher's method is very sensitive to a single strong result
# One study with p=0.001 can make the combined p-value significant
# even if other studies show nothing
```

Stouffer's Method: Weighted P-Value Combination

Stouffer's method converts p-values to z-scores, weights them (typically by the square root of the sample size), and combines:

Z_pooled = sum(w_i * Z_i) / sqrt(sum(w_i^2))

```
# Stouffer's weighted Z method

let study_ps = [
  { pvalue: 0.02,  n: 5000 },
  { pvalue: 0.15,  n: 3200 },
  { pvalue: 0.005, n: 8500 }
]

# Convert p-values to z-scores (one-sided)
let z_scores = study_ps |> map(|s| {
  z: qnorm(1.0 - s.pvalue / 2.0),   # two-sided p to z
  weight: sqrt(s.n),                  # weight by sqrt(n)
  n: s.n
})

let weighted_z_sum = z_scores |> map(|s| s.weight * s.z) |> sum()
let weight_ss = z_scores |> map(|s| s.weight * s.weight) |> sum()
let pooled_z_stouffer = weighted_z_sum / sqrt(weight_ss)
let stouffer_p = 2.0 * (1.0 - pnorm(abs(pooled_z_stouffer)))

println("=== Stouffer's Weighted Z Method ===")
z_scores |> each(|s|
  println("  Z=" + str(round(s.z, 3)) + " weight=" +
          str(round(s.weight, 1)) + " (n=" + str(s.n) + ")")
)
println("Pooled Z: " + str(round(pooled_z_stouffer, 3)))
println("Combined p-value: " + str(round(stouffer_p, 8)))
println("")
println("Stouffer's method is preferred over Fisher's when sample sizes
vary")
println("because it gives more weight to larger studies")
```

Real Example: Combining GWAS Hits from the GWAS Catalog

Now let us apply meta-analysis to real data from the GWAS Catalog. We will find all studies reporting on a specific variant and combine their results.

```
# Find all reports for a well-studied variant
# rs7903146 in TCF7L2 — strongest known T2D risk variant

let target_reports = gwas |> filter(|row|
  row.SNPS == "rs7903146"
)

println("All GWAS reports for rs7903146:")
println("  Total entries: " + str(len(target_reports)))

# Filter to those with reported OR/beta values
let with_effects = target_reports |> filter(|row|
  row.OR_BETA != "" and row.OR_BETA != "NR"
)

println("  With effect sizes: " + str(len(with_effects)))

# Extract and summarize
let effect_summary = with_effects |> mutate(|row| {
  trait: row.DISEASE_TRAIT,
  first_author: row.FIRST_AUTHOR,
  year: row.DATE,
  or_beta: row.OR_BETA,
  pvalue: row.P_VALUE,
  sample_size: row.SAMPLE_SIZE
})

# Group by trait to see which traits this variant is associated with
let trait_counts = effect_summary |> count_by("trait")
println("")
println("Traits associated with rs7903146:")
trait_counts |> sort(|a, b| b.n - a.n) |> head(10) |> each(|t|
  println("  " + t.trait + ": " + str(t.n) + " studies")
)

# Focus on T2D studies for meta-analysis
let t2d_effects = effect_summary |> filter(|row|
  row.trait == "Type 2 diabetes"
)

println("")
println("T2D-specific studies for meta-analysis: " + str(len(t2d_effects)))
```

Publication Bias: The Funnel Plot

Publication bias is the tendency for studies with statistically significant results to be published, while studies with null results remain in file drawers. This biases meta-analyses toward larger effect sizes.

The funnel plot visualizes this: it plots each study's effect size against its precision (1/SE or sample size). In the absence of publication bias, the plot should be symmetric around the pooled estimate. Asymmetry — especially missing small-study null results in the lower left — suggests publication bias.

```
# Create a funnel plot

let funnel_data = log_or_studies |> map(|s| {
  effect: s.effect,          # log(OR) on x-axis
  precision: 1.0 / s.se,    # 1/SE on y-axis (more precise studies at top)
  study: s.name
})

let funnel_table = table(funnel_data)

scatter(funnel_table, "effect", "precision",
  title: "Funnel Plot: " + target_snp,
  x_label: "Effect Size (log OR)",
  y_label: "Precision (1/SE)")
  |> save_plot("funnel_plot.svg")

# Add reference lines
println("Funnel plot interpretation:")
println("  Vertical line at pooled effect = " + str(round(pooled_effect, 4)))
println("  Symmetric distribution: no publication bias")
println("  Missing lower-left: small negative studies unpublished")
println("  Missing lower-right: small positive studies unpublished")

# Egger's test for funnel plot asymmetry
# Regress the standard normal deviate (effect/SE) against precision (1/SE)
# Intercept significantly different from zero suggests bias

let egger_x = log_or_studies |> map(|s| 1.0 / s.se)
let egger_y = log_or_studies |> map(|s| s.effect / s.se)

let egger_result = lm(egger_y, egger_x)
println("")
println("Egger's test for publication bias:")
println("  Intercept: " + str(round(egger_result.intercept, 4)))
println("  P-value for intercept: " + str(round(egger_result.pvalue, 4)))

if egger_result.pvalue < 0.10 then
  println("  -> Evidence of funnel plot asymmetry (potential publication
bias)")
else
  println("  -> No significant asymmetry detected")
end
```

Subgroup Analysis: Exploring Heterogeneity

When heterogeneity is detected, subgroup analysis investigates whether study-level characteristics explain the differences. Common subgroup variables in genomics:

- **Ancestry**: European vs East Asian vs African
- **Study design**: Case-control vs cohort
- **Genotyping platform**: Array vs sequencing
- **Phenotype definition**: Strict vs broad diagnostic criteria

```
# Subgroup analysis by ancestry
# Suppose we have ancestry information for each study

let studies_with_ancestry = [
  { name: "Study A", effect: log(1.35), se: 0.12, ancestry: "European", n:
5000 },
  { name: "Study B", effect: log(1.18), se: 0.15, ancestry: "East Asian", n:
3200 },
  { name: "Study C", effect: log(1.28), se: 0.08, ancestry: "European", n:
8500 },
  { name: "Study D", effect: log(1.42), se: 0.10, ancestry: "European", n:
6200 },
  { name: "Study E", effect: log(1.22), se: 0.18, ancestry: "East Asian", n:
2100 },
  { name: "Study F", effect: log(1.15), se: 0.20, ancestry: "African", n:
1800 }
]

let ancestries = unique(studies_with_ancestry |> map(|s| s.ancestry))

println("=== Subgroup Analysis by Ancestry ===")
ancestries |> each(|anc|
  let subgroup = studies_with_ancestry |> filter(|s| s.ancestry == anc)

  # Fixed-effects meta-analysis within subgroup
  let sg_weights = subgroup |> map(|s| 1.0 / (s.se * s.se))
  let sg_total_w = sg_weights |> sum()
  let sg_pooled = subgroup |> map(|s|
    (1.0 / (s.se * s.se)) * s.effect
  ) |> sum() / sg_total_w
  let sg_se = 1.0 / sqrt(sg_total_w)
  let sg_or = exp(sg_pooled)
  let sg_ci_lo = exp(sg_pooled - 1.96 * sg_se)
  let sg_ci_hi = exp(sg_pooled + 1.96 * sg_se)

  println(anc + " (" + str(len(subgroup)) + " studies):")
  println("  Pooled OR: " + str(round(sg_or, 3)) +
          " [" + str(round(sg_ci_lo, 3)) + ", " +
          str(round(sg_ci_hi, 3)) + "]")
  println("")
)

# Test for interaction between subgroups
# Is the effect significantly different between ancestries?
# This uses a chi-squared test comparing subgroup estimates
println("If subgroup effects differ significantly, ancestry may modify the")
println("variant's effect on disease risk (gene-environment interaction)")
```

The Winner’s Curse

The winner’s curse is a form of selection bias that inflates effect sizes in the discovery cohort. When you test thousands of variants and select the “winners” (most significant), you are preferentially selecting those where random noise happened to increase the apparent effect.

```
# Demonstrate the winner's curse with simulation

# True effect: OR = 1.2 (log OR = 0.182)
let true_log_or = log(1.2)
let true_or = 1.2

# Simulate 100 studies, each with different SE
let simulated = rnorm(100) |> map(|noise|
  let se = 0.05 + abs(rnorm(1)[0]) * 0.15  # random SE between 0.05 and ~0.20
  let observed_effect = true_log_or + noise * se  # noisy observation
  let z = observed_effect / se
  let p = 2.0 * (1.0 - pnorm(abs(z)))
  {
    effect: observed_effect,
    se: se,
    or: exp(observed_effect),
    pvalue: p,
    significant: p < 0.05
  }
)

# Compare significant vs non-significant studies
let sig_studies = simulated |> filter(|s| s.significant)
let nonsig_studies = simulated |> filter(|s| !s.significant)

let sig_mean_or = mean(sig_studies |> map(|s| s.or))
let nonsig_mean_or = mean(nonsig_studies |> map(|s| s.or))
let all_mean_or = mean(simulated |> map(|s| s.or))

println("=== Winner's Curse Demonstration ===")
println("True OR: " + str(true_or))
println("Mean OR across all 100 studies: " + str(round(all_mean_or, 3)))
println("Mean OR in significant studies only: " + str(round(sig_mean_or, 3)))
println("Mean OR in non-significant studies: " + str(round(nonsig_mean_or,
3)))
println("")
println("The significant studies overestimate the true effect by " +
        str(round(100 * (sig_mean_or - true_or) / true_or, 1)) + "%")
println("This is the winner's curse")
println("")
println("Implication: the first GWAS to discover an association will report")
println("an inflated effect size. Replication studies typically show smaller
effects.")
println("Meta-analysis of all studies (including replications) corrects this
bias.")
```

Multi-Trait Meta-Analysis

In genomics, we often want to combine evidence across related traits. For example, a variant may affect both fasting glucose and HbA1c — combining evidence across these correlated traits increases power.

```
# Multi-trait analysis: combine evidence across related phenotypes

let multi_trait = [
  { trait: "Type 2 diabetes", effect: log(1.30), se: 0.06, n: 15000 },
  { trait: "Fasting glucose", effect: 0.08, se: 0.02, n: 45000 },   # beta in
mmol/L
  { trait: "HbA1c", effect: 0.05, se: 0.015, n: 50000 },            # beta in
%
  { trait: "Fasting insulin", effect: 0.03, se: 0.025, n: 25000 }   # beta in
pmol/L
]

# These are on different scales — cannot directly meta-analyze with IVW
# Convert to Z-scores for combination

let trait_zscores = multi_trait |> map(|t| {
  trait: t.trait,
  z: t.effect / t.se,
  pvalue: 2.0 * (1.0 - pnorm(abs(t.effect / t.se))),
  n: t.n
})

println("=== Multi-Trait Evidence ===")
trait_zscores |> each(|t|
  println("  " + t.trait + ": Z=" + str(round(t.z, 2)) +
          " p=" + str(round(t.pvalue, 8)))
)

# Combine using Stouffer's method (weighted by sqrt(n))
let multi_w = trait_zscores |> map(|t| sqrt(t.n))
let multi_wz = trait_zscores |> map(|t| sqrt(t.n) * t.z) |> sum()
let multi_wss = multi_w |> map(|w| w * w) |> sum()
let multi_z_pooled = multi_wz / sqrt(multi_wss)
let multi_p = 2.0 * (1.0 - pnorm(abs(multi_z_pooled)))

println("")
println("Multi-trait combined Z: " + str(round(multi_z_pooled, 2)))
println("Multi-trait combined p: " + str(round(multi_p, 12)))
println("")
println("Caution: traits are correlated, so this overstates significance")
println("Proper multi-trait methods (MTAG, ASSET) account for trait
correlations")
```

Sensitivity Analysis

Robust meta-analyses include sensitivity analyses to check whether the results depend on any single study:

```
# Leave-one-out sensitivity analysis
println("=== Leave-One-Out Analysis ===")
println("Removing each study in turn to check robustness:")
println("")

log_or_studies |> each(|excluded|
  let remaining = log_or_studies |> filter(|s| s.name != excluded.name)

  let loo_w = remaining |> map(|s| s.weight) |> sum()
  let loo_sum = remaining |> map(|s| s.weight * s.effect) |> sum()
  let loo_effect = loo_sum / loo_w
  let loo_se = 1.0 / sqrt(loo_w)
  let loo_or = exp(loo_effect)
  let loo_ci_lo = exp(loo_effect - 1.96 * loo_se)
  let loo_ci_hi = exp(loo_effect + 1.96 * loo_se)

  println("  Excluding " + excluded.name + ": OR=" + str(round(loo_or, 3)) +
          " [" + str(round(loo_ci_lo, 3)) + ", " +
          str(round(loo_ci_hi, 3)) + "]")
)

println("")
println("If the pooled OR changes substantially when any single study is
removed,")
println("the meta-analysis is sensitive to that study — investigate why")
```

Cumulative Meta-Analysis

A cumulative meta-analysis adds studies one at a time (chronologically) to show how the pooled estimate evolved as evidence accumulated.

```
# Cumulative meta-analysis (chronological order)

let chronological = [
  { name: "Study A (2008)", effect: log(1.42), se: 0.15, year: 2008 },
  { name: "Study B (2010)", effect: log(1.35), se: 0.12, year: 2010 },
  { name: "Study C (2012)", effect: log(1.28), se: 0.10, year: 2012 },
  { name: "Study D (2015)", effect: log(1.25), se: 0.08, year: 2015 },
  { name: "Study E (2018)", effect: log(1.22), se: 0.06, year: 2018 },
  { name: "Study F (2022)", effect: log(1.20), se: 0.05, year: 2022 }
]

println("=== Cumulative Meta-Analysis ===")
println("Pooled estimate as each study is added chronologically:")
println("")

let included = []
chronological |> each(|study|
  included = concat(included, [study])

  let cum_w = included |> map(|s| 1.0 / (s.se * s.se)) |> sum()
  let cum_sum = included |> map(|s| (1.0 / (s.se * s.se)) * s.effect) |>
sum()
  let cum_effect = cum_sum / cum_w
  let cum_se = 1.0 / sqrt(cum_w)
  let cum_or = exp(cum_effect)
  let cum_ci_lo = exp(cum_effect - 1.96 * cum_se)
  let cum_ci_hi = exp(cum_effect + 1.96 * cum_se)
  let total_n = included |> len()

  println("  After " + study.name + " (" + str(total_n) + " studies):")
  println("    OR=" + str(round(cum_or, 3)) + " [" +
          str(round(cum_ci_lo, 3)) + ", " + str(round(cum_ci_hi, 3)) + "]")
)

println("")
println("Notice how:")
println("  1. The CI narrows as more studies are added (more precision)")
println("  2. The OR decreases slightly over time (winner's curse
correction)")
println("  3. The first study had the largest OR (discovery bias)")
```

Building a Complete Meta-Analysis Pipeline

```
# Complete meta-analysis pipeline function

# Step 1: Collect study-level data
let all_studies = [
  { name: "Grant 2006", effect: log(1.45), se: 0.10, n: 3500, ancestry:
"European" },
  { name: "Scott 2007", effect: log(1.37), se: 0.08, n: 5500, ancestry:
"European" },
  { name: "Sladek 2007", effect: log(1.40), se: 0.12, n: 2800, ancestry:
"European" },
  { name: "DIAGRAM 2008", effect: log(1.36), se: 0.05, n: 12000, ancestry:
"European" },
  { name: "Takeuchi 2009", effect: log(1.25), se: 0.14, n: 3000, ancestry:
"East Asian" },
  { name: "Unoki 2008", effect: log(1.20), se: 0.11, n: 4200, ancestry: "East
Asian" },
  { name: "Waters 2010", effect: log(1.30), se: 0.18, n: 1500, ancestry:
"African" },
  { name: "DIAGRAM 2012", effect: log(1.32), se: 0.04, n: 22000, ancestry:
"European" },
  { name: "Mahajan 2018", effect: log(1.29), se: 0.03, n: 74000, ancestry:
"Multi-ethnic" }
]

# Step 2: Fixed-effects analysis
let fe_w = all_studies |> map(|s| 1.0 / (s.se * s.se))
let fe_total = fe_w |> sum()
let fe_pooled = all_studies |> map(|s| (1.0 / (s.se * s.se)) * s.effect) |>
sum() / fe_total
let fe_se = 1.0 / sqrt(fe_total)

# Step 3: Heterogeneity
let q = all_studies |> map(|s|
  let w = 1.0 / (s.se * s.se)
  w * pow(s.effect - fe_pooled, 2)
) |> sum()
let df = len(all_studies) - 1
let i2 = max(0, (q - df) / q) * 100.0

# Step 4: Random-effects if needed
let c_val = fe_total - (fe_w |> map(|w| w * w) |> sum()) / fe_total
let tau2 = max(0, (q - df) / c_val)
let re_w = all_studies |> map(|s| 1.0 / (s.se * s.se + tau2))
let re_total = re_w |> sum()
let re_pooled = all_studies |> map(|s|
  (1.0 / (s.se * s.se + tau2)) * s.effect
) |> sum() / re_total
let re_se_val = 1.0 / sqrt(re_total)

# Step 5: Report
println("=== Complete Meta-Analysis Report ===")
println("Variant: rs7903146 (TCF7L2)")
println("Trait: Type 2 Diabetes")
println("Studies: " + str(len(all_studies)))
println("Total sample: " + str(all_studies |> map(|s| s.n) |> sum()))
println("")
println("Fixed-effects: OR=" + str(round(exp(fe_pooled), 3)) +
        " [" + str(round(exp(fe_pooled - 1.96 * fe_se), 3)) + ", " +
        str(round(exp(fe_pooled + 1.96 * fe_se), 3)) + "]")
println("Random-effects: OR=" + str(round(exp(re_pooled), 3)) +
        " [" + str(round(exp(re_pooled - 1.96 * re_se_val), 3)) + ", " +
        str(round(exp(re_pooled + 1.96 * re_se_val), 3)) + "]")
```

```
  println("Heterogeneity: Q=" + str(round(q, 2)) + " I2=" +
          str(round(i2, 1)) + "%")
  println("Tau2=" + str(round(tau2, 6)))

  # Step 6: Forest plot
  let final_fp = all_studies |> map(|s| {
    study: s.name + " (" + s.ancestry + ")",
    estimate: exp(s.effect),
    ci_lower: exp(s.effect - 1.96 * s.se),
    ci_upper: exp(s.effect + 1.96 * s.se),
    weight: round(100.0 * (1.0 / (s.se * s.se)) / fe_total, 1)
  })

  let final_fp = concat(final_fp, [{
    study: "RE Pooled",
    estimate: exp(re_pooled),
    ci_lower: exp(re_pooled - 1.96 * re_se_val),
    ci_upper: exp(re_pooled + 1.96 * re_se_val),
    weight: 100.0
  }])

  forest_plot(table(final_fp), "study", "estimate", "ci_lower", "ci_upper",
    title: "rs7903146 and Type 2 Diabetes: Meta-Analysis",
    x_label: "Odds Ratio",
    null_value: 1.0)
    |> save_plot("tcf7l2_forest_plot.svg")
```

Interpretation

Reading a Forest Plot

The forest plot tells you everything you need to know at a glance:

1. **Individual study squares**: Size proportional to weight. Position shows the point estimate. Horizontal lines are 95% CIs.
2. **The diamond**: The pooled estimate. Width represents the pooled CI.
3. **The null line**: Vertical line at OR=1 (or beta=0). If a CI crosses this line, that study's result is not significant.
4. **Overlap**: If all CIs overlap substantially, there is little heterogeneity. If some CIs are completely separated from others, the studies genuinely disagree.

When Studies Disagree

Heterogeneity is not always bad. It can reveal important biology:

- **Ancestry-specific effects**: A variant may have OR=1.4 in Europeans but OR=1.1 in East Asians due to different linkage disequilibrium patterns or

gene-environment interactions.
- **Phenotype heterogeneity**: "Type 2 diabetes" diagnosed by fasting glucose vs oral glucose tolerance test may capture different patient populations.
- **Winner's curse**: Early studies with smaller samples report larger effects. This is expected and should decrease over time.

```
# Investigating heterogeneity sources

# Stratify by ancestry
let european = all_studies |> filter(|s| s.ancestry == "European")
let east_asian = all_studies |> filter(|s| s.ancestry == "East Asian")

let eur_w = european |> map(|s| 1.0 / (s.se * s.se))
let eur_total = eur_w |> sum()
let eur_pooled = european |> map(|s| (1.0 / (s.se * s.se)) * s.effect) |>
sum() / eur_total

let ea_w = east_asian |> map(|s| 1.0 / (s.se * s.se))
let ea_total = ea_w |> sum()
let ea_pooled = east_asian |> map(|s| (1.0 / (s.se * s.se)) * s.effect) |>
sum() / ea_total

println("Ancestry-stratified results:")
println("  European: OR=" + str(round(exp(eur_pooled), 3)))
println("  East Asian: OR=" + str(round(exp(ea_pooled), 3)))

# Test for difference between subgroups
let diff = eur_pooled - ea_pooled
let diff_se = sqrt(1.0 / eur_total + 1.0 / ea_total)
let diff_z = diff / diff_se
let diff_p = 2.0 * (1.0 - pnorm(abs(diff_z)))

println("  Difference test: Z=" + str(round(diff_z, 2)) +
        " p=" + str(round(diff_p, 4)))

if diff_p < 0.05 then
  println("  -> Effect size differs significantly between ancestries")
else
  println("  -> No significant difference between ancestries")
end
```

Decision

Fixed-Effects vs Random-Effects: Which to Use?

Criterion	Fixed-Effects	Random-Effects
Assumption	All studies estimate the same true effect	True effect varies between studies

Criterion	Fixed-Effects	Random-Effects
When to use	Studies are homogeneous (I^2 < 25%)	Studies are heterogeneous (I^2 > 50%)
CI width	Narrower	Wider
Weight distribution	Dominated by large studies	More balanced across studies
Interpretation	“The” common effect	The average effect across studies

In practice: report both. If they agree, it does not matter which you emphasize. If they disagree, explain why (heterogeneity) and use the random-effects estimate as the primary result.

The Meta-Analysis Checklist

Before publishing a meta-analysis, verify:

```
println("=== Meta-Analysis Quality Checklist ===")
println("")
println("1. Search strategy")
println("   [ ] Systematic search of multiple databases")
println("   [ ] Predefined inclusion/exclusion criteria")
println("   [ ] PRISMA flow diagram")
println("")
println("2. Data extraction")
println("   [ ] Effect sizes on a consistent scale")
println("   [ ] Standard errors (not just p-values)")
println("   [ ] Sample sizes for each study")
println("")
println("3. Analysis")
println("   [ ] Fixed-effects AND random-effects results")
println("   [ ] Heterogeneity assessment (Q, I-squared)")
println("   [ ] Forest plot")
println("   [ ] Publication bias assessment (funnel plot, Egger's test)")
println("")
println("4. Sensitivity")
println("   [ ] Leave-one-out analysis")
println("   [ ] Subgroup analysis (ancestry, phenotype, study design)")
println("   [ ] Cumulative meta-analysis (chronological)")
println("")
println("5. Reporting")
println("   [ ] PRISMA guidelines followed")
println("   [ ] Effect sizes with CIs, not just p-values")
println("   [ ] Heterogeneity statistics reported")
println("   [ ] Limitations discussed (publication bias, heterogeneity)")
```

Pitfalls

Pitfall 1: Combining Incompatible Effect Sizes

You cannot meta-analyze odds ratios from one study with beta coefficients from another without proper conversion. You cannot combine standardized effect sizes (Cohen's d) with unstandardized effect sizes. Always verify that all studies are on the same scale.

```
# Common scale conversions

# Odds ratio to log odds ratio (for meta-analysis)
let or_value = 1.35
let log_or = log(or_value)
println("OR " + str(or_value) + " -> log(OR) = " + str(round(log_or, 4)))

# Cohen's d to log odds ratio (approximate)
let d = 0.5
let log_or_from_d = d * 3.14159 / sqrt(3)
println("Cohen's d " + str(d) + " -> log(OR) ≈ " + str(round(log_or_from_d,
4)))

# Correlation r to Cohen's d (approximate)
let r = 0.3
let d_from_r = 2 * r / sqrt(1 - r * r)
println("r " + str(r) + " -> Cohen's d ≈ " + str(round(d_from_r, 4)))
```

Pitfall 2: Ignoring Study Overlap

If two meta-analyzed studies share participants (e.g., both used UK Biobank), treating them as independent inflates the effective sample size and produces falsely precise pooled estimates. Always check for sample overlap.

Pitfall 3: The File Drawer Problem

You can only meta-analyze published studies. If negative results remain unpublished, your meta-analysis overestimates the true effect. The funnel plot and Egger's test help detect this, but they cannot fix it.

Pitfall 4: Confusing Statistical Significance with Clinical Significance

A meta-analysis of 100,000 participants may produce a p-value of 1e-30 for an odds ratio of 1.05. Statistically significant — but an OR of 1.05 means a 5% increase in risk. Is that clinically meaningful?

```
# Distinguish statistical from clinical significance

let examples = [
  { or: 1.05, p: 1e-30, n: 100000, interpretation: "Tiny effect, huge study"
},
  { or: 1.50, p: 0.04,  n: 200,    interpretation: "Moderate effect, small
study" },
  { or: 2.00, p: 1e-15, n: 5000,   interpretation: "Large effect, well-
powered" }
]

println("Statistical vs clinical significance:")
examples |> each(|e|
  println("  OR=" + str(e.or) + " p=" + str(e.p) + " n=" + str(e.n))
  println("    " + e.interpretation)
  println("")
)
```

Pitfall 5: Cherry-Picking Studies

Including or excluding studies based on their results (rather than predefined criteria) is a form of p-hacking. The inclusion criteria must be defined before looking at the data. This is why systematic reviews require a registered protocol.

Where Next?

Mendelian randomization — A causal inference method that uses genetic variants as “instruments” to estimate causal effects. It is a specialized form of meta-analysis where you combine multiple genetic instruments, each with an exposure effect and an outcome effect, to estimate the causal relationship between exposure and outcome.

Network meta-analysis — When multiple treatments have been compared in different studies (Drug A vs placebo, Drug B vs placebo, Drug A vs Drug B), network meta-analysis combines both direct and indirect comparisons to rank all treatments simultaneously.

Mega-analysis vs meta-analysis — Instead of combining summary statistics from published studies, mega-analysis combines individual-level data from all studies. This is more powerful (you can adjust for individual-level covariates) but requires data sharing agreements. The PGC and UK Biobank increasingly enable mega-analysis.

Multi-ancestry meta-analysis — Methods like MANTRA and MR-MEGA account for ancestry-specific linkage disequilibrium patterns when combining GWAS across ancestries. These are critical for identifying causal variants (which are shared across ancestries) vs tag variants (which are ancestry-specific).

```
# Summary of meta-analysis methods

println("=== Method Selection Guide ===")
println("")
println("Have effect sizes + SEs?")
println("  -> Inverse-variance weighted meta-analysis")
println("  -> Fixed-effects if I2 < 25%, random-effects otherwise")
println("")
println("Have only p-values?")
println("  -> Fisher's method or Stouffer's weighted Z")
println("")
println("Combining across traits?")
println("  -> Stouffer's Z (simple) or MTAG (accounts for correlations)")
println("")
println("Combining across ancestries?")
println("  -> Random-effects (treats ancestry as source of heterogeneity)")
println("  -> MANTRA or MR-MEGA (models LD differences)")
println("")
println("Suspecting publication bias?")
println("  -> Funnel plot + Egger's test")
println("  -> Trim-and-fill method to estimate corrected effect")
```

Prediction Intervals in Random-Effects Models

The confidence interval of a random-effects meta-analysis tells you the uncertainty in the average effect. But the prediction interval tells you the range in which a future study's true effect is likely to fall. The prediction interval is always wider than the confidence interval because it includes both the uncertainty in the mean AND the between-study variance.

```
# Prediction interval for a random-effects meta-analysis
# PI = pooled +/- t(df=k-2) * sqrt(tau2 + SE_pooled^2)

let t_crit = qnorm(0.975)  # approximate for df > 5
let pi_se = sqrt(tau2 + re_se_val * re_se_val)
let pi_lo = exp(re_pooled - t_crit * pi_se)
let pi_hi = exp(re_pooled + t_crit * pi_se)

println("=== Prediction Interval ===")
println("95% CI for the average effect: [" +
        str(round(re_pooled_ci_lo, 3)) + ", " +
        str(round(re_pooled_ci_hi, 3)) + "]")
println("95% PI for a new study: [" +
        str(round(pi_lo, 3)) + ", " + str(round(pi_hi, 3)) + "]")
println("")
println("The CI tells you: where is the average effect?")
println("The PI tells you: what would a new study likely find?")
println("")
println("If the PI includes 1.0 (for OR), a new study might find no effect")
println("even if the pooled estimate is significant")
```

Trim-and-Fill Method for Publication Bias Adjustment

When the funnel plot is asymmetric (suggesting publication bias), the trim-and-fill method estimates how many studies are missing and imputes their results to produce a bias-corrected pooled estimate.

```
# Trim-and-fill concept
# 1. Estimate the number of missing studies from funnel asymmetry
# 2. "Fill" the funnel with imputed mirror-image studies
# 3. Recompute the pooled estimate

# The adjusted estimate is always closer to the null (OR = 1.0)
# than the original estimate

println("Trim-and-fill adjusts for publication bias by:")
println("  1. Identifying the most extreme studies on the asymmetric side")
println("  2. Removing ('trimming') them to estimate the true center")
println("  3. Replacing them and adding their mirror images ('filling')")
println("  4. Recomputing the pooled estimate with all studies (real +
imputed)")
println("")
println("If the original and adjusted estimates are similar,")
println("publication bias is unlikely to change the conclusion")
println("If they differ substantially, the result should be interpreted
cautiously")
```

Handling Correlated Studies

In practice, GWAS meta-analyses often include studies with partially overlapping samples. This violates the independence assumption and produces anti-

conservative (too-small) p-values.

```
# Detecting sample overlap

# Common scenario: Study A uses UK Biobank discovery (n=200,000)
# Study B uses UK Biobank + additional samples (n=250,000)
# These overlap by 200,000 individuals

# Solutions:
# 1. Remove the overlapping study (lose information)
# 2. Use the subset without overlap from Study B
# 3. Use methods designed for correlated studies (e.g., MTAG)

println("Sample overlap detection checklist:")
println("  1. Check if studies used the same biobank or cohort")
println("  2. Check author affiliations and acknowledgments")
println("  3. Check supplementary tables for cohort-level details")
println("  4. When in doubt, contact the study authors")
println("")
println("Approximate correction: reduce effective sample size by")
println("the estimated overlap fraction")

# Example: correcting for 50% overlap between two studies
let n1 = 10000
let n2 = 8000
let overlap = 4000  # shared individuals

# Effective combined sample size (not n1 + n2)
let effective_n = n1 + n2 - overlap
println("Naive combined N: " + str(n1 + n2))
println("Effective N (corrected): " + str(effective_n))
```

Practical Workflow: GWAS Meta-Analysis Pipeline

```
# End-to-end GWAS meta-analysis workflow

# Step 1: Harmonize summary statistics
println("Step 1: Harmonize effect alleles across studies")
println("  - Ensure all studies report effects on the same allele")
println("  - Flip sign of beta (or invert OR) when effect allele differs")
println("  - Align to a common reference (e.g., GRCh38)")
println("")

# Step 2: Quality control per study
println("Step 2: Per-study QC")
println("  - Remove variants with MAF < 0.01")
println("  - Remove variants with imputation quality < 0.3")
println("  - Check lambda (genomic inflation factor) per study")
println("  - Flag outlier studies with lambda > 1.1")
println("")

# Step 3: Meta-analyze
println("Step 3: Meta-analysis")
println("  - Inverse-variance weighted fixed-effects")
println("  - Also compute random-effects for comparison")
println("  - Calculate heterogeneity (I-squared) per variant")
println("")

# Step 4: Post-meta-analysis QC
println("Step 4: Post-meta QC")
println("  - QQ plot of meta-analysis p-values")
println("  - Check genomic inflation of meta-analysis lambda")
println("  - Filter variants present in < 75% of studies")
println("  - Flag variants with I-squared > 75%")
println("")

# Step 5: Identify hits
println("Step 5: Identify genome-wide significant hits")
println("  - Threshold: p < 5e-8")
println("  - Define independent loci (LD clumping)")
println("  - Identify lead SNPs per locus")
println("")

# Step 6: Replication and follow-up
println("Step 6: Replication")
println("  - Test top hits in independent replication cohorts")
println("  - Compare discovery effect sizes to replication")
println("  - Expect regression to the mean (winner's curse)")
```

Effect Size Inflation and Correction

The winner’s curse can be formally corrected using conditional likelihood methods or bootstrap approaches:

```
# Simple winner's curse correction
# The conditional MLE adjusts the effect size for selection bias

# If a variant was selected because |z| > z_threshold:
let z_threshold = qnorm(1.0 - 5e-8 / 2)  # genome-wide significance
println("Genome-wide significance z-threshold: " + str(round(z_threshold,
2)))

# Observed z-score for a hit
let observed_z = 6.5
let observed_se = 0.05
let observed_beta = observed_z * observed_se

# Expected true effect (conditional MLE approximation)
# The true beta is approximately: observed_beta * (1 - threshold^2 / z^2)
let correction_factor = 1.0 - (z_threshold * z_threshold) / (observed_z *
observed_z)
let corrected_beta = observed_beta * max(correction_factor, 0.0)

println("Observed beta: " + str(round(observed_beta, 4)))
println("Correction factor: " + str(round(correction_factor, 4)))
println("Corrected beta: " + str(round(corrected_beta, 4)))
println("Shrinkage: " + str(round(100 * (1 - correction_factor), 1)) + "%")
println("")
println("Variants barely passing the threshold are most inflated")
println("Variants far exceeding the threshold are least affected")
```

Summary

Concept	Key Takeaway
Fixed-effects	Assumes one true effect; appropriate when I^2 < 25%
Random-effects	Allows varying true effects; always has wider CI
I-squared	Quantifies heterogeneity: 0%=none, 50%=moderate, 75%=high
Q statistic	Tests null hypothesis of no heterogeneity
Forest plot	Essential visualization: estimates, CIs, weights, pooled result
Funnel plot	Detects publication bias: asymmetry = concern
Winner’s curse	Discovery studies overestimate true effects
Fisher’s method	Combines p-values; sensitive to single strong result
Stouffer’s method	Weighted p-value combination; better for unequal sample sizes
Leave-one-out	Sensitivity check: is the result driven by one study?

Concept	Key Takeaway
Subgroup analysis	Explores sources of heterogeneity
Prediction interval	Where a new study's effect would fall
Publication bias	The biggest threat to meta-analysis validity

Appendix A: Statistical Decision Flowchart

This appendix provides a practical decision tree for choosing the right statistical test. Start with your question and data type, then follow the branches to the appropriate BioLang function.

Step 1: What Is Your Question?

Before choosing a test, classify your question:

- **Comparing groups**: Is there a difference between two or more groups?
- **Association**: Are two variables related?
- **Prediction**: Can one variable predict another?
- **Structure**: Are there natural groups in the data?
- **Survival**: Does time-to-event differ between groups?
- **Combining evidence**: What is the pooled result across studies?

Step 2: Comparing Two Groups

Are the observations paired or independent?

Paired means each observation in group A has a matched observation in group B (e.g., before/after treatment on the same patient, matched tumor/normal from the same individual).

Independent means the observations in the two groups come from different subjects with no inherent pairing.

Paired observations

If your data is **continuous and approximately normal**:

```
# Paired t-test
# Example: expression before vs after treatment, same patients
let before = [5.2, 4.8, 6.1, 5.5, 4.9, 5.8, 6.3]
let after = [4.1, 3.9, 5.2, 4.6, 3.8, 4.7, 5.1]
let result = ttest_paired(before, after)
println("Paired t-test: p=" + str(round(result.pvalue, 4)))
```

If your data is **continuous but not normal**, or **ordinal**:

```
# Wilcoxon signed-rank test (paired version)
let result = wilcoxon(before, after)
println("Wilcoxon signed-rank: p=" + str(round(result.pvalue, 4)))
```

Independent observations, continuous outcome

If data is **approximately normal** (check with QQ plot, Shapiro-Wilk):

```
# Two-sample t-test (Welch's, unequal variance)
let group_a = [12.3, 14.5, 11.8, 13.2, 15.1]
let group_b = [9.8, 10.2, 8.5, 11.0, 9.3]
let result = ttest(group_a, group_b)
println("Two-sample t-test: p=" + str(round(result.pvalue, 4)))
```

If data is **not normal**, or has **outliers**, or is **ordinal**:

```
# Wilcoxon rank-sum test (Mann-Whitney U)
let result = wilcoxon(group_a, group_b)
println("Wilcoxon rank-sum: p=" + str(round(result.pvalue, 4)))
```

Independent observations, categorical outcome

If you have a **2x2 table** (two groups, two outcomes):

```
# If expected cell counts >= 5: chi-squared test
let observed = [45, 155, 12, 188]  # flattened 2x2
let result = chi_square(observed)
println("Chi-squared: p=" + str(round(result.pvalue, 4)))

# If any expected cell count < 5: Fisher's exact test
let table = [[45, 155], [12, 188]]
let result = fisher_exact(table)
println("Fisher's exact: p=" + str(round(result.pvalue, 4)))
```

One-sample test (comparing to a known value)

```
# One-sample t-test: is the mean different from a hypothesized value?
let measurements = [5.2, 4.8, 5.5, 5.1, 4.9, 5.3]
let result = ttest_one(measurements, mu: 5.0)
println("One-sample t-test: p=" + str(round(result.pvalue, 4)))
```

Step 3: Comparing Three or More Groups

Are the observations independent?

If **independent** and **continuous, approximately normal**:

```
# One-way ANOVA
let group1 = [12.1, 13.4, 11.8, 14.2]
let group2 = [9.5, 10.1, 8.8, 11.3]
let group3 = [15.2, 16.0, 14.5, 15.8]
let result = anova(group1, group2, group3)
println("ANOVA F-test: p=" + str(round(result.pvalue, 4)))
# If significant, follow up with pairwise t-tests + p_adjust
```

If **independent** and **not normal** or **ordinal**:

```
# Kruskal-Wallis test (use ks_test as non-parametric alternative)
# For pairwise comparisons: Wilcoxon with p_adjust
let p1 = wilcoxon(group1, group2).pvalue
let p2 = wilcoxon(group1, group3).pvalue
let p3 = wilcoxon(group2, group3).pvalue
let adjusted = p_adjust([p1, p2, p3], "BH")
println("Adjusted p-values: " + str(adjusted))
```

If **categorical outcome** (contingency table larger than 2x2):

```
# Chi-squared test of independence
let observed = [30, 20, 50, 45, 35, 20]  # flattened contingency table
let result = chi_square(observed)
println("Chi-squared: p=" + str(round(result.pvalue, 4)))
```

Step 4: Association Between Two Variables

Both continuous?

If **linear relationship expected** and **bivariate normal**:

```
# Pearson correlation
let x = [1.2, 2.3, 3.1, 4.5, 5.2]
let y = [2.1, 4.0, 5.8, 8.2, 10.1]
let result = cor(x, y)
println("Pearson r: " + str(round(result, 4)))
```

If **monotonic but not necessarily linear**, or **non-normal**:

```
# Spearman rank correlation
let result = spearman(x, y)
println("Spearman rho: " + str(round(result.rho, 4)) +
        " p=" + str(round(result.pvalue, 4)))
```

If **robustness to outliers** is critical:

```
# Kendall's tau (more robust than Spearman)
let result = kendall(x, y)
println("Kendall tau: " + str(round(result.tau, 4)) +
        " p=" + str(round(result.pvalue, 4)))
```

One continuous, one categorical (2+ categories)?

This is the same as comparing groups (Step 2 or Step 3). The “association” between a continuous variable and a categorical variable is tested by comparing the continuous variable across the categories.

Step 5: Prediction and Modeling

Continuous outcome, one or more continuous predictors?

```
# Linear regression
let outcome = [10, 12, 14, 16, 18]
let predictor = [1, 2, 3, 4, 5]
let result = lm(outcome, predictor)
println("Linear model: R2=" + str(round(result.r_squared, 4)) +
        " p=" + str(round(result.pvalue, 4)))
```

Binary outcome (yes/no, case/control)?

```
# Logistic regression (GLM with binomial family)
let outcome = [0, 0, 1, 1, 1, 0, 1, 1]
let predictor_table = table([
  { age: 45, bmi: 22 },
  { age: 50, bmi: 25 },
  { age: 55, bmi: 28 },
  { age: 60, bmi: 30 },
  { age: 65, bmi: 32 },
  { age: 48, bmi: 23 },
  { age: 62, bmi: 29 },
  { age: 70, bmi: 31 }
])
let result = glm(outcome, predictor_table, family: "binomial")
println("Logistic regression: " + str(result))
```

Count outcome (number of mutations, reads)?

```
# Poisson or negative binomial GLM
let counts = [5, 12, 3, 45, 8, 22]
let result = glm(counts, predictor_table, family: "poisson")
println("Poisson GLM: " + str(result))
```

Time-to-event outcome?

```
# Cox proportional hazards model
let times = [12, 24, 6, 36, 18, 30]
let events = [1, 0, 1, 0, 1, 1]  # 1=event, 0=censored
let covariates = table([{ age: 55, stage: 2 }, { age: 60, stage: 1 },
  { age: 48, stage: 3 }, { age: 65, stage: 1 },
  { age: 52, stage: 2 }, { age: 70, stage: 3 }])
let result = cox_ph(times, events, covariates)
println("Cox PH: " + str(result))

# Kaplan-Meier survival curves (non-parametric)
let km = kaplan_meier(times, events)
println("Median survival: " + str(km.median_survival))
```

Step 6: Multivariate Structure

Dimensionality reduction?

```
# PCA — linear, deterministic, quantitative
let result = pca(data_matrix, n_components: 10)

# UMAP — nonlinear, for visualization
let result = umap(pca_coords, n_neighbors: 15, min_dist: 0.1)

# t-SNE — nonlinear, local structure emphasis
let result = tsne(pca_coords, perplexity: 30)
```

Clustering?

```
# K-means — when you know K, spherical clusters
let result = kmeans(data_matrix, k: 5)

# Leiden — graph-based, no K required, standard for single-cell
let labels = leiden(pca_coords, resolution: 1.0)
```

Step 7: Multiple Testing Correction

Whenever you perform more than one test, adjust p-values:

```
# Benjamini-Hochberg (FDR control) — standard for genomics
let adjusted = p_adjust(pvalues, "BH")

# Bonferroni (FWER control) — conservative, for few tests
let adjusted = p_adjust(pvalues, "bonferroni")
```

Quick Reference Table

Question	Data Type	Normal?	Test	BioLang
2 groups, paired	Continuous	Yes	Paired t-test	`ttest_paired`
2 groups, paired	Continuous	No	Wilcoxon signed-rank	`wilcoxon`
2 groups, independent	Continuous	Yes	Welch's t-test	`ttest`
2 groups, independent	Continuous	No	Wilcoxon rank-sum	`wilcoxon`
2 groups, independent	Categorical	N/A	Chi-squared / Fisher's	`chi_square` / `fisher_exact`
3+ groups, independent	Continuous	Yes	ANOVA	`anova`
3+ groups, independent	Continuous	No	Pairwise Wilcoxon + FDR	`wilcoxon` + `p_adjust`
1 sample vs value	Continuous	Yes	One-sample t-test	`ttest_one`
Association	Both continuous	Yes	Pearson correlation	`cor`
Association	Both continuous	No	Spearman correlation	`spearman`
Association	Both continuous	Robust	Kendall correlation	`kendall`
Prediction	Continuous outcome	-	Linear regression	`lm`

Question	Data Type	Normal?	Test	BioLang
Prediction	Binary outcome	-	Logistic regression	`glm` (binomial)
Prediction	Count outcome	-	Poisson/NB regression	`glm` (poisson)
Time-to-event	Survival	-	Cox PH	`cox_ph`
Time-to-event	Survival curves	-	Kaplan-Meier	`kaplan_meier`
Reduce dimensions	High-D matrix	-	PCA	`pca`
Find groups	Matrix	-	K-means / Leiden	`kmeans` / `leiden`
Multiple tests	P-value list	-	FDR correction	`p_adjust`
Distribution fit	Sample vs theory	-	KS test	`ks_test`

Decision Rules of Thumb

When in doubt about normality: Use the non-parametric alternative. You lose very little power if the data actually is normal (~95% efficiency for Wilcoxon vs t-test), but you avoid catastrophic Type I error if it is not.

When in doubt about the number of groups: Start with the omnibus test (ANOVA or chi-squared), then do pairwise comparisons with FDR correction. Never skip the omnibus test and jump to pairwise comparisons.

When in doubt about multiple testing: Correct. Always. The BH method at FDR < 0.05 is the standard for genomics.

When in doubt about independence: If observations might be correlated (repeated measures, family data, spatial data), standard tests give falsely small p-values. Use mixed models or account for the correlation.

When in doubt about which regression: Start with the simplest model (linear, logistic, or Cox depending on outcome type). Add complexity only if diagnostics indicate the simple model is inadequate.

Appendix B: Distribution Reference

This appendix provides a quick reference for the probability distributions used throughout this book. For each distribution: what it is, its parameters, the BioLang functions, when it appears in genomics, and a concrete example.

Normal (Gaussian) Distribution

Definition: The bell curve. Defined by its mean (mu) and standard deviation (sigma). Most naturally occurring continuous measurements cluster symmetrically around the mean, with approximately 68% within one SD, 95% within two SDs, and 99.7% within three SDs.

Parameters: mu (mean), sigma (standard deviation)

BioLang functions:

```
# Density: probability density at a point
let density = dnorm(1.96, mean: 0, sd: 1)
println("dnorm(1.96) = " + str(round(density, 6)))  # 0.058441

# CDF: probability of being less than or equal to a value
let prob = pnorm(1.96, mean: 0, sd: 1)
println("pnorm(1.96) = " + str(round(prob, 6)))  # 0.975000

# Quantile: value at a given probability
let quantile_val = qnorm(0.975, mean: 0, sd: 1)
println("qnorm(0.975) = " + str(round(quantile_val, 4)))  # 1.9600

# Random generation
let samples = rnorm(1000, mean: 0, sd: 1)
println("Generated " + str(len(samples)) + " standard normal values")
println("Sample mean: " + str(round(mean(samples), 4)))
println("Sample SD: " + str(round(stdev(samples), 4)))
```

When it appears in genomics:

- Log-transformed gene expression values are approximately normal
- Test statistics (t, Z) follow normal or near-normal distributions under the null hypothesis
- QQ plots compare observed quantiles to normal quantiles
- Measurement error in continuous assays (mass spec, qPCR) is often normal

Example: Log2 fold changes from a differential expression analysis are approximately normally distributed around zero under the null hypothesis. Genes

with |log2FC| > 2 are more than 2 standard deviations from the mean, corresponding to roughly the most extreme 5% if the SD is about 1.

Binomial Distribution

Definition: The number of successes in n independent trials, each with probability p of success. Discrete distribution.

Parameters: n (number of trials), p (probability of success per trial)

BioLang functions:

```
# Probability of exactly k successes
let prob = dbinom(3, n: 10, p: 0.3)
println("P(X=3 | n=10, p=0.3) = " + str(round(prob, 6)))

# Cumulative probability: P(X <= k)
let cum_prob = pbinom(3, n: 10, p: 0.3)
println("P(X<=3 | n=10, p=0.3) = " + str(round(cum_prob, 6)))

# Random generation
let samples = rbinom(1000, n: 10, p: 0.3)
println("Sample mean: " + str(round(mean(samples), 2)) + " (expected: 3.0)")
```

When it appears in genomics:

- Variant allele frequency: the number of alternate alleles in 2n chromosomes
- SNP genotype frequencies under Hardy-Weinberg equilibrium
- Success/failure outcomes in enrichment tests (is this gene in the pathway?)
- Coin-flip model for strand bias in sequencing reads

Example: At a heterozygous site, each sequencing read has a 50% chance of carrying the reference or alternate allele. With 100 reads, the number of alternate reads follows Binomial(n=100, p=0.5). Observing 80 alternate reads would be highly unlikely ($p < 0.001$) and suggests allelic imbalance or a copy number variant.

Poisson Distribution

Definition: The number of events in a fixed interval when events occur independently at a constant average rate. Discrete distribution.

Parameters: lambda (rate, which equals both the mean and the variance)

BioLang functions:

```
# Probability of exactly k events
let prob = dpois(5, lambda: 3.0)
println("P(X=5 | lambda=3) = " + str(round(prob, 6)))

# Cumulative probability: P(X <= k)
let cum_prob = ppois(5, lambda: 3.0)
println("P(X<=5 | lambda=3) = " + str(round(cum_prob, 6)))

# Random generation
let samples = rpois(1000, lambda: 3.0)
println("Sample mean: " + str(round(mean(samples), 2)) + " (expected: 3.0)")
println("Sample variance: " + str(round(variance(samples), 2)) + " (expected:
3.0)")
```

When it appears in genomics:

- Read count distribution in low-coverage sequencing
- Number of mutations per gene (somatic mutation models)
- ChIP-seq peak calling (Poisson background model)
- Rare variant counts in a genomic region

Example: If the average somatic mutation rate is 3 mutations per megabase, the number of mutations in a 1 Mb region follows Poisson(lambda=3). The probability of seeing 10 or more mutations in such a region is very small, which would suggest a mutational hotspot or defective DNA repair.

Negative Binomial Distribution

Definition: Like the Poisson, but with an extra parameter to allow the variance to exceed the mean (overdispersion). This is the key distribution for RNA-seq count data.

Parameters: mu (mean), size/r (dispersion — smaller r means more overdispersion). When r approaches infinity, the negative binomial approaches the Poisson.

When it appears in genomics:

- **DESeq2** and **edgeR** model RNA-seq read counts as negative binomial
- The biological variation between replicates causes overdispersion (variance > mean), which the Poisson cannot capture
- scRNA-seq count models (e.g., in scanpy, Seurat)

Example: A gene with mean count of 100 across replicates might have variance of 500 (5x the mean). The Poisson would predict variance = 100. The negative binomial with mu=100 and appropriate dispersion fits the observed variance. This is why DESeq2 uses a negative binomial model — using a Poisson would dramatically underestimate the variance and produce thousands of false positives.

```
# Negative binomial as overdispersed Poisson
# Compare variance in Poisson vs observed RNA-seq counts
let poisson_samples = rpois(100, lambda: 100.0)
let poisson_var = variance(poisson_samples)
println("Poisson(lambda=100) variance: " + str(round(poisson_var, 1)))
println("Expected: ~100")
println("")
println("Typical RNA-seq gene with mean=100:")
println("  Observed variance: ~400-800 (overdispersed)")
println("  Negative binomial handles this; Poisson does not")
```

Uniform Distribution

Definition: All values in the interval [a, b] are equally likely.

Parameters: a (minimum), b (maximum)

When it appears in genomics:

- P-values under the null hypothesis follow Uniform(0, 1) — this is the basis of QQ plots and calibration checks
- Random positions on a chromosome (null model for enrichment)
- Permutation test reference distributions

Example: If no gene is truly differentially expressed, all p-values should be uniformly distributed between 0 and 1. A QQ plot comparing observed p-values to Uniform(0,1) quantiles reveals departure from the null — an excess of small p-values indicates true signal.

```
# P-value calibration check
# Under the null: p-values should be uniform
let null_pvals = rnorm(1000) |> map(|z|
  2.0 * (1.0 - pnorm(abs(z)))  # two-sided p from standard normal z
)

# Check uniformity with a QQ plot
qq_plot(null_pvals,
  title: "P-value QQ Plot (null simulation)",
  x_label: "Expected -log10(p)",
  y_label: "Observed -log10(p)")
  |> save_plot("pvalue_qq_null.svg")
```

Exponential Distribution

Definition: The time between events in a Poisson process. Continuous, always positive, memoryless.

Parameters: rate (lambda = 1/mean)

When it appears in genomics:

- Distances between mutations along a chromosome (if mutations are random)
- Inter-event times in sequencing or processing
- Waiting times in continuous-time Markov models of sequence evolution

Example: If mutations occur at rate 1 per megabase, the distance between consecutive mutations follows Exponential(rate=1/Mb). Clusters of mutations closer together than expected suggest localized hypermutation (kataegis).

Chi-Squared Distribution

Definition: The sum of squares of k independent standard normal variables. Always positive, right-skewed.

Parameters: df (degrees of freedom)

When it appears in genomics:

- Test statistic for chi-squared test of independence
- Cochran's Q statistic in meta-analysis
- Likelihood ratio tests in GLMs
- Hardy-Weinberg equilibrium test

```
# Chi-squared test for Hardy-Weinberg equilibrium
let observed = [210, 480, 310]  # AA, Aa, aa
let n = sum(observed)
let p = (2.0 * observed[0] + observed[1]) / (2.0 * n)
let q = 1.0 - p
let expected = [p * p * n, 2.0 * p * q * n, q * q * n]

let result = chi_square(observed, expected)
println("HWE test: chi2=" + str(round(result.statistic, 3)) +
        " df=" + str(result.df) + " p=" + str(round(result.pvalue, 4)))
```

F Distribution

Definition: The ratio of two chi-squared variables, each divided by their degrees of freedom. Used for comparing variances.

Parameters: df1 (numerator df), df2 (denominator df)

When it appears in genomics:

- ANOVA F-statistic
- Comparing variance between conditions (Levene's test)
- Model comparison in regression (nested models)

```
# ANOVA produces an F-statistic
let brain = [12.1, 13.4, 11.8, 14.2, 12.9]
let liver = [9.5, 10.1, 8.8, 11.3, 10.5]
let lung = [15.2, 16.0, 14.5, 15.8, 14.9]

let result = anova(brain, liver, lung)
println("ANOVA: F=" + str(round(result.f_statistic, 2)) +
        " df_between=" + str(result.df_between) +
        " df_within=" + str(result.df_within) +
        " p=" + str(round(result.pvalue, 6)))
```

Beta Distribution

Definition: Continuous distribution on [0, 1]. Extremely flexible shape controlled by two parameters. Used for modeling probabilities and proportions.

Parameters: alpha (shape1), beta (shape2)

When it appears in genomics:

- Prior distribution for allele frequencies in Bayesian methods
- Methylation beta values (proportion of methylated CpGs)
- Proportion data in general (percent reads mapped, GC content)
- The conjugate prior for the binomial distribution

Example: CpG methylation beta values range from 0 (unmethylated) to 1 (fully methylated). In many tissues, the distribution is bimodal — most CpGs are either fully methylated (~0.9) or fully unmethylated (~0.1), with few in between. This can be modeled as a mixture of two Beta distributions.

Distribution Selection Guide

Data Type	Typical Distribution	BioLang Functions
Continuous measurements	Normal	`dnorm`, `pnorm`, `qnorm`, `rnorm`
Counts (low overdispersion)	Poisson	`dpois`, `ppois`, `rpois`
Counts (high overdispersion)	Negative Binomial	(modeled via `glm`)
Success/failure counts	Binomial	`dbinom`, `pbinom`, `rbinom`
Proportions (0-1)	Beta	(modeled via `glm`)
Times between events	Exponential	(modeled via `glm`)
Test statistics	Chi-squared, F, t	(produced by test functions)
P-values (under null)	Uniform(0,1)	(checked via `qq_plot`)

The t-Distribution

Definition: Similar to the normal distribution but with heavier tails. Arises when estimating the mean of a normally distributed population with unknown variance. As degrees of freedom increase, it approaches the normal.

Parameters: df (degrees of freedom)

When it appears in genomics:

- Test statistic for t-tests (one-sample, two-sample, paired)
- Moderated t-statistics in limma for differential expression
- Confidence intervals for regression coefficients

```
# The t-distribution has heavier tails than the normal
# This matters most for small sample sizes

# Compare critical values: t vs normal
let z_crit = qnorm(0.975)    # 1.96
println("Normal 97.5th percentile: " + str(round(z_crit, 4)))
println("")
println("t-distribution 97.5th percentile by df:")
println("  df=3:  ~3.18 (much wider CI than normal)")
println("  df=5:  ~2.57")
println("  df=10: ~2.23")
println("  df=30: ~2.04")
println("  df=100: ~1.98 (approaching normal)")
println("  df=Inf: 1.96 (equals normal)")
println("")
println("With small samples (n < 30), using the normal instead of t")
println("underestimates uncertainty and produces anti-conservative p-values")
```

Example: When comparing gene expression between 5 tumor samples and 5 normal samples, the t-statistic has df approximately 8. The critical value for significance at alpha=0.05 is 2.31, not 1.96. Using the normal approximation would make results appear more significant than they are.

Key Relationships Between Distributions

- **Binomial(n, p)** approaches **Normal(np, np(1-p))** when n is large
- **Poisson(lambda)** is a special case of **Negative Binomial** as r -> infinity
- **Poisson(lambda)** approaches **Normal(lambda, lambda)** when lambda is large
- **Exponential(rate)** is a special case of **Gamma(1, rate)**
- **Chi-squared(df)** is a special case of **Gamma(df/2, 1/2)**
- **Beta(1, 1)** is the same as **Uniform(0, 1)**
- The **t-distribution** approaches the **Normal** as df increases

These relationships explain why some tests work interchangeably: for large sample sizes, a chi-squared test and a z-test give the same p-value because the binomial counts are approximately normal.

Appendix C: Glossary

A

Adjusted Rand Index (ARI): A measure of agreement between two clusterings, corrected for chance. ARI = 0 means no better than random, ARI = 1 means perfect agreement. Used to compare unsupervised clusters to known labels (e.g., k-means clusters vs PAM50 subtypes). See Chapter 14.

AIC (Akaike Information Criterion): A model selection criterion that balances goodness of fit against model complexity. Lower AIC is better. AIC = 2k - 2ln(L), where k is the number of parameters and L is the likelihood. Prefers simpler models when fit is comparable.

Alternative Hypothesis (H1): The hypothesis that there is an effect or difference. In a two-sample t-test, the alternative hypothesis is that the two group means are not equal. The p-value is computed under the assumption that the null hypothesis is true.

ANOVA (Analysis of Variance): A statistical test comparing means across three or more groups. Tests whether any group mean differs from the others. The F-statistic is the ratio of between-group variance to within-group variance. BioLang: `anova()`. See Chapter 6.

B

Batch Effect: A systematic technical artifact that affects measurements from different experimental batches (sequencing lanes, processing dates, laboratories). Batch effects can be larger than biological signal and must be identified and corrected before analysis. See Chapter 12.

Bayes Factor: The ratio of the likelihood of the data under two competing hypotheses. Unlike p-values, Bayes factors directly compare hypotheses and can provide evidence for the null.

Benjamini-Hochberg (BH) Procedure: The standard method for controlling the false discovery rate in multiple testing. Ranks p-values, then finds the largest rank i where p(i) <= i/m * alpha. All tests with rank <= i are called significant. BioLang: `p_adjust(pvals, "BH")`. See Chapter 8.

BIC (Bayesian Information Criterion): A model selection criterion similar to AIC but with a stronger penalty for model complexity. BIC = k*ln(n) - 2ln(L). Tends to select simpler models than AIC, especially for large n.

Biological Replicate: An independent biological sample (different mouse, different patient, different cell culture). Biological replicates capture biological variability and are essential for valid statistical inference. Contrast with technical replicate.

Bonferroni Correction: The simplest multiple testing correction: multiply each p-value by the number of tests (or equivalently, divide the significance threshold by the number of tests). Controls the family-wise error rate but is very conservative. BioLang: `p_adjust(pvals, "bonferroni")`.

Box Plot: A visualization showing the median, interquartile range (box), and range (whiskers) of a distribution. Useful for comparing distributions across groups. BioLang: `boxplot()`.

C

Censoring: In survival analysis, an observation is censored when the event of interest has not occurred by the end of follow-up. Right censoring (the most common type) means the patient was alive at last contact but their eventual survival time is unknown.

Chi-Squared Test: A test for association between categorical variables, or for goodness of fit to an expected distribution. Compares observed counts to expected counts. BioLang: `chi_square()`. See Chapter 5.

Clustering: Unsupervised method for grouping observations based on similarity. Common methods: k-means, hierarchical, Leiden. See Chapter 14.

Cohen's d: A standardized effect size for the difference between two group means. d = (mean1 - mean2) / pooled_SD. Small: d=0.2, medium: d=0.5, large: d=0.8. Independent of sample size, unlike p-values.

Confidence Interval (CI): A range of values that, with a specified probability (typically 95%), contains the true population parameter. A 95% CI means: if you repeated the experiment many times, 95% of the resulting CIs would contain the true value. It does NOT mean there is a 95% probability the true value is in this specific interval.

Confounder: A variable that is associated with both the exposure and the outcome, distorting the apparent relationship between them. Example: age confounds the association between gray hair and cancer because age causes both. See Chapter 12.

Correlation: A measure of linear association between two variables. Pearson's r ranges from -1 (perfect negative) to +1 (perfect positive). r=0 means no linear relationship. BioLang: `cor()`. See Chapter 9.

Covariance Matrix: A matrix where entry (i,j) is the covariance between variables i and j. The diagonal contains the variances. Used in PCA, multivariate tests, and mixed models. BioLang: `cov_matrix()`.

Cox Proportional Hazards: A semi-parametric regression model for time-to-event data. Estimates hazard ratios for each covariate. Assumes proportional hazards (the ratio of hazards is constant over time). BioLang: `cox_ph()`. See Chapter 11.

D

Degrees of Freedom (df): The number of independent pieces of information used to estimate a parameter. For a t-test with n observations, df = n-1 (or approximately n1+n2-2 for two-sample). More df means more precise estimates and narrower confidence intervals.

Density Plot: A smoothed version of a histogram showing the probability density function. BioLang: `histogram()`.

Differential Expression: Identifying genes whose expression differs between conditions. Uses negative binomial models (DESeq2, edgeR) for count data or linear models for normalized data.

Distance Matrix: A matrix where entry (i,j) is the distance between observations i and j. Used for clustering and dimensionality reduction. BioLang: `dist_matrix()`. Common metrics: Euclidean, Manhattan, correlation.

E

Effect Size: A quantitative measure of the magnitude of a phenomenon. Examples: Cohen's d (standardized mean difference), odds ratio, hazard ratio,

correlation coefficient. Unlike p-values, effect sizes are independent of sample size.

Eigenvalue: In PCA, the eigenvalue of a principal component equals the variance explained by that component. Eigenvalues are obtained from the eigenvalue decomposition of the covariance matrix. BioLang: `eigenvalues()`.

Elbow Plot (Scree Plot): A plot of eigenvalues vs PC number used to choose the number of principal components. The "elbow" is where the curve flattens, indicating that additional PCs capture mostly noise.

Enrichment Analysis: Testing whether a set of genes (e.g., differentially expressed genes) is enriched for a particular pathway or GO term beyond what would be expected by chance. BioLang: `enrich()`. Uses Fisher's exact test or hypergeometric test.

F

False Discovery Rate (FDR): The expected proportion of false positives among all rejected hypotheses. FDR = 5% means that among all genes called significant, 5% are expected to be false positives. Controlled by the Benjamini-Hochberg procedure. See Chapter 8.

Family-Wise Error Rate (FWER): The probability of making at least one false positive across all tests. More conservative than FDR. Controlled by Bonferroni or Holm corrections.

Fisher's Exact Test: An exact test for association in a 2x2 contingency table. Unlike the chi-squared test, it is valid for small sample sizes. BioLang: `fisher_exact()`.

Fixed-Effects Model: In meta-analysis, assumes all studies estimate the same true effect. Appropriate when heterogeneity is low (I-squared < 25%). See Chapter 15.

Fold Change: The ratio of expression in one condition to another. A fold change of 2 means the gene is twice as highly expressed. Often reported on the log2 scale: log2FC = 1 corresponds to a 2-fold increase.

Forest Plot: The standard meta-analysis visualization showing each study's estimate with confidence interval, plus the pooled estimate. BioLang: `forest_plot()`. See Chapter 15.

G

GLM (Generalized Linear Model): An extension of linear regression to non-normal outcomes. Includes logistic regression (binary outcome), Poisson regression (count outcome), and others. Uses a link function to relate the linear predictor to the mean. BioLang: `glm()`. See Chapter 10.

H

Hazard Ratio (HR): In survival analysis, the ratio of hazard rates between two groups. HR=2 means the event rate is twice as high in one group. HR=1 means no difference. Estimated by Cox proportional hazards model.

Heatmap: A matrix visualization where values are represented by color intensity. Used to display expression matrices, correlation matrices, and distance matrices. BioLang: `heatmap()`.

Heterogeneity: In meta-analysis, the degree to which true effect sizes vary across studies. Measured by I-squared and Cochran's Q. High heterogeneity (I-squared > 75%) suggests the studies are measuring different things.

Histogram: A visualization showing the frequency distribution of a continuous variable. BioLang: `histogram()`.

I

I-Squared (I2): The percentage of total variability in a meta-analysis that is due to true between-study heterogeneity (rather than sampling error). 0% = no heterogeneity, 25% = low, 50% = moderate, 75% = high. See Chapter 15.

Independent Filtering: In multiple testing, removing tests with low overall signal (e.g., low-expression genes) before FDR correction. This increases power by reducing the number of tests without biasing the results. See Chapter 8.

K

Kaplan-Meier Estimator: A non-parametric method for estimating the survival function from time-to-event data with censoring. Produces a step function that

decreases at each event time. BioLang: `kaplan_meier()`. See Chapter 11.

K-Means Clustering: An unsupervised method that partitions n observations into k clusters by minimizing within-cluster sum of squares. Requires specifying k in advance. Finds spherical clusters. BioLang: `kmeans()`. See Chapter 14.

L

Leiden Algorithm: A community detection algorithm for graph-based clustering. Widely used in single-cell genomics. Does not require specifying the number of clusters; uses a resolution parameter instead. BioLang: `leiden()`. See Chapter 14.

Linear Regression: Models the relationship between a continuous outcome and one or more predictors as a linear function. Estimates slope (effect size), intercept, and R-squared. BioLang: `lm()`. See Chapter 9.

Link Function: In GLMs, the function that relates the linear predictor to the expected value of the outcome. Logistic regression uses the logit link; Poisson regression uses the log link; linear regression uses the identity link.

Log Transformation: Applying the logarithm (usually log2 or log10) to data. Common in genomics because expression data spans several orders of magnitude and raw counts are right-skewed. Log-transformed data is often approximately normal.

M

Meta-Analysis: The statistical method for combining results from multiple independent studies. Produces a pooled effect estimate with proper uncertainty. See Chapter 15.

Multiple Testing Problem: When performing many simultaneous tests (e.g., testing 20,000 genes), the probability of at least one false positive increases dramatically. At alpha=0.05 with 20,000 tests, expect 1,000 false positives by chance. Must be corrected using FDR or FWER methods. See Chapter 8.

N

Negative Binomial Distribution: A count distribution that allows variance to exceed the mean (overdispersion). The standard model for RNA-seq count data. Used by DESeq2 and edgeR.

Non-Parametric Test: A statistical test that makes no assumptions about the distribution of the data. Examples: Wilcoxon rank-sum, Spearman correlation, Kruskal-Wallis. Generally less powerful than parametric tests when assumptions hold, but more robust when they do not.

Normal Distribution: See Appendix B. The bell curve, parameterized by mean and standard deviation.

Null Hypothesis (H0): The hypothesis of no effect or no difference. In a t-test comparing treatment vs control, H0 is that the true means are equal. The p-value is the probability of observing data as extreme as yours if H0 is true.

O

Odds Ratio (OR): The ratio of the odds of an event in one group to the odds in another. OR=1 means no association. OR>1 means higher odds in the first group. Used in case-control studies and logistic regression.

Overdispersion: When the observed variance exceeds the variance predicted by a model. Common in count data: RNA-seq counts have more variance than the Poisson distribution predicts. Addressed by using the negative binomial distribution.

P

p-value: The probability of observing a test statistic as extreme as (or more extreme than) the one computed from the data, assuming the null hypothesis is true. It is NOT the probability that the null hypothesis is true. It is NOT the probability that the result is due to chance.

Parametric Test: A statistical test that assumes a specific distribution for the data (usually normal). Examples: t-test, ANOVA, Pearson correlation. More powerful than non-parametric tests when assumptions hold.

PCA (Principal Component Analysis): A dimensionality reduction method that finds orthogonal directions of maximum variance. BioLang: `pca()`. See Chapter 13.

Poisson Distribution: See Appendix B. A count distribution where mean equals variance. Used for modeling rare events.

Power: The probability of correctly rejecting the null hypothesis when it is false (i.e., detecting a true effect). Power = 1 - Type II error rate. Depends on sample size, effect size, significance level, and variability. See Chapter 4.

Pseudoreplication: Treating non-independent observations as independent replicates. Example: measuring the same mouse 5 times and calling it n=5 instead of n=1 with 5 technical replicates. Inflates sample size and produces falsely significant p-values.

Publication Bias: The tendency for studies with significant results to be published while null results remain unpublished. Biases meta-analyses toward overestimating effect sizes.

Q

QQ Plot (Quantile-Quantile Plot): A plot comparing observed quantiles to theoretical quantiles (usually normal or uniform). Points on the diagonal indicate the data follows the theoretical distribution. Deviations reveal non-normality, inflation, or true signal. BioLang: `qq_plot()`.

R

R-Squared (R2): The proportion of variance in the outcome explained by the predictor(s). R2 = 0 means no explanatory power, R2 = 1 means perfect prediction. In genomics, R2 is often small (0.01-0.10 for individual SNPs) because genetic effects are typically modest.

Random-Effects Model: In meta-analysis, assumes that true effect sizes vary between studies according to a distribution. Produces wider confidence intervals than fixed-effects. Appropriate when I-squared > 25%. See Chapter 15.

Residual: The difference between an observed value and the value predicted by a model. Examining residuals reveals model misspecification (patterns in residuals indicate the model is wrong).

S

Sensitivity: The probability of a positive test result given that the condition is truly present. Also called the true positive rate or recall. Sensitivity = TP / (TP + FN).

Silhouette Score: A measure of cluster quality. For each observation, it compares the average distance to same-cluster points vs the average distance to the nearest other cluster. Ranges from -1 (wrong cluster) to +1 (well-clustered). See Chapter 14.

Specificity: The probability of a negative test result given that the condition is truly absent. Also called the true negative rate. Specificity = TN / (TN + FP).

Standard Error (SE): The standard deviation of a sampling distribution. For the sample mean, SE = SD / sqrt(n). Measures the precision of an estimate. Smaller SE = more precise estimate.

Survival Analysis: Statistical methods for analyzing time-to-event data, accounting for censoring. Key methods: Kaplan-Meier curves, log-rank test, Cox proportional hazards. BioLang: `kaplan_meier()`, `cox_ph()`. See Chapter 11.

T

t-SNE (t-distributed Stochastic Neighbor Embedding): A nonlinear dimensionality reduction method for visualization. Preserves local structure but NOT global distances or cluster sizes. BioLang: `tsne()`. See Chapter 13.

Technical Replicate: Repeated measurements of the same biological sample (same RNA extraction sequenced twice, same sample run on two lanes). Technical replicates measure measurement precision but do NOT capture biological variability. They cannot substitute for biological replicates in statistical tests.

Type I Error (False Positive): Rejecting the null hypothesis when it is actually true. The significance level (alpha) is the maximum acceptable Type I error rate. Alpha = 0.05 means you accept a 5% chance of a false positive.

Type II Error (False Negative): Failing to reject the null hypothesis when it is actually false. Beta is the Type II error rate. Power = 1 - beta.

U

UMAP (Uniform Manifold Approximation and Projection): A nonlinear dimensionality reduction method that preserves both local and some global structure. Faster than t-SNE and more reproducible. Standard for single-cell visualization. BioLang: `umap()`. See Chapter 13.

V

Variance: The average squared deviation from the mean. Measures the spread of a distribution. BioLang: `variance()`. Standard deviation is the square root of variance.

Volcano Plot: A scatter plot of effect size (log2 fold change) vs significance (-log10 p-value). Points in the upper corners are both statistically significant and biologically meaningful. BioLang: `volcano()`.

W

Wilcoxon Rank-Sum Test (Mann-Whitney U): A non-parametric test for comparing two independent groups. Ranks all observations and compares the sum of ranks between groups. Does not assume normality. BioLang: `wilcoxon()`. See Chapter 7.

Winner's Curse: The phenomenon where the effect size estimated in a discovery study is systematically larger than the true effect. Occurs because studies that detect significant effects (and get published) tend to overestimate the effect size. Replication studies typically show smaller effects. See Chapter 15.